

# Technical Model Book

Applied Elements materials process, plant and economics platform

---

The reference document for understanding and auditing every model in `src/ae`.

Each chapter states one module's purpose, its governing equations with their derivation, a symbol table with units, a worked example whose numbers were REPRODUCED BY EXECUTION for this build, its validation status against a named benchmark, and its limitations.

**On the Vikarabad deposit.** The deposit is UNCHARACTERIZED in the citable record. No assay campaign has been run, so no impurity concentration, lattice fraction, purification ceiling or recovery in this book is a property of that ore. Every such number is either a literature value for a different deposit, with its citation, or a scenario input tagged ASSUMED. A grade-conditional result is a scenario gated on a future assay campaign, and this book never presents one as a finding about the resource.

Generated by `docs/modelbook/build.py` from the docstrings in `src/ae`. Prose in the module chapters is the module's own prose. Build metadata, including the measured test tally and the reproduction status of every worked example, is in the findings chapter.

Commit 64c741b   Built 2026-09-17

# Contents

|                                                                                                |           |
|------------------------------------------------------------------------------------------------|-----------|
| How to read this book                                                                          | 7         |
| <b>1 What the platform cannot do</b>                                                           | <b>9</b>  |
| 1.1 The deposit is uncharacterized, and that bounds everything                                 | 9         |
| 1.2 Limits stated by the models themselves                                                     | 9         |
| 1.2.1 physics/diffusion.py: What this module actually establishes, and what it does not        | 9         |
| 1.2.2 physics/packing.py: The central caveat, stated before any equation                       | 10        |
| 1.2.3 physics/phases.py: What the literature actually supports, and where this brief was wrong | 10        |
| 1.2.4 physics/chlorination.py: Thermochemical data status                                      | 11        |
| 1.2.5 ml/surrogate.py: What a surrogate may and may not be used for                            | 11        |
| 1.3 Classes of question this platform does not answer                                          | 11        |
| 1.4 The state of the test suite, measured                                                      | 12        |
| <b>I Core</b>                                                                                  | <b>13</b> |
| <b>2 core/units.py</b>                                                                         | <b>14</b> |
| 2.1 Purpose, governing equations and limitations                                               | 14        |
| 2.2 Interface                                                                                  | 14        |
| 2.3 Worked examples, reproduced by execution                                                   | 16        |
| 2.4 Validation status                                                                          | 16        |
| <b>3 core/provenance.py</b>                                                                    | <b>17</b> |
| 3.1 Purpose, governing equations and limitations                                               | 17        |
| 3.2 Interface                                                                                  | 17        |
| 3.3 Worked examples, reproduced by execution                                                   | 19        |
| 3.4 Validation status                                                                          | 20        |
| <b>4 core/registry.py</b>                                                                      | <b>21</b> |
| 4.1 Purpose, governing equations and limitations                                               | 21        |
| 4.2 Interface                                                                                  | 21        |
| 4.3 Worked examples, reproduced by execution                                                   | 22        |
| 4.4 Validation status                                                                          | 23        |
| <b>5 core/feedstock.py</b>                                                                     | <b>24</b> |
| 5.1 Purpose, governing equations and limitations                                               | 24        |
| 5.2 Interface                                                                                  | 24        |

|           |                                              |           |
|-----------|----------------------------------------------|-----------|
| 5.3       | Worked examples, reproduced by execution     | 26        |
| 5.4       | Validation status                            | 27        |
| <b>6</b>  | <b>core/site.py</b>                          | <b>28</b> |
| 6.1       | Purpose, governing equations and limitations | 28        |
| 6.2       | Interface                                    | 28        |
| 6.3       | Worked examples, reproduced by execution     | 29        |
| 6.4       | Validation status                            | 30        |
| <b>II</b> | <b>Physics</b>                               | <b>31</b> |
| <b>7</b>  | <b>physics/comminution.py</b>                | <b>32</b> |
| 7.1       | Purpose, governing equations and limitations | 32        |
| 7.2       | Interface                                    | 35        |
| 7.3       | Worked examples, reproduced by execution     | 39        |
| 7.4       | Validation status                            | 39        |
| <b>8</b>  | <b>physics/psd.py</b>                        | <b>41</b> |
| 8.1       | Purpose, governing equations and limitations | 41        |
| 8.2       | Interface                                    | 44        |
| 8.3       | Worked examples, reproduced by execution     | 47        |
| 8.4       | Validation status                            | 48        |
| <b>9</b>  | <b>physics/liberation.py</b>                 | <b>49</b> |
| 9.1       | Purpose, governing equations and limitations | 49        |
| 9.2       | Interface                                    | 51        |
| 9.3       | Worked examples, reproduced by execution     | 54        |
| 9.4       | Validation status                            | 54        |
| <b>10</b> | <b>physics/impurity_location.py</b>          | <b>56</b> |
| 10.1      | Purpose, governing equations and limitations | 56        |
| 10.2      | Interface                                    | 59        |
| 10.3      | Worked examples, reproduced by execution     | 62        |
| 10.4      | Validation status                            | 62        |
| <b>11</b> | <b>physics/separation.py</b>                 | <b>64</b> |
| 11.1      | Purpose, governing equations and limitations | 64        |
| 11.2      | Interface                                    | 67        |
| 11.3      | Worked examples, reproduced by execution     | 73        |
| 11.4      | Validation status                            | 74        |

|           |                                                        |            |
|-----------|--------------------------------------------------------|------------|
| <b>12</b> | <b>physics/leaching.py</b>                             | <b>76</b>  |
| 12.1      | Purpose, governing equations and limitations . . . . . | 76         |
| 12.2      | Interface . . . . .                                    | 79         |
| 12.3      | Worked examples, reproduced by execution . . . . .     | 83         |
| 12.4      | Validation status . . . . .                            | 84         |
| <b>13</b> | <b>physics/reagents.py</b>                             | <b>85</b>  |
| 13.1      | Purpose, governing equations and limitations . . . . . | 85         |
| 13.2      | Interface . . . . .                                    | 88         |
| 13.3      | Worked examples, reproduced by execution . . . . .     | 91         |
| 13.4      | Validation status . . . . .                            | 91         |
| <b>14</b> | <b>physics/chlorination.py</b>                         | <b>93</b>  |
| 14.1      | Purpose, governing equations and limitations . . . . . | 93         |
| 14.2      | Interface . . . . .                                    | 96         |
| 14.3      | Worked examples, reproduced by execution . . . . .     | 98         |
| 14.4      | Validation status . . . . .                            | 99         |
| <b>15</b> | <b>physics/diffusion.py</b>                            | <b>100</b> |
| 15.1      | Purpose, governing equations and limitations . . . . . | 100        |
| 15.2      | Interface . . . . .                                    | 105        |
| 15.3      | Worked examples, reproduced by execution . . . . .     | 108        |
| 15.4      | Validation status . . . . .                            | 109        |
| <b>16</b> | <b>physics/thermal.py</b>                              | <b>110</b> |
| 16.1      | Purpose, governing equations and limitations . . . . . | 110        |
| 16.2      | Interface . . . . .                                    | 113        |
| 16.3      | Worked examples, reproduced by execution . . . . .     | 118        |
| 16.4      | Validation status . . . . .                            | 120        |
| <b>17</b> | <b>physics/phases.py</b>                               | <b>121</b> |
| 17.1      | Purpose, governing equations and limitations . . . . . | 121        |
| 17.2      | Interface . . . . .                                    | 124        |
| 17.3      | Worked examples, reproduced by execution . . . . .     | 130        |
| 17.4      | Validation status . . . . .                            | 131        |
| <b>18</b> | <b>physics/packing.py</b>                              | <b>132</b> |
| 18.1      | Purpose, governing equations and limitations . . . . . | 132        |
| 18.2      | Interface . . . . .                                    | 135        |
| 18.3      | Worked examples, reproduced by execution . . . . .     | 138        |
| 18.4      | Validation status . . . . .                            | 139        |

|            |                                                        |            |
|------------|--------------------------------------------------------|------------|
| <b>III</b> | <b>Plant</b>                                           | <b>141</b> |
| <b>19</b>  | <b>plant/streams.py</b>                                | <b>142</b> |
| 19.1       | Purpose, governing equations and limitations . . . . . | 142        |
| 19.2       | Interface . . . . .                                    | 143        |
| 19.3       | Worked examples, reproduced by execution . . . . .     | 145        |
| 19.4       | Validation status . . . . .                            | 146        |
| <b>20</b>  | <b>plant/capacity.py</b>                               | <b>147</b> |
| 20.1       | Purpose, governing equations and limitations . . . . . | 147        |
| 20.2       | Interface . . . . .                                    | 148        |
| 20.3       | Worked examples, reproduced by execution . . . . .     | 150        |
| 20.4       | Validation status . . . . .                            | 151        |
| <b>21</b>  | <b>plant/scheduling.py</b>                             | <b>152</b> |
| 21.1       | Purpose, governing equations and limitations . . . . . | 152        |
| 21.2       | Interface . . . . .                                    | 153        |
| 21.3       | Worked examples, reproduced by execution . . . . .     | 155        |
| 21.4       | Validation status . . . . .                            | 156        |
| <b>22</b>  | <b>plant/yield_cascade.py</b>                          | <b>158</b> |
| 22.1       | Purpose, governing equations and limitations . . . . . | 158        |
| 22.2       | Interface . . . . .                                    | 159        |
| 22.3       | Worked examples, reproduced by execution . . . . .     | 161        |
| 22.4       | Validation status . . . . .                            | 162        |
| <b>IV</b>  | <b>Economics</b>                                       | <b>164</b> |
| <b>23</b>  | <b>econ/unit_economics.py</b>                          | <b>165</b> |
| 23.1       | Purpose, governing equations and limitations . . . . . | 165        |
| 23.2       | Interface . . . . .                                    | 166        |
| 23.3       | Worked examples, reproduced by execution . . . . .     | 167        |
| 23.4       | Validation status . . . . .                            | 167        |
| <b>24</b>  | <b>econ/capex.py</b>                                   | <b>169</b> |
| 24.1       | Purpose, governing equations and limitations . . . . . | 169        |
| 24.2       | Interface . . . . .                                    | 170        |
| 24.3       | Worked examples, reproduced by execution . . . . .     | 173        |
| 24.4       | Validation status . . . . .                            | 174        |
| <b>25</b>  | <b>econ/valuation.py</b>                               | <b>176</b> |

|                                                             |            |
|-------------------------------------------------------------|------------|
| 25.1 Purpose, governing equations and limitations . . . . . | 176        |
| 25.2 Interface . . . . .                                    | 177        |
| 25.3 Worked examples, reproduced by execution . . . . .     | 180        |
| 25.4 Validation status . . . . .                            | 181        |
| <b>26 econ/uncertainty.py</b>                               | <b>182</b> |
| 26.1 Purpose, governing equations and limitations . . . . . | 182        |
| 26.2 Interface . . . . .                                    | 183        |
| 26.3 Worked examples, reproduced by execution . . . . .     | 186        |
| 26.4 Validation status . . . . .                            | 187        |
| <br>                                                        |            |
| <b>V Surrogates and decisions</b>                           | <b>188</b> |
| <br>                                                        |            |
| <b>27 ml/surrogate.py</b>                                   | <b>189</b> |
| 27.1 Purpose, governing equations and limitations . . . . . | 189        |
| 27.2 Interface . . . . .                                    | 190        |
| 27.3 Worked examples, reproduced by execution . . . . .     | 192        |
| 27.4 Validation status . . . . .                            | 194        |
| <br>                                                        |            |
| <b>28 agent/decisions.py</b>                                | <b>195</b> |
| 28.1 Purpose, governing equations and limitations . . . . . | 195        |
| 28.2 Interface . . . . .                                    | 196        |
| 28.3 Worked examples, reproduced by execution . . . . .     | 198        |
| 28.4 Validation status . . . . .                            | 199        |
| <br>                                                        |            |
| <b>A Build findings</b>                                     | <b>200</b> |
| A.1 Worked examples . . . . .                               | 200        |
| A.2 Reproduction result . . . . .                           | 200        |
| A.3 Malformed DOIs in the validation record . . . . .       | 200        |
| A.4 Symbol-table consistency . . . . .                      | 203        |
| A.5 Test suite, measured . . . . .                          | 203        |
| <br>                                                        |            |
| <b>B Worked-example fixtures</b>                            | <b>204</b> |

# How to read this book

Every chapter in Parts I to V documents one module of the platform and follows the same order: purpose, governing equations with their derivation, a symbol table, a worked example, validation status, and limitations. The prose is the module’s own docstring text, converted to LaTeX by `docs/modelbook/rst2tex.py`. That is deliberate: a model book that paraphrases the code drifts from it, whereas this one is regenerated from the code and cannot.

**What the numbers in the worked examples are.** Every worked example was EXECUTED during this build and the output printed below it is what the interpreter actually returned, captured verbatim. A documented output that failed to reproduce is not corrected silently; it is recorded in the findings chapter (Chapter A). This build executed 57 doctest blocks from the module docstrings and 19 supplementary worked examples, across 27 modules.

**What this book is not.** It is not a user guide and not a business case. It documents what each model computes, on what evidence, and where it stops being trustworthy. The chapter on what the platform cannot do (Chapter 1) is not a disclaimer appended for form: it is the part a reader deciding whether to rely on a result should read first.

## The provenance tag scheme

Every stored value in the platform carries a tag recording where it came from. The tags are an enumeration in `ae.core.provenance`, ordered from strongest to weakest evidence, and they are load-bearing rather than decorative: a coverage report counts how much of a model rests on each, and a value tagged ASSUMED without a stated basis fails construction.

| Tag      | Meaning                                                                                                                                        |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------|
| MEASURED | Obtained from a physical measurement on a specific sample, with the method recorded. No value in this build is MEASURED on Vikarabad material. |
| SOURCED  | Taken from a named external source with a resolvable DOI or URL, an access date and a reliability tier.                                        |
| COMPUTED | Produced by an equation in this platform from other tagged values, deterministically.                                                          |
| DERIVED  | Obtained by inference or fitting rather than direct calculation, and requires a stated basis.                                                  |
| ASSUMED  | A scenario input. Requires a <code>basis</code> string giving the reason, or construction fails.                                               |

Source reliability is recorded separately from the tag, in three tiers: tier 1 for peer-reviewed literature, standard references and government or agency data; tier 2 for company filings, patents and industry association data; tier 3 for trade press and vendor material. A tier 3 source may never be sole evidence, and the registry coverage report lists every parameter that rests on one.

Uncertainty is carried on the value, not bolted on later. A `Value` holds a distribution, and `kind=point` means no uncertainty is CLAIMED, which is a different statement from an uncertainty of zero: a point-estimate parameter is excluded from every sensitivity result, which can make a model look more robust than it is. The registry therefore reports the count of point-estimate parameters alongside the count of assumed ones.

## Symbols used across the book

The 40 rows below are harvested from the per-module symbol tables in the docstrings rather than written by hand. Rows are keyed on the symbol together with its unit, so a symbol used by several modules with

the same unit appears once listing all of them, and a symbol used with DIFFERENT units appears once per unit with the disagreement recorded in the findings chapter (Chapter A) rather than resolved silently. Symbols local to a single derivation are defined where they are used rather than repeated here. This build found no such disagreement.

| Symbol           | Unit                                | Range or meaning                   | Modules                           |
|------------------|-------------------------------------|------------------------------------|-----------------------------------|
| $\alpha$         | dimensionless                       | (0, 1] evaporation coefficient     | chlorination                      |
| $\Delta G_r$     | J mol <sup>-1</sup>                 | any sign                           | chlorination                      |
| $\Delta H_r$     | J mol <sup>-1</sup>                 | any sign                           | chlorination                      |
| $\Delta H_{vap}$ | J mol <sup>-1</sup>                 | > 0                                | chlorination                      |
| $\Delta S_r$     | J mol <sup>-1</sup> K <sup>-1</sup> | any sign                           | chlorination                      |
| $\eta_{excess}$  | dimensionless                       | >= 1 (reagent excess)              | reagents                          |
| $\mathcal{R}$    | J mol <sup>-1</sup> K <sup>-1</sup> | 8.314462618 (CODATA 2018)          | leaching, diffusion               |
| Fo               | dimensionless                       | >= 0 (Fourier number)              | diffusion                         |
| $\nu_{i,r}$      | dimensionless                       | > 0 (mol acid per mol i)           | reagents                          |
| $\rho_B$         | mol m <sup>-3</sup>                 | > 0                                | leaching                          |
| $\tau$           | s                                   | > 0 (time for complete conversion) | leaching                          |
| $b$              | dimensionless                       | > 0                                | leaching                          |
| $C_A$            | mol m <sup>-3</sup>                 | > 0                                | leaching                          |
| $C_i$            | kg kg <sup>-1</sup> (as ppm mass)   | >= 0                               | reagents                          |
| $C$              | mol m <sup>-3</sup>                 | >= 0                               | diffusion                         |
| $D_0$            | m <sup>2</sup> s <sup>-1</sup>      | > 0 (pre-exponential)              | diffusion                         |
| $D_e$            | m <sup>2</sup> s <sup>-1</sup>      | > 0 (effective pore diffusivity)   | leaching                          |
| $D$              | m <sup>2</sup> s <sup>-1</sup>      | > 0                                | diffusion                         |
| $E_a$            | J mol <sup>-1</sup>                 | > 0                                | leaching, diffusion               |
| $f_{diss}$       | dimensionless                       | [0, 1] silica dissolved            | reagents                          |
| $J_{evap}$       | mol m <sup>-2</sup> s <sup>-1</sup> | >= 0                               | chlorination                      |
| $J$              | mol m <sup>-2</sup> s <sup>-1</sup> | any sign                           | diffusion                         |
| $k_g$            | m s <sup>-1</sup>                   | > 0 (film coefficient)             | leaching                          |
| $k_s$            | m s <sup>-1</sup>                   | > 0 (surface rate constant)        | leaching                          |
| $L$              | m                                   | >= 0 (diffusion length)            | diffusion                         |
| $M_i$            | kg mol <sup>-1</sup>                | > 0                                | reagents                          |
| $m_{ore}$        | kg                                  | > 0                                | reagents                          |
| $M$              | kg mol <sup>-1</sup>                | > 0                                | chlorination                      |
| $n_i$            | mol                                 | >= 0                               | reagents                          |
| $n_{H^+}$        | mol                                 | >= 0                               | reagents                          |
| $p^{sat}$        | Pa                                  | > 0                                | chlorination                      |
| $p_{ref}$        | Pa                                  | 101325 (1 atm reference)           | chlorination                      |
| $R_{grain}$      | m                                   | > 0                                | diffusion                         |
| $R$              | m                                   | 1e-7 to 1e-1 (0.1 um to 100 mm)    | leaching                          |
| $T_b$            | K                                   | > 0 (normal boiling point)         | chlorination                      |
| $T$              | K                                   | > 0                                | leaching, chlorination, diffusion |
| $t$              | s                                   | >= 0                               | leaching, diffusion               |
| $X$              | dimensionless                       | [0, 1]                             | leaching                          |
| $x$              | m                                   | >= 0                               | diffusion                         |
| $z_i$            | dimensionless                       | ion charge, any sign               | reagents                          |

## CHAPTER 1

# What the platform cannot do

A model is trustworthy in proportion to how precisely its failure modes are stated. This chapter collects the limits that are structural rather than incidental: they are not gaps awaiting a later commit, they are statements about what the available evidence can and cannot support. The text is taken verbatim from the modules that enforce each limit, so a limit cannot be softened here without changing the code.

## 1.1 The deposit is uncharacterized, and that bounds everything

The platform’s first gate is a characterization campaign of 20 to 30 samples labelled AE-Q-####. It has not been run. Consequently:

- No impurity concentration, lattice fraction, fluid-inclusion density or mineral-inclusion assemblage for the Vikarabad deposit exists in the citable record, so none appears in this book as a measurement.
- Every number in a worked example that describes a feed is a SCENARIO INPUT tagged ASSUMED, in several cases set at the published HPQ reference limits so the arithmetic is exercised on a plausible feed. Those are not predictions about the ore.
- The ceiling grade of the deposit is set by lattice-bound Al, Ti, Li and B, which cannot be removed by acid leaching at any residence time. That ceiling is therefore unknown, and no route selection is defensible until it is measured.
- A grade-conditional statement in this book is a scenario gated on that future campaign. The platform is built to rank which measurement to buy first, which is the only decision genuinely available now.

## 1.2 Limits stated by the models themselves

### 1.2.1 physics/diffusion.py: What this module actually establishes, and what it does not

The result is regime dependent, and this module refuses to overstate it. Under the most generous mobility bound that any accessible source supports for a cation in the SiO<sub>2</sub> lattice (MOST\_MOBILE\_BOUND:  $D_0 = 10^{-4}$  m<sup>2</sup> s<sup>-1</sup>,  $E_a = 90$  kJ mol<sup>-1</sup>, the low end of the Liu et al. 2026 range and therefore an overestimate of mobility for substitutional Al), for a 100 micrometre grain radius:

- ATMOSPHERIC-PRESSURE ACID LEACHING (up to about 100 degC): ESTABLISHED INFEASIBLE. At 80 degC and 6 h the diffusion length is 0.325  $\mu$ m against a 100  $\mu$ m radius, a factor of 308 (2.49 orders of magnitude). At 95 degC and 24 h it is 1.21  $\mu$ m, a factor of 82. Since this is computed with an overestimate of mobility, the true deficit is larger. Conclusion robust.
- CHLORINATION ROASTING, Ti SPECIFICALLY: ESTABLISHED INFEASIBLE. Using the Ti4+ activation energy Liu et al. (2026) report for lattice diffusion in SiO<sub>2</sub> (about 400 kJ mol<sup>-1</sup>), at 1200 degC and 6 h the diffusion length is 119 nm for  $D_0 = 10^{-4}$  and 0.12 nm for  $D_0 = 10^{-10}$ , i.e. a deficit of 2.9 to 5.9 orders of magnitude across the entire plausible prefactor range. Fractional extraction from the sphere is 4.0e-3. Conclusion robust because it does not depend on the unknown prefactor.
- HOT PRESSURE ACID LEACHING (200 to 300 degC) and CHLORINATION ROASTING FOR Al: NOT ESTABLISHED BY THIS MODEL. At 250 degC and 6 h the generous bound gives a diffusion length of 47  $\mu$ m against a 100  $\mu$ m radius, a factor of only 2.1, and at 300 degC it exceeds the radius.

At 1200 degC and 6 h the activation energy at which the diffusion length equals the grain radius is 65.8 kJ mol<sup>-1</sup> at the low end of the prefactor sweep ( $D_0 = 10^{-10}$  m<sup>2</sup> s<sup>-1</sup>) and 235.1 kJ mol<sup>-1</sup> at the high end ( $D_0 = 10^{-4}$  m<sup>2</sup> s<sup>-1</sup>) (`critical_activation_energy`), and the true barrier for Al is unmeasured: it lies somewhere between the Na+ value (about 90 kJ mol<sup>-1</sup>) and the Ti4+ value (about 400 kJ mol<sup>-1</sup>) reported by Liu et al. (2026), which straddles the threshold. A diffusion-length argument therefore CANNOT decide the Al case at roasting temperature, and this module says so rather than choosing a prefactor that produces the desired answer.

The platform's conclusion that lattice Al sets the purity ceiling consequently rests on the EMPIRICAL result, not on this diffusion calculation: Xia et al. (2024, doi 10.3390/min14070727) took a vein quartz from 128.86 ug/g total trace impurities to 24.23 ug/g (81.20 percent removal) through crushing, ultrasonic desliming, flotation, calcination, water quenching, hot pressure acid leaching AND chlorination roasting, and identified the residual as lattice-bound Al, Ti and Li. The diffusion model explains why the leaching stages cannot reach the lattice and why Ti survives the roast; it does not independently prove that Al survives a roast, and the Liu et al. (2026) abstract in fact announces an alkali-first, Al-follows coupled diffusion mechanism for aluminium removal plus a temperature-staged, atmosphere-segmented roasting strategy, which would be incoherent if Al diffusion at roasting temperature were negligible.

### 1.2.2 physics/packing.py: The central caveat, stated before any equation

Every maximum-packing number in this module is a GEOMETRIC UPPER BOUND on the solids fraction of a poured and vibrated dry powder. A real formulation does not reach it, and is not trying to. Three reasons, in descending order of importance:

1. VISCOSITY. The Krieger-Dougherty relation (equation 5) shows relative viscosity diverging as the solids fraction approaches the maximum packing fraction. A molding compound has to flow through a gate and fill a cavity in seconds, so it must sit well below the divergence. At a maximum packing fraction of 0.70, loading to 0.65 already costs a factor of roughly 101 in relative viscosity against a factor of roughly 9 at 0.50. An 11-fold viscosity penalty for a 15 percent loading gain is the trade that actually sets commercial loading.
2. WETTING AND BINDER DEMAND. Every particle surface needs resin. Fine fractions that help geometric packing raise the specific surface area (see `ae.physics.psd`) and so raise the resin demand, which is why the geometric optimum is not the rheological one.
3. PACKING MODELS ASSUME SPHERES POURED AND VIBRATED, NOT DISPERSED IN A MELT. McGeary 1961 (doi 10.1111/j.1151-2916.1961.tb13716.x) obtained his numbers with "spherical metal shot" packed "by mechanical vibration" in glass containers. Angular ground silica packs worse; a melt-dispersed system behaves differently again.

This module therefore reports the geometric bound, the viscosity at a chosen loading, and refuses to present the bound as an achievable formulation.

### 1.2.3 physics/phases.py: What the literature actually supports, and where this brief was wrong

The platform brief states the alpha to beta inversion carries "roughly 3.7 vol% expansion". That number is not the discontinuity at 573 degC. Two distinct quantities are conflated in the trade literature:

- (a) The CUMULATIVE transition-related expansion of alpha-quartz on heating from room temperature to the inversion. The HP2011 Landau calibration gives 4.21 vol% (see equation 2 above). Values quoted in the 3.5 to 4.5 vol% band, including the brief's 3.7 vol%, belong to this quantity. (b) The STEP change at the inversion itself, which Ringdalen 2015 (doi 10.1007/s11837-014-1149-y) reports as "an increase in volume of around 0.4%".

Both are real and they differ by an order of magnitude. This module returns (a) from `alpha_beta_cumulative_volume_strain` and (b) from `TRANSITIONS`, each separately labelled, because a thermal-shock model fed the wrong one is wrong by 10x.

The mechanistic claim in the brief, that the alpha to beta inversion is "what cracks inclusion trails and is why calcination works", is only partly supported. The much larger strain available on heating is the quartz to cristobalite conversion: Ringdalen 2015 measured up to 37% volume expansion across her sample set and found that specific surface area rose with cristobalite content, attributing it to "cracking when the volume increases". The evidence for the 0.4% inversion step being the dominant cracking mechanism is weak; this module therefore reports strains and does not assert a cracking mechanism.

#### 1.2.4 `physics/chlorination.py`: Thermochemical data status

NIST-JANAF ([janaf.nist.gov](http://janaf.nist.gov)) and the NIST Chemistry WebBook ([webbook.nist.gov](http://webbook.nist.gov)) are both outside this sandbox's network allowlist and returned proxy errors on 2026-09-16. No request for access was made, per the task instruction to log the attempt and continue. Standard formation enthalpies and entropies for the oxide-to-chloride reactions of Al, Ti and B could therefore NOT be sourced. Consequences, all visible in the code rather than papered over:

- THERMO contains only the species whose formation data could be retrieved from PubChem (which aggregates HSDB and other compendia, each carrying its own reference). Entropies are almost entirely absent from that source, so `gibbs_of_reaction` requires the caller to supply a complete table and raises when any species is missing.
- There is NO built-in oxide-to-chloride  $\Delta G_r$  for Al, Ti or B in this build. The reductant requirement is enforced from the Liu et al. (2026) finding as a RULE, not recomputed from thermodynamics.
- CHLORIDES boiling and sublimation points ARE sourced (PubChem), and the volatility screen based on them is the part of this module that rests on retrieved data.

#### 1.2.5 `ml/surrogate.py`: What a surrogate may and may not be used for

Legitimate: replacing a slow physics call inside a 10,000-draw Monte Carlo after the surrogate's leave-one-deposit-out error has been shown small relative to the input uncertainty; screening a large design space to shortlist candidates for full simulation.

NOT legitimate: predicting a lattice impurity ceiling for an uncharacterized deposit. Lattice-bound Al, Ti, Li and B are set by crystallisation conditions and are not recoverable from bulk assay or process response, so a model fitted on other deposits has no mechanism by which to know them. The platform states this as a limitation rather than relying on the OOD flag to catch it, because a new deposit can sit inside the training hull on every measured feature and still have a different lattice chemistry.

### 1.3 Classes of question this platform does not answer

- **It does not predict a lattice impurity ceiling from a bulk assay.** Lattice-bound trace elements are set by crystallisation conditions and are not recoverable from bulk chemistry or from process response. A surrogate fitted on other deposits has no mechanism by which to know them, and a new deposit can sit inside the training hull on every measured feature while having different lattice chemistry.
- **It does not conduct the reasoning.** There is no autonomous loop and no language-model call inside any model. The decision module computes the quantities a decision needs so that the reasoning is auditable arithmetic rather than narrative.

- **It does not produce a bankable capital estimate.** Capital cost is factored from purchased-equipment cost with an accuracy class reported alongside, and the band is wide enough to change an investment decision.
- **It does not model reaction kinetics at the surface, gas-film resistance, or pore diffusion in an agglomerated charge,** and it does not model chloride condensation in a cooler downstream zone, which is how  $\text{AlCl}_3$  fouls equipment in practice.
- **It does not model silica chlorination.**  $\text{SiCl}_4$  forms from  $\text{SiO}_2$  under aggressive chlorination and is a yield loss of the product itself. A complete flowsheet model must include it; this one does not.
- **It does not price anything.** No market price, contract structure or customer qualification timeline is modelled as an outcome. Prices enter as tagged inputs with sources and dates.

## 1.4 The state of the test suite, measured

Measured for this build by a full run: 1802 passed, 14 failed, 18 skipped, out of 1834 collected. The failures are confined to 3 prose-audit modules: `test_test_docstrings` (11), `test_golden_vectors` (2), `test_registry_export` (1).

Those files audit documentation rather than model behaviour: they check that a number appearing in a test docstring is reproduced by an assertion in that same test body, that an in-text author-year citation resolves to a reference block, and that docstring arithmetic is self-consistent. Their failures are therefore a statement about the completeness of the prose, not about the correctness of the models. Every model test passes. The distinction matters in both directions: a reader should not discount the model results because of the failure count, and should not treat the prose audit as clean.

Of the 18 skipped tests, one group is the citation audit skipping modules that make no in-text author-year citation, which is a correct skip rather than a gap. The workbook tests skip because recalculating the Excel mirror needs a formula engine that is not installed here, so that claim is unchecked in this environment rather than checked and passing.

## PART I

# Core

Units, provenance, the parameter registry, and the feedstock and site objects every model takes as input.

## CHAPTER 2

### core/units.py

Module `ae.core.units`, 285 lines. Chapter text is this module's own documentation.

## 2.1 Purpose, governing equations and limitations

Project unit registry and dimensional-analysis enforcement.

Every physical quantity in this platform is a `pint.Quantity`. Bare floats are accepted only where a quantity is genuinely dimensionless (mass fractions, recovery, efficiencies), and those are still range-checked.

Rationale for a single shared registry: `pint` quantities created by two different `UnitRegistry` instances cannot be combined, and the resulting error message is opaque. Import `UREG` (or the `Q_` constructor) from this module everywhere.

### Concentration convention

Trace impurity levels in quartz are reported in the literature as mass ratios: "ppm" almost always means micrograms of element per gram of solid (ug/g) and "ppb" means nanograms per gram (ng/g). `pint`'s built-in `ppm` is a bare dimensionless  $1e-6$ , which silently conflates mass ratio with mole ratio and with volume ratio.

Note carefully: `pint` reduces ug/g, umol/mol and mL/L ALL to `dimensionless`, so a dimensionality check cannot separate them. The guard is therefore `ratio_basis`, which inspects unit composition, and the `to_ppm_mass/to_ppb_mass` helpers that call it:

```
>>> from ae.core.units import Q_, to_ppm_mass
>>> to_ppm_mass(Q_(30.0, "ug/g")).magnitude
30.0
```

`ppm_mass` and `ppb_mass` are defined as aliases of ug/g and ng/g so that the numeric value a paper reports can be entered verbatim without a conversion step.

## 2.2 Interface

### Functions

*ratio\_basis*

```
ratio_basis(q: Quantity) -> str
```

Classify a dimensionless ratio as "mass", "mole", "volume", "bare" or "mixed" by inspecting its unit composition.

This exists because `**pint cannot tell these apart by dimensionality**`: it reduces ug/g, umol/mol and mL/L all to `dimensionless`, so a dimensional check passes a mole ratio into a mass-ratio slot silently. That is not a hypothetical: trace-element data is published in all three bases and the numeric values differ by the ratio of molar masses. For Al in SiO<sub>2</sub>,  $M(\text{SiO}_2)/M(\text{Al}) = 60.08/26.98 = 2.227$ , so 30 ppm Al BY MASS is 66.8 umol per mol of SiO<sub>2</sub> formula units, and the mole-basis figure is LARGER than the mass-basis one. Worse, the two mole conventions in use disagree with each other by a further factor of 3: expressed per

mole of ATOMS (SiO<sub>2</sub> contributing three atoms per formula unit) the same material reads 22.3  $\mu\text{mol/mol}$ . A single reported "30 ppm" can therefore mean any of three numbers spanning 3x, which is why the basis is checked rather than assumed.

"bare" means a true dimensionless number with no unit trace, i.e. a mass fraction entered as 0.0022 rather than as 2200  $\mu\text{g/g}$ . Callers decide whether to accept it; `to_ppm_mass` does, because a bare fraction in this codebase is a mass fraction by convention.

*to\_ppm\_mass*

```
to_ppm_mass(q: Quantity) -> Quantity
```

Convert a mass-ratio concentration to ppm by mass ( $\mu\text{g/g}$ ).

Accepts the `ppm_mass/ppb_mass` aliases, any mass-per-mass unit ( $\mu\text{g/g}$ ,  $\text{mg/kg}$ ), and a bare dimensionless mass fraction.

**Raises.** `ValueError` If `q` is a mole ratio or a volume ratio. `pint` would convert these without complaint because all three reduce to dimensionless, so this check is explicit rather than dimensional. Convert with an explicit molar-mass calculation instead, so the stoichiometry is visible.

*to\_ppb\_mass*

```
to_ppb_mass(q: Quantity) -> Quantity
```

Convert a mass-ratio concentration to ppb by mass ( $\text{ng/g}$ ).

*require\_dimensionality*

```
require_dimensionality(q: Quantity, kind: str, name: str='value') -> Quantity
```

Assert that `q` has the dimensionality registered under `kind`.

**Parameters.** `q` Quantity to check. `kind` Key into DIMS, e.g. `"specific_energy_mass"`. `name` Name used in the error message, so a failure names the offending field.

**Returns.** Quantity `q` unchanged, to allow use inline in an expression.

*require\_fraction*

```
require_fraction(x: float, name: str='fraction', lo: float=0.0, hi: float=1.0) -> float
```

Validate a genuinely dimensionless fraction (recovery, efficiency, yield).

These are the one class of quantity carried as bare floats, because wrapping a recovery of 0.92 in a unit adds noise without adding a check. The range check is the substitute for the dimensional check.

*as\_dimensionless*

```
as_dimensionless(q: Quantity | float) -> float
```

Reduce a dimensionless quantity to a float, raising if it carries units.

## 2.3 Worked examples, reproduced by execution

The 1 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

### core.units

```
>>> from ae.core.units import Q_, to_ppm_mass
>>> to_ppm_mass(Q_(30.0, "ug/g")).magnitude
30.0
```

### Mass ratio versus mole ratio for 30 ppm Al in SiO2

```
from ae.core.units import Q_, to_ppm_mass, ratio_basis

q = Q_(30.0, "ug/g")
print("basis of ug/g          :", ratio_basis(q))
print("to_ppm_mass(30 ug/g)   :", to_ppm_mass(q).magnitude, "ppm by mass")

# The same material on a mole basis, computed explicitly rather than by unit
# conversion, so the stoichiometry is visible.
M_SiO2, M_Al = 60.08, 26.98
print("M(SiO2)/M(Al)          :", round(M_SiO2 / M_Al, 4))
print("30 ppm mass as umol/mol :", round(30.0 * M_SiO2 / M_Al, 2))
print("per mole of ATOMS       :", round(30.0 * M_SiO2 / M_Al / 3.0, 2))

# A mole ratio offered where a mass ratio is required must be refused.
try:
    to_ppm_mass(Q_(30.0, "umol/mol"))
except ValueError as e:
    print("mole ratio rejected   :", str(e)[:56])
```

Output:

```
basis of ug/g          : mass
to_ppm_mass(30 ug/g)   : 30.0 ppm by mass
M(SiO2)/M(Al)          : 2.2268
30 ppm mass as umol/mol : 66.81
per mole of ATOMS       : 22.27
mole ratio rejected     : expected a mass-ratio concentration, got a mole-basis ra
```

The three readings of one reported 30 ppm span a factor of three. pint reduces all three to dimensionless, so the basis is checked by inspecting unit composition rather than inferred from dimensionality.

## 2.4 Validation status

No test in the suite is marked golden or benchmark against this module. It is exercised by unmarked unit tests only, which check behaviour but make no claim of validation against an external datapoint or a closed form.

## CHAPTER 3

# core/provenance.py

Module `ae.core.provenance`, 323 lines. Chapter text is this module's own documentation.

## 3.1 Purpose, governing equations and limitations

Provenance tagging and uncertainty for every value in the platform.

Hard rule from the platform brief: every stored datapoint carries source, DOI or URL, access date, source tier and extraction method, and every value is tagged MEASURED / SOURCED / COMPUTED / DERIVED / ASSUMED with a confidence. Missing values are recorded as missing and raise on use rather than silently defaulting.

The central type is `Value`, a quantity bundled with its uncertainty and its origin. Models accept `Value` (or a plain `Quantity` for intermediates) and Monte Carlo sampling reads the distribution off the `Value` directly, so an uncertain input cannot be used as though it were exact.

### Design note on why ASSUMED values do not raise

An assumption is legitimate input to a scenario; an *\*undocumented\** assumption is not. `Value` therefore requires a basis string for every ASSUMED entry, so an assumption without a stated reason fails construction rather than review.

## 3.2 Interface

### Types

**Tag.** Origin of a value. Ordered from strongest to weakest evidence.

**Tier.** Source reliability tier from the platform brief.

**MissingValueError.** Raised when a value recorded as missing is used in a calculation.

**Source.** Bibliographic and retrieval provenance for one datapoint.

**Distribution.** Uncertainty on a value, as a sampleable distribution.

`kind="point"` means no uncertainty is claimed, which is different from an uncertainty of zero: it is recorded so that a Sobol analysis can report the value as unswept rather than as certain.

**Absolute versus relative.** By default all parameters are ABSOLUTE, expressed in the `Value`'s own unit: `Distribution(kind="normal", loc=100.0, scale=5.0)` on a `Value` in USD/tonne is a mean of 100 USD/tonne with a 5 USD/tonne standard deviation.

Setting `relative=True` makes the parameters MULTIPLICATIVE factors on the `Value`'s nominal magnitude instead, which is the natural form for a "plus or minus 20 percent" input. It must be set explicitly: an earlier version of this class inferred relative semantics from `loc == 0.0`, which silently turned a legitimate

zero-mean absolute normal (say a temperature offset with a 5 K standard deviation) into a 500 percent multiplicative spread. The flag removes the ambiguity, and `Value.sample` dispatches on it alone.

```
sample(self, n: int, rng: np.random.Generator, point: float) -> np.ndarray
```

Draw `n` samples. `point` is the `Value`'s nominal magnitude.

```
relative_normal(cls, frac: float) -> Distribution
```

Symmetric normal with standard deviation `frac` TIMES the nominal.

The common "plus or minus 20 percent" input: `relative_normal(0.20)`. Carries `relative=True` so the multiplicative reading is explicit in the object rather than inferred from a sentinel value.

```
relative_uniform(cls, lo_frac: float, hi_frac: float) -> Distribution
```

Uniform between `lo_frac` and `hi_frac` times the nominal.

For an asymmetric scenario band such as "between 0.7x and 1.5x".

**Value.** A quantity with its uncertainty and its provenance.

### Examples.

```
>>> from ae.core.units import Q_
>>> import datetime as dt
>>> v = Value(
...     quantity=Q_(30.0, "ppm_mass"),
...     tag=Tag.SOURCED,
...     source=Source(citation="Muller et al. 2012, Quartz: Deposits, Mineralogy and Analytics",
...                     tier=Tier.T1, url="https://doi.org/10.1007/978-3-642-22161-3",
...                     accessed=dt.date(2026, 9, 16)),
... )
>>> v.magnitude
30.0
```

```
magnitude(self) -> float
```

```
units(self) -> str
```

```
to(self, unit: str) -> Quantity
```

```
sample(self, n: int, rng: np.random.Generator) -> Quantity
```

Draw `n` samples as a `Quantity` array in this `Value`'s own unit.

Dispatch is on `dist.relative` alone, never on the parameter values:

- `relative=False` (default): the distribution parameters are absolute, in this `Value`'s own unit, and are sampled directly.
- `relative=True`: the parameters are multiplicative factors. A normal is applied as `nominal * (1 + N(loc, scale))` so a zero-mean normal centres on the nominal; a uniform or triangular is applied as `nominal * U(low, high)`.

### 3.3 Worked examples, reproduced by execution

The 1 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

#### core.provenance.Value

```
>>> from ae.core.units import Q_
>>> import datetime as dt
>>> v = Value(
...     quantity=Q_(30.0, "ppm_mass"),
...     tag=Tag.SOURCED,
...     source=Source(citation="Muller et al. 2012, Quartz: Deposits, Mineralogy and Analytics",
...                     tier=Tier.T1, url="https://doi.org/10.1007/978-3-642-22161-3",
...                     accessed=dt.date(2026, 9, 16)),
... )
>>> v.magnitude
30.0
```

#### A sourced value, a rejected assumption, and a missing value

```
import datetime as dt
import numpy as np
from ae.core.units import Q_
from ae.core.provenance import (Distribution, MISSING, MissingValueError, Source,
                                Tag, Tier, Value)

v = Value(quantity=Q_(30.0, "ppm_mass"), tag=Tag.SOURCED,
          source=Source(citation="Goetze and Moeckel (eds) 2012, Quartz: Deposits, "
                              "Mineralogy and Analytics",
                              tier=Tier.T1, url="https://doi.org/10.1007/978-3-642-22161-3",
                              accessed=dt.date(2026, 9, 16)))

print("sourced value      :", v)

# An ASSUMED value with no stated basis must fail construction, not review.
try:
    Value(quantity=Q_(0.7, "dimensionless"), tag=Tag.ASSUMED)
except Exception as e:
    print("assumed, no basis      :", type(e).__name__)

a = Value(quantity=Q_(0.7, "dimensionless"), tag=Tag.ASSUMED,
          basis="placeholder cascade yield pending the assay campaign",
          dist=Distribution.relative_normal(0.20))
s = a.sample(20000, np.random.default_rng(0))
print("nominal              :", a.magnitude)
print("relative_normal(0.20) sd:", round(float(np.std(s.magnitude)), 5),
      " expected 0.7 x 0.20 =", 0.7 * 0.20)

try:
    _ = MISSING + 1.0
except MissingValueError as e:
    print("MISSING in arithmetic  :", type(e).__name__)
```

Output:

```
sourced value      : 30.0 ppm_mass [SOURCED] (Goetze and Moeckel (eds) 2012, Quartz: Deposits, Miner
\ alogy and Analytics, T1)
assumed, no basis  : ValueError
nominal           : 0.7
relative_normal(0.20) sd: 0.13945 expected 0.7 x 0.20 = 0.13999999999999999
MISSING in arithmetic : MissingValueError
```

A 20 percent relative normal is multiplicative on the nominal, so its standard deviation is 0.7 times 0.20. An absolute distribution with the same parameters would mean something entirely different, which is why the flag is explicit rather than inferred from a zero mean.

### 3.4 Validation status

No test in the suite is marked golden or benchmark against this module. It is exercised by unmarked unit tests only, which check behaviour but make no claim of validation against an external datapoint or a closed form.

## CHAPTER 4

# core/registry.py

Module `ae.core.registry`, 225 lines. Chapter text is this module's own documentation.

## 4.1 Purpose, governing equations and limitations

Queryable registry of every parameter the platform uses.

One place holds every number, its unit, its uncertainty, its origin tag and its source. Models look parameters up by key rather than embedding literals, so that

- the provenance of any result can be traced to the values that produced it,
- a coverage report can count how much of the model rests on ASSUMED values,
- a Monte Carlo run can sweep every uncertain parameter without hunting for them,
- and re-sourcing a value updates every model at once.

Registry keys are dotted paths: `comminution.work_index.quartzite`, `site.US-NM-ABQ.power.energy_price`. Lookups are exact; there is deliberately no fuzzy matching, because silently resolving a typo to a neighbouring parameter is worse than failing.

### Coverage reporting

`Registry.coverage` returns the count of parameters by tag and by source tier. The brief requires honesty about how much of a model is assumption, and a number is easier to argue with than an adjective.

## 4.2 Interface

### Types

`ParameterNotFound`. Raised when a key is absent. Carries near-miss keys to catch typos.

`DuplicateParameter`. Raised on a second registration of the same key without `overwrite=True`.

`Registry`. A mutable, serializable collection of provenance-tagged parameters.

```
add(self, key: str, value: Value, overwrite: bool=False) -> Value
```

Register value under key.

```
add_many(self, mapping: dict[str, Value], overwrite: bool=False) -> None
```

```
get(self, key: str) -> Value
```

Fetch a `Value`, raising with near-miss suggestions if absent.

```
quantity(self, key: str, unit: str | None=None) -> Quantity
```

Fetch a parameter as a Quantity, optionally converted to unit.

```
magnitude(self, key: str, unit: str) -> float
```

Fetch a parameter's magnitude in an EXPLICIT unit.

The unit is required rather than optional: reading a magnitude without naming the unit is how a kWh/short-ton work index becomes a kWh/tonne one.

```
find(self, prefix: str) -> dict[str, Value]
```

All parameters whose key starts with prefix.

```
uncertain_keys(self) -> list[str]
```

Keys carrying a non-point distribution, i.e. the Monte Carlo inputs.

```
sample(self, keys: list[str], n: int, rng: np.random.Generator) -> dict[str, Quantity]
```

Draw n correlated-free samples for each requested key.

```
coverage(self) -> dict[str, Any]
```

Count parameters by tag and source tier, and list the weak spots.

Returns a dict with exactly these keys: `n_parameters`, `by_tag`, `by_tier`, `assumed_keys`, `assumed_fraction`, `point_estimate_keys` and `tier3_keys`.

`point_estimate_keys` matters because a parameter with no uncertainty is excluded from every sensitivity result, which can make a model look more robust than it is. `tier3_keys` matters because a tier 3 source may never be sole evidence.

```
to_records(self) -> list[dict[str, Any]]
```

Flatten to rows for a CSV or a DataFrame, one parameter per row.

```
save(self, path: str | Path) -> Path
```

Write the registry to JSON, preserving units as strings.

```
load(cls, path: str | Path) -> Registry
```

Read a registry back, reconstructing units and provenance.

## 4.3 Worked examples, reproduced by execution

### Coverage report over a three-parameter registry

```
import datetime as dt
from ae.core.units import Q_
from ae.core.provenance import Distribution, Source, Tag, Tier, Value
from ae.core.registry import ParameterNotFound, Registry
```

```

r = Registry()
r.add("comminution.work_index.quartzite",
      Value(quantity=Q_(13.57, "kWh/tonne"), tag=Tag.SOURCED,
            source=Source(citation="Bond work index compilation, quartzite",
                          tier=Tier.T2, url="https://www.911metallurgist.com/blog/bond-work-index/",
                          accessed=dt.date(2026, 9, 16))))
r.add("plant.cascade_yield",
      Value(quantity=Q_(0.70, "dimensionless"), tag=Tag.ASSUMED,
            basis="placeholder pending the AE-Q characterization campaign",
            dist=Distribution.relative_normal(0.15)))
r.add("site.power.energy_price",
      Value(quantity=Q_(0.0541, "USD/kWh"), tag=Tag.SOURCED,
            source=Source(citation="EIA Electric Power Monthly Table 5.6.B", tier=Tier.T1,
                          url="https://www.eia.gov/electricity/monthly/",
                          accessed=dt.date(2026, 9, 16))))

c = r.coverage()
print("n_parameters          :", c["n_parameters"])
print("by_tag                :", dict(sorted(c["by_tag"].items())))
print("by_tier               :", dict(sorted(c["by_tier"].items())))
print("assumed_fraction      :", round(c["assumed_fraction"], 6))
print("assumed_keys          :", c["assumed_keys"])
print("point_estimate_keys   :", len(c["point_estimate_keys"]), "of", c["n_parameters"])
print("uncertain_keys        :", r.uncertain_keys())
print("magnitude, named unit  :", r.magnitude("comminution.work_index.quartzite", "kWh/tonne"))

try:
    r.get("comminution.work_index.quartzit")
except ParameterNotFound as e:
    print("typo, near miss offered :", "quartzite" in str(e))

```

Output:

```

n_parameters          : 3
by_tag                : {'ASSUMED': 1, 'SOURCED': 2}
by_tier               : {'NO_SOURCE': 1, 'T1': 1, 'T2': 1}
assumed_fraction      : 0.333333
assumed_keys          : ['plant.cascade_yield']
point_estimate_keys   : 2 of 3
uncertain_keys        : ['plant.cascade_yield']
magnitude, named unit : 13.57
typo, near miss offered : True

```

One parameter of three is ASSUMED, so a third of this registry rests on assumption. Two of three carry no distribution and are therefore invisible to any sensitivity result, which is the second number a reviewer should ask about.

## 4.4 Validation status

No test in the suite is marked golden or benchmark against this module. It is exercised by unmarked unit tests only, which check behaviour but make no claim of validation against an external datapoint or a closed form.

## CHAPTER 5

### core/feedstock.py

Module `ae.core.feedstock`, 403 lines. Chapter text is this module's own documentation.

#### 5.1 Purpose, governing equations and limitations

**FEEDSTOCK:** an ore, described well enough to run every downstream model.

**Platform principle:** nothing is hardcoded to one deposit. Every model takes a **FEEDSTOCK** and a **SITE**, so a new ore is a new object rather than a new code path.

##### Impurity convention

Impurities are stored as **ELEMENTAL** mass ratios (ppm by mass), not as oxides. Royalty and whole-rock assays are usually reported as oxides ( $\text{Al}_2\text{O}_3$ ,  $\text{Fe}_2\text{O}_3$ ), and converting by eye is a recurring error, so `oxide_to_element` does it with explicit stoichiometry and tags the result **DERIVED** with the factor used.

##### Why the lattice fraction is a first-class field

Lattice-substituted Al, Ti, Li and B cannot be removed by any acid leach, so the lattice inventory sets the purity ceiling of the deposit no matter how good the flowsheet is. A **FEEDSTOCK** that reports a total impurity level without the lattice split cannot answer the only question that matters for HPQ, and the schema makes that explicit: `ImpurityProfile.lattice_fraction` may be **MISSING**, and the purification models raise rather than guess when it is.

#### 5.2 Interface

##### Types

**OreType.** Ore classes this platform models. Extend, do not overload.

**InclusionType.**

**ImpurityProfile.** Per-element impurity inventory with the location split that sets the ceiling.

`total` is the bulk concentration. `lattice_fraction` is the fraction of that element structurally substituted into the quartz lattice, which no leach or flotation step can touch. Both are per element.

```
total_ppm(self, element: str) -> float
```

Bulk concentration in ppm by mass, raising if never measured.

```
lattice_ppm(self, element: str) -> float
```

Lattice-bound concentration in ppm by mass.

**Raises.** `MissingValueError` If the lattice split was not measured. This is deliberate: guessing the split would fabricate the deposit's purity ceiling, which is the single most decision-relevant number in the platform.

```
sum_ppm(self, elements: tuple[str, ...] | None=None) -> float
```

Total trace sum over measured elements, excluding OH.

OH is excluded because it is reported on a different basis in the literature (as water content or by FTIR absorbance) and adding it to a cation sum double-counts hydrogen-compensated Al substitution.

**InclusionCharacter.** Fluid and mineral inclusion population, which drives calcination response.

**PhysicalProperties.**

**Feedstock.** One characterized (or uncharacterized) ore.

`sample_id` encodes mineral, country, deposit and sample number, so data from several deposits and sites can share one table without collision: AE-Q-IN-VKB-001 is quartz, India, Vikarabad, sample 1.

```
characterization_tier(self) -> CharacterizationTier
```

How well this ore is known, on four tiers rather than a binary flag.

The tiers exist because a binary characterized flag blocked all early screening: a candidate ore with an XRF scan is not "characterized", but it is not unknown either, and refusing to model it at all is the wrong response. Each tier unlocks different outputs (see `permits_output`).

`unmeasured` No impurity measurement of any kind. Supports no output; every grade-conditional result is a scenario. `screened` Any measurement at all, typically XRF. Gives major-element composition and a Fe indication. `bulk_quantified` The full element suite (Al, Ti, Li, Fe, Na, K, B) by a bulk method with adequate detection limits, i.e. ICP-MS after digestion or GDMS. This gives TOTAL content per element at ppm-to-ppb limits. `located` The full suite by a spatially resolved method (LA-ICP-MS), with the lattice fraction measured rather than assumed, plus inclusion work. Only this tier supports a purification ceiling.

WHY XRF IS INSUFFICIENT, stated correctly. A previous version of this module claimed bulk XRF "cannot resolve lattice impurities", implying it misses lattice-bound atoms. That is wrong: XRF measures TOTAL element content irrespective of where the atoms sit. Its actual limitations are that it cannot measure Li or B at all (both too light for practical XRF), its detection limits for Ti and the alkalis are poor at the ppm level that matters here, and, like every bulk method, it reports no spatial information. The last point applies equally to bulk ICP-MS, which does achieve the detection limits: the reason bulk analysis cannot set a purification ceiling is that it cannot tell you WHERE the impurities are, not that it cannot see them.

```
characterization_gap(self) -> dict[str, object]
```

What is missing to reach the next tier, as a checklist.

Written to be actionable by a campaign planner: it names the elements and the measurement, not a score.

```
permits_output(self, output: str) -> bool
```

Whether this ore's characterization tier supports a given output.

The gate the binary flag was trying to be, at the right granularity: a screened ore may be ranked and costed on mass yield, but only a located ore may have a purification ceiling or a product grade claimed.

```
require_output(self, output: str) -> None
```

Raise unless the tier supports the output, naming the gap.

```
id_parts(self) -> dict[str, str]
```

## Functions

*oxide\_to\_element*

```
oxide_to_element(oxide: str, oxide_ppm: float) -> tuple[str, float, float]
```

Convert an oxide mass concentration to its elemental equivalent.

**Parameters.** `oxide` Formula as written in the assay, e.g. "Al<sub>2</sub>O<sub>3</sub>". `oxide_ppm` Oxide concentration in ppm by mass (equivalently, wt% times 10000).

**Returns.** (`element`, `element_ppm`, `factor`) The factor is the multiplier applied, returned so it can be recorded in the DERIVED value's basis string rather than left implicit.

**Examples.** 0.22 wt% Al<sub>2</sub>O<sub>3</sub> is 2200 ppm oxide, and Al is 52.9 percent of Al<sub>2</sub>O<sub>3</sub> by mass:

```
>>> el, ppm, f = oxide_to_element("Al2O3", 2200.0)
>>> el, round(ppm), round(f, 4)
('Al', 1164, 0.5293)
```

## 5.3 Worked examples, reproduced by execution

The 1 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

`core.feedstock.oxide_to_element`

```
>>> el, ppm, f = oxide_to_element("Al2O3", 2200.0)
>>> el, round(ppm), round(f, 4)
('Al', 1164, 0.5293)
```

## Lattice-bound versus leachable impurities, and the ceiling they set

```
f = scenario_feedstock()
imp = f.impurities
els = ("Al", "Ti", "Li", "Fe", "Na", "K", "B")
print(f"{el:3s} {'total':>8s} {'lattice f':>10s} {'lattice':>9s} {'leachable':>10s}")
for el in els:
    tot = imp.total_ppm(el)
    lat = imp.lattice_ppm(el)
    lf = imp.lattice_fraction[el].magnitude
    print(f"{el:3s} {tot:8.1f} {lf:10.2f} {lat:9.2f} {tot - lat:10.2f}")
print()
print("sum of all traces      :", round(imp.sum_ppm(els), 2), "ppm")
print("lattice-bound subtotal :",
      round(sum(imp.lattice_ppm(e) for e in els), 2), "ppm")
```

```

print("acid-accessible      :",
      round(imp.sum_ppm(els) - sum(imp.lattice_ppm(e) for e in els), 2), "ppm")
print()
print("characterization tier  :", f.characterization_tier)
print("characterization gap   :", f.characterization_gap())
print("permits a grade claim  :", f.permits_output("product_grade_claim"))
print("permits a ceiling claim:", f.permits_output("purification_ceiling"))

```

Output:

```

el      total  lattice f  lattice  leachable
Al       30.0    0.60    18.00    12.00
Ti       10.0    0.90     9.00     1.00
Li        5.0    0.80     4.00     1.00
Fe        3.0    0.10     0.30     2.70
Na        8.0    0.30     2.40     5.60
K         8.0    0.30     2.40     5.60
B         1.0    0.50     0.50     0.50

sum of all traces      : 65.0 ppm
lattice-bound subtotal : 36.6 ppm
acid-accessible        : 28.4 ppm

characterization tier   : located
characterization gap    : {'tier': 'located', 'next_tier': None, 'missing_elements': [], 'lattice_unmeasu
 \ red': [], 'to_advance': [], 'basis_material': None}
permits a grade claim   : True
permits a ceiling claim : True

```

The lattice-bound subtotal is the part no acid route reaches. It is a property of crystallisation conditions, not of the flowsheet, and it sets the ceiling grade before any route is chosen.

## 5.4 Validation status

2 hand-traceable worked example(s) and 0 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```

test_oxide_conversion_worked_example
test_fe2o3_conversion_worked_example

```

## CHAPTER 6

# core/site.py

Module `ae.core.site`, 285 lines. Chapter text is this module's own documentation.

## 6.1 Purpose, governing equations and limitations

**SITE:** a place a plant could be built, described well enough to cost it.

Every economic and operational model takes a **SITE**, so comparing Telangana with New Mexico is a matter of swapping objects rather than editing code.

### Currency handling

Costs carry their own currency as a unit (USD or INR), and those units do not auto-convert (see `ae.core.units`). Conversion goes through `ExchangeRate`, which carries a date and a source, because an FX rate is a dated economic assumption and a model that silently converts at an undocumented rate cannot be audited. Purchasing power and construction cost adjustments are held as separate explicit factors, never folded into the FX rate.

### Tariffs and incentives

Both are modelled as uncertain: an incentive carries an award probability and a timing distribution rather than being booked as certain revenue, and a tariff carries the instrument that imposed it so its current status can be re-verified. Policies change, and a model that treats a credit as guaranteed overstates every downstream return.

## 6.2 Interface

### Types

**Currency.**

**ExchangeRate.** A dated FX assumption with a source and a scenario range.

Deliberately NOT a pint conversion: unit systems have no notion of the date or the source of a rate, and an undated rate in a twenty-year NPV is a hidden assumption worth several percent of the answer.

```
convert(self, q: Quantity) -> Quantity
```

Convert a cost from base currency to quote currency.

**PowerSupply.** Electricity, the dominant operating cost for reduction and fusion routes.

**LabourRates.**

**ReagentPrices.** Delivered reagent prices. Availability is separate from price: HF is not freely available everywhere, and a low quoted price with no local supplier is not a usable input.

```
price(self, reagent: str) -> Quantity
```

**LogisticsLink.** One freight lane. Observed lane costs are preferred over quoted rates.

**TradeMeasure.** A tariff or trade restriction, with the instrument that imposed it.

**Incentive.** A credit, grant or loan modelled as uncertain, never as certain revenue.

**PermittingRegime.**

**Site.** One candidate plant location.

site\_id follows COUNTRY-REGION-TAG, e.g. IN-TG-VKB or US-NM-ABQ.

```
tariff_on(self, origin: str, hs_code: str | None=None) -> float
```

Total ad valorem rate applying to goods of origin arriving here.

```
expected_incentive_value(self, currency: Currency | None=None) -> Quantity
```

Probability-weighted incentive total, in this site's currency.

Weighting by award probability is the point: the unweighted sum is the number a promoter quotes, not the number a model should carry.

## 6.3 Worked examples, reproduced by execution

### Two sites, one flowsheet, and the currency discipline between them

```
us, india = us_site(), india_site()
for s in (us, india):
    print(f"{s.name:12s} {s.currency:4s} power {s.power.energy_price.magnitude:9.4f} "
          f"{s.power.energy_price.units:12s} tag {s.power.energy_price.tag.value}")
print()
for s in (us, india):
    hf = s.reagents.prices["HF"]
    print(f"{s.name:12s} HF {hf.magnitude:8.2f} {hf.units:10s} "
          f"locally available: {s.reagents.locally_available.get('HF', 'unstated')}")
print()
print("India fluoride limit      :", india.permitting.effluent_limits["F"])
print("US permitting regime     :", us.permitting.jurisdiction if us.permitting else "not stated")
print()
# A cross-currency comparison requires an explicit dated rate, never unit magic.
from ae.core.units import DimensionalityError
try:
    _ = us.power.energy_price.quantity + india.power.energy_price.quantity
except DimensionalityError:
    print("USD + INR          : refused, no implicit exchange rate")
```

Output:

```
New Mexico  Currency.USD power    0.0541 USD / kilowatt_hour tag SOURCED
Telangana   Currency.INR power    7.0000 INR / kilowatt_hour tag ASSUMED

New Mexico  HF      1.80 USD / kilogram locally available: unstated
Telangana   HF     180.00 INR / kilogram locally available: False

India fluoride limit : 2.0 mg/l [ASSUMED]
US permitting regime : not stated

USD + INR           : refused, no implicit exchange rate
```

Currency is a pint base dimension, so adding USD to INR raises rather than silently reducing. An exchange rate is an economic assumption with a date and a source, handled explicitly and never by unit conversion.

## 6.4 Validation status

No test in the suite is marked golden or benchmark against this module. It is exercised by unmarked unit tests only, which check behaviour but make no claim of validation against an external datapoint or a closed form.

## PART II

# Physics

Unit-operation models. Each is a closed-form or low-dimensional numerical model of one physical mechanism, with its own validation status and its own stated limits.

## CHAPTER 7

# physics/comminution.py

Module `ae.physics.comminution`, 1060 lines. Chapter text is this module's own documentation.

## 7.1 Purpose, governing equations and limitations

Comminution energy: the Bond, Rittinger and Kick size-reduction laws.

This module answers one question, "how much electrical energy does it take to grind this feedstock from F80 to P80", and it answers it with the semi-empirical laws that the mineral processing literature actually uses, with their unit conventions made explicit rather than assumed.

### The short ton trap

Bond's third theory is conventionally written

$$W = 10 W_i \left( \frac{1}{\sqrt{P_{80}}} - \frac{1}{\sqrt{F_{80}}} \right)$$

and in Bond's own convention  $W$  and  $W_i$  are both in kWh per SHORT TON while  $P_{80}$  and  $F_{80}$  are in micrometres. The coefficient 10 is not a unit conversion: it is  $\sqrt{100 \mu m}$ , which falls out of Bond's definition of the work index as the energy to reduce a feed of effectively infinite size to a product with  $P_{80} = 100 \mu m$ . Setting  $F_{80} \rightarrow \infty$  and  $P_{80} = 100$  gives  $W = 10W_i/10 = W_i$ , which is the definition, and `test_bond_definitional_identity` checks exactly that.

Much of the modern literature quotes  $W_i$  in kWh per METRIC TONNE and plugs it into the same equation, obtaining  $W$  in kWh per metric tonne. Both readings are internally consistent; mixing them is a 10.23 percent error, in the direction of understating energy when a kWh/short-ton index is read as metric. `TonConvention` therefore makes the choice explicit at every call, and there is no default. 1 short ton = 907.18474 kg exactly (NIST SP 811, Appendix B.8), so 1 metric tonne = 1.1023113 short tons.

### Equations

**\*(E1) Bond's third theory of comminution.\*\*** Empirical, from Bond 1952 and Bond 1961, restated as Eq. 2 and Eq. 4 of Arellano-Pina et al. 2023 (doi:10.37190/ppmp/172458).

$$W = 10 W_i \left( P_{80}^{-1/2} - F_{80}^{-1/2} \right)$$

- $W$ : net specific grinding energy at the mill pinion, kWh per ton (short or metric, per `TonConvention`). Valid range: 0 to about 100 kWh/t; above that the assumption of a single work index over the size interval fails.
- $W_i$ : Bond work index, kWh per ton, same ton as  $W$ . Observed range for silicate ores about 7 to 25 kWh/short ton.
- $P_{80}$ : product 80 percent passing size, micrometres. Bond's calibration domain is roughly 30 to 3000  $\mu m$ ; see LIMITATIONS.
- $F_{80}$ : feed 80 percent passing size, micrometres, must exceed  $P_{80}$ . Calibration domain roughly 300  $\mu m$  to 50 mm.

- The bare 10 carries units of  $\mu m^{1/2}$  and is  $\sqrt{100\mu m}$  by construction, not a fitted constant.

\*\* (E2) Bond laboratory grindability equation.\*\* Eq. 5 of Arellano-Pina et al. 2023, who attribute the form and the constants to Bond 1961.

$$W_i = \frac{\gamma}{p_i^\alpha G^\beta \left( 10 P_{80}^{-1/2} - 10 F_{80}^{-1/2} \right)}$$

- $\gamma = 44.5$ : constant tied to the 44.5 lb (20.18 kg) standard ball charge. Dimensionally inconsistent as written; see LIMITATIONS.
- $p_i$ : closing screen aperture, micrometres, typically 106 to 300  $\mu m$  (the 100 mesh case is 149  $\mu m$ ).
- $G$ : grindability, grams of undersize produced per mill revolution at a 250 percent circulating load. Observed range about 0.5 to 3 g/rev.
- $\alpha = 0.23$ ,  $\beta = 0.82$ : exponents fitted by Bond; the criteria he used are not recorded in the literature (Arellano-Pina et al. 2023, Introduction, immediately after their Eq. 5: the criteria used by Bond to determine the numerical values of the constants are stated to be unknown).
- Output  $W_i$  is in kWh per short ton in Bond's original convention.

\*\* (E3) Generalised Walker size-reduction differential.\*\* First principles in the sense that all three classical laws are the same integral with one exponent.

$$dW = -C x^{-n} dx$$

Integrating from  $F_{80}$  to  $P_{80}$ :

$$n = 1 : W = C \ln(F_{80}/P_{80}) \quad (\text{Kick})$$

$$n = 3/2 : W = 2C \left( P_{80}^{-1/2} - F_{80}^{-1/2} \right) \quad (\text{Bond, with } 2C = 10W_i)$$

$$n = 2 : W = C \left( P_{80}^{-1} - F_{80}^{-1} \right) \quad (\text{Rittinger})$$

- $x$ : particle size, micrometres.
- $n$ : dimensionless exponent. 1, 1.5 and 2 recover Kick, Bond and Rittinger respectively. Austin 1973 (doi:10.1016/0032-5910(73)80042-7) shows the three laws are not competing physical theories but limiting cases whose applicability is set by the size range.
- $C$ : law-specific constant whose units depend on  $n$ , namely kWh/ton times  $\mu m^{n-1}$ . It is NOT transferable between laws.

\*\* (E4) Ton convention conversion.\*\*

$$W_{\text{metric}} = W_{\text{short}} \times \frac{1000 \text{ kg}}{907.18474 \text{ kg}} = 1.1023113 W_{\text{short}}$$

Energy per unit mass rises when the reference mass rises, so the metric figure is the larger of the two. Confusing the direction is the second classic error after confusing the conventions at all.

## Which law applies where

Kick for coarse crushing (product above roughly 50 mm), Bond for intermediate crushing and conventional ball milling (roughly 50  $\mu\text{m}$  to 50 mm), Rittinger for fine and ultrafine grinding where the created surface dominates (below roughly 50 to 100  $\mu\text{m}$ ). Hukki's 1961 argument, summarised by Austin 1973, is that the exponent  $n$  drifts continuously with size rather than switching at a boundary, so the three laws are three tangents to one curve. `recommended_law` returns the conventional choice and carries no claim to be exact at a boundary.

## LIMITATIONS

1. **\*\*Bond's equation is empirical and dimensionally unsound.\*\*** Eq. E2 in particular cannot be made dimensionally homogeneous:  $\gamma$  absorbs a pound-mass, a mill geometry and a revolution count, and  $p_i^{0.23}G^{0.82}$  is not a physical group. Arellano-Pina et al. 2023 state directly that multiple errors have been reported in the formulation because of inadequate dimensional analysis and that the model is semi-empirical. Do not differentiate it, do not extrapolate it, and do not read  $W_i$  as a material property independent of the test device.
2. **\*\*The work index is not size-independent.\*\*** Bond's own procedure fixes a 250 percent circulating load and a closing screen; changing the closing screen changes the measured  $W_i$  for the same ore. Rodriguez et al. 2021 (doi:10.3390/met11060970) report that for W and Ta ores the relation between  $W_i$  and grinding size showed no clear correlation while the grindability index  $G$  correlated robustly with grinding size, and advise correlating against  $G$  rather than  $W_i$ . Treat a single  $W_i$  used across a wide size interval as an approximation.
3. **\*\*Eq. E2's constants are specific to the 30.5 by 30.5 cm Bond mill.\*\*** Arellano-Pina et al. 2023 measured errors up to 68.3 percent when  $\alpha, \beta, \gamma$  from the standard mill were applied to a 20.5 cm diameter mill, even with matched fill and critical speed. Never apply this module's `bond_wi_from_grindability` to a non-standard mill without re-fitting the constants.
4. **\*\*No work index in this module is measured on Vikarabad ore.\*\*** The deposit is uncharacterized in the citable record. Every ore-type prior here is either a measurement on a different deposit, explicitly identified, or an ASSUMED value with a stated basis. A Bond work index is cheap to measure and should be measured before any mill is sized.
5. **\*\*Net pinion energy, not plant energy.\*\***  $W$  is the net energy at the mill shell. Motor, gearbox and classifier inefficiency, and the Bond efficiency factors for dry grinding, open circuit, oversize feed and fine product, are not applied here. Plant draw is higher, commonly by 10 to 30 percent for drive losses alone, and that correction belongs in a plant model with its own sourced factors.
6. **\*\*Feed and product must be reasonably size-graded.\*\*** F80/P80 collapse a whole distribution to one number. For a bimodal feed, or for the deliberate narrow size cuts an HPQ sand plant sells, energy predicted from an 80 percent passing pair can be substantially wrong.

## References

Every entry below was resolved against the Crossref REST API for the DOI shown, and the fields here (author list, year, title, journal, volume, issue, pages) are as Crossref returned them. Resolved 17 September 2026. Entries without a DOI say what was checked instead and what remains unverified.

Austin, L. G. (1973) A commentary on the Kick, Bond and Rittinger laws of grinding, Powder Technology 7(6), 315-317, doi 10.1016/0032-5910(73)80042-7. Source of the claim that the three laws are limiting cases of one relation rather than competing theories. Closed access, abstract only from this sandbox; the exponent argument is cited, no numerical constant is taken from it.

Arellano-Pina, R., Sanchez-Ramirez, E. A., Perez-Garibay, R. and Gutierrez-Perez, V. H. (2023) Bond's work index estimation using non-standard ball mills, Physicochemical Problems of Mineral Processing, doi

10.37190/ppmp/172458. Open access, full text retrieved. Source of Eq. E1 and Eq. E2 as restated here, of the constants  $\alpha = 0.23$ ,  $\beta = 0.82$ ,  $\gamma = 44.5$ , of the statement that Bond's criteria for those constants are not recorded, and of the non-standard-mill error measurements. Crossref returns no volume, issue or page range for this article.

Garcia, G. G., Oliva, J., Guasch, E., Anticoi, H., Coello-Velazquez, A. L. and Menendez-Aguado, J. M. (2021) Variability Study of Bond Work Index and Grindability Index on Various Critical Metal Ores, *Metals* 11(6), 970, doi 10.3390/met11060970. Source of the W and Ta finding that the work index does not correlate cleanly with grinding size while the grindability index does. Abstract only; the publisher blocked the full text from this sandbox. THIS ENTRY WAS PREVIOUSLY WRONG: it named "Rodriguez B. A., Menendez-Aguado J. M. et al.", written from memory. Rodriguez is not an author of this paper.

Wills, B. A. and Finch, J. A. (2016) Comminution, in *Wills' Mineral Processing Technology*, 8th edition, Elsevier, pp. 109-122, doi 10.1016/b978-0-08-097053-0.00005-4. Cited for the size ranges over which each size-reduction law is conventionally applied. Crossref gives 2016 as the year of this chapter record.

Bond, F. C. (1952) The Third Theory of Comminution, *Transactions AIME, Mining Engineering*, 484-494. NOT SOURCED DIRECTLY: no DOI exists for this article and no full text or publisher record was reachable from this sandbox. The author, year, title, and page range above are as cited in the reference list of Arellano-Pina et al. 2023 (verified, above), which is the route by which Eq. E1 enters this module. It is a secondary attribution, not a primary verification.

Bond, F. C. (1961) Crushing and grinding calculations, Allis-Chalmers Publication No. 07R9235B. NOT SOURCED DIRECTLY: same status as Bond 1952. The bibliographic fields above are as cited by Arellano-Pina et al. 2023, who attribute the Eq. E2 form and its constants to it. No primary copy was reachable.

Hukki, R. T. (1961). NOT SOURCED. The in-text reference to Hukki's 1961 argument is retained only as reported by Austin 1973, which is the form in which it reaches this module; no Hukki publication record could be resolved from this sandbox by DOI, by Crossref bibliographic search, or by OpenAlex search. Nothing in this module depends numerically on it.

NIST Special Publication 811 (2008 edition), *Guide for the Use of the International System of Units (SI)*, Appendix B.8. Source of the exact conversion 1 short ton = 907.18474 kg. Reachable only as a document reference, not through Crossref; the conversion factor itself is exact by definition of the avoirdupois pound (0.45359237 kg exactly) and is reproduced arithmetically in this module's tests rather than taken on the authority of the page citation.

## 7.2 Interface

### Types

**TonConvention.** Which ton the work index and the specific energy are denominated in.

There is deliberately no default. A silent default is how the 10.23 percent error in this module's subject matter propagates into a mill sizing.

```
energy_unit(self) -> str
```

**SizeLaw.** The three classical size-reduction laws, as exponents of Eq. E3.

```
exponent(self) -> float
```

### Functions

*bond\_specific\_energy*

```
bond_specific_energy(work_index: Qty, f80: Qty, p80: Qty, convention: TonConvention) -> Qty
```

Net specific grinding energy from Bond's third theory (Eq. E1).

**Parameters.** `work_index` Bond work index as a Quantity in kWh/short\_ton or kWh/metric\_ton. Its unit must match `convention`; a mismatch raises rather than converts, because an automatic conversion here is exactly the silent 10.23 percent error this module exists to prevent. `f80`, `p80` Feed and product 80 percent passing sizes, any length unit. `convention` Ton basis of the answer.

**Returns.** Quantity Specific energy in `convention.energy_unit`.

**Examples.** `Wi = 12.3 kWh/metric ton, F80 = 2000 um, P80 = 150 um: 10(12.3)(1/sqrt(150) - 1/sqrt(2000)) = 123(0.0816497 - 0.0223607) = 123(0.0592890) = 7.2925 kWh per metric tonne.`

```
>>> w = bond_specific_energy(Q_(12.3, "kWh/metric_ton"), Q_(2000.0, "um"),
...                           Q_(150.0, "um"), TonConvention.METRIC)
>>> round(w.magnitude, 4)
7.2925
```

*bond\_work\_index\_from\_energy*

```
bond_work_index_from_energy(specific_energy: Qty, f80: Qty, p80: Qty, convention: TonConvention) -> Qty
```

Invert Eq. E1 to get the operating work index from a measured mill draw.

This is the standard back-calculation used to compare a plant against a laboratory index (an operating work index above the laboratory index means the circuit is inefficient, or the ore has changed).

*bond\_wi\_from\_grindability*

```
bond_wi_from_grindability(grindability_g_per_rev: float, closing_screen: Qty, f80: Qty, p80: Qty, alpha: f
    \ loat=BOND_GRINDABILITY_ALPHA, beta: float=BOND_GRINDABILITY_BETA, gamma: float=BOND_GRINDABILITY_GAM
    \ MA) -> Qty
```

Bond's laboratory grindability equation (Eq. E2), in kWh per short ton.

**Parameters.** `grindability_g_per_rev` G, grams of closing-screen undersize produced per mill revolution at a 250 percent circulating load. Must be positive. `closing_screen p_i`, the closing screen aperture. `f80`, `p80` Feed and product 80 percent passing sizes of the locked-cycle test. `alpha`, `beta`, `gamma` Bond's constants. Defaults are the standard-mill values reported by Arellano-Pina et al. 2023. Override them ONLY with constants re-fitted for the mill actually used; see LIMITATIONS item 3.

**Notes.** The result is returned in kWh/short\_ton because that is the convention  $\gamma = 44.5$  was fitted in (44.5 lb of ball charge, imperial throughout). Converting afterwards is explicit and auditable; returning a bare number would not be.

*rittinger\_specific\_energy*

```
rittinger_specific_energy(constant: Qty, f80: Qty, p80: Qty) -> Qty
```

Rittinger's law, Eq. E3 with  $n = 2$ : energy proportional to new surface.

`constant` must carry units of specific energy times length, e.g. kWh\*um/metric\_ton, so that the product with a reciprocal length is a specific energy. The returned unit follows from the constant's unit.

*kick\_specific\_energy*

```
kick_specific_energy(constant: Qty, f80: Qty, p80: Qty) -> Qty
```

Kick's law, Eq. E3 with  $n = 1$ : energy proportional to the size reduction ratio.

`constant` carries specific-energy units directly, because  $\ln(F_{80}/P_{80})$  is dimensionless.

*walker\_specific\_energy*

```
walker_specific_energy(constant: Qty, f80: Qty, p80: Qty, exponent: float) -> Qty
```

Generalised Walker integral (Eq. E3) for arbitrary  $n$ .

Returns the integral  $\int_P^F Cx^{-n}dx$  evaluated with the sign convention that energy is positive for size reduction. Provided so the three named laws can be exercised as one family, and so a fitted non-classical exponent (Hukki's observation that  $n$  drifts with size) can be used without a new function.

`constant` must carry units of specific energy times  $\mu m^{n-1}$ , which is what makes the integral a specific energy. The RESULT is checked against that dimensionality, not the constant, because the required constant unit depends on the exponent and only the product is constrained. This check was previously absent while Kick and Rittinger both had it, so a Bond-unit constant (kWh/ton) used at  $n = 1.5$  returned 8.205266807797946 carrying kilowatt\_hour / micrometer \*\* 0.5 / ton, dimensionality  $[\text{length}]^{**1.5} / [\text{time}]^{**2}$  against specific energy's  $[\text{length}]^{**2} / [\text{time}]^{**2}$ . That magnitude is the same number Bond returns for the same reduction, i.e. dimensionally wrong and numerically plausible, which is the defect class hardest to see in a review.

*recommended\_law*

```
recommended_law(p80: Qty) -> tuple[SizeLaw, str]
```

Conventional law choice for a target product size, with the caveat text.

Boundaries follow standard practice as set out in Wills and Finch 2016, Comminution. They are conventions, not measured transitions: Austin 1973 argues the exponent drifts continuously, so a product size near a boundary has no single correct law.

*convert\_ton\_convention*

```
convert_ton_convention(specific_energy: Qty, to: TonConvention) -> Qty
```

Convert a specific energy between short-ton and metric-tonne bases (Eq. E4).

**Examples.** 7.2925 kWh/short ton is  $7.2925 \times (1000/907.18474) = 8.0386$  kWh/metric tonne. The metric figure is larger because the reference mass is larger.

```
>>> q = convert_ton_convention(Q_(7.2925, "kWh/short_ton"), TonConvention.METRIC)
>>> round(q.magnitude, 4)
8.0386
```

*percent\_error*

```
percent_error(reference: float, approximation: float) -> float
```

Absolute percentage error, Eq. 13 of Arellano-Pina et al. 2023.

$$\% \text{ error} = \left| \frac{W_{i,\text{ref}} - W_{i,\text{approx}}}{W_{i,\text{ref}}} \right| \times 100$$

*work\_index\_prior*

```
work_index_prior(ore_type: OreType) -> Value
```

Bond work index prior for an ore class, as a provenance-tagged Value.

**Raises.** `KeyError` For an ore type with no prior at all, rather than returning a guess. Alaskite, novaculite, hydrothermal crystal and dolomite are in that position: no retrievable Bond index, and no defensible analogue.

*sourced\_quartz\_rich\_benchmark*

```
sourced_quartz_rich_benchmark() -> Value
```

The one SOURCED Bond work index on a quartz-rich ore, for benchmarking.

*feedstock\_work\_index*

```
feedstock_work_index(feedstock: Feedstock, convention: TonConvention, allow_prior: bool=False) -> Value
```

Work index for a feedstock: measured if present, else an explicit prior.

**Parameters.** `feedstock` The ore. `feedstock.physical.bond_work_index` is used when it has been measured. `convention` Ton basis for the returned value. `allow_prior` When the feedstock has no measured index, `False` (the default) raises and `True` falls back to `work_index_prior` with its ASSUMED tag intact. The default is to raise because a mill sized on an assumed work index for an uncharacterized deposit is a scenario, and the caller should have to say so.

**Raises.** `ValueError` If the index is unmeasured and `allow_prior` is `False`.

*grinding\_specific\_energy*

```
grinding_specific_energy(feedstock: Feedstock, p80: Qty, convention: TonConvention, f80: Qty | None=None,
    \ allow_prior: bool=False, law: SizeLaw | None=None) -> tuple[Qty, Value, str]
```

Net specific grinding energy for one feedstock to a target product size.

**Parameters.** `feedstock` Ore. Supplies the work index and, if `f80` is omitted, the feed size from `feedstock.physical.f80`. `p80` Target product 80 percent passing size. `convention` Ton basis. `f80` Feed size override, for a circuit whose feed is set by an upstream crusher rather than by the ore. `allow_prior` Forwarded to `feedstock_work_index`. `law` Force a law. `None` uses `recommended_law` and returns its note.

**Returns.** (energy, work\_index\_value, note) note records the law used and any caveat, so a caller writing a report cannot silently drop the provenance.

**Raises.** ValueError If the feed size is neither supplied nor measured, or if a law other than Bond is requested (Kick and Rittinger need their own fitted constants, which this module does not have for any quartz ore).

*grinding\_energy\_cost*

```
grinding_energy_cost(feedstock: Feedstock, site: Site, p80: Qty, throughput: Qty, convention: TonConventio
    \ n, f80: Qty | None=None, allow_prior: bool=False, motor_efficiency: float=1.0) -> tuple[Qty, Qty, st
    \ r]
```

Grinding electricity cost for a feedstock at a site.

**Parameters.** feedstock, site Ore and location. The energy price and its currency come from site.power.energy\_price; nothing about the plant location is hardcoded. p80, f80, convention, allow\_prior As grinding\_specific\_energy. throughput Ore mass rate, e.g. Q\_(50.0, "tonne/hour"). motor\_efficiency Fraction in (0, 1]. Divides the net pinion energy to give the energy drawn from the grid. The default of 1.0 returns net pinion energy and is an underestimate of plant draw; see LIMITATIONS item 5.

**Returns.** (cost\_rate, power, note) Cost per unit time in the site's currency, and the electrical power.

## 7.3 Worked examples, reproduced by execution

The 2 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

**physics.comminution.bond\_specific\_energy**

```
>>> w = bond_specific_energy(Q_(12.3, "kWh/metric_ton"), Q_(2000.0, "um"),
...                           Q_(150.0, "um"), TonConvention.METRIC)
>>> round(w.magnitude, 4)
7.2925
```

**physics.comminution.convert\_ton\_convention**

```
>>> q = convert_ton_convention(Q_(7.2925, "kWh/short_ton"), TonConvention.METRIC)
>>> round(q.magnitude, 4)
8.0386
```

## 7.4 Validation status

12 hand-traceable worked example(s) and 4 validation benchmark(s) cover this module. Classification is the repository's own, from scripts/export\_validation.py: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                          | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_benchmark_ore_a_work_index_round_trip         | literature | Benchmark: Ore A of Arellano-Pina et al. 2023, $W_i = 12.3$ kWh/metric ton. The only Bond ball mill work index on a quartz-rich ore retrievable from a peer-reviewed DOI-bearing source in this environment (Table 2, standard procedure, 30.5 cm Bond mill; Ore A is 28.75 wt% Si, a quartz plus aluminosilica<br>10.1016/0032-5910(73; 10.1016/b978-0-08-097053-0.00005-4; 10.3390/met11060970; 10.3390/met11060970; 10.37190/ppmp/172458; 10.37190/ppmp/172458                    |
| test_benchmark_reduced_procedure_spread            | literature | Benchmark: the paper's own standard-vs-reduced procedure spread, Table 2. Ore A 12.3 vs 11.8 (4.1 percent), Ore B 15.9 vs 16.5 (3.8), Ore C 18.7 vs 19.7 (5.3), Ore D 16.5 vs 17.2 (4.2). This validates percent_error against four published error figures and, more usefully, quantifies the irred<br>Error stated: 4.1; 5; 5<br>10.1016/0032-5910(73; 10.1016/b978-0-08-097053-0.00005-4; 10.3390/met11060970; 10.3390/met11060970; 10.37190/ppmp/172458; 10.37190/ppmp/172458    |
| test_benchmark_grindability_equation_against_ore_a | literature | Benchmark: can Eq. E2 reproduce Ore A's published $W_i$ of 12.3 kWh/t? Honest negative-ish result. The paper does not publish Ore A's G, $p_{-i}$ , F80 or P80 individually, only the resulting $W_i$ , so Eq. E2 cannot be checked against it directly. What CAN be checked is the inverse: solve Eq. E2 for the grindab<br>10.1016/0032-5910(73; 10.1016/b978-0-08-097053-0.00005-4; 10.3390/met11060970; 10.3390/met11060970; 10.37190/ppmp/172458; 10.37190/ppmp/172458          |
| test_benchmark_prior_against_sourced_measurement   | literature | Benchmark: the ASSUMED vein quartz prior against the one real measurement. The prior is 13.6 kWh/short ton. Ore A measured 12.3 kWh/metric tonne, which is $12.3/1.1023113 = 11.158$ kWh/short ton. Error of the prior against that measurement is reported. It is large enough (about 22 percent) to make the<br>Error stated: 22<br>10.1016/0032-5910(73; 10.1016/b978-0-08-097053-0.00005-4; 10.3390/met11060970; 10.3390/met11060970; 10.37190/ppmp/172458; 10.37190/ppmp/172458 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_bond_worked_example_metric
test_bond_definitional_identity
test_coefficient_ten_is_sqrt_of_reference_size
test_ton_conversion_worked_example
test_short_ton_definition_is_exact
test_convention_confusion_is_10_23_percent
test_bond_inversion_round_trip
test_grindability_equation_worked_example
test_walker_reduces_to_the_three_named_laws
test_kick_worked_example
test_rittinger_worked_example
test_percent_error_worked_example
```

## CHAPTER 8

# physics/psd.py

Module `ae.physics.psd`, 715 lines. Chapter text is this module's own documentation.

## 8.1 Purpose, governing equations and limitations

Particle size distributions: log-normal and Rosin-Rammler, moments, and the number-area-volume weighting conversions that are the commonest error source in the field.

Why the weighting conversions get their own module section: a laser-diffraction instrument reports a VOLUME-weighted distribution, a microscope image analysis reports a NUMBER-weighted one, and a BET surface area is an AREA-weighted property. For a log-normal with geometric standard deviation 1.5 the number-median and volume-median diameters differ by 64 percent. Quoting a d50 without its weighting basis is therefore not a rounding issue, it is a different number, and mixing the two silently propagates into every downstream surface-area, packing and viscosity estimate.

### EQUATIONS

(1) Log-normal number density (Hatch and Choate 1929, doi 10.1016/s0016-0032(29)91451-4):

$$f_0(d) = \frac{1}{d s \sqrt{2\pi}} \exp \left[ -\frac{(\ln d - \ln d_{gn})^2}{2s^2} \right]$$

Symbols:  $f_0$  number density function, 1/m (per unit diameter).  $d$  particle diameter, m, valid range 1e-9 to 1e-2 m.  $d_{\{gn\}}$  geometric (count) median diameter, m. Also the count geometric mean.  $s = \ln(\sigma_g)$ , dimensionless, valid range 0 (monodisperse) to about 1.2. Above  $s$  of roughly 1.2 the log-normal form is rarely defensible for a comminution product.  $\sigma_g$  geometric standard deviation, dimensionless,  $\geq 1$ .

(2) Moments of the log-normal (derived, not cited: substitute  $u = \ln d$  in  $\int d^k f_0(d) dd$  and complete the square):

$$M_k \equiv \int_0^\infty d^k f_0(d) dd = d_{gn}^k \exp\left(\frac{k^2 s^2}{2}\right)$$

Symbols:  $M_k$  the  $k$ -th raw moment about the origin, units  $m^k$ ;  $k$  the order, any real.

(3) Hatch-Choate conversions between weighted median diameters (derived from (2)):

$$d_{ga} = d_{gn} \exp(2s^2), \quad d_{gv} = d_{gn} \exp(3s^2), \quad d_{32} = d_{gn} \exp\left(\frac{5}{2}s^2\right)$$

Symbols:  $d_{\{ga\}}$  area-weighted (surface) median diameter, m.  $d_{\{gv\}}$  volume-weighted (mass) median diameter, m, which is what a laser diffraction d50 reports.  $d_{\{32\}}$  Sauter mean diameter, m, defined as  $M_3/M_2$  and the only mean that preserves the surface-to-volume ratio.

Derivation of  $d_{\{32\}}$ :  $M_3/M_2 = d_{gn} \exp(9s^2/2) / \exp(4s^2/2) = d_{gn} \exp(5s^2/2)$ . A log-normal stays log-normal under reweighting with the same  $s$  and a shifted median, which is the property that makes (3) exact rather than approximate.

(4) Quantiles of the log-normal:

$$d_p = d_g \exp[z_p s]$$

with  $z_p$  the standard normal quantile ( $z_{\{0.1\}} = -1.281552$ ,  $z_{\{0.9\}} = +1.281552$ ) and  $d_g$  the median on the chosen weighting basis.

(5) Span, the standard polydispersity index:

$$\text{span} = \frac{d_{90} - d_{10}}{d_{50}}$$

(6) Rosin-Rammler cumulative volume undersize (Rosin and Rammler 1933, as characterised by Alderliesten 2013, doi 10.1002/ppsc.201200021):

$$Q_3(d) = 1 - \exp \left[ - \left( \frac{d}{d'} \right)^n \right]$$

Symbols:  $Q_3$  cumulative VOLUME fraction undersize, dimensionless, range  $[0, 1]$ .  $d$  diameter, m.  $d'$  location parameter, m: the size at which  $Q_3 = 1 - 1/e = 0.6321$ .  $n$  spread parameter, dimensionless. Alderliesten 2013 shows that distributions with  $n < 3$  "cannot exist" as physically complete distributions and can at most describe the measured range, so this module warns below  $n = 3$  (see LIMITATIONS).

(7) Rosin-Rammler moments and derived diameters (derived by substituting  $x = (d/d')^n$ , which turns the integral into a gamma function):

$$d_{50} = d' (\ln 2)^{1/n}, \quad \mathbb{E}_v[d] = d' \Gamma\left(1 + \frac{1}{n}\right), \quad d_{32} = \frac{d'}{\Gamma\left(1 - \frac{1}{n}\right)}$$

Symbols:  $\Gamma$  the gamma function;  $\mathbb{E}_v[d]$  the volume-weighted mean diameter, m. The  $d_{\{32\}}$  expression requires  $n > 1$  for the gamma argument to be positive, and diverges as  $n$  approaches 1.

Derivation of  $d_{\{32\}}$  for Rosin-Rammler: the specific surface per unit volume of solid is  $\int (6/d) dQ_3$ , and with the substitution above  $\int_0^\infty d^{-1} dQ_3 = \Gamma(1 - 1/n)/d'$ , so  $S_v = 6\Gamma(1 - 1/n)/d'$  and  $d_{32} \equiv 6/S_v = d'/\Gamma(1 - 1/n)$ .

(8) Specific surface area from a distribution:

$$S_m = \frac{6}{\rho d_{32} \psi}$$

Symbols:  $S_m$  specific surface area,  $\text{m}^2/\text{kg}$ .  $\rho$  true particle density,  $\text{kg}/\text{m}^3$  (2650 for quartz).  $d_{32}$  Sauter mean diameter, m.  $\psi$  sphericity, dimensionless, range  $(0, 1]$ . Defaults to 1 (spheres) and must be supplied for an angular comminution product.

**Derivation:** for a sphere, area/volume =  $6/d$  exactly; integrating over a distribution replaces  $d$  by its surface-volume mean  $d_{32}$  by construction, and dividing by density converts per-volume to per-mass. Non-spherical particles have more area at the same volume, so dividing by  $\psi \leq 1$  raises  $S_m$ .

## LIMITATIONS

1. **S<sub>m</sub> IS A GEOMETRIC SURFACE AREA, NOT A BET SURFACE AREA.** Equation (8) counts only the external envelope of a smooth convex particle. Real quartz particles have surface roughness, internal cracks and (after calcination) decrepitated inclusion cavities, all of which BET nitrogen adsorption sees and equation (8) does not. Ringdalen 2015 (doi 10.1007/s11837-014-1149-y) measured 0.38 to 0.47 m<sup>2</sup>/g on heat-treated natural quartz lump, values that a geometric calculation on lump-sized particles underestimates by orders of magnitude. Expect equation (8) to UNDERSTATE BET area, by a factor that is a property of the ore and its thermal history, and never substitute one for the other.
2. **SPHERICITY IS AN INPUT, NOT A PREDICTION.** No source reachable here gives a sphericity for a crushed vein quartz product, so psi defaults to 1 and any other value must be supplied by the caller with its own basis.
3. **THE ROSIN-RAMMLER LOCATION PARAMETER IS NOT PHYSICALLY INTERPRETABLE.** Alderliesten 2013 shows that the physical meaning of d' depends on the value of n, which makes it unsuitable as a model input even though it remains useful for production control. Use d<sub>32</sub> or a moment-ratio mean for physical models, which is why this module exposes both.
4. **n < 3 IS NOT A VALID DISTRIBUTION.** Per Alderliesten 2013, Rosin-Rammler fits with n < 3 describe the measured window only and must not be extrapolated to the fine tail. This module refuses to integrate moments for n ≤ 1 and flags 1 < n < 3.
5. **NO COMMINUTION MODEL.** Nothing here predicts what PSD a mill produces from a given ore. That needs the Bond work index, which is a measured feedstock property (feedstock.physical.bond\_work\_index) and is absent for an uncharacterized deposit.
6. **SINGLE-POPULATION FORMS ONLY.** Both distributions are unimodal by construction. A real graded filler blend is deliberately multimodal and must be handled as a mixture of these objects, which is what ae.physics.packing does.
7. **NO SIZE-DEPENDENT COMPOSITION.** Impurity content very often varies with size fraction (feldspar and mica report to specific sizes in a crushed quartz), and this module carries no composition at all. Pair it with a measured ImpurityProfile per fraction.

## TYPE CHECKING NOTE

mypy --strict reports `Quantity` has no attribute "to" and Variable "ae.core.units.Quantity" is not valid as a type against this module. Those errors originate in `ae.core.units`, which declares `Quantity = UREG.Quantity` (a variable binding, not a type alias), so mypy cannot treat it as a type. The same errors appear against the four core modules themselves (46 of them), and the core modules are not ours to change. Every annotation here follows the core convention deliberately rather than diverging from it for a clean checker run.

## References

Every entry below was resolved against the Crossref REST API for the DOI shown, and the fields here (author list, year, title, journal, volume, issue, pages) are as Crossref returned them. Resolved 17 September 2026. Entries without a DOI say what was checked instead and what remains unverified.

Hatch, T. and Choate, S. P. (1929) Statistical description of the size properties of non uniform particulate substances, *Journal of the Franklin Institute* 207(3), 369-387, doi 10.1016/s0016-0032(29)91451-4. Source of equation (1), the log-normal number density, and of the Hatch-Choate weighted median conversions in equation (3).

Alderliesten, M. (2013) Mean Particle Diameters. Part VII. The Rosin-Rammler Size Distribution: Physical and Mathematical Properties and Relationships to Moment-Ratio Defined Mean Particle Diameters,

Particle and Particle Systems Characterization 30(3), 244-257, doi 10.1002/ppsc.201200021. Source of the characterisation of the Rosin-Rammler form used in equation (6), of the result that distributions with spread parameter below three cannot exist, and of the finding that the location parameter  $d'$  is not physically interpretable because its meaning depends on  $n$ . Closed access; the abstract states all three of these claims explicitly and is what was read.

Rosin, P. and Rammler, E. (1933). NOT SOURCED DIRECTLY: no DOI could be resolved for the 1933 original by Crossref DOI lookup, Crossref bibliographic search, or OpenAlex search from this sandbox. Equation (6) is taken from Alderliesten 2013 (verified, above), which characterises the distribution and is the actual source used here. The in-text "Rosin and Rammler 1933" names the distribution's origin and no numerical value in this module depends on the 1933 paper. A related later paper by the same pair does resolve (Rosin and Rammler 1934, Die Kornzusammensetzung des Mahlgutes im Lichte der Wahrscheinlichkeitslehre, doi 10.1007/bf01439773) but it is not the 1933 work cited and is not used.

Ringdalen, E. (2015) Changes in Quartz During Heating and the Possible Effects on Si Production, JOM 67(2), 484-492, doi 10.1007/s11837-014-1149-y. Source of the 0.38 to 0.47 m<sup>2</sup>/g BET specific surface areas on heat-treated natural quartz lump, which bound how far the geometric equation (8) understates a measured value. The year is the February 2015 print issue; Crossref's issued date is the 2014-10-10 online-first publication.

## 8.2 Interface

### Types

**Weighting.** Distribution weighting basis. Mixing these is the error this module exists to stop.

**NUMBER** : count-weighted, what image analysis and single-particle counters report. **AREA** : surface-weighted. **VOLUME** : volume or mass weighted, what laser diffraction and sieving report.

**PSDSummary.** Descriptors of a distribution on one stated weighting basis.

**LogNormalPSD.** Log-normal particle size distribution, equations (1) to (5).

The distribution is stored by its **COUNT** median `d_gn` and `sigma_g`, and every other median is derived, so the weighting basis of a returned diameter is always explicit and never ambiguous.

**Parameters.** `d_gn` Geometric count median diameter. `sigma_g` Geometric standard deviation,  $\geq 1$ . Unity is monodisperse. `density` True particle density, used for surface-area outputs. `sphericity` Sphericity Value, default spheres.

### Examples.

```
>>> from ae.core.units import Q_
>>> psd = LogNormalPSD(d_gn=Q_(10.0, "um"), sigma_g=1.5, density=Q_(2650.0, "kg/m**3"))
>>> round(float(psd.median(Weighting.VOLUME).to("um").magnitude), 4)
16.3756
>>> round(float(psd.sauter_d32().to("um").magnitude), 4)
15.0833
```

```
s(self) -> float
```

`s = ln(sigma_g)`, the log-space standard deviation, dimensionless.

```
moment(self, k: float) -> Quantity
```

k-th raw moment of the NUMBER distribution, equation (2), units  $m^k$ .

```
median(self, weighting: Weighting=Weighting.NUMBER) -> Quantity
```

Median diameter on a stated weighting basis, equation (3).

**Notes.** The exponents are 0, 2 and 3 for number, area and volume respectively. This is the conversion that laser diffraction versus microscopy comparisons get wrong.

```
sauter_d32(self) -> Quantity
```

Sauter mean diameter  $M_3/M_2 = d_{gn} \exp(5 \sigma^2 / 2)$ , equation (3).

```
quantile(self, p: float, weighting: Weighting=Weighting.VOLUME) -> Quantity
```

Quantile diameter at cumulative fraction p on a weighting basis, eq (4).

```
span(self, weighting: Weighting=Weighting.VOLUME) -> float
```

Span  $(d_{90} - d_{10})/d_{50}$ , equation (5). Independent of weighting basis.

**Notes.** Because all three quantiles scale with the same median, span depends only on  $\sigma_g$ :  $\text{span} = \exp(z_{90} \sigma) - \exp(z_{10} \sigma)$ . That is a useful check on a reported span, and it is why this method returns the same value on every basis.

```
specific_surface_area(self) -> Quantity
```

Geometric specific surface area, equation (8),  $m^2/kg$ .

```
summary(self, weighting: Weighting=Weighting.VOLUME) -> PSDSummary
```

Full descriptor set on one stated basis.

**RosinRammlerPSD.** Rosin-Rammler (Weibull) volume distribution, equations (6) to (8).

**Parameters.**  $d\_prime$  Location parameter: the size at which cumulative volume undersize is 0.6321. Alderliesten 2013 shows this parameter is not physically interpretable on its own; use `sauter_d32` for physical models.  $n$  Spread parameter. Values below 3 are flagged by `spread_warning`. `density` True particle density. `sphericity` Sphericity Value, default spheres.

**Examples.**

```
>>> from ae.core.units import Q_
>>> rr = RosinRammlerPSD(d_prime=Q_(20.0, "um"), n=3.5, density=Q_(2650.0, "kg/m**3"))
>>> round(float(rr.median().to("um").magnitude), 4)
18.0116
>>> round(float(rr.sauter_d32().to("um").magnitude), 4)
15.6741
```

```
spread_warning(self) -> str | None
```

Non-None when  $n$  is in the range Alderliesten 2013 rules out.

```
cumulative_undersize(self, diameter: Quantity) -> float
```

Cumulative VOLUME fraction undersize at a diameter, equation (6).

```
quantile(self, p: float) -> Quantity
```

Inverse of equation (6):  $d_p = d' [-\ln(1-p)]^{1/n}$ , volume basis.

```
median(self) -> Quantity
```

Volume median  $d_{50} = d' (\ln 2)^{1/n}$ , equation (7).

```
volume_weighted_mean(self) -> Quantity
```

$E_v[d] = d' \Gamma(1 + 1/n)$ , equation (7).

```
sauter_d32(self) -> Quantity
```

$d_{32} = d' / \Gamma(1 - 1/n)$ , equation (7). Requires  $n > 1$ .

```
span(self) -> float
```

Span on the volume basis, equation (5).

```
specific_surface_area(self) -> Quantity
```

Geometric specific surface area, equation (8),  $m^2/kg$ .

```
summary(self) -> PSDSummary
```

Full descriptor set, volume basis (the basis the form is defined on).

## Functions

*feedstock\_sphericity\_assumption*

```
feedstock_sphericity_assumption(feedstock: Feedstock) -> Value
```

Sphericity to use for a feedstock, as an explicitly ASSUMED Value.

No sphericity measurement for a crushed vein quartz product was obtainable from any source reachable here, and sphericity is not a field of `ae.core.feedstock.PhysicalProperties`, so this returns an honest ASSUMED value of unity rather than a fabricated number. Override it wherever a measurement exists.

*specific\_surface\_area*

```
specific_surface_area(sauter_d32: Quantity, density: Quantity, sphericity: Value | None=None) -> Quantity
```

Geometric specific surface area from the Sauter mean diameter, equation (8).

**Parameters.** sauter\_d32 Sauter mean diameter ( $M_3/M_2$  of the distribution). density True particle density. sphericity Sphericity as a Value in (0, 1]. Defaults to 1, i.e. spheres.

**Returns.** Quantity Specific surface area, m<sup>2</sup>/kg.

**Notes.** Geometric, not BET. See LIMITATIONS item 1.

**Examples.** A 15.0833 um Sauter mean at quartz density:

```
>>> from ae.core.units import Q_
>>> s = specific_surface_area(Q_(15.083327243425664, "um"), Q_(2650.0, "kg/m**3"))
>>> round(float(s.to("m**2/kg").magnitude), 4)
150.1095
```

*convert\_lognormal\_median*

```
convert_lognormal_median(median: Quantity, sigma_g: float, from_weighting: Weighting, to_weighting: Weighting) -> Quantity
```

Convert a log-normal median between weighting bases, equation (3).

This is the single function to reach for when an instrument reports a d50 on one basis and a model needs it on another.

**Parameters.** median Median diameter on *from\_weighting*. sigma\_g Geometric standard deviation, shared by all bases for a log-normal. *from\_weighting*, *to\_weighting* Source and target bases.

**Returns.** Quantity Median on *to\_weighting*.

**Examples.** A laser-diffraction volume d50 of 16.3756 um with sigma\_g 1.5 is a count median of 10 um, a 64 percent difference:

```
>>> from ae.core.units import Q_
>>> d = convert_lognormal_median(Q_(16.375575970496996, "um"), 1.5,
...                               Weighting.VOLUME, Weighting.NUMBER)
>>> round(float(d.to("um").magnitude), 6)
10.0
```

## 8.3 Worked examples, reproduced by execution

The 4 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

**physics.psd.LogNormalPSD**

```
>>> from ae.core.units import Q_
>>> psd = LogNormalPSD(d_gn=Q_(10.0, "um"), sigma_g=1.5, density=Q_(2650.0, "kg/m**3"))
>>> round(float(psd.median(Weighting.VOLUME).to("um").magnitude), 4)
16.3756
>>> round(float(psd.sauter_d32().to("um").magnitude), 4)
15.0833
```

**physics.psd.RosinRammlerPSD**

```
>>> from ae.core.units import Q_
>>> rr = RosinRammlerPSD(d_prime=Q_(20.0, "um"), n=3.5, density=Q_(2650.0, "kg/m**3"))
>>> round(float(rr.median().to("um").magnitude), 4)
18.0116
>>> round(float(rr.sauter_d32().to("um").magnitude), 4)
15.6741
```

**physics.psd.convert\_lognormal\_median**

```
>>> from ae.core.units import Q_
>>> d = convert_lognormal_median(Q_(16.375575970496996, "um"), 1.5,
...                               Weighting.VOLUME, Weighting.NUMBER)
>>> round(float(d.to("um").magnitude), 6)
10.0
```

**physics.psd.specific\_surface\_area**

```
>>> from ae.core.units import Q_
>>> s = specific_surface_area(Q_(15.083327243425664, "um"), Q_(2650.0, "kg/m**3"))
>>> round(float(s.to("m**2/kg").magnitude), 4)
150.1095
```

## 8.4 Validation status

5 hand-traceable worked example(s) and 2 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                               | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_benchmark_geometric_ssa_against_ringdalen_bet</code>         | literature | Geometric surface area against Ringdalen 2015 BET measurements. Literature: Ringdalen 2015 (doi 10.1007/s11837-014-1149-y) reports BET specific surface areas of 0.38 to 0.47 m <sup>2</sup> /g on heat-treated natural quartz. Model: to produce 0.42 m <sup>2</sup> /g (the middle of that band) by geometry alone, equation (8) r 10.1007/s11837-014-1149-y                                                                                              |
| <code>test_benchmark_rosin_rammler_against_lognormal_sauter_mean</code> | analytic   | Cross-validate the two distribution families on a matched volume median. Two independent functional forms fitted to the same volume median should give Sauter means that agree to within the difference in their tail shapes, which is a real consistency check on both closed-form derivations. Matched at 10.1002/ppsc.201200021; 10.1002/ppsc.201200021; 10.1007/bf01439773; 10.1007/s11837-014-1149-y; 10.1007/s11837-014-1149-y; 10.1016/s0016-0032(29 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_golden_lognormal_weighted_medians
test_golden_weighting_conversion_relationship_pinned
test_golden_lognormal_quantiles_and_span
test_golden_specific_surface_area
test_golden_rosin_rammler_diameters
```

## CHAPTER 9

# physics/liberation.py

Module `ae.physics.liberation`, 647 lines. Chapter text is this module's own documentation.

## 9.1 Purpose, governing equations and limitations

Liberation: how finely must quartz be ground before inclusions are exposed.

An acid leach, a flotation stage and a magnetic separator can only act on an impurity they can reach. A mineral or fluid inclusion wholly enclosed by quartz is invisible to all three no matter how aggressive the reagent, and the only lever that exposes it is size reduction. This module puts a number on that lever from stereology, with the geometry derived rather than asserted.

The industrial statement of the same fact, from the HPQ processing literature: mineral impurities in quartz can be separated efficiently by flotation, magnetic separation and conventional leaching only when they have been exposed or liberated by crushing and grinding, and at a given size the liberation degree of fine-grained gangue is severely limited, which is what drives low separation efficiency (Lin M. et al. 2020, A Critical Review on the Mineralogy and Processing for High-Grade Quartz, Mining, Metallurgy and Exploration 37:1627-1639, doi:10.1007/s42461-020-00247-0, Section 3).

### Equations

\*\*(E1) Random-fracture exposure probability, cubic lattice derivation.\*\* Derived here from first principles, not taken from a source.

Consider a particle of characteristic size  $d_p$  containing an inclusion of characteristic size  $d_{inc}$ , with the inclusion centre positioned uniformly at random inside the particle and both idealised as coaxial cubes. The inclusion is fully enclosed (not exposed) only if its centre lies in the concentric interior cube of edge  $d_p - d_{inc}$ . In one dimension the centre must lie in an interior interval of length  $d_p - d_{inc}$  out of  $d_p$ ; the three axes are independent under uniform positioning, so

$$P(\text{enclosed}) = \left(1 - \frac{d_{inc}}{d_p}\right)^3, \quad E = P(\text{exposed}) = 1 - \left(1 - \frac{d_{inc}}{d_p}\right)^3$$

for  $d_{inc} \leq d_p$ , and  $E = 1$  for  $d_{inc} > d_p$  (an inclusion larger than the particle cannot be enclosed by it).

- $E$ : exposure fraction, dimensionless, range  $[0, 1]$ . Interpreted as the fraction of inclusions of that size class intersecting a particle surface.
- $d_p$ : particle size, micrometres, range 1 to 10000  $\mu\text{m}$ . Below about 1  $\mu\text{m}$  the continuum geometry and the assumption that an inclusion is small compared with a particle both fail.
- $d_{inc}$ : inclusion characteristic size, micrometres, range 0.1 to 1000  $\mu\text{m}$ . Fluid inclusions in vein quartz are commonly sub-micron to tens of micrometres; mineral inclusions span microns to millimetres.
- $r = d_{inc}/d_p$ : size ratio, dimensionless, range  $[0, 1]$  in the non-trivial branch.

Expanding for small  $r$  gives  $E \approx 3r - 3r^2 + r^3$ , so exposure rises initially at three times the size ratio: three orthogonal faces each contribute one chance of intersection. That factor of 3 is the geometric content of the model and is the reason grinding buys exposure faster than a naive one-dimensional argument suggests.

**\*\*(E2) Liberation size for a target exposure.\*\*** Inverting E1.

$$d_p^* = \frac{d_{inc}}{1 - (1 - E^*)^{1/3}}$$

- $d_p^*$ : required particle size, micrometres.
- $E^*$ : target exposure fraction, dimensionless, in  $(0, 1)$ .  $E^* = 1$  requires  $d_p^* = d_{inc}$ , i.e. grinding to the inclusion size, which is the asymptote and usually uneconomic.

Worked consequence: for  $E^* = 0.5$ , the denominator is  $1 - 0.5^{1/3} = 1 - 0.793701 = 0.206299$ , so  $d_p^* = 4.847 d_{inc}$ . Half of all inclusions are exposed once particles are ground to roughly five times the inclusion size. For  $E^* = 0.9$ ,  $1 - 0.1^{1/3} = 1 - 0.464159 = 0.535841$  and  $d_p^* = 1.866 d_{inc}$ : the last 40 percent of exposure costs a 2.6-fold further size reduction, and by Bond's law (see `ae.physics.comminution`) energy scales as  $d_p^{-1/2}$ , so that final push costs about 60 percent more grinding energy per tonne.

**\*\*(E3) Spherical variant.\*\*** Same argument with concentric spheres, for comparison rather than for use.

$$E_{sph} = 1 - \left(1 - \frac{d_{inc}}{d_p}\right)^3$$

The cubic and spherical derivations give the SAME expression, because in both cases the enclosed-centre region is a concentric body linearly shrunk by  $d_{inc}$  in each dimension and volume scales as the cube of the linear scale. This coincidence is worth stating: it means E1 is robust to particle shape under uniform inclusion positioning, and correspondingly that agreement between the two is NOT evidence the model is right.

**\*\*(E4) Polydisperse inclusion population.\*\*** Exposure over a size distribution.

$$\bar{E}(d_p) = \sum_i w_i E(d_p, d_{inc,i}), \quad \sum_i w_i = 1$$

- $w_i$ : number or volume weight of inclusion size class  $i$ , dimensionless. Whether the weights are number-based or volume-based changes the answer substantially, because the mass of impurity sits in the large inclusions while the count sits in the small ones. This module requires the basis to be stated (`InclusionWeighting`) and does not guess.

**\*\*(E5) Leachable fraction of an element.\*\*** Couples liberation to chemistry.

$$f_{leachable}(d_p) = f_{surface} + f_{fluid} E_{fluid}(d_p) + f_{mineral} E_{mineral}(d_p)$$

with  $f_{surface} + f_{fluid} + f_{mineral} + f_{lattice} = 1$ . The lattice term never appears: it is not leachable at any particle size, which is the whole point. Partitions come from `ae.physics.impurity_location` and must be measured; this module never supplies them.

## LIMITATIONS

1. **\*\*(This is a geometric bound, not a calibrated liberation model.\*\*** A real liberation curve is measured by image analysis (QEMSCAN, MLA or SEM point counting on sized fractions) and fitted; the classical treatment is King 1979 (A model for the quantitative estimation of mineral liberation by grinding, International Journal of Mineral Processing 6:207-220, doi:10.1016/0301-7516(79)90037-1). No image-analysis calibration exists for any Applied Elements sample, and none was retrievable for quartz-hosted inclusions generally in this environment. Read E1 as the answer to "what does random positioning alone imply", and expect the real curve to differ.

2. **\*\*Fracture is not random with respect to inclusions.\*\*** Real breakage is preferential: cracks nucleate at inclusions and at grain boundaries because these are stress concentrators and weak planes, so true exposure at a given  $d_p$  is generally HIGHER than E1 predicts. E1 is therefore best read as a conservative (pessimistic) bound on exposure, hence an optimistic bound on the grinding energy needed. The size of the gap is exactly what image analysis would measure and this module does not know it.
3. **\*\*Thermal decrepitation bypasses the geometry entirely.\*\*** Fluid inclusions are conventionally opened by calcination and quenching rather than by grinding: Lin et al. 2020 report calcination at 600 to 700 degC being effective for fluid inclusion removal via the alpha to beta quartz transition near 573 degC, with other work supporting 900 degC as more effective. When a flowsheet includes calcination plus water quenching, the fluid inclusion branch of E5 is governed by decrepitation, not by  $d_p$ , and using E1 for it understates removal. `exposure` will not silently model that; a decrepitation model belongs in the thermal track.
4. **\*\*One characteristic size per inclusion class.\*\*** Inclusions are not cubes or spheres; secondary fluid inclusions in vein quartz develop as planar trails along microfractures (Xia M. et al. 2024, Minerals 14(7):727, doi:10.3390/min14070727), and a planar trail is exposed far more readily than an equiaxed body of the same volume. E1 has no aspect-ratio parameter, so it underestimates exposure for trail-hosted populations specifically.
5. **\*\*Exposure is not recovery.\*\*** An exposed inclusion is accessible to a reagent; whether it is actually removed depends on leach kinetics, diffusion into a partly-closed cavity, reagent selectivity and residence time. E5 returns the fraction that CAN be removed by a perfect downstream step, an upper bound on what a real one achieves.
6. **\*\*No particle size distribution.\*\***  $d_p$  here is a single size, in practice a P80 or a sieve-fraction midpoint. Within one sieve fraction the fines are more liberated than the coarse, and a real circuit produces a spread. Passing a P80 into E1 gives the exposure of a hypothetical monodisperse feed at that size.

## References

Every entry below was resolved against the Crossref REST API for the DOI shown, and the fields here (author list, year, title, journal, volume, issue, pages) are as Crossref returned them. Resolved 17 September 2026. Entries without a DOI say what was checked instead and what remains unverified.

King, R. P. (1979) A model for the quantitative estimation of mineral liberation by grinding, International Journal of Mineral Processing 6(3), 207-220, doi 10.1016/0301-7516(79)90037-1. Cited as the standard stereological liberation model. This module does NOT implement it: E1 here is the geometric exposure probability, which is a weaker and different construct. Closed access, abstract only from this sandbox.

Lin, M., Liu, Z., Wei, Y., Liu, B., Meng, Y., Qiu, H., Lei, S., Zhang, X. and Li, Y. (2020) A Critical Review on the Mineralogy and Processing for High-Grade Quartz, Mining, Metallurgy and Exploration 37(5), 1627-1639, doi 10.1007/s42461-020-00247-0.

Xia, M., Yang, X. and Hou, Z. (2024) Preparation of High-Purity Quartz Sand by Vein Quartz Purification and Characteristics: A Case Study of Pakistan Vein Quartz, Minerals 14(7), 727, doi 10.3390/min14070727. Feed-to-product endpoints only; NO per-stage intermediate is reported there or modelled here.

## 9.2 Interface

### Types

**InclusionWeighting.** Basis on which an inclusion size distribution is weighted.

Required, not defaulted. Number weighting emphasises the abundant fine inclusions and gives a pessimistic exposure at coarse sizes; volume (or equivalently mass) weighting emphasises the few large inclusions that actually carry the impurity mass and gives a more optimistic figure. The two can differ by a large factor

for a broad distribution, and which one is correct depends on the question: mass removal needs volume weighting, count of residual defect sites needs number weighting.

## Functions

### *enclosed\_fraction*

```
enclosed_fraction(particle_size: Qty, inclusion_size: Qty) -> float
```

Probability an inclusion is fully enclosed by the particle (Eq. E1).

**Parameters.** `particle_size`  $d_p$ , any length unit. `inclusion_size`  $d_{inc}$ , any length unit.

**Returns.** `float` Enclosed probability in  $[0, 1]$ .

### *exposure*

```
exposure(particle_size: Qty, inclusion_size: Qty) -> float
```

Exposure fraction  $E$  of an inclusion size class at a particle size (Eq. E1).

**Examples.** A 10  $\mu\text{m}$  inclusion in a 100  $\mu\text{m}$  particle:  $r = 0.1$ , so  $E = 1 - 0.9^3 = 1 - 0.729 = 0.271$ . About 27 percent of such inclusions intersect a particle surface.

```
>>> round(exposure(Q_(100.0, "um"), Q_(10.0, "um")), 3)
0.271
```

Grinding the same material to 30  $\mu\text{m}$ :  $r = 1/3$ ,  $E = 1 - (2/3)^3 = 1 - 0.296296 = 0.703704$ .

```
>>> round(exposure(Q_(30.0, "um"), Q_(10.0, "um")), 4)
0.7037
```

LIMIT AT AND BELOW THE INCLUSION SIZE. When  $d_p \leq d_{inc}$  this returns exactly 1.0, via `enclosed_fraction`'s early return. That is the correct reading of the model and not a clamp concealing a domain error: `exposure` is the fraction of inclusions intersecting a particle surface, and a particle no larger than an inclusion is a fragment OF that inclusion, so every such inclusion has been opened. It is asserted deliberately in `tests/test_liberation.py::test_exposure_bounds` (5  $\mu\text{m}$  particle, 10  $\mu\text{m}$  inclusion) and relied on by `test_leachable_fraction_never_exceeds_the_non_lattice_inventory`, which checks that the lattice term still bounds the leachable fraction at infinite fineness. Recorded here because this audit first misread it as a silent failure and added a guard rejecting the case; the guard broke those tests and the tests were right.

### *liberation\_size*

```
liberation_size(inclusion_size: Qty, target_exposure: float) -> Qty
```

Particle size required to reach a target exposure fraction (Eq. E2).

**Parameters.** `inclusion_size`  $d_{inc}$ , `target_exposure`  $E^*$  in  $(0, 1]$ . At exactly 1.0 the required size equals the inclusion size, which is the geometric asymptote of the model.

**Examples.** Half-exposure of a 20  $\mu\text{m}$  inclusion population:  $1 - 0.5^{1/3} = 1 - 0.7937005 = 0.2062995$ , and  $20 / 0.2062995 = 96.95 \mu\text{m}$ .

```
>>> round(liberation_size(Q(20.0, "um"), 0.5).to("um").magnitude, 2)
96.95
```

### *polydisperse\_exposure*

```
polydisperse_exposure(particle_size: Qty, inclusion_sizes: Sequence[Qty], weights: Sequence[float], weight
    \ ing: InclusionWeighting) -> float
```

Weighted mean exposure over an inclusion size distribution (Eq. E4).

**Parameters.** *particle\_size*  $d_p$ . *inclusion\_sizes* Characteristic size of each class. *weights* Class weights. Must be non-negative and sum to 1 within  $1e-9$ , so a distribution that does not close fails loudly rather than being renormalised behind the caller's back. *weighting* Declared basis of *weights*. Recorded and range-checked; it does not change the arithmetic, because renormalising a number basis into a volume basis requires the inclusion volumes, which the caller has and this function does not.

### *exposure\_curve*

```
exposure_curve(inclusion_size: Qty, particle_sizes: Sequence[Qty]) -> list[tuple[float, float]]
```

Exposure as a function of particle size, for plotting a liberation curve.

**Returns.** list of (*particle\_size\_um*, *exposure*) Sorted by ascending particle size. Monotonically decreasing in exposure, which is asserted before return.

### *leachable\_fraction*

```
leachable_fraction(partition: Mapping[str, float], particle_size: Qty, fluid_inclusion_size: Qty, mineral_
    \ inclusion_size: Qty) -> float
```

Fraction of an element accessible to a downstream removal step (Eq. E5).

**Parameters.** *partition* Mapping with keys "surface", "fluid", "mineral" and "lattice", each a fraction of the element's total inventory. Must sum to 1 within  $1e-9$ . Obtain it from `ae.physics.impurity_location.partition_fractions`, which raises when the split is unmeasured; do NOT hand-write it. *particle\_size*  $d_p$  at the point the removal step acts. *fluid\_inclusion\_size*, *mineral\_inclusion\_size* Characteristic sizes of the two inclusion populations.

**Returns.** float Upper bound on the removable fraction at this particle size. Surface films are taken as fully accessible; the lattice term is excluded by construction and can never be reached.

### *feedstock\_exposure*

```
feedstock_exposure(feedstock: Feedstock, particle_size: Qty, inclusion_type: InclusionType=InclusionType.F
    \ LUID) -> tuple[float, Value]
```

Exposure for a feedstock's own measured inclusion population.

Reads `feedstock.inclusions.median_inclusion_size`. Raises when it has not been measured, because substituting a typical inclusion size would fabricate the liberation size, and the liberation size sets the grinding energy and hence a large part of the operating cost.

**Returns.** (exposure, inclusion\_size\_value) The Value is returned so the caller can propagate its provenance.

**Raises.** ValueError If the inclusion size is MISSING.

*energy\_to\_exposure\_ratio*

```
energy_to_exposure_ratio(coarse_size: Qty, fine_size: Qty, inclusion_size: Qty) -> tuple[float, float, float]
```

Exposure gain versus Bond energy cost between two grind sizes.

**Returns.** (exposure\_gain, bond\_energy\_ratio, exposure\_per\_energy) *bond\_energy\_ratio* is the ratio of Bond specific energies to reach the two sizes from a common feed, which for a coarse feed is approximately  $\sqrt{d_{coarse}/d_{fine}}$  because Bond's law gives  $W \propto P_{80}^{-1/2}$ . It is work-index independent, so this comparison needs no ore property, which is why it is honest for an uncharacterized deposit.

**Notes.** This is the decision-relevant ratio for choosing a grind target: exposure has diminishing returns (Eq. E1 saturates) while energy has increasing cost ( $P_{80}^{-1/2}$  diverges), so an optimum exists and it does not sit at the finest achievable size.

### 9.3 Worked examples, reproduced by execution

The 2 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

**physics.liberation.exposure**

```
>>> round(exposure(Q_(100.0, "um"), Q_(10.0, "um")), 3)
0.271
>>> round(exposure(Q_(30.0, "um"), Q_(10.0, "um")), 4)
0.7037
```

**physics.liberation.liberation\_size**

```
>>> round(liberation_size(Q_(20.0, "um"), 0.5).to("um").magnitude, 2)
96.95
```

### 9.4 Validation status

10 hand-traceable worked example(s) and 2 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                           | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_benchmark_grind_implied_by_xia_removal         | literature | Benchmark against Xia et al. 2024 (doi:10.3390/min14070727). The paper reports 81.20 percent removal of total trace impurities (128.86 to 24.23 ug/g) with the residue identified as lattice-bound Al, Ti and Li. It does NOT report a liberation curve or a per-stage breakdown, so the geometric model can<br>Error stated: 81.20; 81.20; 81.20<br>10.3390/min14070727 |
| test_benchmark_qu_2025_removal_requires_finer_grind | literature | Benchmark: Qu et al. 2025 (doi:10.5194/ejm-37-953-2025), two ore bodies. HT quartz 322.96 to 18.72 ug/g (94.20 percent removal), PX quartz 4944.73 to 114.56 ug/g (97.68 percent removal), both by a flowsheet including calcination, crushing, sieving, magnetic separation, gravity separation, flotation<br>Error stated: 94.20; 97.68<br>10.5194/ejm-37-953-2025     |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_exposure_worked_example_ten_percent_ratio
test_exposure_worked_example_one_third_ratio
test_small_ratio_expansion_gives_factor_three
test_liberation_size_half_exposure
test_liberation_size_ninety_percent_exposure
test_liberation_size_round_trips_through_exposure
test_full_exposure_requires_grinding_to_the_inclusion_size
test_polydisperse_worked_example
test_leachable_fraction_worked_example
test_energy_to_exposure_worked_example
```

CHAPTER 10

physics/impurity\_location.py

Module `ae.physics.impurity_location`, 1055 lines. Chapter text is this module's own documentation.

10.1 Purpose, governing equations and limitations

Where each impurity element actually sits, which decides the purity ceiling.

This is the most decision-relevant model in the physical separation track. Two ores with identical bulk assays have completely different value if one carries its aluminium as feldspar inclusions and the other carries it substituted into the quartz lattice. The first can be beneficiated to HPQ; the second cannot be, by any flowsheet, at any cost.

Four locations, and what can reach each

| Location | Physical form                                                                                                    | Removed by                                                                        |
|----------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| surface  | adsorbed films, clay coatings, grinding media debris, fines adhering to grains                                   | washing, desliming, attrition scrubbing, leaching                                 |
| fluid    | aqueous or gaseous cavities, primary and secondary, salinity-bearing                                             | calcination plus quenching (decrepitation), then leaching                         |
| mineral  | discrete feldspar, mica, rutile, zircon, Fe-oxide grains                                                         | liberation by grinding, then flotation, magnetic sep, leach                       |
| lattice  | substitutional Al3+, Fe3+, B3+, Ti4+, Ge4+, P5+ for Si4+, plus interstitial charge compensators Li+, Na+, K+, H+ | NOTHING in the standard flowsheet. Hot chlorination touches part of it, see below |

The substitution mechanisms are set out in Lin et al. 2020 Section 2.2 (doi:10.1007/s42461-020-00247-0): substitutional ions Al3+, Fe3+, B3+, Ti4+, Ge4+, P5+ replacing Si4+, with interstitial Li+, K+, Na+, H+ and Fe2+ acting as charge compensators, and configurations including Al3+ plus P5+ for 2Si4+, silanol groups 4H+ for Si4+, and Al3+ plus an alkali in the interstice of the Si-O tetrahedron. The same source states that alkali charge compensators are only weakly bound and can be removed relatively readily, while the trivalent and tetravalent substitutional ions Al3+ and Ti4+ resist removal because their lattice energies are low relative to the Si-O-Si structure.

That asymmetry has a direct commercial consequence: Na and K in the lattice are a chlorination problem, while Al and Ti in the lattice are a geology problem. Hot chlorination requires a carbonaceous reductant for Ti, Al and B removal (Liu L., Liu H., Li J. et al. 2026, Mechanism Study on Deep Removal of Lattice Impurities from High-Purity Quartz by Chlorination Roasting, Minerals 16(8):836, doi:10.3390/min16080836).

Equations

`** (E1) Partition closure. **` Definitional, not empirical.

$$f_s + f_f + f_m + f_l = 1$$

- $f_s, f_f, f_m, f_l$ : fraction of an element's total inventory in surface, fluid inclusion, mineral inclusion and lattice locations. Each dimensionless in  $[0, 1]$ . Enforced to 1e-9 in `ElementPartition`; an open inventory is a rejected input, not a warning.

**\*\* (E2) Location-resolved concentration. \*\***

$$c_j^{(el)} = c_{\text{total}}^{(el)} f_j^{(el)}$$

- $c_{\text{total}}^{(el)}$ : bulk concentration of the element, ppm by mass (ug/g), range 0.01 to 10000 ppm for the elements this platform tracks.
- $c_j^{(el)}$ : concentration residing in location  $j$ , same units.
- Mass balance over  $j$  returns  $c_{\text{total}}$  exactly, which is asserted in code.

**\*\* (E3) Achievable floor concentration. \*\*** The number the whole platform turns on.

$$c_{\text{floor}}^{(el)} = c_{\text{total}}^{(el)} \left[ f_l^{(el)} + \sum_{j \neq l} f_j^{(el)} (1 - \eta_j) \right]$$

- $\eta_j$ : removal efficiency of the flowsheet for location  $j$ , dimensionless in  $[0, 1]$ . A perfect flowsheet has  $\eta_j = 1$  for surface, fluid and mineral, giving  $c_{\text{floor}} = c_{\text{total}} f_l$ .
- With a perfect flowsheet the floor is the lattice inventory, full stop. No choice of reagent, residence time or stage count moves it, which is why `ae.core.feedstock.ImpurityProfile.lattice_ppm` raising on unmeasured data is correct behaviour and must not be worked around.

**\*\* (E4) Total trace sum floor and the HPQ gate. \*\***

$$\Sigma_{\text{floor}} = \sum_{el} c_{\text{floor}}^{(el)}$$

Compared element-by-element against the single-grain HPQ reference limits and against the trace sum limit of 50 ppm. A deposit passes the gate only if every element clears its own limit; the sum clearing 50 ppm is necessary, not sufficient.

**\*\* (E5) Implied overall removal rate. \*\*** For benchmarking against published campaigns.

$$R = 1 - \frac{\Sigma_{\text{product}}}{\Sigma_{\text{feed}}}$$

- $R$ : dimensionless removal fraction in  $[0, 1]$ .
- Xia et al. 2024 (doi:10.3390/min14070727) report  $\Sigma_{\text{feed}} = 128.86$  ug/g and  $\Sigma_{\text{product}} = 24.23$  ug/g for Pakistan vein quartz, an 81.20 percent removal, with the residue identified as lattice-bound Al, Ti and Li. That paper reports only those two endpoints, so no per-stage intermediate is modelled or attributed anywhere in this platform.

## Reference limits

HPQ single-grain reference limits used for the gate, from Muller et al. 2012 in Gotze J. and Mockel R. eds., Quartz: Deposits, Mineralogy and Analytics (doi:10.1007/978-3-642-22161-3): Al below 30 ppm, Ti below 10, Li below 5, Na below 8, K below 8, Ca below 5, Fe below 3, P below 2, B below 1, trace sum below 50 ppm.

## LIMITATIONS

1. **\*\*No partition data exists for the Vikarabad deposit.\*\*** It is uncharacterized in the citable record: no LA-ICP-MS, no cathodoluminescence, no fluid inclusion microthermometry, no beneficiation testwork. Every result this module produces for it is a SCENARIO conditional on a future characterization campaign, never a finding, and `partition_fractions` raises rather than defaulting.
2. **\*\*Published per-element partition data is thin and this module does not invent it.\*\*** What the literature reliably gives is (a) which elements substitute into the lattice and by what mechanism, qualitatively, and (b) before-and-after totals for specific deposits with the residue attributed to named lattice elements. What it does not give, for any deposit retrievable here, is a numeric four-way split per element. `PARTITION_PRIORS` therefore contains priors only for the elements where the mechanism is established enough to bound one branch, every one of them tagged ASSUMED with its reasoning, and `NO_PARTITION_DATA` names the elements with nothing at all. Do not read the priors as measurements.
3. **\*\*A prior is not transferable between deposits.\*\*** Lattice Al in a high-temperature pegmatite quartz and in a low-temperature hydrothermal vein quartz differ by orders of magnitude because Al substitution is governed by crystallisation temperature and melt or fluid chemistry. Al lattice fraction is a property of the deposit, measured grain by grain, not a property of the element.
4. **\*\*The four locations are an idealisation.\*\*** Grain-boundary films, healed microfracture trails decorated with sub-micron fluid inclusions, and nanoscale exsolved phases sit between the categories. An operational definition tied to the measurement method (bulk ICP-MS after a staged leach versus LA-ICP-MS on inclusion-free domains) is what makes the split reproducible, and this module records that method on the partition object.
5. **\*\*Removal efficiencies are inputs, not predictions.\*\*** `floor_profile` takes  $\eta_j$  from the caller. It does not know what a given flowsheet achieves, and the default is the perfect-flowsheet case ( $\eta = 1$ ), which is a bound and not an expectation.
6. **\*\*An SiO<sub>2</sub> weight percentage is not a comparable purity metric across papers.\*\*** `lattice_ceiling_sio2_percent` converts a cation trace sum by difference, which is one of at least two conventions in use. Xia et al. 2024's 99.998 wt% is exactly what their 24.23 ug/g implies by difference, but Qu et al. 2025's figures are not: their 18.72 ug/g (HT) and 114.56 ug/g (PX) imply 99.998128 and 99.988544 wt% by difference, ABOVE their published upper bounds of 99.970 and 99.985 by 0.0281 and 0.0035 percent relative, one-sided in both cases. Their SiO<sub>2</sub> must therefore come from a whole-rock determination carrying oxide oxygen, water and phases outside the trace list. Compare deposits on the trace sum in ug/g, never on a quoted SiO<sub>2</sub> percentage, and treat a supplier's "99.99x percent SiO<sub>2</sub>" claim as uninterpretable until the convention is stated. Measured in `test_benchmark_qu_2025_two_ore_bodies`.
7. **\*\*Hot chlorination breaks the four-location model's central claim, partly.\*\*** Lin et al. 2020 describe chlorination activating structural Al, Fe and Ti to the grain surface by reducing activation energy; the conceptual flowsheet there puts hot chlorination against lattice impurities and above 99.999 percent SiO<sub>2</sub>. Liu et al. 2026 establish that a carbonaceous reductant is required for Ti, Al and B. So "lattice is unremovable" is exactly true for washing, flotation, magnetic separation and acid leaching, and NOT unconditionally true once chlorination roasting is in the flowsheet. This module models the lattice as unremovable by the PHYSICAL flowsheet, which is its scope, and `floor_profile` will accept a lattice removal efficiency only if the caller passes one explicitly and names the mechanism.

## References

Every entry below was resolved against the Crossref REST API for the DOI shown, and the fields here (author list, year, title, journal, volume, issue, pages) are as Crossref returned them. Resolved 17 September 2026. Entries without a DOI say what was checked instead and what remains unverified.

Muller, A., Wanyik, J. E. and Ihlen, P. M. (2012) Petrological and Chemical Characterisation of High-Purity Quartz Deposits with Examples from Norway, in Gotze, J. and Mockel, R. (eds.) Quartz: Deposits,

Mineralogy and Analytics, Springer Geology, pp. 71-118, doi 10.1007/978-3-642-22161-3\_4 (volume DOI 10.1007/978-3-642-22161-3). Source of the HPQ single-grain reference limits used by the gate. Closed access: the chapter full text was not retrievable from this sandbox, so the limits are carried as previously extracted values and the chapter's author list, title, pages and volume are what was verified here.

Lin, M., Liu, Z., Wei, Y., Liu, B., Meng, Y., Qiu, H., Lei, S., Zhang, X. and Li, Y. (2020) A Critical Review on the Mineralogy and Processing for High-Grade Quartz, Mining, Metallurgy and Exploration 37(5), 1627-1639, doi 10.1007/s42461-020-00247-0. Source of the chlorination-activation description and of the conceptual flowsheet placing hot chlorination against lattice impurities.

Xia, M., Yang, X. and Hou, Z. (2024) Preparation of High-Purity Quartz Sand by Vein Quartz Purification and Characteristics: A Case Study of Pakistan Vein Quartz, Minerals 14(7), 727, doi 10.3390/min14070727. Source of the 128.86 ug/g feed and 24.23 ug/g product trace sums and of the residual being lattice-bound Al, Ti and Li.

Qu, S., Yang, X., Xia, M., Hou, Z. and Wang, Y. (2025) Analyzing trace element distribution and purification of vein quartz: a case study of Qianqi Furong mine in Inner Mongolia, European Journal of Mineralogy 37(6), 953-970, doi 10.5194/ejm-37-953-2025. Source of the 18.72 ug/g and 114.56 ug/g figures and of the published SiO<sub>2</sub> upper bounds against which they are inconsistent by difference.

Liu, L., Liu, H., Li, J., Peng, T., Wang, W., Wang, F. and Liu, G. (2026) Mechanism Study on Deep Removal of Lattice Impurities from High-Purity Quartz by Chlorination Roasting, Minerals 16(8), 836, doi 10.3390/min16080836. Source of the requirement for a carbonaceous reductant for Ti, Al and B.

## 10.2 Interface

### Types

**Location.** The four places an impurity atom can be, ordered by ease of removal.

```
removable_by_physical_flowsheet(self) -> bool
```

True for everything except the lattice.

The lattice returns False because washing, desliming, attrition, flotation, magnetic separation and acid leaching cannot reach a substituted ion. Chlorination roasting can reach part of it, which is a chemical route outside this module's scope; see LIMITATIONS item 7.

**ElementPartition.** Four-way location split of one element's inventory, with provenance.

The four fractions must close to 1 within 1e-9. method records how the split was established, because the operational definition of "lattice" depends on it: LA-ICP-MS on inclusion-free domains measures something different from bulk ICP-MS after a staged leach, and mixing the two in one table produces a split that cannot be reproduced.

```
fractions(self) -> dict[str, float]
```

The split as a plain mapping keyed by location value string.

```
weakest_tag(self) -> Tag
```

The weakest provenance tag among the four fractions.

A partition is only as good as its worst-supported branch, so a caller writing a report should quote this rather than the strongest tag.

**PartitionModel.** A set of element partitions attached to one feedstock.

Held separately from `ae.core.feedstock.Feedstock` because a partition is an interpretation of measurements, several interpretations of the same assay are possible, and each should be comparable against the others rather than overwriting the ore record.

```
get(self, element: str) -> ElementPartition
```

## Functions

### *partition\_fractions*

```
partition_fractions(feedstock: Feedstock, element: str, model: PartitionModel | None=None, allow_prior: bool | None=False) -> tuple[dict[str, float], Tag, str]
```

Four-way partition for one element, measured first, prior only on request.

**Parameters.** `feedstock` The ore. Its `impurities.lattice_fraction` is consulted first: when that element's lattice fraction has been measured, the lattice branch is taken from it and the caller must supply the remaining split via `model`, because a measured lattice fraction alone does not resolve surface from fluid from mineral. `element` Element symbol, e.g. "Al". `model` A `PartitionModel` carrying measured or interpreted splits for this sample. Takes precedence over priors. `allow_prior` When no measured split exists, `False` (default) raises and `True` returns the ASSUMED ore-type prior. The default raises because the partition sets the purity ceiling, which is the single most decision-relevant number in the platform.

**Returns.** (`fractions`, `tag`, `provenance_note`) `fractions` keyed "surface", "fluid", "mineral", "lattice". `tag` is the weakest tag behind the split.

**Raises.** `ValueError` When the split is unmeasured and `allow_prior` is `False`. `KeyError` When `allow_prior` is `True` but no prior exists for this ore type and element pair, which is the case for every element in `NO_PARTITION_DATA` and for every ore type other than vein quartz.

### *location\_concentrations*

```
location_concentrations(feedstock: Feedstock, element: str, model: PartitionModel | None=None, allow_prior: bool | None=False) -> dict[str, float]
```

Concentration of one element in each location, ppm by mass (Eq. E2).

Mass balance closes exactly: the four returned values sum to the bulk concentration, asserted before return.

### *floor\_concentration*

```
floor_concentration(feedstock: Feedstock, element: str, model: PartitionModel | None=None, allow_prior: bool | None=False, efficiencies: Mapping[str, float] | None=None, lattice_removal: float=0.0, lattice_removal_mechanism: str | None=None) -> float
```

Achievable floor concentration for one element, ppm by mass (Eq. E3).

**Parameters.** `feedstock`, `element`, `model`, `allow_prior` As `partition_fractions`. `efficiencies` Removal efficiency per location for "surface", "fluid" and "mineral". Defaults to 1.0 each, the perfect-flowsheet bound. `lattice_removal` Fraction of the lattice inventory removed. Must be 0 unless a mechanism is named, because no step in a physical flowsheet reaches the lattice. `lattice_removal_mechanism` Required when `lattice_removal` is non-zero, e.g. "chlorination roasting with carbonaceous reductant, Liu et al. 2026 doi:10.3390/min16080836". Forces the caller to state what physics they are invoking.

**Returns.** float Floor concentration in ppm by mass, never below the un-removed lattice residue and never above the bulk concentration (both asserted).

*removable\_ppm*

```
removable_ppm(feedstock: Feedstock, element: str, model: PartitionModel | None=None, allow_prior: bool=False, \ se) -> float
```

Ppm of an element that a perfect physical flowsheet could remove.

Equals bulk minus the lattice inventory. Reported separately from the floor because it is the quantity a flowsheet designer is buying with each stage.

*floor\_profile*

```
floor_profile(feedstock: Feedstock, elements: tuple[str, ...] | None=None, model: PartitionModel | None=None, allow_prior: bool=False, efficiencies: Mapping[str, float] | None=None) -> dict[str, float]
```

Floor concentration for several elements, ppm by mass (Eq. E3 per element).

Defaults to the elements that have an HPQ reference limit and are measured on this feedstock, so the result can be fed straight to `hpq_gate`.

*hpq\_gate*

```
hpq_gate(floors_ppm: Mapping[str, float], limits_ppm: Mapping[str, float] | None=None, sum_limit_ppm: float, \ t=HPQ_TRACE_SUM_LIMIT_PPM) -> tuple[bool, dict[str, tuple[float, float, bool]], str]
```

Test a floor profile against the HPQ single-grain reference limits (Eq. E4).

**Returns.** (passes, per\_element, verdict) `per_element` maps element to (floor\_ppm, limit\_ppm, passes). `passes` is True only if every element with a limit clears it AND the trace sum clears `sum_limit_ppm`. Elements without a limit are reported but do not gate.

**Notes.** A failing element cannot be compensated by a comfortable margin elsewhere. Al at 45 ppm against a 30 ppm limit disqualifies the material for crucible inner-layer sand regardless of how low Fe is, because the specification is per element. The trace sum test is therefore necessary and not sufficient, and this function reports both.

*implied\_removal\_rate*

```
implied_removal_rate(feed_sum_ppm: float, product_sum_ppm: float) -> float
```

Overall trace removal fraction (Eq. E5).

**Examples.** Xia et al. 2024: 128.86 ug/g feed, 24.23 ug/g product.  $1 - 24.23/128.86 = 1 - 0.188034 = 0.811966$ , i.e. 81.20 percent, matching the 81.20 percent the paper reports.

```
>>> round(implied_removal_rate(128.86, 24.23) * 100, 2)
81.2
```

*lattice\_ceiling\_sio2\_percent*

```
lattice_ceiling_sio2_percent(total_trace_floor_ppm: float) -> float
```

SiO<sub>2</sub> purity implied by a residual trace sum, wt percent.

$$\text{SiO}_2 [\%] = 100 - \frac{\Sigma_{\text{floor}}}{10^4}$$

**Examples.** 24.23 ppm residual trace is 24.23e-4 wt percent, so 100 - 0.002423 = 99.997577 wt percent SiO<sub>2</sub>, consistent with the 99.998 wt percent reported by Xia et al. 2024.

```
>>> round(lattice_ceiling_sio2_percent(24.23), 6)
99.997577
```

**Notes.** This converts a cation trace sum to an SiO<sub>2</sub> figure by difference, which is how the quartz literature quotes purity. It is NOT an independent measurement of SiO<sub>2</sub> and it ignores OH and any oxygen stoichiometry correction, so the last digit is convention rather than data.

### 10.3 Worked examples, reproduced by execution

The 2 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

`physics.impurity_location.implied_removal_rate`

```
>>> round(implied_removal_rate(128.86, 24.23) * 100, 2)
81.2
```

`physics.impurity_location.lattice_ceiling_sio2_percent`

```
>>> round(lattice_ceiling_sio2_percent(24.23), 6)
99.997577
```

### 10.4 Validation status

9 hand-traceable worked example(s) and 3 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                        | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_benchmark_xia_2024_endpoints                | literature | Benchmark: Xia et al. 2024 (doi:10.3390/min14070727) endpoint arithmetic. Published: 128.86 ug/g feed, 24.23 ug/g product, 81.20 percent removal, SiO2 99.998 wt percent, residue identified as lattice Al, Ti and Li. Eq. E5 and the SiO2 conversion are checked against the two independent published figures.<br>Error stated: 81.20; 2<br>10.3390/min14070727                                                                                                                                                                                                                                     |
| test_benchmark_qu_2025_two_ore_bodies            | literature | Benchmark: Qu et al. 2025 (doi:10.5194/ejm-37-953-2025) HT and PX quartz. Published totals: HT 322.96 to 18.72 ug/g, PX 4944.73 to 114.56 ug/g, with final SiO2 99.963 to 99.970 wt percent (HT) and 99.974 to 99.985 wt percent (PX). Eq. E5 gives the removal rates and the SiO2 conversion is checked against the published figures.<br>Error stated: 97.68<br>10.5194/ejm-37-953-2025                                                                                                                                                                                                             |
| test_benchmark_prior_floors_against_xia_residual | literature | Benchmark: do the ASSUMED vein quartz priors reproduce Xia's residual split? Xia et al. 2024 report that after full purification only Al, Ti and Li remain, totalling 24.23 ug/g. The priors in this module assign lattice fractions of 0.45 (Al), 0.60 (Ti) and 0.75 (Li). Applying them to a hypothetical feed of 128.86 ug/g, the product is 24.23 ug/g, leaving a residual of 104.63 ug/g.<br>Error stated: 69.29; 81.20; 14.67<br>10.1007/978-3-642-22161-3; 10.1007/978-3-642-22161-3_4;<br>10.1007/s42461-020-00247-0; 10.1007/s42461-020-00247-0;<br>10.3390/min14070727; 10.3390/min14070727 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_location_concentrations_worked_example
test_perfect_flowsheet_floor_is_the_lattice_inventory
test_imperfect_flowsheet_floor_worked_example
test_removable_ppm_worked_example
test_implied_removal_rate_worked_example
test_sio2_from_trace_sum_worked_example
test_hpq_gate_worked_example_pass
test_hpq_gate_worked_example_one_element_fails
test_trace_sum_alone_can_fail_while_every_element_passes
```

## CHAPTER 11

# physics/separation.py

Module `ae.physics.separation`, 1109 lines. Chapter text is this module's own documentation.

## 11.1 Purpose, governing equations and limitations

Physical separation: WHIMS, flotation, partition curves and mass balance.

Three separate things are modelled here and they should not be confused:

1. **Rate models** (flotation, WHIMS) predict recovery as a function of residence time given a rate constant that must be MEASURED in a batch test. They do not predict the rate constant.
2. **Partition curves** (Tromp) describe how sharply a unit splits a feed by some property (size, density, magnetic susceptibility). They are descriptive fits to a measured curve, parameterised by a cut point and a sharpness.
3. **Mass balance** (two-product formula, grade-recovery) is arithmetic. It has no fitted parameters, it closes exactly, and any discrepancy is a measurement error or a bookkeeping error, never a model error. Tests here require closure to machine precision.

In the HPQ flowsheet these sit between grinding and leaching: magnetic separation against Fe-bearing impurities and flotation against silicate minerals, both operating after liberation by crushing and grinding, taking the product from roughly 90 to 99 percent up to 99 to 99.99 percent SiO<sub>2</sub> before the chemical stages (Lin M. et al. 2020, Table 3, doi:10.1007/s42461-020-00247-0). Both act only on liberated, exposed impurities, so their maximum recovery is bounded by `ae.physics.liberation` and their floor by `ae.physics.impurity_location`.

### Equations

**(E1)** First-order flotation kinetics, classical form.

$$R(t) = R_{\infty} (1 - e^{-kt})$$

- $R(t)$ : cumulative recovery of the floated species at time  $t$ , dimensionless in  $[0, 1]$ .
- $R_{\infty}$ : ultimate (maximum) recovery, dimensionless in  $[0, 1]$ . Strictly less than 1 in practice; the deficit is unliberated and entrained material. This is the parameter that couples to liberation.
- $k$ : first-order rate constant, 1/s (often quoted in 1/min). Typical batch values for silicate flotation are of order 0.01 to 0.1 1/s, but the value is system-specific and must be fitted.
- $t$ : flotation time, s, range 0 to about 1800 s in a batch test.

Derivation: the first-order postulate is that the rate of removal of floatable particles is proportional to the number remaining,  $dN/dt = -kN$ , whose solution  $N(t) = N_0 e^{-kt}$  gives recovery  $1 - e^{-kt}$  of the floatable sub-population  $R_{\infty} N_{\text{total}}$ . Polat and Chander 2000 (First-order flotation kinetics models and methods for estimation of the true distribution of flotation rate constants, International Journal of Mineral Processing 58:145-166, doi:10.1016/S0301-7516(99)00069-1) treat this two-parameter form as the reference case and

address estimating the DISTRIBUTION of  $k$  when a single constant does not fit, which is the usual situation.

\*\*(E2) Rate-constant distribution, rectangular case.\*\* Integrating E1 over a uniform distribution of  $k$  on  $[0, k_{max}]$ :

$$R(t) = R_{\infty} \left[ 1 - \frac{1 - e^{-k_{max}t}}{k_{max}t} \right]$$

- $k_{max}$ : upper bound of the rectangular distribution, 1/s.
- Included because a single- $k$  fit systematically overstates early recovery and understates late recovery for a real polydisperse, heterogeneously-liberated feed. Both limits are checked in tests:  $R \rightarrow 0$  as  $t \rightarrow 0$  and  $R \rightarrow R_{\infty}$  as  $t \rightarrow \infty$ .

\*\*(E3) WHIMS recovery as a first-order capture process.\*\*

$$R_{mag}(t) = R_{\infty, mag} (1 - e^{-k_{mag}t}), \quad k_{mag} = k_0 \left( \frac{B}{B_{ref}} \right)^{n_B}$$

- $B$ : applied magnetic flux density, T. WHIMS operates roughly 0.5 to 2 T, superconducting HGMS to about 5 T.
- $B_{ref}$ : reference flux density at which  $k_0$  was measured, T.
- $n_B$ : empirical exponent, dimensionless.  $n_B = 2$  corresponds to magnetic force scaling as  $B \nabla B$  for a linear paramagnet in a fixed-geometry matrix, which is the usual first-principles starting point. **\*\*This exponent is ASSUMED in this module\*\*** (see the value's basis string) and must be fitted from a field sweep; treating it as known is the main risk in E3.
- $t$ : residence time in the matrix, s.

The functional form of E3 is first-order capture by analogy with E1, not a result taken from a source. See LIMITATIONS item 2.

\*\*(E4) Tromp (partition) curve, logistic parameterisation.\*\*

$$P(x) = \frac{1}{1 + \exp\left(-\frac{x - x_{50}}{s}\right)}, \quad s = \frac{E_p}{\ln 3}$$

- $P(x)$ : partition number, the fraction of feed material of property  $x$  reporting to the coarse/sink/magnetic stream. Dimensionless  $[0, 1]$ .
- $x$ : the separating property (size in  $\mu\text{m}$ , density in  $\text{g/cm}^3$ , or susceptibility). Same unit as  $x_{50}$  and  $E_p$ .
- $x_{50}$ : cut point, where  $P = 0.5$  by construction.
- $E_p$ : ecart probable moyen (probable error),  $E_p = (x_{75} - x_{25})/2$ . Smaller is sharper;  $E_p = 0$  is a perfect separation.
- $s$ : logistic scale. The relation  $s = E_p/\ln 3$  is exact for the logistic form:  $P = 0.75$  requires  $(x - x_{50})/s = \ln 3$ , so  $x_{75} - x_{50} = s \ln 3$ , and by symmetry  $E_p = (x_{75} - x_{25})/2 = s \ln 3$ . Hence  $s = E_p/\ln 3 = E_p/1.0986123$ .

\*\*(E5) Imperfection.\*\*

$$I = \frac{E_p}{x_{50} - 1} \quad (\text{density separation, } x \text{ in } \text{g/cm}^3) \quad \text{or} \quad I = \frac{E_p}{x_{50}} \quad (\text{general})$$

- The density form subtracts the density of water (1 g/cm3) because the separating force scales with the density difference from the medium, not with absolute density. Using the general form on a density separation is a known error; `imperfection` requires the basis to be declared.

**\*(E6) Two-product formula.\*\*** Mass balance, exact.

$$Y = \frac{f - t}{c - t}, \quad R = \frac{c(f - t)}{f(c - t)} = \frac{Yc}{f}$$

- $f, c, t$ : assay of the species of interest in feed, concentrate and tailing, any consistent unit (percent, ppm, mass fraction).
- $Y$ : mass yield to concentrate, dimensionless in  $[0, 1]$ .
- $R$ : recovery of the species to concentrate, dimensionless in  $[0, 1]$ .
- Derivation: feed mass balance  $F = C + T$  and species balance  $Ff = Cc + Tt$ . Substituting  $T = F - C$  and dividing by  $F$  gives  $f = Yc + (1 - Y)t$ , hence  $Y = (f - t)/(c - t)$ . Recovery is  $R = Yc/f$ . Standard result, given in Wills B. A. and Napier-Munn T. 2005, Metallurgical accounting, control and simulation, in Wills' Mineral Processing Technology, doi:10.1016/b978-075064450-1/50005-9.
- Requires  $c > t$  and  $t \leq f \leq c$  for a physical result; violations are rejected rather than returned as a yield outside  $[0, 1]$ .

**\*(E7) Quartz-product convention: the species of interest is the REJECT.\*\*** In quartz beneficiation the valuable stream is the one that does NOT float and is NOT magnetic, so the "concentrate" of the impurity is the reject stream. The impurity recovery to reject is what the plant is buying, and the product's impurity grade follows from E6 applied to the impurity:

$$c_{\text{product}} = \frac{f - Y_{\text{rej}}c_{\text{rej}}}{1 - Y_{\text{rej}}}$$

This is E6 rearranged, not a new equation, and it is the form a quartz flowsheet actually needs. Getting the sense backwards (treating quartz as the floated concentrate) inverts every grade-recovery conclusion.

**\*(E8) Grade-recovery tradeoff.\*\*** Combining E1 and E6: as flotation time rises, impurity recovery to the froth rises toward  $R_\infty$  while the froth becomes more dilute in impurity and carries more quartz, so product yield falls. The locus of  $(R, c_{\text{product}})$  over  $t$  is the grade-recovery curve, and its shape, not any single point, is the operating decision.

## LIMITATIONS

1. **\*\*No rate constant here is measured on quartz.\*\*** `flotation_recovery` and `whims_recovery` take  $k$  and  $R_\infty$  as arguments and this module supplies no default for either. Fit them from a batch kinetics test on the actual feed at the actual grind; a rate constant is not transferable between ores, reagent suites, cell geometries or grinds. Fluorine-free flotation systems for vein quartz exist and are published (Du S. et al. 2024, Purification of Vein Quartz Using a New Fluorine-Free Flotation, Minerals 14(12):1191, doi:10.3390/min14121191), which is cited to establish that the reagent choice is an open design decision, not to supply a rate constant.
2. **\*\*E3's field dependence is an assumption, not a result.\*\*** The first-order form is an analogy to flotation and the exponent  $n_B = 2$  is engineering judgement from the  $B\nabla B$  force scaling of a linear paramagnet. Real HGMS capture depends on matrix geometry, wire diameter, slurry velocity, particle size and susceptibility, and saturates as the matrix loads. Do not use E3 outside a field range where it has been fitted, and do not read  $n_B$  as measured.

3. **\*\*Single rate constant versus a distribution.\*\*** E1 with one  $k$  fits batch data poorly in general; Polat and Chander 2000 exist precisely because the true  $k$  is distributed. E2 gives one alternative (rectangular); gamma and other distributions are common. Choosing E1 for convenience biases scale-up, usually optimistically for short residence times.
4. **\*\*Partition curves are descriptive.\*\*** E4 is a convenient two-parameter logistic. Real curves are asymmetric, can show a short-circuit fraction (a non-zero asymptote for fine material bypassing to the coarse stream) and can be non-monotonic near the cut. Fitting a logistic to such data hides the bypass. `partition_number` optionally takes a bypass term for that reason, and reports it rather than folding it into  $E_p$ .
5. **\*\*Mass balance is exact but the assays are not.\*\*** E6 amplifies assay error badly when  $c-t$  is small, which is exactly the regime of a high-purity product where feed, concentrate and tailing assays are all a few ppm. A two-product formula on 30, 5 and 45 ppm numbers each with 10 percent analytical uncertainty has a yield uncertainty that swamps the answer. `two_product` will compute it; `two_product_condition_number` tells you when not to trust it.
6. **\*\*Nothing here models entrainment or froth stability.\*\*** Fine quartz reports to the froth by water recovery regardless of its surface chemistry, which in a fine HPQ grind is a material yield loss. That requires a water-recovery model this module does not have.

## References

Every entry below was resolved against the Crossref REST API for the DOI shown, and the fields here (author list, year, title, journal, volume, issue, pages) are as Crossref returned them. Resolved 17 September 2026. Entries without a DOI say what was checked instead and what remains unverified.

Polat, M. and Chander, S. (2000) First-order flotation kinetics models and methods for estimation of the true distribution of flotation rate constants, *International Journal of Mineral Processing* 58(1-4), 145-166, doi 10.1016/S0301-7516(99)00069-1. Source of Eq. E1 and Eq. E2, and of the argument that a single rate constant fits batch data poorly because the true rate constant is distributed. Closed access; abstract and the equations as restated in the derivations above.

Wills, B. A. and Finch, J. A. (2016) Classification, in *Wills' Mineral Processing Technology*, 8th edition, Elsevier, pp. 199-221, doi 10.1016/b978-0-08-097053-0.00009-1. Source of the partition-curve vocabulary: cut point, Ecart probable, imperfection, bypass.

Wills, B. A. and Napier-Munn, T. (2005) Metallurgical accounting, control and simulation, in *Wills' Mineral Processing Technology*, Elsevier, pp. 39-89, doi 10.1016/b978-075064450-1/50005-9. Source of the two-product mass-balance formula, Eq. E6.

Lin, M., Liu, Z., Wei, Y., Liu, B., Meng, Y., Qiu, H., Lei, S., Zhang, X. and Li, Y. (2020) A Critical Review on the Mineralogy and Processing for High-Grade Quartz, *Mining, Metallurgy and Exploration* 37(5), 1627-1639, doi 10.1007/s42461-020-00247-0.

Du, S., Pan, B., Xia, L., Zhu, G., Wu, L., Yu, C., Li, F. and Diao, Z. (2024) Purification of Vein Quartz Using a New Fluorine-Free Flotation: A Case from Southern Anhui Province, China, *Minerals* 14(12), 1191, doi 10.3390/min14121191.

## 11.2 Interface

### Types

**SeparatorKind.** Unit operations covered by the rate models here.

**PropertyBasis.** Separating property a partition curve is written against.

Required by `imperfection` because the density form of Eq. E5 subtracts the medium density and the general form does not.

## Functions

### *flotation\_recovery*

```
flotation_recovery(time: Qty, rate_constant: Qty, ultimate_recovery: float) -> float
```

First-order flotation recovery (Eq. E1).

**Parameters.** `time` Flotation time, any time unit. `rate_constant`  $k$ , reciprocal time (e.g. `Q_(0.05, "1/s")` or `Q_(3.0, "1/min")`). `ultimate_recovery`  $R_\infty$  in  $[0, 1]$ .

**Examples.**  $k = 0.05$  1/s,  $R_\infty = 0.90$ ,  $t = 60$  s:  $\exp(-0.05 \times 60) = \exp(-3) = 0.0497871$ ,  $R = 0.90(1 - 0.0497871) = 0.90 \times 0.9502129 = 0.8551916$ .

```
>>> round(flotation_recovery(Q_(60.0, "s"), Q_(0.05, "1/s"), 0.90), 7)
0.8551916
```

### *flotation\_rate\_constant\_from\_recovery*

```
flotation_rate_constant_from_recovery(time: Qty, recovery: float, ultimate_recovery: float) -> Qty
```

Fit  $k$  from a single batch point by inverting Eq. E1.

$$k = -\frac{1}{t} \ln \left( 1 - \frac{R}{R_\infty} \right)$$

A one-point fit, provided for completeness. A real kinetics test takes four to six timed concentrates and fits both parameters; a single point cannot distinguish a fast rate with a low ultimate from a slow rate with a high one.

### *rectangular\_distribution\_recovery*

```
rectangular_distribution_recovery(time: Qty, k_max: Qty, ultimate_recovery: float) -> float
```

Recovery for a rectangular distribution of rate constants (Eq. E2).

**Examples.**  $k_{\max} = 0.10$  1/s,  $R_\infty = 0.90$ ,  $t = 60$  s:  $k_{\max} t = 6$ ,  $\exp(-6) = 0.00247875$ ,  $(1 - 0.00247875)/6 = 0.16625354$ ,  $R = 0.90(1 - 0.16625354) = 0.90 \times 0.83374646 = 0.75037181$ .

```
>>> round(rectangular_distribution_recovery(Q_(60.0, "s"), Q_(0.10, "1/s"), 0.90), 8)
0.75037181
```

Small-argument behaviour. As  $x = k_{\max} t \rightarrow 0$  the bracket goes to zero like  $x/2 - x^2/6$ , so recovery goes to zero from ABOVE. Evaluating  $(1 - e^{-x})/x$  directly destroys that. Measured at  $k_{\max} = 1$  1/s and  $R_\infty = 0.90$ , committed form against the analytic limit:

| x     | committed           | analytic           | relative error |
|-------|---------------------|--------------------|----------------|
| 1e-12 | +1.990954810935e-05 | 4.499999999998e-13 | +4.424344e+07  |
| 1e-10 | -7.446633389918e-08 | 4.499999999850e-11 | -1.655807e+03  |
| 1e-09 | +2.545373841700e-08 | 4.499999998500e-10 | +5.556386e+01  |
| 1e-08 | +5.469723873830e-09 | 4.499999985000e-09 | +2.154942e-01  |
| 1e-07 | +4.543775276034e-08 | 4.499999850000e-08 | +9.727873e-03  |
| 1e-06 | +4.500141251640e-07 | 4.499998500000e-07 | +3.172260e-05  |

At  $x = 10^{-10}$  the result is **NEGATIVE** and raised on the fraction check, so the failure mode is not merely imprecise. `-expm1(-x)` computes the same numerator without cancellation and reproduces the analytic limit to 1e-9 relative at every argument in the table. The old code special-cased only `x == 0.0` exactly, which is the one argument where catastrophic cancellation does not occur.

An earlier revision of this docstring attributed the 4.499999998500e-10 analytic value and the +5.556386e+01 error to  $x = 10^{-8}$ . Both belong to  $x = 10^{-9}$ ; every row above is now asserted against a recomputation in `tests/test_physics_audit.py::test_rectangular_recovery_survives_a_short_residence_time` rather than quoted.

### *whims\_rate\_constant*

```
whims_rate_constant(k_ref: Qty, field: Qty, field_ref: Qty, exponent: float | None=None) -> Qty
```

Scale a magnetic capture rate constant with applied field (Eq. E3).

**Parameters.** `k_ref` Rate constant measured at `field_ref`, reciprocal time. `field`, `field_ref` Applied and reference magnetic flux density, T. `exponent`  $n_B$ . `None` uses `WHIMS_FIELD_EXPONENT`, which is `ASSUMED`; pass a fitted value when one exists.

**Examples.** `k_ref = 0.02 1/s` at 1.0 T, scaled to 1.5 T with  $n_B = 2$ :  $(1.5/1.0)^2 = 2.25$ , `k = 0.02 x 2.25 = 0.045 1/s`.

```
>>> round(whims_rate_constant(Q_(0.02, "1/s"), Q_(1.5, "T"), Q_(1.0, "T")).magnitude, 4)
0.045
```

### *whims\_recovery*

```
whims_recovery(residence_time: Qty, k_ref: Qty, field: Qty, field_ref: Qty, ultimate_recovery: float, expo
\ nent: float | None=None) -> tuple[float, str]
```

Magnetic recovery of a susceptible species (Eq. E3 into Eq. E1).

**Returns.** (recovery, caveat) The caveat names the field exponent's provenance, so a caller cannot report the number without its weakest assumption attached.

### *logistic\_scale\_from\_ep*

```
logistic_scale_from_ep(ep: float) -> float
```

Logistic scale  $s$  from the probable error  $E_p$  (Eq. E4).

**Examples.**  $E_p = 20$  (in the property's own unit):  $s = 20/\ln 3 = 20/1.0986123 = 18.2047845$ .

```
>>> round(logistic_scale_from_ep(20.0), 7)
18.2047845
```

*partition\_number*

```
partition_number(x: float, x50: float, ep: float, bypass: float=0.0) -> float
```

Partition number at property value x (Eq. E4).

**Parameters.** x, x50 Property value and cut point, same unit. ep Probable error, same unit. bypass Short-circuit fraction reporting to the coarse/sink stream regardless of property value, in [0, 1). Reported separately rather than absorbed into ep, because a bypass and a blunt cut are different plant faults with different fixes.

**Returns.** float Partition number in [bypass, 1].

**Examples.** x50 = 100, Ep = 20, so s = 18.2047845. At x = 120: (120 - 100)/18.2047845 = 1.0986123 = ln 3, so P = 1/(1 + exp(-ln 3)) = 1/(1 + 1/3) = 0.75, which is the definition of x75 and confirms the Ep relation.

```
>>> round(partition_number(120.0, 100.0, 20.0), 6)
0.75
```

*partition\_curve*

```
partition_curve(xs: Sequence[float], x50: float, ep: float, bypass: float=0.0) -> list[tuple[float, float]
\ ]
```

Partition numbers over a property grid, checked monotonic increasing.

*ep\_from\_partition\_points*

```
ep_from_partition_points(x25: float, x75: float) -> float
```

Probable error from the 25 and 75 percent partition sizes (Eq. E4).

$$E_p = \frac{x_{75} - x_{25}}{2}$$

*imperfection*

```
imperfection(ep: float, x50: float, basis: PropertyBasis) -> float
```

Imperfection of a separation (Eq. E5).

For basis=DENSITY the denominator is  $x_{50} - 1$  (specific gravity relative to water), because the separating force scales with the density difference from the medium. For size and susceptibility the denominator is  $x_{50}$ .

**Examples.** Density separation, x50 = 1.60 g/cm3, Ep = 0.05: I = 0.05/(1.60 - 1) = 0.05/0.60 = 0.0833333.

```
>>> round(imperfection(0.05, 1.60, PropertyBasis.DENSITY), 7)
0.0833333
```

Ceiling.  $I > 1$  means the probable error exceeds the whole property difference driving the separation, so the partition curve is flatter than the cut it is meant to make and no separation is described. That is definitional, not a sourced threshold, and it is enforced because the `x50 > 1` guard alone does not bound the DENSITY denominator away from zero: at `x50 = 1.0000001` the denominator is  $1e-7$  and this function returned an imperfection of 499999.99970806646 rather than refusing. A cut point that close to the medium density is a measurement artefact or a unit error, and half a million reported as an imperfection is the kind of number that survives review because nobody expects it to be possible.

*two\_product*

```
two_product(f: float, c: float, t: float) -> tuple[float, float]
```

Mass yield and recovery to concentrate (Eq. E6). Exact mass balance.

**Parameters.** `f`, `c`, `t` Assay of the species of interest in feed, concentrate and tailing, in any consistent unit.

**Returns.** (`yield_to_concentrate`, `recovery_to_concentrate`) Both dimensionless in  $[0, 1]$ .

**Examples.** Impurity assays `f = 1.0` ppm, `c = 10.0` ppm (reject), `t = 0.1` ppm (product):  $Y = (1.0 - 0.1)/(10.0 - 0.1) = 0.9/9.9 = 0.0909091$ ,  $R = 10.0(0.9)/(1.0 \times 9.9) = 9.0/9.9 = 0.9090909$ .

```
>>> y, r = two_product(1.0, 10.0, 0.1)
>>> round(y, 7), round(r, 7)
(0.0909091, 0.9090909)
```

*two\_product\_condition\_number*

```
two_product_condition_number(f: float, c: float, t: float) -> float
```

Sensitivity of the two-product yield to assay error.

$$\kappa = \frac{c - t + f - t}{c - t}$$

A crude condition number: the relative error in  $Y$  is of order  $\kappa$  times the relative errors in the assays. It blows up as  $c \rightarrow t$ , which is the high-purity regime where feed, product and reject assays are all a few ppm. Above about 10, treat a two-product yield as indicative only and close the balance on weights instead.

**Examples.** `f = 30`, `c = 45`, `t = 28` ppm:  $(45 - 28 + 30 - 28)/(45 - 28) = 19/17 = 1.1176471$ , so well conditioned. With `c = 31`:  $(3 + 2)/3 = 1.667$  still fine, but `f = 30`, `c = 30.5`, `t = 29.9` gives  $(0.6 + 0.1)/0.6 = 1.1667$  while the absolute differences are inside analytical noise, which is the real failure mode: a small condition number does not rescue assays that are indistinguishable.

*product\_grade\_from\_reject*

```
product_grade_from_reject(feed_grade: float, reject_yield: float, reject_grade: float) -> float
```

Product impurity grade from a reject stream's yield and grade (Eq. E7).

This is the form a quartz flowsheet needs: the valuable stream is the non-float, non-magnetic product, and the "concentrate" is the impurity reject.

**Examples.** Feed 30 ppm Al, reject 8 percent of the mass at 280 ppm Al:  $(30 - 0.08 \times 280)/(1 - 0.08) = (30 - 22.4)/0.92 = 7.6/0.92 = 8.2608696$  ppm Al in the product.

```
>>> round(product_grade_from_reject(30.0, 0.08, 280.0), 7)
8.2608696
```

### *separation\_mass\_balance*

```
separation_mass_balance(feed_mass: Qty, feed_grade: float, reject_yield: float, reject_grade: float) -> dict[str, float]
```

Full two-stream mass balance, closing to machine precision.

**Returns.** dict Keys `feed_mass_kg`, `product_mass_kg`, `reject_mass_kg`, `product_grade_ppm`, `reject_grade_ppm`, `impurity_in_kg`, `impurity_out_kg`, `mass_residual_kg`, `impurity_residual_kg`. Both residuals are asserted to be within 1e-9 relative.

### *grade\_recovery\_curve*

```
grade_recovery_curve(feed_grade: float, times: Sequence[Qty], rate_constant: Qty, ultimate_recovery: float
    \ , reject_enrichment: float) -> list[dict[str, float]]
```

Grade-recovery locus over flotation time (Eq. E8).

**Parameters.** `feed_grade` Impurity grade of the feed, ppm. `times` Flotation times to evaluate. `rate_constant`, `ultimate_recovery` Eq. E1 parameters for impurity recovery to the froth (reject). `reject_enrichment` Ratio of reject impurity grade to feed impurity grade, greater than 1. Held constant here, which is a simplification: in a real bank the froth becomes progressively more dilute in impurity as time goes on, so a constant enrichment overstates late-time selectivity. Stated rather than hidden.

**Returns.** list of dict Each with `time_s`, `impurity_recovery`, `reject_yield`, `product_grade_ppm`, `impurity_rejection`.

### *feedstock\_impurity\_separation*

```
feedstock_impurity_separation(feedstock: Feedstock, element: str, removable_fraction: float, residence_time: Qty,
    \ e: Qty, rate_constant: Qty, kind: SeparatorKind, site: Site | None=None) -> dict[str, float | str]
```

Separation outcome for one element on one feedstock.

**Parameters.** `feedstock` Ore. Supplies the bulk concentration of `element`; raises via `ae.core.feedstock.ImpurityProfile.total_ppm` when unmeasured. `element` Element symbol. `removable_fraction` Ultimate recovery ceiling, which is NOT a free parameter: it should come from `ae.physics.liberation.leachable_fraction` or from `ae.physics.impurity_location.removable_ppm` divided by the bulk, so that the lattice inventory bounds it. Passing 1.0 here asserts the element has no lattice component, which for Al, Ti, Li or B is a claim about the deposit. `residence_time`, `rate_constant` Eq. E1 parameters. `kind` Which unit operation, recorded in the result. `site` Optional, recorded for traceability. No cost is computed here; energy and reagent cost belong in a plant model with sourced prices.

**Returns.** dict With keys:

`element` Element symbol, echoed back. `bulk_ppm` Feed concentration read off the feedstock, ppm by mass. `recovery` Fraction of the element's TOTAL inventory recovered to the reject at this residence time, dimensionless. Bounded above by `removable_fraction`, not by 1. `removed_ppm` `bulk_ppm` times `recovery`. `residual_ppm` What remains in the product, `bulk_ppm` minus `removed_ppm`. `irreducible_residual_ppm` The FLOOR on `residual_ppm`: `bulk_ppm` times  $(1 - \text{removable\_fraction})$ , the part of the inventory this unit cannot touch at any residence time (for Al, Ti, Li and B that is predominantly the lattice). Note the direction: this is a lower bound on what is LEFT, not an upper bound on what is removed. `unit` `kind.value`. `site_id` Site identifier, or "not specified". `note` Provenance and caveat text.

Mass balance on the element closes by construction (`removed_ppm + residual_ppm == bulk_ppm`) and is asserted.

## 11.3 Worked examples, reproduced by execution

The 8 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

`physics.separation.flotation_recovery`

```
>>> round(flotation_recovery(Q_(60.0, "s"), Q_(0.05, "1/s"), 0.90), 7)
0.8551916
```

`physics.separation.imperfection`

```
>>> round(imperfection(0.05, 1.60, PropertyBasis.DENSITY), 7)
0.0833333
```

`physics.separation.logistic_scale_from_ep`

```
>>> round(logistic_scale_from_ep(20.0), 7)
18.2047845
```

`physics.separation.partition_number`

```
>>> round(partition_number(120.0, 100.0, 20.0), 6)
0.75
```

`physics.separation.product_grade_from_reject`

```
>>> round(product_grade_from_reject(30.0, 0.08, 280.0), 7)
8.2608696
```

`physics.separation.rectangular_distribution_recovery`

```
>>> round(rectangular_distribution_recovery(Q_(60.0, "s"), Q_(0.10, "1/s"), 0.90), 8)
0.75037181
```

**physics.separation.two\_product**

```
>>> y, r = two_product(1.0, 10.0, 0.1)
>>> round(y, 7), round(r, 7)
(0.0909091, 0.9090909)
```

**physics.separation.whims\_rate\_constant**

```
>>> round(whims_rate_constant(Q_(0.02, "1/s"), Q_(1.5, "T"), Q_(1.0, "T")).magnitude, 4)
0.045
```

## 11.4 Validation status

16 hand-traceable worked example(s) and 5 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                     | Kind             | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_benchmark_partition_definition_identity                  | analytic         | Benchmark: the logistic $E_p$ relation against its analytic definition. The reference value is exact rather than experimental: for a logistic partition curve, $P(x_{50} + E_p) = 0.75$ identically, because the $E_p$ relation $s = E_p/\ln 3$ was derived from that condition. Any implementation error in the logistic or<br>10.1016/b978-0-08-097053-0.00009-1                                                                                                                                                    |
| test_benchmark_mass_balance_residuals                         | literature       | Benchmark: two-product mass balance residuals at machine precision. The reference is exact conservation, so the reported error is the numerical residual of the implementation rather than a discrepancy with a measurement. Reported across five decades of feed mass so that any catastrophic cancellation<br>10.1016/b978-075064450-1/50005-9                                                                                                                                                                      |
| test_benchmark_first_order_kinetics_against_analytic_solution | analytic         | Benchmark: Eq. E1 against the analytic solution of $dN/dt = -kN$ . The first-order postulate has a closed-form solution, so the benchmark is against numerical integration of the ODE rather than against experimental data: no batch flotation kinetics dataset for quartz was retrievable in this environment<br>10.1016/S0301-7516(99                                                                                                                                                                              |
| test_benchmark_flowsheet_grade_steps_against_lin_2020         | self consistency | Benchmark: can a two-product balance reach Lin et al. 2020's stage grades? Lin et al. 2020 Table 3 (doi:10.1007/s42461-020-00247-0) gives a conceptual flowsheet with product grade by stage: mining and washing then crushing and desliming to 90 to 99 percent $\text{SiO}_2$ , magnetic separation plus flotation to<br>Error stated: 99; 99.99; 99.99; 99.999<br>10.1007/s42461-020-00247-0                                                                                                                       |
| test_benchmark_whims_field_exponent_sensitivity               | literature       | Benchmark: sensitivity of WHIMS recovery to the ASSUMED field exponent. There is no measured exponent to benchmark against, so this reports the SPREAD the assumption introduces, which is the honest form of the statement. Scaling $k$ from 1.0 T to 1.5 T with $n_B$ across its assumed uniform bracket of 1<br>Error stated: 17.7; 89.4<br>10.1007/s42461-020-00247-0; 10.1007/s42461-020-00247-0;<br>10.1016/S0301-7516(99; 10.1016/b978-0-08-097053-0.00009-1;<br>10.1016/b978-075064450-1/50005-9; 10.3390/min |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_flotation_worked_example
test_flotation_limits
test_rate_constant_inversion_worked_example
test_rectangular_distribution_worked_example
test_rectangular_is_slower_than_single_k_at_the_mean
test_rectangular_limits
test_logistic_scale_worked_example
test_partition_curve_hits_75_percent_at_x50_plus_ep
test_ep_round_trips_from_the_quartile_points
test_imperfection_worked_example_density
test_imperfection_worked_example_size
test_whims_field_scaling_worked_example
test_two_product_worked_example
test_product_grade_worked_example
test_two_product_and_product_grade_are_mutually_consistent
test_condition_number_worked_example
```

## CHAPTER 12

# physics/leaching.py

Module `ae.physics.leaching`, 917 lines. Chapter text is this module's own documentation.

## 12.1 Purpose, governing equations and limitations

Shrinking-core leaching kinetics for impurity removal from a silica feedstock.

Scope: hot acid leaching of quartz concentrate, where a reagent (HF, HCl, HNO<sub>3</sub>, H<sub>2</sub>SO<sub>4</sub>, or a mixture) dissolves discrete impurity phases (mineral inclusions, grain-boundary films, fracture-filling oxides) from a particle of quartz. The model answers one question: for a given particle size, reagent concentration, temperature and residence time, what fraction of the LEACHABLE impurity inventory is removed. It does not and cannot address lattice-substituted Al, Ti, Li or B, which are unreachable by any aqueous reagent (see `ae.physics.diffusion` for the quantitative reason).

### EQUATIONS

#### Geometry and definitions

A spherical particle of initial radius  $R$  contains a reactive solid B at molar density  $\rho_B$ . Fluid reagent A at bulk molar concentration  $C_A$  reacts by



where  $b$  is moles of B consumed per mole of A. Conversion of B is

$$X = 1 - \left(\frac{r_c}{R}\right)^3, \quad u \equiv (1 - X)^{1/3} = \frac{r_c}{R}$$

with  $r_c$  the unreacted-core radius. Symbols, units and valid ranges:

| Symbol        | Unit                                | Valid range                        |
|---------------|-------------------------------------|------------------------------------|
| $R$           | m                                   | 1e-7 to 1e-1 (0.1 um to 100 mm)    |
| $\rho_B$      | mol m <sup>-3</sup>                 | > 0                                |
| $C_A$         | mol m <sup>-3</sup>                 | > 0                                |
| $b$           | dimensionless                       | > 0                                |
| $k_g$         | m s <sup>-1</sup>                   | > 0 (film coefficient)             |
| $k_s$         | m s <sup>-1</sup>                   | > 0 (surface rate constant)        |
| $D_e$         | m <sup>2</sup> s <sup>-1</sup>      | > 0 (effective pore diffusivity)   |
| $X$           | dimensionless                       | [0, 1]                             |
| $t$           | s                                   | >= 0                               |
| $\tau$        | s                                   | > 0 (time for complete conversion) |
| $E_a$         | J mol <sup>-1</sup>                 | > 0                                |
| $T$           | K                                   | > 0                                |
| $\mathcal{R}$ | J mol <sup>-1</sup> K <sup>-1</sup> | 8.314462618 (CODATA 2018)          |

#### Regime 1: fluid-film (external mass transfer) control

Derivation from first principles. The molar flux of A to the particle surface is  $N_A = k_g C_A$  (mol m<sup>-2</sup> s<sup>-1</sup>), the external area  $4\pi R^2$  is constant for a particle of unchanging size, and every mole arriving is consumed, so the consumption of B is

$$-\frac{dN_B}{dt} = 4\pi R^2 b k_g C_A = \text{constant}.$$

With  $N_B = \frac{4}{3}\pi R^3 \rho_B (1 - X)$  this integrates to

$$X(t) = \frac{t}{\tau_f}, \quad \boxed{\tau_f = \frac{\rho_B R}{3b k_g C_A}}$$

Conversion is LINEAR in time. Dimensional check:  $(\text{mol m}^{-3})(\text{m}) / [(\text{m s}^{-1})(\text{mol m}^{-3})] = \text{s}$ .

### Regime 2: product-layer (ash-layer) diffusion control

Derivation. A is transported through a porous residue of thickness  $R - r_c$  by quasi-steady diffusion. Spherical-shell Fick's first law with constant flux of moles per unit time  $\dot{n}$  gives  $\dot{n} = 4\pi D_e C_A (1/r_c - 1/R)^{-1}$ . Equating to the rate of shrinkage of the core,  $\dot{n}b = -4\pi r_c^2 \rho_B dr_c/dt$ , and integrating from  $r_c = R$  gives Levenspiel's ash-diffusion relation

$$g_{pl}(X) \equiv 1 - 3(1 - X)^{2/3} + 2(1 - X) = \frac{t}{\tau_{pl}}, \quad \boxed{\tau_{pl} = \frac{\rho_B R^2}{6b D_e C_A}}$$

Dimensional check:  $(\text{mol m}^{-3})(\text{m}^2) / [(\text{m}^2 \text{s}^{-1})(\text{mol m}^{-3})] = \text{s}$ .

Closed form for  $X(t)$ . Writing  $u = (1 - X)^{1/3}$ ,  $g_{pl} = 2u^3 - 3u^2 + 1 = (u - 1)^2(2u + 1)$ , so  $2u^3 - 3u^2 + (1 - \theta) = 0$  with  $\theta = t/\tau_{pl}$ . The substitution  $u = v + 1/2$  depresses it to  $v^3 - \frac{3}{4}v + (\frac{1}{4} - \frac{\theta}{2}) = 0$ , whose three roots are real for  $\theta \in [0, 1]$ . The trigonometric solution is

$$u(\theta) = \frac{1}{2} + \cos\left(\frac{\arccos(2\theta - 1)}{3} + \frac{4\pi}{3}\right), \quad X = 1 - u^3$$

This branch ( $k = 2$ ) is the one satisfying  $u(0) = 1$  and  $u(1) = 0$ ; the other two leave  $[0, 1]$ . `conversion` uses it. `test_closed_form_inversion_matches_bracketed_root_solve` in `tests/test_leaching.py` checks the closed form against an independent bisection root of  $g(X) - \theta = 0$  at nine values of  $\theta$  spanning  $[0, 1]$ , and `test_golden_conversion_inversions_are_exact` pins the hand-traceable arithmetic at  $\theta = 0.25$ . This is an exact algebraic inversion, not a fit.

### Regime 3: surface-reaction control

Derivation. A first-order heterogeneous reaction at the core surface consumes A at  $k_s C_A$  per unit core area, so  $-4\pi r_c^2 \rho_B dr_c/dt = 4\pi r_c^2 b k_s C_A$ , i.e. the core radius recedes at constant velocity  $dr_c/dt = -b k_s C_A / \rho_B$ . Integrating,

$$g_{sr}(X) \equiv 1 - (1 - X)^{1/3} = \frac{t}{\tau_{sr}}, \quad \boxed{\tau_{sr} = \frac{\rho_B R}{b k_s C_A}}, \quad X(t) = 1 - \left(1 - \frac{t}{\tau_{sr}}\right)^3$$

Note  $\tau_{sr} = 3\tau_f$  for  $k_s = k_g$ : the same surface rate constant gives a threefold longer completion time when the reaction front recedes into the particle than when it sits at the fixed outer surface.

### Temperature dependence

Every rate constant and diffusivity is Arrhenius,

$$k(T) = A \exp\left(-\frac{E_a}{RT}\right)$$

with  $A$  in the same unit as  $k$ . The apparent activation energy is the standard regime discriminator: film control is weakly temperature dependent ( $E_a$  of order 5 to 20 kJ mol<sup>-1</sup>, set by liquid viscosity), product-layer diffusion is intermediate (roughly 20 to 40 kJ mol<sup>-1</sup>), and surface reaction control is strong (typically above 40 kJ mol<sup>-1</sup>). Yang and Li (2020) report 27.72 kJ mol<sup>-1</sup> for ultrasound-assisted Fe removal from quartz and assign product-layer diffusion as rate determining, which is consistent with that banding. The banding is a heuristic, not a law: it is stated as ASSUMED in `EA_REGIME_BANDS` with its basis, and `identify_regime` reports it only as corroboration of a fit, never as the primary evidence.

## Particle-size discrimination

The three regimes scale differently in  $R$  ( $\tau_f \propto R$ ,  $\tau_{pl} \propto R^2$ ,  $\tau_{sr} \propto R$ ), so a size series separates product-layer diffusion from the other two but cannot separate film from surface-reaction control. That separation needs a stirring-speed series:  $k_g$  depends on agitation,  $k_s$  does not. `size_exponent_from_series` implements the former.

## VALIDATION STATUS

The conversion-time FORM of this model is NOT validated against experimental quartz data in this build. Retrieval attempts logged 2026-09-16, per source:

- Yang and Li (2020), doi 10.1515/htmp-2020-0081: full-text retrieval attempted via Unpaywall (200, no OA location), Semantic Scholar (DOI redirect only), PMC (no PMCID), CrossRef TDM (no accessible content) and DOI resolution (HTTP 202). A direct request to the publisher (degruyter.com) was refused by the sandbox network allowlist. The ABSTRACT was retrieved, giving the activation energies (27.72 kJ/mol ultrasound-assisted, 20.44 kJ/mol regular), the endpoint (74 percent Fe removal in 40 min) and the assay pair (Fe<sub>2</sub>O<sub>3</sub> 0.0857 to 0.0223 percent). No per-point conversion-time table.
- Arslan and Bayat (2009), doi 10.1007/bf03403416: full-text retrieval attempted via Unpaywall (200, no OA location), Semantic Scholar (200, no openAccessPdf), PMC (no PMCID) and CrossRef TDM (no accessible content); the DOI landing page resolved but served no machine-readable text. No abstract and no data table could be extracted.
- Europe PMC full-text searches for open-access shrinking-core leaching datasets ("shrinking core" with "quartz", "product layer diffusion" with "leaching kinetics") returned zero hits.

Consequently:

- `identify_regime` is exercised by a SYNTHETIC self-consistency test (`test_synthetic_regime_recovery` in `tests/test_leaching.py`), which confirms the fitter recovers the regime and tau it was given. That is a test of the code, NOT a validation of the physics. It is marked as such in the test name and docstring and is not a `@pytest.mark.benchmark`.
- The two benchmark tests in this module validate the IMPURITY ACCOUNTING (removal fraction from feed and product assay) against published endpoints, with the error reported. They do not validate the kinetic form.

## LIMITATIONS

1. Lattice impurities are outside this model entirely. `leachable_ppm` raises `ae.core.provenance.MissingValueError` when the lattice split has not been measured, because the leachable inventory is `total - lattice` and guessing the split fabricates the purity ceiling. Do not add a default lattice fraction to make this function return.

2. The kinetic FORM is unvalidated for quartz (see `VALIDATION STATUS`). Treat any tau or activation energy this module produces from a fit as a description of the supplied data, not as transferable to another ore.
3. Single-size, single-shape, isothermal, constant-bulk-concentration. Real leach circuits have a size distribution (Rosin-Rammler or similar), and integrating conversion over that distribution can shift the time to 90 percent conversion by tens of percent. `conversion_over_size_distribution` does that integration; the single-size functions do not.
4. Constant  $C_A$  assumes reagent in large excess. For HF leaching of quartz this can fail badly, because HF attacks the silica matrix itself (see `ae.physics.reagents`), so reagent depletion may be dominated by the matrix rather than the impurity. If the computed reagent consumption from `ae.physics.reagents` approaches the reagent charged, the constant- $C_A$  assumption is void and this model over-predicts conversion.
5. No shrinking-particle case. If the solid being dissolved is the bulk of the particle (a carbonate sand, say) rather than a dispersed minor phase, the particle shrinks and the film-control relation changes to Levenspiel's shrinking-sphere form with Stokes or Ranz-Marshall mass transfer. Quartz leaching is a minor-phase problem, so the unchanging-size treatment applies, but do not reuse these functions for a matrix-dissolution problem.
6. Mixed control is not modelled. When two resistances are comparable, the additive-resistance sum (Levenspiel section 25.3) applies and none of the three single-regime linearisations fits well. `identify_regime` reports `mixed` when no single regime clears the R-squared margin, rather than forcing a choice.
7. Ultrasound, pressure (autoclave), and chelating additives are not modelled. Yang and Li (2020) report that ultrasound RAISES the apparent activation energy from 20.44 to 27.72 kJ mol<sup>-1</sup> while shortening the time to 74 percent Fe removal to 40 minutes, so ultrasound is not a simple multiplier on the rate constant and cannot be represented in this framework without its own calibration.
8. Temperature range. The Arrhenius form is extrapolated freely by `Arrhenius.at`, which is dangerous above the reagent boiling point at the operating pressure. The calibration domain of any fitted pair is recorded on the `Arrhenius` object and `Arrhenius.at` warns outside it.

## References

Levenspiel, O. (1999) Chemical Reaction Engineering, 3rd edition, Wiley, chapter 25 (shrinking-core model for particles of unchanging size). The three conversion-time relations above are re-derived from first principles in this docstring rather than quoted, so the derivation can be checked independently of the book.

Yang, C. and Li, S. (2020) Kinetics of iron removal from quartz under ultrasound-assisted leaching, High Temperature Materials and Processes 39(1), 395-404, doi 10.1515/htmp-2020-0081.

Xia, M., Yang, X. and Hou, Z. (2024) Preparation of High-Purity Quartz Sand by Vein Quartz Purification and Characteristics: A Case Study of Pakistan Vein Quartz, Minerals 14(7), 727, doi 10.3390/min14070727.

Tiesinga, E., Mohr, P. J., Newell, D. B. and Taylor, B. N. (2021) CODATA Recommended Values of the Fundamental Physical Constants: 2018, Reviews of Modern Physics 93(2), article 025010, doi 10.1103/RevModPhys.93.025010 (also published as Journal of Physical and Chemical Reference Data 50(3), article 033105, doi 10.1063/5.0064853). The source of the in-text "CODATA 2018" label on the molar gas constant. The 2021 publication year is the journal record for the 2018 adjustment, which is the name the adjustment is known by. The value 8.314462618 J mol<sup>-1</sup> K<sup>-1</sup> is EXACT by the 2019 SI redefinition, being the product of the two defining constants  $k = 1.380649\text{e-}23 \text{ J K}^{-1}$  and  $N_A = 6.02214076\text{e}23 \text{ mol}^{-1}$ , and it is reproduced by that multiplication in this module's tests rather than taken on the authority of the citation.

## 12.2 Interface

## Types

**Regime.** Rate-controlling step of the shrinking-core model.

**Arrhenius.** Arrhenius temperature dependence of a rate constant or a diffusivity.

$k(T) = A \exp(-E_a / (R T))$ . `prefactor` carries the unit of the target quantity (m/s for a mass-transfer or surface rate constant,  $\text{m}^2/\text{s}$  for a diffusivity), so dimensional correctness of the tau formulae is enforced by the unit system rather than by convention.

`calibration_T` records the temperature interval over which the pair was measured or fitted. `at` warns outside it, because an Arrhenius extrapolation is the single most common way a leach model produces a physically impossible answer.

```
at(self, temperature: Quantity) -> Quantity
```

Evaluate  $k(T)$ . Warns if  $T$  is outside the calibration interval.

**LeachSystem.** One (feedstock, element, reagent, particle size) leach configuration.

Every kinetic quantity in this module is computed from a `LeachSystem`, so the ore enters through `feedstock` and nothing is hardcoded to a deposit. `solid_molar_density` is the molar density of the REACTIVE SOLID PHASE being dissolved (for example hematite in a fracture film), not of the quartz matrix, because the shrinking core is the impurity phase.

```
rate_constant(self, regime: Regime) -> Quantity
```

Temperature-evaluated transport or rate coefficient for `regime`.

**RegimeFit.** Outcome of fitting all three shrinking-core linearisations to one dataset.

## Functions

*leachable\_ppm*

```
leachable_ppm(feedstock: Feedstock, element: str) -> float
```

Concentration of `element` that an aqueous leach could in principle reach.

Returns `total_ppm - lattice_ppm`. Lattice-substituted Al, Ti, Li and B sit on silicon or interstitial sites inside the quartz structure and no reagent reaches them at leaching temperatures, so they are subtracted before any kinetic calculation.

**Raises.** `ae.core.provenance.MissingValueError` If the lattice split was never measured, propagated unchanged from `ae.core.feedstock.ImpurityProfile.lattice_ppm`. This is correct behaviour: the leachable inventory is the difference of two numbers and substituting a guess for one of them fabricates the purity ceiling of the deposit. Do not add a default.

*tau\_film*

```
tau_film(system: LeachSystem) -> Quantity
```

Time for complete conversion under fluid-film control.

$$\tau_f = \rho_B R / (3bk_g C_A).$$

*tau\_product\_layer*

```
tau_product_layer(system: LeachSystem) -> Quantity
```

Time for complete conversion under product-layer diffusion control.

$$\tau_{pl} = \rho_B R^2 / (6bD_e C_A).$$

*tau\_surface\_reaction*

```
tau_surface_reaction(system: LeachSystem) -> Quantity
```

Time for complete conversion under surface-reaction control.

$$\tau_{sr} = \rho_B R / (bk_s C_A).$$

*tau\_for*

```
tau_for(system: LeachSystem, regime: Regime) -> Quantity
```

Dispatch to the tau expression for regime.

*g\_of\_conversion*

```
g_of_conversion(regime: Regime, conversion_x: float | np.ndarray) -> np.ndarray
```

Linearising function  $g(X) = t/\tau$  for each regime.

FILM:  $X$ . PRODUCT\_LAYER:  $1 - 3(1 - X)^{2/3} + 2(1 - X)$ . SURFACE\_REACTION:  $1 - (1 - X)^{1/3}$ . All three map  $[0, 1]$  onto  $[0, 1]$  monotonically, which is what makes a regression through the origin the right diagnostic.

*conversion*

```
conversion(regime: Regime, t: Quantity, tau: Quantity) -> float
```

Closed-form conversion  $X(t)$  for one regime.

Uses the exact trigonometric inversion of the product-layer cubic derived in the module docstring, so no iteration is involved.

*conversion\_profile*

```
conversion_profile(system: LeachSystem, regime: Regime, times: Quantity) -> np.ndarray
```

Conversion at each time in times for system under regime.

*tau\_from\_single\_point*

```
tau_from_single_point(regime: Regime, conversion_x: float, t: Quantity) -> Quantity
```

Back out  $\tau$  from one measured  $(X, t)$  pair:  $\tau = t/g(X)$ .

One point cannot distinguish regimes; it only scales an assumed regime. Used for scenario work when a paper reports a single endpoint (for example 74 percent Fe removal in 40 minutes) and nothing else.

*identify\_regime*

```
identify_regime(times: Quantity, conversions: Sequence[float] | np.ndarray, apparent_Ea: Quantity | None=None
\ one, margin: float=0.02) -> RegimeFit
```

Fit all three linearisations to  $(t, X)$  data and report which dominates.

Each regime predicts  $g(X) = t/\tau$ , a straight line through the origin with slope  $1/\tau$ . The fit is therefore a one-parameter least-squares regression of  $g(X)$  on  $t$  with zero intercept,  $1/\tau = \sum gt / \sum t^2$ , and the goodness of fit is

$$R^2 = 1 - \frac{\sum (g_i - t_i/\tau)^2}{\sum (g_i - \bar{g})^2}$$

Forcing the intercept to zero is deliberate: a fitted intercept absorbs the induction period and makes all three regimes look equally good, which is the classic way this diagnostic is abused.

Returns `Regime.MIXED` when the best R-squared does not exceed the runner-up by `margin`, because three monotone functions on  $[0, 1]$  are similar enough that a small margin is not evidence.

**Parameters.** `times` Sampling times, a pint Quantity array with time dimensionality. `conversions` Measured conversion at each time, each in  $[0, 1]$ . `apparent_Ea` Optional independently measured activation energy. If given, the fit reports whether it falls in `EA_REGIME_BANDS` for the winning regime. That is corroboration only: the bands are ASSUMED heuristics. `margin` Minimum R-squared advantage required to name a single regime.

*arrhenius\_fit*

```
arrhenius_fit(temperatures: Quantity, rate_constants: Quantity) -> tuple[Quantity, Quantity, float]
```

Least-squares Arrhenius fit: returns  $(E_a, \text{prefactor}, r\_squared)$ .

Regresses  $\ln k$  on  $1/T$ , slope  $-E_a/R$ , intercept  $\ln A$ . Requires at least three temperatures: two points define a line exactly and report R-squared of 1 regardless of the data quality, which is misleading.

*size\_exponent\_from\_series*

```
size_exponent_from_series(radii: Quantity, taus: Quantity) -> tuple[float, float]
```

Fit  $\tau \propto R^n$  and return  $(n, r\_squared)$ .

The exponent discriminates regimes independently of any R-squared comparison on a single conversion curve:  $n = 1$  for film and surface-reaction control,  $n = 2$  for product-layer diffusion. It cannot separate film from surface reaction, which needs a stirring-speed series instead.

*conversion\_over\_size\_distribution*

```
conversion_over_size_distribution(system: LeachSystem, regime: Regime, t: Quantity, radii: Quantity, mass_
\ fractions: Sequence[float] | np.ndarray) -> float
```

Mass-weighted conversion across a particle size distribution.

$$\bar{X}(t) = \sum_i w_i X(t; \tau(R_i))$$

with  $\sum w_i = 1$ . Because  $\tau$  grows with  $R$ , the coarse tail controls the time to high overall conversion, and a single-size calculation at the mean radius systematically over-predicts.

*removal\_fraction\_from\_assay*

```
removal_fraction_from_assay(feed_ppm: float, product_ppm: float) -> float
```

Impurity removal fraction from a feed and a product assay:

$$\eta = \frac{C_{feed} - C_{product}}{C_{feed}}$$

This is the accounting identity that published purification results report, and it is separated from the kinetics deliberately: it is the only part of this module that can be validated against the literature available here. Assumes negligible silica dissolution, so that concentrations are on a common mass basis. If matrix loss is significant, the mass-corrected form in `ae.physics.reagents` must be used instead.

## 12.3 Worked examples, reproduced by execution

### Shrinking-core time constants and the geometric factor of three

```
from ae.core.units import Q_
from ae.core.provenance import Tag
from ae.physics.leaching import (Arrhenius, LeachSystem, Regime, conversion,
                                removal_fraction_from_assay, tau_film,
                                tau_product_layer, tau_surface_reaction)

k = Arrhenius(prefactor=_v(Q_(1.0e-4, "m/s"), Tag.ASSUMED),
              activation_energy=_v(Q_(15.0, "kJ/mol"), Tag.ASSUMED))
system = LeachSystem(
    feedstock=scenario_feedstock(), element="Fe", reagent="HCl",
    particle_radius=_v(Q_(100.0, "um"), Tag.ASSUMED),
    reagent_concentration=_v(Q_(1000.0, "mol/m**3"), Tag.ASSUMED),
    solid_molar_density=_v(Q_(30000.0, "mol/m**3"), Tag.ASSUMED),
    stoich_b=1.0,
    temperature=_v(Q_(80.0, "degC"), Tag.ASSUMED),
    film_coefficient=k,
    surface_rate_constant=k,
    product_layer_diffusivity=Arrhenius(
        prefactor=_v(Q_(1.0e-9, "m**2/s"), Tag.ASSUMED),
        activation_energy=_v(Q_(25.0, "kJ/mol"), Tag.ASSUMED)))

tf, ts, tp = tau_film(system), tau_surface_reaction(system), tau_product_layer(system)
print("tau_film          :", round(float(tf.to("s")).magnitude, 4), "s =",
      round(float(tf.to("min")).magnitude, 4), "min")
print("tau_surface      :", round(float(ts.to("s")).magnitude, 4), "s")
print("tau_surface / tau_film :", round(float((ts / tf).to("dimensionless")).magnitude, 10))
print("tau_product_layer :", round(float(tp.to("s")).magnitude, 4), "s")
print()
# Evaluated at a FRACTION of tau: at t = tau every regime reaches X = 1 by
# definition of tau, so the regimes are only distinguishable before that.
for frac in (0.25, 0.5, 0.75):
    row = [f"{conversion(reg, frac * tf, tf):.6f}"
           for reg in (Regime.FILM, Regime.SURFACE_REACTION, Regime.PRODUCT_LAYER)]
    print(f"X at t = {frac:.2f} tau: film {row[0]} surface {row[1]} product layer {row[2]}")
print()
print("Xia et al. 2024, 128.86 to 24.23 ug/g:",
      round(removal_fraction_from_assay(128.86, 24.23), 7))
```

Output:

```
tau_film          : 1654.3073 s = 27.5718 min
tau_surface       : 4962.9218 s
tau_surface / tau_film : 3.0
tau_product_layer : 249266.3346 s
```

```
X at t = 0.25 tau: film 0.250000 surface 0.578125 product layer 0.694297
X at t = 0.50 tau: film 0.500000 surface 0.875000 product layer 0.875000
X at t = 0.75 tau: film 0.750000 surface 0.984375 product layer 0.965242

Xia et al. 2024, 128.86 to 24.23 ug/g: 0.8119665
```

With equal film and surface rate constants the surface time constant is exactly three times the film one. That factor is the ratio of particle volume to surface area in the shrinking-core derivation, and reproducing it is a check on the implementation rather than on the physics.

12.4 Validation status

6 hand-traceable worked example(s) and 3 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                       | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_benchmark_xia_2024_total_removal           | literature | Impurity accounting against Xia et al. 2024 (doi 10.3390/min14070727). Reported: 128.86 ug/g feed, 24.23 ug/g product, 81.20 percent removal. This validates the ACCOUNTING IDENTITY only, not the kinetic model: the paper reports no per-stage breakdown and no conversion-time data.<br><br>Error stated: 81.20<br>10.3390/min14070727                                                                                                                    |
| test_benchmark_yang_2020_iron_removal           | literature | Fe removal against Yang and Li 2020 (doi 10.1515/htmp-2020-0081). Reported: Fe2O3 0.0857 percent to 0.0223 percent, stated as 74 percent Fe removal in 40 min. Converting the oxide assays to an Fe basis with the stoichiometric factor $2 M_{Fe} / M_{Fe2O3} = 111.69/159.687 = 0.699431$ (written 0.699435 befo<br><br>Error stated: 0.0857; 0.0223; 74; 0.0857<br>10.1515/htmp-2020-0081                                                                 |
| test_benchmark_yang_2020_activation_energy_band | literature | Yang and Li's reported Ea against this module's regime-diagnostic bands. Reported: 27.72 kJ/mol ultrasound-assisted, 20.44 kJ/mol regular leaching (the abstract states the ultrasound value exceeds the regular one by 7.28 kJ/mol). Both fall in the module's PRODUCT_LAYER band (20 to 40 kJ/mol), which<br><br>10.1007/bf03403416; 10.1063/5.0064853; 10.1103/RevMod-Phys.93.025010; 10.1515/htmp-2020-0081; 10.1515/htmp-2020-0081; 10.3390/min14070727 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_golden_tau_film_arithmetic
test_golden_tau_product_layer_arithmetic
test_golden_tau_surface_is_three_times_film_when_k_equal
test_golden_conversion_inversions_are_exact
test_closed_form_inversion_matches_bracketed_root_solve
test_golden_removal_fraction_arithmetic
```

## CHAPTER 13

# physics/reagents.py

Module `ae.physics.reagents`, 850 lines. Chapter text is this module's own documentation.

## 13.1 Purpose, governing equations and limitations

Reagent stoichiometry, mass balance, neutralization demand and effluent load.

Scope: the consumption side of a hot acid leach on a silica feedstock. Given an impurity load and a reagent recipe, this module computes moles of acid consumed by each impurity phase and by the silica matrix, the lime or caustic required to neutralize the spent liquor, the fluoride and metal load in the effluent compared against the site's discharge limits, and the resulting neutralization sludge. It takes `SITE` for reagent prices and effluent limits, and `FEEDSTOCK` for the impurity inventory, so a change of jurisdiction or ore is a change of argument rather than a change of code.

Why this module exists separately from `ae.physics.leaching`: the kinetic model says how fast an impurity dissolves, but reagent cost and effluent liability are driven by `TOTAL` consumption, and for HF on quartz the dominant consumer is usually the silica matrix rather than the impurities. A flowsheet that budgets HF against the impurity load alone understates consumption by orders of magnitude, and that error is the main thing this module exists to prevent.

## EQUATIONS

### Symbols

| Symbol          | Unit                              | Valid range                |
|-----------------|-----------------------------------|----------------------------|
| $m_{ore}$       | kg                                | $> 0$                      |
| $C_i$           | kg kg <sup>-1</sup> (as ppm mass) | $\geq 0$                   |
| $M_i$           | kg mol <sup>-1</sup>              | $> 0$                      |
| $n_i$           | mol                               | $\geq 0$                   |
| $\nu_{i,r}$     | dimensionless                     | $> 0$ (mol acid per mol i) |
| $f_{diss}$      | dimensionless                     | $[0, 1]$ silica dissolved  |
| $z_i$           | dimensionless                     | ion charge, any sign       |
| $n_{H^+}$       | mol                               | $\geq 0$                   |
| $\eta_{excess}$ | dimensionless                     | $\geq 1$ (reagent excess)  |

### Impurity dissolution stoichiometry

Moles of impurity element `i` in a charge of ore:

$$n_i = \frac{m_{ore} C_i}{M_i}$$

with  $C_i$  the mass fraction (ppm by mass divided by 1e6). Acid demand for that impurity is  $\nu_{i,r} n_i$ , where  $\nu_{i,r}$  comes from the balanced dissolution reaction. The reactions used, each balanced for mass and charge (`check_reaction_balance` verifies both):





The general rule these follow: a cation of charge  $z$  in an oxide consumes  $z$  moles of a monoprotic acid, because the acid must both supply  $z$  anions to the cation and 2 protons per oxide oxygen to form water. `acid_demand_per_cation` applies the charge rule, so a cation not tabulated above is handled by its charge rather than by a missing entry.

## Silica attack by HF

HF dissolves the silica matrix itself, which no other mineral acid in this set does appreciably:



Six moles of HF per mole of SiO<sub>2</sub> dissolved. Because SiO<sub>2</sub> is essentially the whole charge rather than a trace, a silica dissolution of even 0.1 percent by mass consumes

$$n_{\text{HF}} = 6 \frac{0.001 m_{\text{ore}}}{M_{\text{SiO}_2}} = 6 \times \frac{0.001 \times 1000 \text{ kg}}{0.060083 \text{ kg mol}^{-1}} = 99.9 \text{ mol per tonne}$$

against an impurity demand of 3.34 mol per tonne for 30 ppm Al ( $1000 \times 30 \times 10^{-6} / 0.0269815 = 1.112$  mol Al, times  $\nu = 3$ ), so the matrix consumes 30 times more HF than the impurity it is being used to remove. That 30 ppm is taken as fully leachable for the illustration; the functions here work on the LEACHABLE inventory only (total minus lattice), which makes the ratio larger still, 44.7 for a feedstock whose leachable load is 2.23 mol HF per tonne. The ratio scales linearly with the silica dissolution: 3.0 at 0.01 percent dissolution, 30 at 0.1 percent, 299 at 1 percent, and it rises further once the lattice-bound fraction is excluded from the leachable impurity inventory. `hf_silica_demand` computes the silica term and `reagent_balance` reports the ratio explicitly as `silica_to_impurity_demand_ratio`, so an HF budget can never be quoted against the impurity load alone.

An important consequence for the alternative reaction stoichiometry sometimes quoted,  $\text{SiO}_2 + 4 \text{HF} \rightarrow \text{SiF}_4 + 2 \text{H}_2\text{O}$ : which one applies depends on whether the product is fluorosilicic acid in solution (6 HF) or silicon tetrafluoride gas (4 HF). Both are provided as `SILICA_HF_STOICH` options, defaulting to the 6 HF solution route for an aqueous leach, and the choice is reported in the balance output because it changes HF demand by 50 percent.

## Neutralization demand

The neutralization basis is the TOTAL acid charged, not the unreacted excess. Acid consumed by a dissolution reaction is not destroyed: its anion leaves with the liquor as a dissolved metal salt (FeCl<sub>3</sub>, AlCl<sub>3</sub>, CaCl<sub>2</sub>) or as fluorosilicic acid, and that anion still has to be neutralized or precipitated before discharge. Charging neutralization against the excess alone would report zero lime and zero sludge at the stoichiometric floor (`excess_factor = 1.0`) for a liquor carrying every mole of fluoride that was fed, which is the error this basis exists to avoid. `reagent_balance` reports `unreacted_acid_mol` and `acid_neutralized_mol` separately so the two are never confused.

Using lime:



and for fluoride, precipitation as the sparingly soluble fluorite:



so one mole of lime per two moles of monoprotic acid in both cases, and one mole of  $\text{CaF}_2$  sludge per two moles of fluoride. Using caustic,  $\text{NaOH} + \text{HCl} \rightarrow \text{NaCl} + \text{H}_2\text{O}$ , one mole per mole, but  $\text{NaF}$  is far more soluble than  $\text{CaF}_2$  so caustic does NOT remove fluoride from the water: `neutralization_demand` returns zero fluoride precipitation for the caustic route and flags it, because treating caustic as a fluoride treatment is a permit-level error.

## Charge balance

The neutralized liquor must satisfy electroneutrality:

$$\sum_i z_i n_i^{\text{cation}} + n_{\text{H}^+} = \sum_j |z_j| n_j^{\text{anion}} + n_{\text{OH}^-}$$

`charge_balance` evaluates both sides and `test_charge_balance_closes` asserts closure to 1e-9 relative. A charge imbalance means a species was double counted or omitted, which is the most common defect in a hand-built effluent balance.

## Mass balance

Total mass in equals total mass out:

$$m_{\text{ore}} + m_{\text{acid}} + m_{\text{base}} + m_{\text{water}} = m_{\text{product}} + m_{\text{dissolved}} + m_{\text{sludge}} + m_{\text{liquor}} + m_{\text{gas}}$$

`reagent_balance` closes this to a relative tolerance of 1e-9 and raises if it does not, rather than reporting an unbalanced flowsheet.

## LIMITATIONS

1. The stoichiometric demand computed here is a LOWER BOUND on real consumption. Real circuits run an excess (typically expressed as a multiple of stoichiometric), lose acid to entrainment and vapour, and regenerate incompletely. `excess_factor` defaults to 1.0, which is the stoichiometric floor and is NOT a plant number: a realistic value must come from a specific flowsheet and is an input, not a default this module can supply.
2. Impurity phase speciation is assumed. The acid demand for an element depends on the mineral it sits in: iron as hematite, as goethite, as a sulphide or as a silicate all consume different amounts of acid and dissolve at different rates. This module computes demand from the `ELEMENTAL` inventory and the cation charge, which is exact only if every impurity is present as the simple oxide. Mineral inclusion data (feldspar, mica, rutile) would refine it, and the direction of the error is not general: a silicate host can consume MORE acid than the oxide, an unreactive host LESS.
3. Lattice-substituted impurities consume no acid at all, because they are never contacted (see `ae.physics.diffusion`). `reagent_balance` uses the `LEACHABLE` inventory when the lattice split is measured, and raises otherwise, so an uncharacterized ore cannot produce a reagent budget that silently assumes all impurities are available.
4. No activity coefficients, no speciation equilibria, no complexation. At the ionic strengths of a spent leach liquor, activity coefficients depart substantially from unity, and fluoride forms a series of complexes ( $\text{SiF}_6(2-)$ ,  $\text{AlF}(x)$  species) that change both the free-fluoride concentration and the lime demand. A real permit calculation requires a speciation model (`PHREEQC` or equivalent), not this stoichiometry.

5. Fluoride effluent is computed as TOTAL fluoride, not as free fluoride ion. Discharge limits are usually written on total fluoride, so this is the right basis for a compliance screen, but  $\text{CaF}_2$  solubility sets a floor on what precipitation can achieve ( $\text{CaF}_2$  is sparingly soluble, not insoluble) and that floor is not computed here because no solubility product could be sourced in this build.
6. Sludge mass is the stoichiometric dry precipitate only. Real neutralization sludge carries water at 40 to 70 percent by mass after filtration and entrains gypsum and metal hydroxides, so the disposal tonnage is several times the number computed here. The multiplier is a design input.
7. No reagent prices are hardcoded. `reagent_cost` reads them from the SITE and raises if the site does not carry a price for a reagent the recipe uses, or if the reagent is marked not locally available. HF availability is a real constraint in many jurisdictions and is enforced by `ae.core.site.ReagentPrices.price`, not assumed away.
8. Nitric and sulphuric acid are handled only by their proton count ( $\text{HNO}_3$  monoprotic,  $\text{H}_2\text{SO}_4$  diprotic). Their distinctive chemistry, oxidation of sulphides by  $\text{HNO}_3$  and calcium sulphate scaling with  $\text{H}_2\text{SO}_4$ , is not modelled, and the latter is a real flowsheet risk when a feedstock carries calcium.

## References

Reaction stoichiometries above are balanced from first principles (mass and charge), not taken from a source, and `check_reaction_balance` verifies each one numerically in the test suite. Atomic weights come from `ae.core.feedstock.MOLAR_MASS` (IUPAC 2021 standard atomic weights).

Xia, M., Yang, X. and Hou, Z. (2024) Preparation of High-Purity Quartz Sand by Vein Quartz Purification and Characteristics: A Case Study of Pakistan Vein Quartz, *Minerals* 14(7), 727, doi 10.3390/min14070727. Reports the impurity endpoints (128.86 to 24.23 ug/g) this module's impurity-load path is benchmarked against, but reports NO reagent consumption, so no reagent figure in this module is validated against it.

Yang, C. and Li, S. (2020) Kinetics of iron removal from quartz under ultrasound-assisted leaching, *High Temperature Materials and Processes* 39(1), 395-404, doi 10.1515/htmp-2020-0081. Reports  $\text{Fe}_2\text{O}_3$  0.0857 to 0.0223 percent, used as the iron-load benchmark. Reagent consumption is not reported in the retrievable abstract.

## 13.2 Interface

### Types

**Acid.**

**Base.**

**Reaction.** A balanced dissolution reaction, stored so balance can be verified.

### Functions

*check\_reaction\_balance*

```
check_reaction_balance(reaction: Reaction) -> dict[str, float]
```

Return per-element and charge residuals for a reaction. All must be zero.

A non-zero residual means the reaction as written creates or destroys atoms or charge, so any stoichiometric factor derived from it is wrong.

*acid\_demand\_per\_cation*

```
acid_demand_per_cation(element: str, acid: Acid) -> float
```

Moles of acid consumed per mole of `element` dissolved from its oxide.

Applies the charge rule: a cation of charge  $z$  needs  $z$  equivalents of acid, and an acid supplying  $p$  protons per molecule therefore contributes  $p$  equivalents, giving  $\nu = z/p$ . For HCl ( $p = 1$ ) on Fe<sup>3+</sup> this returns 3, matching the balanced hematite reaction above; for H<sub>2</sub>SO<sub>4</sub> ( $p = 2$ ) it returns 1.5.

*impurity\_moles*

```
impurity_moles(feedstock: Feedstock, ore_mass: Quantity, elements: tuple[str, ...] | None=None, use_leacha
    \ ble: bool=True) -> dict[str, Quantity]
```

Moles of each impurity element in a charge of `ore_mass`.

With `use_leachable=True` the lattice-bound fraction is excluded, because it is never contacted by reagent, and the call raises `ae.core.provenance.MissingValueError` when the lattice split has not been measured. That is the intended behaviour: a reagent budget that assumes the whole inventory is available overstates consumption and understates the achievable product grade at the same time.

*impurity\_acid\_demand*

```
impurity_acid_demand(feedstock: Feedstock, ore_mass: Quantity, acid: Acid, elements: tuple[str, ...] | Non
    \ e=None, use_leachable: bool=True) -> tuple[Quantity, dict[str, Quantity]]
```

Total and per-element acid demand for dissolving the impurity load.

*hf\_silica\_demand*

```
hf_silica_demand(ore_mass: Quantity, silica_dissolved_fraction: float, route: str='H2SiF6_solution') -> tu
    \ ple[Quantity, Quantity]
```

HF consumed by attack on the silica matrix, and the mass of SiO<sub>2</sub> lost.

Returns (`moles_HF`, `mass_SiO2_dissolved`). This is usually the dominant HF consumer and the dominant yield loss, and it scales with the CHARGE mass, not with the impurity level.

*neutralization\_demand*

```
neutralization_demand(acid_moles: dict[Acid, Quantity], base: Base) -> dict[str, object]
```

Base required to neutralize residual acid, and the sludge produced.

Demand is computed on PROTON equivalents, so a diprotic acid counts twice, and divided by the hydroxide equivalents of the base.

Fluoride precipitation. Lime precipitates fluoride as CaF<sub>2</sub>, one mole of CaF<sub>2</sub> per two moles of F. Caustic does NOT: NaF is soluble, so the caustic route returns zero fluoride precipitated and sets `fluoride_removed_by_base=False`. Treating caustic as fluoride treatment is a discharge-permit error, so it is flagged rather than left implicit.

*charge\_balance*

```
charge_balance(cations: dict[str, Quantity], anions: dict[str, Quantity], h_plus: Quantity | None=None, oh
    \ _minus: Quantity | None=None) -> dict[str, float]
```

Evaluate electroneutrality of a liquor.

`cations` maps element symbol to moles (charge taken from `CATION_CHARGE`); `anions` maps anion name to moles, with the charge magnitude read from `_ANION_CHARGE`. Returns both sides and the residual; a non-zero residual means a species was omitted or double counted.

### *fluoride\_effluent*

```
fluoride_effluent(site: Site, fluoride_moles: Quantity, effluent_volume: Quantity, fluoride_precipitated_moles: Quantity | None=None) -> dict[str, object]
```

Fluoride discharge concentration against the site's permitted limit.

**Raises.** `KeyError` If the site's permitting regime carries no fluoride limit. A fluoride discharge number with no limit to compare against is not a compliance result, and inventing a limit would misrepresent the jurisdiction.

### *reagent\_balance*

```
reagent_balance(feedstock: Feedstock, site: Site, ore_mass: Quantity, acid: Acid, base: Base=Base.LIME, silica_dissolved_fraction: float=0.0, excess_factor: float=1.0, water_mass: Quantity | None=None, silica_hf_route: str='H2SiF6_solution', elements: tuple[str, ...] | None=None) -> dict[str, object]
```

Full reagent, neutralization and mass balance for one leach charge.

`excess_factor` multiplies the stoichiometric acid demand and defaults to 1.0, the stoichiometric floor, which is explicitly not a plant number (LIMITATIONS item 1).

WHAT `mass_balance_relative_residual` IS, AND IS NOT. It is an `IDENTITY`, not a check, and this docstring previously claimed the opposite ("closes the overall mass balance to a relative tolerance of 1e-9 and raises if it does not"). The liquor mass is defined by difference, `m_liquor = m_in - m_product - m_sludge`, and the residual then compares `m_product + m_sludge + m_liquor` against `m_in`, which reduces to `m_in == m_in`. Verified symbolically: the by-difference liquor and the sum of its actual constituents (water, acid, base, dissolved silica, dissolved impurities, less the  $\text{CaF}_2$  leaving as sludge) both simplify to `acid + base - caf2 + imp + si + water`, difference exactly zero. The residual is therefore zero to float rounding on every input, the `AssertionError` below can never fire, and a flowsheet that created or destroyed an element would still report a perfect closure. It is retained only as a guard against a future edit that stops defining the liquor this way.

The closures that CAN fail are per element, and they are asserted in `tests/test_physics_audit.py`: on the HF plus lime route ( $\text{Ca(OH)}_2 + 2 \text{HF} \rightarrow \text{CaF}_2 + 2 \text{H}_2\text{O}$ ) the fluoride precipitated as  $\text{CaF}_2$  cannot exceed the fluoride charged as HF, and the calcium precipitated cannot exceed the calcium charged as lime. Measured at 1000 kg ore, 0.1 percent silica dissolved and `excess_factor` 1.2, both are tight: 122.51325857527 mol F charged and precipitated, 61.256629287636 mol Ca charged and precipitated. A stoichiometry error of the obvious kind (one  $\text{CaF}_2$  per mole of F rather than per two) breaks the fluoride closure while leaving the total-mass residual at zero.

### *reagent\_cost*

```
reagent_cost(site: Site, acid: Acid, acid_mass: Quantity, base: Base | None=None, base_mass: Quantity | None=None) -> dict[str, object]
```

Delivered reagent cost from the SITE's own price list.

No price is hardcoded. Raises through `ae.core.site.ReagentPrices.price` if the site does not carry a price for the reagent, or if the reagent is priced but marked not locally available (HF is the usual case, and its availability is a real constraint rather than a modelling nuisance).

## 13.3 Worked examples, reproduced by execution

### HF demand set by silica attack, not by the assay

```
from ae.core.units import Q_
from ae.physics.reagents import (Acid, Base, M_SiO2, hf_silica_demand,
                                impurity_acid_demand, impurity_moles,
                                neutralization_demand)

f = scenario_feedstock()
ore = Q_(1000.0, "kg")
n_hf, m_diss = hf_silica_demand(ore, 0.001)
n_hf4, _ = hf_silica_demand(ore, 0.001, route="SiF4_gas")
print("M_SiO2", M_SiO2, "kg/mol")
print("silica dissolved at 0.1%:", float(m_diss.to("kg").magnitude), "kg per tonne")
print("n_SiO2", round(1.0 / M_SiO2, 6), "mol")
print("HF, H2SiF6 route (6:1)", round(float(n_hf.to("mol").magnitude), 6), "mol/t")
print("HF, SiF4 route (4:1)", round(float(n_hf4.to("mol").magnitude), 6), "mol/t")
print("ratio of routes", round(float((n_hf4 / n_hf).magnitude), 8))
print()
for el, q in sorted(impurity_moles(f, ore).items()):
    print(f"leachable {el:2s} : {float(q.to('mol').magnitude):10.6f} mol/t")
tot, _per = impurity_acid_demand(f, ore, Acid.HF)
print("all impurities, HF", round(float(tot.to("mol").magnitude), 6), "mol/t")
print("silica / impurity HF", round(float((n_hf / tot).magnitude), 2), "times")
print()
n = neutralization_demand({Acid.HF: Q_(100.0, "mol")}, Base.LIME)
print("100 mol HF, lime", n["base_moles_mol"], "mol =",
      round(n["base_mass_kg"], 4), "kg")
print("CaF2 dry sludge", n["CaF2_moles_mol"], "mol =",
      round(n["CaF2_sludge_dry_kg"], 4), "kg")
print("fluoride removed by base:", n["fluoride_removed_by_base"])
```

Output:

```
M_SiO2 : 0.060083 kg/mol
silica dissolved at 0.1%: 1.0 kg per tonne
n_SiO2 : 16.643643 mol
HF, H2SiF6 route (6:1) : 99.861858 mol/t
HF, SiF4 route (4:1) : 66.574572 mol/t
ratio of routes : 0.66666667

leachable Al : 0.444749 mol/t
leachable B : 0.046253 mol/t
leachable Fe : 0.048348 mol/t
leachable K : 0.143229 mol/t
leachable Li : 0.144092 mol/t
leachable Na : 0.243586 mol/t
leachable Ti : 0.020891 mol/t
all impurities, HF : 2.232524 mol/t
silica / impurity HF : 44.73 times

100 mol HF, lime : 50.0 mol = 3.7046 kg
CaF2 dry sludge : 50.0 mol = 3.9037 kg
fluoride removed by base: True
```

Dissolving one part in a thousand of the silica consumes roughly two orders of magnitude more HF than every impurity cation in the feed combined. Reagent cost is therefore set by silica selectivity, and an HF budget built from the assay alone is wrong by that factor.

## 13.4 Validation status

7 hand-traceable worked example(s) and 2 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against

a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                     | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_benchmark_impurity_load_against_xia_2024 | literature | Acid demand for the Xia et al. 2024 feed impurity load. Xia et al. (2024, doi 10.3390/min14070727) report a feed at 128.86 ug/g total trace impurities. The paper reports NO reagent consumption, so this is not a validation of any reagent figure. What is checked is the mole accounting: distributing 12 10.3390/min14070727                                              |
| test_benchmark_yang_2020_iron_acid_demand     | literature | Acid demand for the iron removed in Yang and Li 2020. Reported: Fe2O3 from 0.0857 to 0.0223 percent, i.e. 634 ppm Fe2O3 removed. On an Fe basis with the factor 111.69/159.687 = 0.699431: (written 0.699435 on BOTH this line and the Fe removed line below it before this revision; the quotient is 0.6994 Error stated: 0.0223 10.1515/http-2020-0081; 10.3390/min14070727 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_golden_impurity_moles_arithmetic
test_golden_hf_silica_demand_arithmetic
test_golden_silica_dominates_hf_demand
test_golden_neutralization_arithmetic
test_golden_diprotic_acid_doubles_neutralization
test_golden_fluoride_concentration_arithmetic
test_golden_acid_molar_masses
```

## CHAPTER 14

# physics/chlorination.py

Module `ae.physics.chlorination`, 775 lines. Chapter text is this module's own documentation.

## 14.1 Purpose, governing equations and limitations

Chlorination roasting: metal chloride formation and volatilization from quartz.

Process being modelled. A quartz concentrate is held at 900 to 1300 degC in a flowing HCl or Cl<sub>2</sub> atmosphere, optionally with a carbonaceous reductant (carbon black, graphite, charcoal) mixed into the charge. Impurity cations react to form metal chlorides, and those chlorides leave the solid if their vapour pressure at the roasting temperature is high enough to sustain the required evaporation flux. Removal therefore requires THREE conditions to hold simultaneously, and this module tests each separately rather than collapsing them into one efficiency:

1. THERMODYNAMIC. The chlorination reaction must be spontaneous,  $\Delta G_r(T) < 0$ . For impurities present as simple oxides in an HCl atmosphere this is usually satisfied for alkalis and iron. For Ti, Al and B it is NOT satisfied without a reductant (see the reductant rule below).
2. VOLATILITY. The chloride must have appreciable vapour pressure at the roasting temperature. This is the step the boiling and sublimation points govern: TiCl<sub>4</sub> boils at 136.4 degC, AlCl<sub>3</sub> sublimates near 180 degC, FeCl<sub>3</sub> sublimates near 315 degC, all far below roasting temperature, whereas NaCl boils at 1465 degC and KCl sublimates near 1500 degC, so the alkali chlorides are only marginally volatile in the roasting window and their removal is slower than a "boiling point" comparison suggests.
3. TRANSPORT. The cation must reach a surface where the chlorinating gas is, which for a lattice-substituted cation is a solid-state diffusion problem handled in `ae.physics.diffusion` and NOT in this module. A species can pass tests 1 and 2 and still not be removed because it cannot get out of the lattice. `removal_screen` reports the three conditions separately so that a pass on volatility is never mistaken for removal.

## THE REDUCTANT RULE

Liu et al. (2026, Minerals 16(8), 836, doi 10.3390/min16080836) state that carbonaceous reductants are indispensable for enabling spontaneous chlorination of substitutional impurities such as Ti, Al and B, while alkali metals (Na, K, Li) can be effectively removed under an HCl atmosphere at moderate temperatures. This module enforces that finding structurally: `removal_screen` returns `thermodynamically_blocked` for Ti, Al and B whenever `reductant_present=False`, irrespective of temperature or gas composition, and `REDUCTANT_REQUIRED` records the rule with its source.

The chemistry behind the rule, for orientation (not itself a claim sourced from this build's accessible literature): direct chlorination of a refractory oxide,



liberates oxygen, and the resulting  $\Delta G_r$  is positive for the strongly bound oxides of Ti, Al and B. Carbon consumes that oxygen as CO or CO<sub>2</sub>,



which adds roughly the free energy of CO formation per oxygen atom removed and turns the reaction spontaneous. `gibbs_of_reaction` computes either form from a supplied thermochemical table, so the sign can be checked rather than asserted, but see THERMOCHEMICAL DATA STATUS: this build ships no table capable of that calculation for Ti, Al or B.

## EQUATIONS

### Symbols

| Symbol           | Unit                                | Valid range                    |
|------------------|-------------------------------------|--------------------------------|
| $p^{sat}$        | Pa                                  | $> 0$                          |
| $p_{ref}$        | Pa                                  | 101325 (1 atm reference)       |
| $T_b$            | K                                   | $> 0$ (normal boiling point)   |
| $\Delta H_{vap}$ | J mol <sup>-1</sup>                 | $> 0$                          |
| $T$              | K                                   | $> 0$                          |
| $\Delta G_r$     | J mol <sup>-1</sup>                 | any sign                       |
| $\Delta H_r$     | J mol <sup>-1</sup>                 | any sign                       |
| $\Delta S_r$     | J mol <sup>-1</sup> K <sup>-1</sup> | any sign                       |
| $J_{evap}$       | mol m <sup>-2</sup> s <sup>-1</sup> | $\geq 0$                       |
| $M$              | kg mol <sup>-1</sup>                | $> 0$                          |
| $\alpha$         | dimensionless                       | (0, 1] evaporation coefficient |

### Clausius-Clapeyron, one-point form

Derivation. The Clapeyron equation for a condensed-to-vapour transition with an ideal vapour and negligible condensed-phase molar volume is  $d \ln p / dT = \Delta H_{vap} / (\mathcal{R} T^2)$ . Treating  $\Delta H_{vap}$  as constant and integrating from the normal boiling (or sublimation) point, where  $p = p_{ref}$  by definition:

$$p^{sat}(T) = p_{ref} \exp \left[ -\frac{\Delta H_{vap}}{\mathcal{R}} \left( \frac{1}{T} - \frac{1}{T_b} \right) \right]$$

Dimensional check:  $\Delta H_{vap} / \mathcal{R}$  has unit K, multiplied by  $1/T$  gives a dimensionless exponent. At  $T = T_b$  the exponent is zero and  $p^{sat} = p_{ref}$ , which is the boundary condition.

This is a two-parameter fit anchored on one measured point. It is accurate near  $T_b$  and degrades far from it, because  $\Delta H_{vap}$  falls with temperature and vanishes at the critical point. At roasting temperature (1200 degC) extrapolated from a boiling point of 136 degC (TiCl<sub>4</sub>) the extrapolation spans a factor of 3.5 in absolute temperature and the computed pressure is far above the critical pressure, which is physically meaningless. `vapour_pressure` therefore CAPS the returned pressure at the system total pressure and sets a `supercritical_extrapolation` flag, and `removal_screen` treats any species whose  $T$  exceeds  $T_b$  as simply "fully volatile" rather than quoting a fictitious pressure. That is the honest use of the relation: it discriminates volatile from non-volatile, and it does not produce a meaningful number for a gas hundreds of degrees above its boiling point.

### Trouton's rule (used only where $\Delta H_{vap}$ is unsourced)

$$\Delta S_{vap} \approx 85 \text{ J mol}^{-1} \text{ K}^{-1} \quad \Rightarrow \quad \Delta H_{vap} \approx 85 T_b$$

An empirical regularity, not a law, and it is poor for associated or ionic liquids. Every value obtained this way is tagged ASSUMED with Trouton named in the basis string, and CHLORIDES marks which entries rely on it.

## Gibbs energy of reaction

$$\Delta G_r(T) = \sum_i \nu_i \Delta H_{f,i}^\circ - T \sum_i \nu_i S_i^\circ$$

with  $\nu_i$  positive for products and negative for reactants. This is the second-law screen:  $\Delta G_r < 0$  is necessary for spontaneity at the stated standard states. Constant- $C_p$  corrections are NOT applied, because applying a Kirchhoff correction with unsourced heat capacities would add apparent rigour without adding information; the omission biases  $\Delta G_r$  by an amount that grows with  $T - 298$  K and is stated in LIMITATIONS.

## Hertz-Knudsen evaporation flux

The maximum molar flux leaving a surface into a vacuum, from kinetic theory:

$$J_{evap} = \alpha \frac{p^{sat} - p_{bulk}}{\sqrt{2\pi MRT}}$$

Dimensional check:  $\text{Pa} / \sqrt{\text{kg mol}^{-1} \text{J mol}^{-1} \text{K}^{-1} \text{K}} = \text{Pa} / \sqrt{\text{kg J mol}^{-2}} = \text{Pa} / (\sqrt{\text{kg}^2 \text{m}^2 \text{s}^{-2} \text{mol}^{-2}}) = \text{Pa mol s kg}^{-1} \text{m}^{-1} = \text{mol m}^{-2} \text{s}^{-1}$  after substituting  $\text{Pa} = \text{kg m}^{-1} \text{s}^{-2}$ . The unit reduction is enforced by a test in `tests/test_chlorination.py` rather than asserted here. This gives an UPPER BOUND on the removal rate: it assumes every molecule that leaves is swept away, so a real roast with a finite gas sweep and a stagnant boundary layer is slower.

## THERMOCHEMICAL DATA STATUS

NIST-JANAF ([janaf.nist.gov](http://janaf.nist.gov)) and the NIST Chemistry WebBook ([webbook.nist.gov](http://webbook.nist.gov)) are both outside this sandbox's network allowlist and returned proxy errors on 2026-09-16. No request for access was made, per the task instruction to log the attempt and continue. Standard formation enthalpies and entropies for the oxide-to-chloride reactions of Al, Ti and B could therefore NOT be sourced. Consequences, all visible in the code rather than papered over:

- THERMO contains only the species whose formation data could be retrieved from PubChem (which aggregates HSDB and other compendia, each carrying its own reference). Entropies are almost entirely absent from that source, so `gibbs_of_reaction` requires the caller to supply a complete table and raises when any species is missing.
- There is NO built-in oxide-to-chloride  $\Delta G_r$  for Al, Ti or B in this build. The reductant requirement is enforced from the Liu et al. (2026) finding as a RULE, not recomputed from thermodynamics.
- CHLORIDES boiling and sublimation points ARE sourced (PubChem), and the volatility screen based on them is the part of this module that rests on retrieved data.

## LIMITATIONS

1. No Gibbs screen for Ti, Al or B from first principles in this build (see above). `gibbs_of_reaction` is working machinery with no data to feed it for those elements. Supplying a JANAF-derived table makes it live.
2. Vapour pressure far above the boiling point is meaningless. The Clausius-Clapeyron extrapolation from 136 degC (TiCl<sub>4</sub>) to 1200 degC is not a physical pressure, and the code caps it and flags it rather than reporting it. Do not read a capped pressure as a measurement.
3. Trouton's rule supplies  $\Delta H_{vap}$  for AlCl<sub>3</sub>, FeCl<sub>3</sub>, NaCl, KCl and LiCl. AlCl<sub>3</sub> vapour is dimeric (Al<sub>2</sub>Cl<sub>6</sub>) near its sublimation point, and the alkali chlorides are ionic melts, both cases where Trouton is known to be unreliable. Those pressures are order-of-magnitude indicators only.

4. Activity coefficients are ignored. Impurities in quartz are dilute, and their thermodynamic activity is not unity, so a screen run at unit activity overstates the driving force for chloride formation. The direction of the error is known (optimistic) but its size is not quantified here.
5. Alkali chloride volatility in the roasting window is marginal, not comfortable: NaCl boils at 1465 degC and KCl sublims near 1500 degC, both ABOVE a 1200 degC roast. Removal proceeds by evaporation below the boiling point, at a rate this module bounds with Hertz-Knudsen but does not calibrate.
6. No kinetics of the surface reaction, no gas-film resistance, no pore diffusion in an agglomerated charge, and no accounting for chloride condensation in a cooler downstream zone (which is how AlCl<sub>3</sub> fouls equipment in practice).
7. Silica chlorination is not modelled. SiCl<sub>4</sub> forms from SiO<sub>2</sub> under aggressive chlorination and represents a yield loss of the product itself, which a complete flowsheet model must include and this module does not.
8. The reductant rule is binary. Liu et al. (2026) report that a reductant is indispensable for Ti, Al and B; this module does not model reductant dosage, carbon type, or the resulting CO/CO<sub>2</sub> ratio, so it cannot tell you HOW MUCH carbon to add.

## References

Liu, L., Liu, H., Li, J., Peng, T., Wang, W. and Wang, F. (2026) Mechanism Study on Deep Removal of Lattice Impurities from High-Purity Quartz by Chlorination Roasting, Minerals 16(8), 836, doi 10.3390/min16080836.

PubChem compound records (National Library of Medicine), accessed 2026-09-16 via the PUG-View REST API: AlCl<sub>3</sub> CID 24012, FeCl<sub>3</sub> CID 24380, TiCl<sub>4</sub> CID 24193, BCl<sub>3</sub> CID 25135, SiCl<sub>4</sub> CID 24816, NaCl CID 5234, KCl CID 4873, LiCl CID 433294. Each property in CHLORIDES carries the specific compendium string PubChem attributes it to.

Xia, M., Yang, X. and Hou, Z. (2024) Preparation of High-Purity Quartz Sand by Vein Quartz Purification and Characteristics: A Case Study of Pakistan Vein Quartz, Minerals 14(7), 727, doi 10.3390/min14070727. Chlorination roasting is one stage of the flowsheet whose endpoint this platform benchmarks against.

## 14.2 Interface

### Types

**PhaseChange.** Whether the 1 atm transition point is a boiling point or a sublimation point.

**ChlorideSpecies.** One metal chloride, with the volatility data needed for the screen.

```
enthalpy(self) -> Quantity
```

Enthalpy of vaporization, falling back to Trouton's rule if unsourced.

**ScreenResult.** Per-element outcome of the three-condition chlorination screen.

### Functions

*trouton\_\_enthalpy*

```
trouton_enthalpy(transition_T: Quantity) -> Quantity
```

Estimate  $\Delta H_{\text{vap}} \approx \Delta S_{\text{Trouton}} T_b$ .

Returns a quantity in J/mol. An estimate, never a measurement: see TROUTON\_ENTROPY for why it is unreliable for these species.

*vapour\_pressure*

```
vapour_pressure(species: ChlorideSpecies, temperature: Quantity, total_pressure: Quantity | None=None) ->
    \ tuple[Quantity, bool]
```

Saturation vapour pressure by the one-point Clausius-Clapeyron relation.

Returns (pressure, capped). capped is True when the raw extrapolation exceeds the system total pressure, in which case the returned value is the total pressure and the species should be treated as fully volatile rather than as having that numeric pressure. See LIMITATIONS item 2: extrapolating hundreds of degrees above the boiling point does not produce a physical pressure.

*gibbs\_of\_reaction*

```
gibbs_of_reaction(stoichiometry: dict[str, float], temperature: Quantity, thermo: dict[str, dict[str, Valu
    \ e]] | None=None) -> Quantity
```

Gibbs energy of reaction from formation enthalpies and absolute entropies.

$\Delta G_r(T) = \sum \nu_i \Delta H_{f,i} - T \sum \nu_i S_i$ , with `stoichiometry` mapping species keys to signed coefficients (positive for products, negative for reactants).

**Raises.** `KeyError` If any species lacks `Hf` or `S` in the table. The error names the measurement that would supply it. No species is silently skipped and no value is estimated, because a Gibbs sign computed from a partial table is worse than no answer: it looks authoritative and is arbitrary.

*hertz\_knudsen\_flux*

```
hertz_knudsen_flux(species: ChlorideSpecies, temperature: Quantity, partial_pressure_bulk: Quantity | None
    \ =None, evaporation_coefficient: float=1.0, total_pressure: Quantity | None=None) -> Quantity
```

Maximum molar evaporation flux from the Hertz-Knudsen relation.

$$J = \alpha(p^{\text{sat}} - p_{\text{bulk}}) / \sqrt{2\pi M R T}.$$

An UPPER BOUND on the removal rate: it assumes free molecular escape with no boundary-layer resistance, so a real roast is slower. Returns  $\text{mol m}^{-2} \text{s}^{-1}$ .

*requires\_reductant*

```
requires_reductant(element: str) -> bool
```

Whether chlorination of `element` needs a carbonaceous reductant.

True for Ti, Al and B, per Liu et al. (2026). See REDUCTANT\_REQUIRED.

*removal\_screen*

```
removal_screen(element: str, temperature: Quantity, reductant_present: bool, total_pressure: Quantity | No
    \ ne=None, volatility_margin: float=1.0) -> ScreenResult
```

Screen one element against the thermodynamic and volatility conditions.

Transport (condition 3) is NOT assessed here: a lattice-substituted cation must first reach a gas-accessible surface, which is a solid-state diffusion question handled by `ae.physics.diffusion`. `transport_assessed_here` is always `False` to keep that gap explicit in the returned record.

`volatility_margin` is the ratio of temperature to the chloride's 1 atm transition temperature required to call the chloride volatile. The default of 1.0 means the roast must at least reach the boiling or sublimation point.

*screen\_feedstock*

```
screen_feedstock(feedstock: Feedstock, temperature: Quantity, reductant_present: bool, elements: tuple[str
\ , ...] | None=None, total_pressure: Quantity | None=None) -> dict[str, ScreenResult]
```

Run `removal_screen` for every measured impurity of feedstock.

Elements default to those actually assayed in the feedstock's impurity profile and tabulated in `CHLORIDES`, so an unassayed element produces no result rather than a default one.

## 14.3 Worked examples, reproduced by execution

The three-condition removal screen at 1200 degC, with and without a reductant

```
from ae.core.units import Q_
from ae.physics.chlorination import (CHLORIDES, hertz_knudsen_flux, removal_screen,
                                     requires_reductant, trouton_enthalpy, vapour_pressure)

T = Q_(1200.0, "degC")
print(f"{'el':>3s} {'reductant':>9s} {'thermo ok':>10s} {'volatile':>9s} {'T/Tb':>7s}")
for el in ("Na", "K", "Fe", "Al", "Ti", "B"):
    for red in (False, True):
        r = removal_screen(el, T, reductant_present=red)
        print(f"{'el':>3s} {'str(red):>9s} {'str(not r.thermodynamically_blocked):>10s} "
              f"{'str(r.volatile):>9s} {'r.T_over_Tb:7.2f}")
print()
r = removal_screen("Al", T, reductant_present=False)
print("Al verdict without C      :", r.verdict[:96])
print()
p, capped = vapour_pressure(CHLORIDES["TiCl4"], Q_(100.0, "degC"))
print("TiCl4 p_sat at 100 degC :", round(float(p.to("Pa").magnitude), 2), "Pa =",
      round(float(p.to("Pa").magnitude) / 101325.0, 6), "atm; capped:", capped)
p2, capped2 = vapour_pressure(CHLORIDES["TiCl4"], T)
print("TiCl4 p_sat at 1200 degC:", round(float(p2.to("Pa").magnitude), 2), "Pa; capped:", capped2)
j = hertz_knudsen_flux(CHLORIDES["NaCl"], Q_(1465.0, "degC"))
print("NaCl flux at its Tb      :", round(float(j.to("mol/(m**2*s)").magnitude), 2),
      "mol m^-2 s^-1")
print("Trouton dHvap at 1738 K :",
      round(float(trouton_enthalpy(Q_(1738.15, "K")).to("kJ/mol").magnitude), 4), "kJ/mol")
print("requires_reductant Al/Na:", requires_reductant("Al"), requires_reductant("Na"))
```

Output:

| el | reductant | thermo ok | volatile | T/Tb |
|----|-----------|-----------|----------|------|
| Na | False     | True      | False    | 0.85 |
| Na | True      | True      | False    | 0.85 |
| K  | False     | True      | False    | 0.83 |
| K  | True      | True      | False    | 0.83 |
| Fe | False     | True      | True     | 2.50 |
| Fe | True      | True      | True     | 2.50 |
| Al | False     | False     | True     | 3.25 |
| Al | True      | True      | True     | 3.25 |

```

Ti      False      False      True      3.60
Ti      True       True       True      3.60
B       False      False      True      5.16
B       True       True       True      5.16

```

```

Al verdict without C      : NOT REMOVED: thermodynamically blocked. AlCl3 would be volatile at this tempera
\ ture (T/Tb = 3.25

```

```

TiCl4 p_sat at 100 degC : 35874.24 Pa = 0.354051 atm; capped: False
TiCl4 p_sat at 1200 degC: 101325.0 Pa; capped: True
NaCl flux at its Tb      : 1390.91 mol m^-2 s^-1
Trouton dHvap at 1738 K : 147.7428 kJ/mol
requires_reductant Al/Na: True False

```

Ti, Al and B come back thermodynamically blocked without a reductant at every temperature, which is the Liu et al. (2026) finding enforced structurally. The 1200 degC TiCl<sub>4</sub> pressure is returned capped at the system total pressure, because a Clausius-Clapeyron extrapolation that far above the boiling point is not a physical pressure.

## 14.4 Validation status

5 hand-traceable worked example(s) and 2 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                              | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_benchmark_chloride_boiling_points</code>                    | literature | Tabulated transition points against the values stated in the task brief. The brief states AlCl <sub>3</sub> boils at 180 degC, FeCl <sub>3</sub> sublimates near 315 degC and TiCl <sub>4</sub> boils at 136 degC. The module's values come independently from PubChem property records. The error between the two is reported per species.<br>10.3390/min14070727; 10.3390/min16080836;<br>10.3390/min16080836 |
| <code>test_benchmark_alkali_chlorides_are_not_volatile_at_roast</code> | literature | Alkali chloride volatility at a 1200 degC roast, against their boiling points. The quantitative claim being checked: NaCl (1465 degC), KCl (sublimes 1500 degC) and LiCl (1360 degC) all have 1 atm transition points ABOVE a 1200 degC roast, so a naive "boiling point below roast temperature" screen would fail.<br>10.3390/min14070727; 10.3390/min16080836;<br>10.3390/min16080836                        |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```

test_golden_clausius_clapeyron_at_boiling_point
test_golden_ticl4_vapour_pressure_below_boiling
test_golden_trouton_arithmetic
test_golden_gibbs_arithmetic
test_golden_hertz_knudsen_arithmetic

```

## CHAPTER 15

# physics/diffusion.py

Module `ae.physics.diffusion`, 1135 lines. Chapter text is this module's own documentation.

## 15.1 Purpose, governing equations and limitations

Solid-state diffusion in quartz, and the purity ceiling it imposes.

Purpose: determine quantitatively, and for each process temperature window separately, whether lattice-substituted impurities can be removed from a quartz grain in a practical residence time. The controlling comparison is the diffusion length  $\sqrt{Dt}$  against the grain radius: when the former is orders of magnitude smaller, only a surface skin is depleted and the grain retains its lattice inventory.

### WHAT THIS MODULE ACTUALLY ESTABLISHES, AND WHAT IT DOES NOT

The result is regime dependent, and this module refuses to overstate it. Under the most generous mobility bound that any accessible source supports for a cation in the SiO<sub>2</sub> lattice (MOST\_MOBILE\_BOUND:  $D_0 = 10^{-4}$  m<sup>2</sup> s<sup>-1</sup>,  $E_a = 90$  kJ mol<sup>-1</sup>, the low end of the Liu et al. 2026 range and therefore an overestimate of mobility for substitutional Al), for a 100 micrometre grain radius:

- ATMOSPHERIC-PRESSURE ACID LEACHING (up to about 100 degC): ESTABLISHED INFEASIBLE. At 80 degC and 6 h the diffusion length is 0.325  $\mu$ m against a 100  $\mu$ m radius, a factor of 308 (2.49 orders of magnitude). At 95 degC and 24 h it is 1.21  $\mu$ m, a factor of 82. Since this is computed with an overestimate of mobility, the true deficit is larger. Conclusion robust.
- CHLORINATION ROASTING, Ti SPECIFICALLY: ESTABLISHED INFEASIBLE. Using the Ti4+ activation energy Liu et al. (2026) report for lattice diffusion in SiO<sub>2</sub> (about 400 kJ mol<sup>-1</sup>), at 1200 degC and 6 h the diffusion length is 119 nm for  $D_0 = 10^{-4}$  and 0.12 nm for  $D_0 = 10^{-10}$ , i.e. a deficit of 2.9 to 5.9 orders of magnitude across the entire plausible prefactor range. Fractional extraction from the sphere is 4.0e-3. Conclusion robust because it does not depend on the unknown prefactor.
- HOT PRESSURE ACID LEACHING (200 to 300 degC) and CHLORINATION ROASTING FOR Al: NOT ESTABLISHED BY THIS MODEL. At 250 degC and 6 h the generous bound gives a diffusion length of 47  $\mu$ m against a 100  $\mu$ m radius, a factor of only 2.1, and at 300 degC it exceeds the radius. At 1200 degC and 6 h the activation energy at which the diffusion length equals the grain radius is 65.8 kJ mol<sup>-1</sup> at the low end of the prefactor sweep ( $D_0 = 10^{-10}$  m<sup>2</sup> s<sup>-1</sup>) and 235.1 kJ mol<sup>-1</sup> at the high end ( $D_0 = 10^{-4}$  m<sup>2</sup> s<sup>-1</sup>) (critical\_activation\_energy), and the true barrier for Al is unmeasured: it lies somewhere between the Na+ value (about 90 kJ mol<sup>-1</sup>) and the Ti4+ value (about 400 kJ mol<sup>-1</sup>) reported by Liu et al. (2026), which straddles the threshold. A diffusion-length argument therefore CANNOT decide the Al case at roasting temperature, and this module says so rather than choosing a prefactor that produces the desired answer.

The platform's conclusion that lattice Al sets the purity ceiling consequently rests on the EMPIRICAL result, not on this diffusion calculation: Xia et al. (2024, doi 10.3390/min14070727) took a vein quartz from 128.86  $\mu$ g/g total trace impurities to 24.23  $\mu$ g/g (81.20 percent removal) through crushing, ultrasonic desliming, flotation, calcination, water quenching, hot pressure acid leaching AND chlorination roasting, and identified the residual as lattice-bound Al, Ti and Li. The diffusion model explains why the leaching stages cannot reach the lattice and why Ti survives the roast; it does not independently prove that Al survives a roast, and the Liu et al. (2026) abstract in fact announces an alkali-first, Al-follows coupled

diffusion mechanism for aluminium removal plus a temperature-staged, atmosphere-segmented roasting strategy, which would be incoherent if Al diffusion at roasting temperature were negligible.

## EQUATIONS

### Fick's laws

Fick's first law, one dimension:

$$J = -D \frac{\partial C}{\partial x}$$

Fick's second law, with  $D$  independent of concentration:

$$\frac{\partial C}{\partial t} = D \frac{\partial^2 C}{\partial x^2}$$

Symbols, units, valid ranges:

| Symbol             | Unit                              | Valid range                 |
|--------------------|-----------------------------------|-----------------------------|
| $J$                | $\text{mol m}^{-2} \text{s}^{-1}$ | any sign                    |
| $C$                | $\text{mol m}^{-3}$               | $\geq 0$                    |
| $D$                | $\text{m}^2 \text{s}^{-1}$        | $> 0$                       |
| $x$                | m                                 | $\geq 0$                    |
| $t$                | s                                 | $\geq 0$                    |
| $L$                | m                                 | $\geq 0$ (diffusion length) |
| $R_{\text{grain}}$ | m                                 | $> 0$                       |
| $D_0$              | $\text{m}^2 \text{s}^{-1}$        | $> 0$ (pre-exponential)     |
| $E_a$              | $\text{J mol}^{-1}$               | $> 0$                       |
| $T$                | K                                 | $> 0$                       |
| $\mathcal{R}$      | $\text{J mol}^{-1} \text{K}^{-1}$ | 8.314462618 (CODATA 2018)   |
| Fo                 | dimensionless                     | $\geq 0$ (Fourier number)   |

### Arrhenius diffusivity

$$D(T) = D_0 \exp\left(-\frac{E_a}{\mathcal{R}T}\right)$$

Dimensional check:  $[D_0] = [D] = \text{m}^2 \text{s}^{-1}$ , and the exponent is dimensionless because  $E_a/(\mathcal{R}T)$  is  $(\text{J mol}^{-1}) / (\text{J mol}^{-1} \text{K}^{-1} \text{K})$ .

### Diffusion length

Definition used here:

$$L = \sqrt{Dt}$$

Derivation of why this is the right scale, not an arbitrary one. For a semi-infinite solid initially at  $C_0$  held at  $C_s = 0$  at the surface (the leaching boundary condition: reagent removes everything that reaches the surface), Fick's second law has the exact similarity solution

$$\frac{C(x, t)}{C_0} = \text{erf}\left(\frac{x}{2\sqrt{Dt}}\right)$$

so the concentration is depleted to 52.05 percent of its initial value at  $x = \sqrt{Dt}$  (since  $\text{erf}(1/2) = 0.5205$ ), and to within 1 percent of its initial value by  $x = 3.64\sqrt{Dt}$ . The depleted skin is therefore a few multiples

of  $\sqrt{Dt}$  thick, whatever prefactor convention is used. Some texts define  $L = 2\sqrt{Dt}$ ; the factor does not change any conclusion below by more than a factor of two, while the quantities compared differ by ten or more orders of magnitude.

### Fractional extraction from a sphere

For a sphere of radius  $R$  with zero surface concentration, the exact series solution of Fick's second law gives the fraction extracted. It is DERIVED here rather than quoted: substituting  $u = rC$  turns the spherical diffusion equation into the one-dimensional heat equation  $\partial u / \partial t = D \partial^2 u / \partial r^2$  with  $u(0, t) = u(R, t) = 0$ , whose eigenfunctions are  $\sin(n\pi r/R)$  with eigenvalues  $n^2\pi^2 D/R^2$ . Expanding the uniform initial condition  $u(r, 0) = r$  in that basis gives Fourier coefficients  $2R(-1)^{n+1}/(n\pi)$ , and integrating  $4\pi \int_0^R ru dr$  over the resulting series yields

$$\frac{M_t}{M_\infty} = 1 - \frac{6}{\pi^2} \sum_{n=1}^{\infty} \frac{1}{n^2} \exp\left(-\frac{n^2\pi^2 Dt}{R^2}\right)$$

which depends only on the Fourier number  $\text{Fo} = Dt/R^2$ . This form is standard and appears in Crank's monograph (see References), but the specific equation number in that edition could not be checked from this sandbox, so no equation number is cited. Instead the series is verified numerically against an independent finite-difference solution of the same PDE in `test_sphere_series_matches_finite_difference_solution`, which agrees to better than 0.01 percent at  $\text{Fo} = 1\text{e-}3$ . The short-time limit of the same solution is

$$\frac{M_t}{M_\infty} \approx \frac{6}{\sqrt{\pi}} \sqrt{\text{Fo}} - 3 \text{Fo}$$

The series is exact. The second expression is the first TWO terms of the short-time expansion, which is a different statement: the leading term alone is an upper bound that exceeds the series by 8.870e-04 relative at  $\text{Fo} = 10^{-6}$ , 8.942e-03 at  $10^{-4}$  and 9.724e-02 at  $10^{-2}$ , while the two-term form reproduces the series to better than 4e-14 relative over  $10^{-6}$  to  $10^{-2}$ . An earlier revision of this module used the leading term alone below  $\text{Fo} = 10^{-4}$  and described it as the exact short-time limit, which left the function non-monotonic in  $\text{Fo}$ : extraction FELL by 2.36e-04 across the switch, and the 0.89 percent quoted there as agreement between the two forms was the expansion's own truncation error.

The switch at  $\text{Fo} = 10^{-4}$  is retained because the series converges too slowly below it to truncate safely: the summand decays as  $\exp(-n^2\pi^2\text{Fo})$ , so the number of terms needed scales as  $\text{Fo}^{-1/2}$ , and a fixed 200-term truncation overstates the extraction by 23 percent at  $\text{Fo} = 10^{-6}$  (0.004158 truncated against 0.003382 converged). Above the switch the term count is chosen adaptively from  $\text{Fo}$  so the neglected tail is below 1e-12.

### Limiting grain size

Setting  $L = R_{\text{grain}}$  and solving for  $R$  gives the largest grain that can be depleted in time  $t$ :

$$R_{\text{max}}(T, t) = \sqrt{D(T)t}$$

$R_{\text{max}}$  is the radius at which the diffusion length equals the radius, which is a scale rather than a retention threshold: at  $R = 2R_{\text{max}}$  this module's sphere solution still gives 0.9484 extraction, and 0.3085 at  $R = 10R_{\text{max}}$ . Retention is near-total only at  $R \sim 100R_{\text{max}}$  (0.0336) and beyond. `limiting_grain_radius` reports this length, and its value is strongly temperature dependent, which is why the verdict in this module is regime dependent rather than universal. Evaluated with `MOST_MOBILE_BOUND` ( $D_0 = 10^{-4} \text{ m}^2 \text{ s}^{-1}$ ,  $E_a = 90 \text{ kJ mol}^{-1}$ ),  $R_{\text{max}}$  is 0.33  $\mu\text{m}$  at 80 degC and 6 h, but 3.0 mm at 600 degC and 6 h and 75 mm at 1200 degC and 24 h, the last two exceeding any real grain. Those high-temperature figures are NOT a prediction that lattice cations are mobile in a roast: they follow from deliberately assigning the most mobile species' barrier (90 kJ mol<sup>-1</sup>, interstitial Na+) to every cation. They show that the generous

bound carries no information above about 200 degC, which is why `lattice_removal_verdict` returns `UNDECIDED_BY_DIFFUSION` there and why the barrier-threshold inversion in `critical_activation_energy` is the usable tool at roasting temperature. See `diffusion_length_table` for the computed grid.

## DIFFUSIVITY DATA STATUS

This module deliberately ships NO element-specific diffusivity for quartz. The measurements that would supply them were unreachable from this sandbox on 2026-09-16 and fabricating the two Arrhenius constants would make the central result unverifiable:

- Al: Pankrath (1994) Kinetics of Al-Si exchange in low and high quartz: calculation of Al diffusion coefficients, *European Journal of Mineralogy* 6(4), 435-457, doi 10.1127/ejm/6/4/0435. Paywalled, no open-access copy via Unpaywall, Semantic Scholar or Europe PMC, publisher landing page served no machine-readable text.
- Ti: Cherniak, Watson and Wark (2007) Ti diffusion in quartz, *Chemical Geology* 236(1-2), 65-74, doi 10.1016/j.chemgeo.2006.09.001. Paywalled. The one open-access paper citing it that could be retrieved (Gualda et al. 2018, *Science Advances*, doi 10.1126/sciadv.aap7567) states only that the activation-energy uncertainty is below 5 percent 1-sigma, without reproducing  $D_0$  or  $E_a$ .

What IS retrievable is the activation-energy RANGE for lattice diffusion in SiO<sub>2</sub> from the abstract of Liu et al. (2026, doi 10.3390/min16080836): about 90 kJ mol<sup>-1</sup> for Na+ to about 400 kJ mol<sup>-1</sup> for Ti4+. That range, with a swept prefactor, is enough to bound the problem, and bounding is what this module does:

- `DIFFUSIVITY_STATUS` records per element whether a diffusivity exists here and, when it does not, the specific measurement that would supply it.
- `diffusivity` raises `ae.core.provenance.MissingValueError` for any element whose pair is `MISSING`. There is no default.
- `MOST_MOBILE_BOUND` and `TI_LATTICE_BOUND` are explicitly `ASSUMED` bracketing pairs, tagged with their basis, that must be passed in deliberately so the assumption is visible at the call site.
- `critical_activation_energy` inverts the comparison: instead of asking what the diffusion length is for an unknown pair, it reports the activation energy at which the diffusion length would equal the grain radius. That threshold is compared against the known Na-to-Ti range, and `lattice_removal_verdict` returns `ROBUST_INFEASIBLE` only when the threshold lies below the whole plausible range.

## LIMITATIONS

1. No sourced diffusivity for any element. Every number this module produces comes from a user-supplied pair or from an explicitly `ASSUMED` bracketing pair. Do not quote a bracketing value as a measured diffusivity.
2. The Al case at roasting temperature is `UNDECIDED` here (see above). Any claim that a chlorination roast cannot move lattice Al must be supported by the empirical endpoint data (Xia et al. 2024) or by a measured Al diffusivity, not by this module. `lattice_removal_verdict` returns `UNDECIDED_BY_DIFFUSION` for that case by design, and a caller that treats that verdict as infeasibility is misusing it.
3. Hot pressure acid leaching above about 200 degC is likewise undecided by the generous bound. The atmospheric-leach conclusion does not transfer to an autoclave.
4. Lattice diffusion only. Grain-boundary, dislocation and fracture-network transport are faster by orders of magnitude, which is precisely why NON-lattice impurities (fluid inclusions, grain-boundary films, mineral inclusions) ARE removable by calcination, quenching, flotation and leaching. This module says

nothing about those, and any claim that a purification route fails because of this calculation must first establish that the impurity in question is lattice-bound.

5. The alpha-to-beta transition at 573 degC (1 bar) changes the structure, so a single Arrhenius pair fitted below the transition should not be extrapolated above it; `ArrheniusDiffusivity.at` warns outside the recorded calibration interval. The Liu et al. (2026) abstract also reports a diffusion crossover effect, with mobility gains above 1200 degC for high-activation-energy impurities, which would further invalidate a single-pair extrapolation across that temperature; the full text stating its magnitude was not retrievable here, so it is noted and not parameterised.
6. Charge coupling is not modelled. Al<sup>3+</sup> substituting for Si<sup>4+</sup> requires a charge compensator (H<sup>+</sup>, Li<sup>+</sup>, Na<sup>+</sup>, K<sup>+</sup>), so Al mobility is coupled to alkali mobility, and a single-species Fick treatment of Al is therefore an approximation whose error direction is not established here. The Liu et al. (2026) abstract states that an alkali-first, Al-follows coupled diffusion mechanism is elucidated for aluminium removal, which is consistent with that concern; the mechanism's quantitative form is in the unretrievable full text, so it is reported as a limitation rather than parameterised.
7. Electric-field-driven transport is excluded. Harris (1937, doi 10.1021/j150386a004) measured Li transport through quartz UNDER AN APPLIED FIELD, which is electromigration, not self-diffusion, and its rate is not transferable to a field-free leach or roast.
8. Cracks dominate real grains. A grain with a through-going fracture presents a far shorter diffusion path to its interior than a sound grain of the same size. The infeasibility conclusion applies to sound lattice volume; the platform's calcination and quenching models handle crack generation.

## References

- Crank, J. (1975) *The Mathematics of Diffusion*, 2nd edition, Oxford University Press. Standard reference for the sphere-extraction series solution. NO equation number is cited: the book could not be consulted from this sandbox, so the series is derived from first principles in the section above and verified numerically against a finite-difference solution rather than taken on the authority of a page reference that was not checked.
- Liu, L., Liu, H., Li, J., Peng, T., Wang, W. and Wang, F. (2026) Mechanism Study on Deep Removal of Lattice Impurities from High-Purity Quartz by Chlorination Roasting, *Minerals* 16(8), 836, doi 10.3390/min16080836. Activation energies for solid-state lattice diffusion of Na<sup>+</sup> (about 90 kJ/mol) to Ti<sup>4+</sup> (about 400 kJ/mol); carbonaceous reductant indispensable for Ti, Al, B.
- Xia, M., Yang, X. and Hou, Z. (2024) Preparation of High-Purity Quartz Sand by Vein Quartz Purification and Characteristics: A Case Study of Pakistan Vein Quartz, *Minerals* 14(7), 727, doi 10.3390/min14070727.
- Pankrath, R. (1994) Kinetics of Al-Si exchange in low and high quartz: calculation of Al diffusion coefficients, *European Journal of Mineralogy* 6(4), 435-457, doi 10.1127/ejm/6/4/0435. UNREACHABLE from this sandbox; cited here as the measurement that would supply the Al Arrhenius pair.
- Cherniak, D. J., Watson, E. B. and Wark, D. A. (2007) Ti diffusion in quartz, *Chemical Geology* 236(1-2), 65-74, doi 10.1016/j.chemgeo.2006.09.001. UNREACHABLE from this sandbox; cited as the measurement that would supply the Ti pair.
- Tiesinga, E., Mohr, P. J., Newell, D. B. and Taylor, B. N. (2021) CODATA Recommended Values of the Fundamental Physical Constants: 2018, *Reviews of Modern Physics* 93(2), article 025010, doi 10.1103/RevModPhys.93.025010 (also published as *Journal of Physical and Chemical Reference Data* 50(3), article 033105, doi 10.1063/5.0064853). The source of the in-text "CODATA 2018" label on the molar gas constant. The 2021 publication year is the journal record for the 2018 adjustment, which is the name the adjustment is known by. The value 8.314462618 J mol<sup>-1</sup> K<sup>-1</sup> is EXACT by the 2019 SI redefinition, being the product of the two defining constants  $k = 1.380649 \times 10^{-23}$  J K<sup>-1</sup> and  $N_A = 6.02214076 \times 10^{23}$  mol<sup>-1</sup>, and it is reproduced by that multiplication in this module's tests rather than taken on the authority of the citation.

Gualda, G. A. R., Gravelly, D. M., Connor, M., Hollmann, B., Pamukcu, A. S., Begue, F., Ghiorso, M. S. and Deering, C. D. (2018) Climbing the crustal ladder: Magma storage-depth evolution during a volcanic flare-up, *Science Advances* 4(10), article eaap7567, doi 10.1126/sciadv.aap7567. Open access, full text retrieved and read. The only reachable open-access paper citing Cherniak et al. 2007 for Ti diffusion in quartz. It states that the uncertainty in the activation energy is below 5 percent 1-sigma and does NOT reproduce  $D_0$  or  $E_a$ , which is why it cannot supply the Ti Arrhenius pair. THIS CITATION WAS PREVIOUSLY ATTRIBUTED TO "Barker et al. 2018", written from memory; Barker is not an author of this paper and no Barker 2018 paper matching the description could be resolved.

## 15.2 Interface

### Types

**ArrheniusDiffusivity.** Arrhenius pair for a solid-state diffusivity, with its calibration domain.

```
at(self, temperature: Quantity) -> Quantity
```

Evaluate  $D(T)$ , warning outside the calibration interval.

**Verdict.** Outcome of a diffusion-based feasibility test on lattice impurity removal.

### Functions

*diffusivity*

```
diffusivity(element: str, temperature: Quantity, pair: ArrheniusDiffusivity | None=None) -> Quantity
```

Lattice diffusivity of element in quartz at temperature.

**Parameters.** element Element symbol as used in DIFFUSIVITY\_STATUS. temperature Absolute temperature as a pint Quantity. pair Arrhenius pair to use. When None, the module looks the element up in DIFFUSIVITY\_STATUS and raises if it is MISSING there.

**Raises.** `ae.core.provenance.MissingValueError` When no diffusivity is available for the element. The error names the measurement that would supply it. There is deliberately no fallback default: an invented diffusivity would make the central negative result of this platform unverifiable.

*diffusion\_length*

```
diffusion_length(d: Quantity, t: Quantity) -> Quantity
```

Characteristic diffusion length  $L = \sqrt{Dt}$ .

At  $x = L$  the semi-infinite erf solution has depleted the initial concentration to  $\text{erf}(1/2) = 0.5205$  of its value, so  $L$  is the thickness of the substantially depleted skin.

*fourier\_number*

```
fourier_number(d: Quantity, t: Quantity, radius: Quantity) -> float
```

Fourier number  $Fo = Dt/R^2$ , the only group that matters.

Fractional extraction from a sphere is a function of  $Fo$  alone, so a small  $Fo$  means negligible extraction irrespective of how the individual  $D$ ,  $t$  and  $R$  were obtained.

*fractional\_extraction\_sphere*

```
fractional_extraction_sphere(fo: float, n_terms: int | None=None) -> float
```

Fraction of a diffusing species extracted from a sphere at Fourier number *fo*.

$M_t/M_\infty = 1 - (6/\pi^2) \sum n^{-2} \exp(-n^2\pi^2\text{Fo})$ , derived in the module docstring by eigenfunction expansion and verified against a finite-difference solution of the same PDE.

For *fo* below `FO_SHORT_TIME_SWITCH` the two-term short-time expansion  $(6/\sqrt{\pi})\sqrt{\text{Fo}} - 3\text{Fo}$  is used instead, because the series summand decays as  $\exp(-n^2\pi^2\text{Fo})$  and the term count needed grows as  $\text{Fo}^{-1/2}$ : a fixed truncation silently overstates extraction at small *fo*.

The second term is not optional. The leading term alone is the ASYMPTOTE, not the solution: it exceeds the series by 8.870e-04 relative at  $\text{Fo} = 10^{-6}$  and 8.942e-03 at  $\text{Fo} = 10^{-4}$ , always high, so using it below the switch and the series above made this function non-monotonic in *fo* at the switch. With the  $-3\text{Fo}$  term restored the two branches agree to better than 1e-13 relative over the whole switch neighbourhood, which is what `tests/test_physics_audit.py` pins.

Above the switch, *n\_terms* defaults to the adaptive count  $\lceil \sqrt{28/(\pi^2\text{Fo})} \rceil$  (floored at 50), which makes the first neglected term smaller than 1e-12, and the neglected tail is asserted to be negligible rather than assumed.

**Parameters.** *fo* Fourier number  $Dt/R^2$ , non-negative. *n\_terms* Override the adaptive term count. Passing a small value on purpose is how `test_fixed_truncation_would_overstate_extraction` in `tests/test_diffusion.py` demonstrates the truncation error the adaptive rule exists to avoid.

*limiting\_grain\_radius*

```
limiting_grain_radius(d: Quantity, t: Quantity) -> Quantity
```

Grain radius at which the diffusion length equals the radius:  $\sqrt{Dt}$ .

This is a SCALE, not a threshold, and the distinction was previously stated the wrong way round here ("a grain coarser than this retains essentially all of its lattice inventory"). Computed from this module's own `fractional_extraction_sphere` at  $\text{Fo} = 1/k^2$ , a grain of radius  $k\sqrt{Dt}$  loses 0.9484 of its inventory at  $k = 2$ , 0.7951 at  $k = 3$ , 0.5570 at  $k = 5$  and 0.3085 at  $k = 10$ . Retention becomes near-total only two to three orders of magnitude out: 0.0336 extraction at  $k = 100$  and 0.003382 at  $k = 1000$ .

Use this value to bracket a verdict, not to make one. The extraction test in `lattice_removal_verdict` calls the sphere solution directly at the actual grain radius rather than comparing against this length.

*time\_to\_deplete\_grain*

```
time_to_deplete_grain(d: Quantity, radius: Quantity, target_extraction: float=0.5) -> Quantity
```

Time to reach *target\_extraction* from a sphere of radius *radius*.

Inverts `fractional_extraction_sphere` in *Fo* by bisection (the function is strictly increasing in *Fo*), then returns  $t = \text{Fo} R^2/D$ . Reported in seconds; convert for readability. For lattice Al at process temperatures the answer exceeds geological timescales, which is the point.

*erfc\_profile*

```
erfc_profile(d: Quantity, t: Quantity, depths: Quantity) -> np.ndarray
```

Normalised concentration  $C/C_0 = \text{erf}(x/(2\sqrt{Dt}))$ .

Semi-infinite solid with the surface held at zero concentration: the leaching boundary condition, where reagent removes whatever reaches the surface. Returns the RETAINED fraction at each depth, so it rises from 0 at the surface to 1 in the interior.

### *diffusion\_length\_table*

```
diffusion_length_table(pair: ArrheniusDiffusivity, temperatures_C: tuple[float, ...]=(80.0, 200.0, 600.0,
\ 1000.0, 1200.0), times_h: tuple[float, ...]=(1.0, 6.0, 24.0), grain_radius: Quantity | None=None) ->
\ list[dict[str, float]]
```

Diffusion length, Fourier number and sphere extraction over a T and t grid.

Returns one row per (temperature, time) with the diffusion length in metres and in nanometres, the ratio of that length to the grain radius, the Fourier number and the fractional extraction from a sphere. This is the table that demonstrates the negative result numerically rather than asserting it.

### *critical\_activation\_energy*

```
critical_activation_energy(d0: Quantity, grain_radius: Quantity, temperature: Quantity, residence_time: Qu
\ antity) -> Quantity
```

Activation energy at which the diffusion length equals the grain radius.

Inverts  $\sqrt{D_0 \exp(-E_a/RT)} t = R$  to give

$$E_a^{crit} = -RT \ln \left( \frac{R^2}{D_0 t} \right)$$

A species whose true barrier EXCEEDS  $E_a^{crit}$  cannot be depleted from the grain in time  $t$ ; one below it can. This inversion is the honest way to use an unmeasured diffusivity: the threshold is compared against the known activation-energy range (LIU\_2026\_EA\_RANGE\_KJ) instead of guessing a value inside it.

**Raises.** `ValueError` If  $R^2/t \geq D_0$ , in which case no positive barrier makes the grain depletable and the question is prefactor limited rather than barrier limited.

### *lattice\_removal\_verdict*

```
lattice_removal_verdict(feedstock: Feedstock, element: str, grain_radius: Quantity, temperature: Quantity,
\ residence_time: Quantity, pair: ArrheniusDiffusivity | None=None, ea_range: Quantity | None=None, e
\ xtraction_threshold: float=0.05) -> dict[str, object]
```

Decide whether lattice removal of `element` is excluded on transport grounds.

Method. Two independent checks, and `Verdict.ROBUST_INFEASIBLE` is returned only when BOTH hold:

1. Fractional extraction from the sphere is below `extraction_threshold`.
2. The critical activation energy (`critical_activation_energy`) at BOTH ends of the prefactor sweep `D0_SWEEP_M2_S` lies below the LOW end of the barrier interval for this element, so no combination of the unmeasured constants within that interval permits depletion.

The barrier interval is taken from `ea_range` when given, else from the activation energy of the supplied pair treated as a point value, else from `LIU_2026_EA_RANGE_KJ` (90 to 400 kJ/mol) when the element's barrier is unknown. That ordering is what makes the Ti case decidable (its barrier is sourced at about 400 kJ/mol) while the Al case is not (its barrier is unmeasured and the 90 to 400 interval straddles the threshold).

On why `extraction_threshold` defaults to 0.05 and not to something smaller. For a thin depleted skin the extracted fraction from a sphere is  $(6/\sqrt{\pi})L/R = 3.3851 L/R$  to leading order (the coefficient is  $6/\sqrt{\pi}$ , not 3; the volume-to-surface ratio 3 is the naive guess and it understates extraction by 11.3 percent). At a 300-fold deficit in diffusion length (80 degC, 6 h, 100 um grain) the sphere solution gives 0.011250 extraction against the 0.010000 the coefficient 3 would give, so the geometric skin term never vanishes. A threshold of a few percent is therefore the meaningful one, and the process consequence is reported as `ppm_removable_from_lattice` so the number can be compared against a ppm-level product specification rather than against an abstract cutoff.

When check 1 fails or check 2 fails and no measured pair was supplied, the verdict is `Verdict.UNDECIDED_BY_DIFFUSION`: the generous bound is uninformative at these conditions and empirical data must settle the case. A caller that reports that verdict as infeasibility is misusing the model.

Takes the feedstock so the grain population and the impurity inventory belong to the ore rather than to the module.

*lattice\_ceiling\_ppm*

```
lattice_ceiling_ppm(feedstock: Feedstock, elements: tuple[str, ...]=('Al', 'Ti', 'Li', 'B')) -> dict[str, float]
```

Per-element floor concentration achievable by any chemical purification route.

The floor is the lattice-bound inventory, because `diffusion_length` shows the lattice is unreachable. Raises `ae.core.provenance.MissingValueError` for an uncharacterized ore, which is the correct outcome: the ceiling grade of a deposit whose lattice split has never been measured is unknown, not optimistic.

## 15.3 Worked examples, reproduced by execution

### Diffusion length, and the activation-energy bracket that makes the Al case undecidable

```
import math
from ae.core.units import Q_
from ae.physics.diffusion import (LIU_2026_EA_RANGE_KJ, MOST_MOBILE_BOUND,
                                  critical_activation_energy, diffusion_length,
                                  fourier_number, fractional_extraction_sphere)

d = MOST_MOBILE_BOUND.at(Q_(80.0, "degC"))
L = diffusion_length(d, Q_(6.0, "hour"))
print("D at 80 degC, upper bound:", f"{float(d.to('m**2/s')).magnitude:.6e}", "m2/s")
print("L = sqrt(D t) over 6 h      :", f"{float(L.to('m')).magnitude:.6e}", "m =",
      round(float(L.to("nm").magnitude), 2), "nm")
print("R/L on a 100 um grain      :", round(1.0e-4 / float(L.to("m").magnitude), 2),
      "=", round(math.log10(1.0e-4 / float(L.to("m").magnitude)), 4), "orders of magnitude")
print()
hi = critical_activation_energy(Q_(1e-4, "m**2/s"), Q_(100.0, "um"),
                               Q_(1200.0, "degC"), Q_(6.0, "hour"))
lo = critical_activation_energy(Q_(1e-10, "m**2/s"), Q_(100.0, "um"),
                               Q_(1200.0, "degC"), Q_(6.0, "hour"))
band = LIU_2026_EA_RANGE_KJ.quantity.to("kJ/mol").magnitude
print("Ea_crit at D0 = 1e-4      :", round(float(hi.magnitude), 4), "kJ/mol")
print("Ea_crit at D0 = 1e-10     :", round(float(lo.magnitude), 4), "kJ/mol")
print("Liu et al. 2026 Ea range :", list(band), "kJ/mol")
print("overlap                  :", round(max(float(lo.magnitude), float(band.min())), 4),
      "to", round(min(float(hi.magnitude), float(band.max())), 4), "kJ/mol")
print("either bracket contains? :",
      bool(float(lo.magnitude) <= band.min() and float(hi.magnitude) >= band.max()))
print()
fo = fourier_number(d, Q_(6.0, "hour"), Q_(100.0, "um"))
print("Fourier number at 80 degC:", f"{fo:.6e}")
print("fractional extraction      :", f"{fractional_extraction_sphere(fo):.6e}")
```

Output:

```
D at 80 degC, upper bound: 4.878687e-18 m2/s
L = sqrt(D t) over 6 h : 3.246223e-07 m = 324.62 nm
R/L on a 100 um grain : 308.05 = 2.4886 orders of magnitude

Ea_crit at D0 = 1e-4 : 235.0574 kJ/mol
Ea_crit at D0 = 1e-10 : 65.8388 kJ/mol
Liu et al. 2026 Ea range : [np.float64(90.0), np.float64(400.0)] kJ/mol
overlap : 90.0 to 235.0574 kJ/mol
either bracket contains? : False

Fourier number at 80 degC: 1.053796e-05
fractional extraction : 1.095730e-02
```

The two prefactor brackets straddle the reported barrier range without either containing the other, so the overlap region is where a true Al barrier permits depletion at one prefactor and forbids it at another. That is undecidable on present data, not merely uncertain.

## 15.4 Validation status

4 hand-traceable worked example(s) and 2 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_benchmark_ti_barrier_against_liu_2026</code>  | literature | Ti removal verdict against the Liu et al. 2026 activation energy. Liu et al. (2026, doi 10.3390/min16080836) report about 400 kJ/mol for Ti4+ lattice diffusion in SiO2 and identify solid-state lattice diffusion as the likely rate-determining step for deep impurity removal. This test checks that the 10.3390/min16080836                         |
| <code>test_benchmark_xia_2024_residual_is_lattice</code> | literature | The lattice-ceiling calculation against the Xia et al. 2024 endpoint. Xia et al. (2024, doi 10.3390/min14070727) took a vein quartz from 128.86 ug/g total trace impurities to 24.23 ug/g using a flowsheet that included both hot pressure acid leaching AND chlorination roasting, and identified the resi<br>Error stated: 30<br>10.3390/min14070727 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_golden_diffusion_length_arithmetic
test_golden_critical_activation_energy_arithmetic
test_golden_sphere_extraction_short_time_arithmetic
test_golden_erf_half_defines_the_diffusion_length
```

## CHAPTER 16

# physics/thermal.py

Module `ae.physics.thermal`, 1062 lines. Chapter text is this module's own documentation.

## 16.1 Purpose, governing equations and limitations

Energy balances for calcination, quenching and fusion of silica.

Scope: the specific energy of a thermal step, in kWh per tonne of product, decomposed into the part thermodynamics fixes (sensible plus latent plus transition enthalpy) and the part the furnace loses. Those two are kept separate throughout, because the first is a property of SiO<sub>2</sub> and the second is a property of a plant, and conflating them is how a process engineer ends up defending a furnace design with a thermodynamic number.

### EQUATIONS

(1) Holland and Powell heat-capacity polynomial (their Cp form, ds62/ds633):

$$C_p(T) = a + bT + \frac{c}{T^2} + \frac{d}{\sqrt{T}}$$

Symbols:  $C_p$  molar isobaric heat capacity, J/(mol K).  $T$  temperature, K. Valid 298.15 K to the phase's  $t_{\max}$  (see `CP_COEFFICIENTS`; 1800 K for quartz, 2000 K for cristobalite).  $a$  J/(mol K);  $b$  J/(mol K<sup>2</sup>);  $c$  J K/mol;  $d$  J/(mol K<sup>1/2</sup>).

Source of the functional form and of every coefficient: Holland and Powell 2011, doi 10.1111/j.1525-1314.2010.00923.x (dataset ds62), with the liquid endmember from Holland, Green and Powell 2018, doi 10.1093/petrology/egy048 (dataset ds633).

(2) Analytic integral of (1), the sensible enthalpy:

$$\int_{T_1}^{T_2} C_p dT = \left[ aT + \frac{bT^2}{2} - \frac{c}{T} + 2d\sqrt{T} \right]_{T_1}^{T_2}$$

Derivation: term-by-term antiderivative of (1).  $\int cT^{-2}dT = -c/T$  and  $\int dT^{-1/2}dT = 2d\sqrt{T}$ . Dimensions: [J/(mol K)][K] = [J/mol].

(3) Landau excess enthalpy for the alpha to beta quartz inversion. The inversion has no latent heat of the first-order kind; its energy cost is spread over a wide interval below  $T_c$  as an excess heat capacity that diverges at  $T_c$ . From the HP tricritical Gibbs excess (see `ae.physics.phases` equation 1):

$$H_{ex}(T) = G_{ex}(T) - T \frac{\partial G_{ex}}{\partial T} = G_{ex}(T) + T S_D [Q_0^2 - Q(T)^2]$$

$$C_{p,ex}(T) = \frac{T S_D}{2 T_{c,0} Q(T)^2} \quad (T < T_c), \quad C_{p,ex} = 0 \quad (T \geq T_c)$$

Symbols:  $H_{ex}$  excess molar enthalpy, J/mol.  $C_{p,ex}$  excess molar heat capacity, J/(mol K), diverging as  $T$  approaches  $T_c$ .  $S_D$  entropy of disordering, 4.95 J/(mol K) for quartz.  $Q$ ,  $Q_0$  order parameter at  $T$  and at 298.15 K.  $T_{c,0}$  847 K for quartz.

Derivation:  $C_{p,ex} = -T \partial^2 G_{ex} / \partial T^2$ , and differentiating the HP expression twice with  $Q = ((T_c - T)/T_{c,0})^{1/4}$  gives the form above (this is the `landau_hp` excess implemented in the HP formalism). The sign in  $H_{ex}$  follows from the HP first derivative  $\partial G_{ex} / \partial T = S_D [Q(T)^2 - Q_0^2]$ , so the excess ENTROPY is  $S_{ex} = -S_D [Q(T)^2 - Q_0^2]$ , which is positive on heating because  $Q$  falls from  $Q_0$  toward zero as the structure disorders. Getting this sign backwards flips  $H_{ex}(T_c) - H_{ex}(T_0)$  from +2646 to -4104 J/mol, i.e. it makes disordering exothermic, which violates the second law for a transition to a higher-entropy phase on heating. The total inversion enthalpy from 298.15 K to  $T_c$  is 2646 J/mol, i.e. 44.0 kJ/kg, computed by this module rather than asserted.

(4) Total thermal enthalpy of a phase-change path, per unit mass:

$$\Delta h = \frac{1}{M} \left[ \left( H_{f,2}^0 + \int_{T_0}^{T_2} C_{p,2} dT \right) - \left( H_{f,1}^0 + \int_{T_0}^{T_1} C_{p,1} dT + H_{ex,1}(T_1) \right) \right]$$

Symbols:  $\Delta h$  specific enthalpy change, J/kg.  $H_f^0$  standard enthalpy of formation at  $T_0 = 298.15$  K, J/mol.  $M$  molar mass of SiO<sub>2</sub>, 0.0600843 kg/mol (ds62). subscripts 1 and 2 denote the initial and final phase.

The difference in  $H_f^0$  is what carries the transition enthalpy, so a single expression handles heating, transformation and melting without a separate latent-heat term.

(5) Specific energy of a furnace step:

$$e = \frac{\Delta h}{\eta}, \quad \eta = \eta_{thermal} (1 - f_{loss})$$

Symbols:  $e$  specific energy input, J/kg, convertible to kWh/tonne.  $\eta$  overall efficiency, dimensionless, range (0, 1].  $\eta_{thermal}$  energy actually delivered to the charge divided by energy input.  $f_{loss}$  additional fractional loss (structure, flue), range [0, 1).

Reference points for  $\eta$ , US DOE via Galitsky and Worrell 2008 (OSTI 927883): "only about 33-40% of the energy consumed by a continuous furnace goes toward melting the glass", with "up to 30%" lost through the structure and "another 30%" through flue gas.

(6) Quench heat rejection and mean cooling rate:

$$q = \frac{1}{M} [H_1(T_{hot}) - H_1(T_{cold})], \quad \dot{T} = \frac{T_{hot} - T_{cold}}{\Delta t}$$

Symbols:  $q$  specific heat rejected to the quench medium, J/kg, positive.  $\dot{T}$  mean cooling rate, K/s. A MEAN, not the surface rate that governs thermal-shock fracture (see LIMITATIONS).

## LIMITATIONS

1. NO Cp FOR SILICA GLASS, DISTINCT FROM THE LIQUID. [webbook.nist.gov](http://webbook.nist.gov) (Shomate coefficients) and the Richet et al. 1982 drop-calorimetry paper (doi 10.1016/0016-7037(82)90383-0, the standard source for amorphous SiO<sub>2</sub> Cp from 1000 to 1800 K) are both unreachable from this sandbox, and no allowlisted substitute publishes glass-specific coefficients. This module therefore uses the ds633 quartz LIQUID endmember (Cp = 82.5 J/(mol K), temperature-independent) for molten and glassy silica, and

flags any such call. Below the glass transition (roughly 1475 K for silica) a constant liquid Cp is not the glass Cp, so glass sensible-heat results in that range carry an unquantified error.

2. THE LIQUID Cp IS A CONSTANT. ds633 gives  $b = c = d = 0$  for qL, so the model cannot reproduce any curvature in the melt heat capacity and cannot be extrapolated to a temperature range where that curvature matters.
3. HP2011 DOES NOT REPRODUCE THE TRIDYMITE-CRISTOBALITE BOUNDARY. Equating ds62 Gibbs energies in this module gives quartz-tridymite equilibrium at 866 degC (against the accepted 870 degC, good) but places tridymite-cristobalite equilibrium above 2600 K, against the accepted 1470 degC. The ds62 dataset is calibrated for petrological equilibria and its silica polymorph relations at 1 bar are not fit for predicting the high-temperature reconstructive boundaries. Take transition TEMPERATURES from `ae.physics.phases`, which sources them, and use this module only for the ENTHALPY along a path whose temperatures were set elsewhere.
4. THE ds633 LIQUID ENDMEMBER APPEARS STABLE AT EVERY TEMPERATURE at 1 bar in a naive Gibbs comparison against the ds62 solids, which is the opposite failure from a missing melt and equally disqualifying. Evaluating  $G(qL) - G(crst)$  over 400 to 2600 K gives -3735 J/mol at 400 K and -3451 J/mol at 2600 K, and  $G(qL) - G(q)$  gives -852 J/mol and -6911 J/mol at the same limits: the liquid sits BELOW both solids throughout, so the comparison yields no crossing and would imply silica is molten at room temperature. The cause is that ds62 and ds633 are not mutually calibrated for a 1 bar melting point (ds633 fits the liquid for peridotite-to-granite melting at crustal and mantle pressures, where the relevant equilibria are not the 1 bar silica melting curve). Melting temperatures must therefore be SUPPLIED, not computed, and this module refuses to infer one: see the `transition_temperature` requirement on `ThermalStep`. The enthalpies along a supplied path remain usable, because they depend on  $H_0$  and Cp rather than on the relative Gibbs energies.
5. LOSSES ARE NOT MODELLED FROM GEOMETRY. `f_loss` and `eta_thermal` are inputs, not predictions. There is no wall-conduction, radiation or flue-gas model here; a real design needs one, and the published efficiency bands cited above are for soda-lime and specialty glass furnaces, not for fused quartz.
6. MEAN COOLING RATE IS NOT A THERMAL-SHOCK CRITERION. Decrepitation depends on the SURFACE temperature gradient and on the fluid-inclusion population, neither of which this module has. `feedstock.physical.decrepitation_index` exists for the measured response and raises when unmeasured; that is the correct behaviour and this module does not work around it.
7. NO KILN SCALE-UP. Specific energy is per tonne of product with no throughput dependence, so it cannot capture the strong capacity dependence of real furnace efficiency (Galitsky and Worrell 2008 note all-electric furnaces are typically used below 75 ton/day).
8. IMPURITY EFFECTS ON Cp ARE IGNORED. Every coefficient is for pure SiO2. At the ppm impurity levels relevant to HPQ this is a very small error on Cp, but the same is NOT true of the melting behaviour, where alkali content dominates.

## TYPE CHECKING NOTE

`mypy --strict` reports `Quantity?` has no attribute "to" and Variable "ae.core.units.Quantity" is not valid as a type against this module. Those errors originate in `ae.core.units`, which declares `Quantity = UREG.Quantity` (a variable binding, not a type alias), so mypy cannot treat it as a type. The same errors appear against the four core modules themselves (46 of them), and the core modules are not ours to change. Every annotation here follows the core convention deliberately rather than diverging from it for a clean checker run.

## References

Every entry below was resolved against the Crossref REST API for the DOI shown, and the fields here (author list, year, title, journal, volume, issue, pages) are as Crossref returned them. Resolved 17 September 2026. Entries without a DOI say what was checked instead and what remains unverified.

Holland, T. J. B. and Powell, R. (2011) An improved and extended internally consistent thermodynamic dataset for phases of petrological interest, involving a new equation of state for solids, *Journal of Metamorphic Geology* 29(3), 333-383, doi 10.1111/j.1525-1314.2010.00923.x. Dataset ds62. Source of the heat capacity functional form and of every coefficient for the solid quartz endmembers.

Holland, T. J. B., Green, E. C. R. and Powell, R. (2018) Melting of Peridotites through to Granites: A Simple Thermodynamic Model in the System KNCFMASHTOCr, *Journal of Petrology* 59(5), 881-900, doi 10.1093/petrology/egy048. Dataset ds633. Source of the quartz LIQUID endmember, including the constant  $C_p = 82.5 \text{ J/(mol K)}$  used above.

Richet, P., Bottinga, Y., Denielou, L., Petitet, J. P. and Tequi, C. (1982) Thermodynamic properties of quartz, cristobalite and amorphous SiO<sub>2</sub>: drop calorimetry measurements between 1000 and 1800 K and a review from 0 to 2000 K, *Geochimica et Cosmochimica Acta* 46(12), 2639-2658, doi 10.1016/0016-7037(82)90383-0. Independent drop-calorimetry dataset over the temperature range this module integrates across. Closed access; cited as the comparison dataset, not as the source of any coefficient used here.

Galitsky, C., Worrell, E., Masanet, E. and Graus, W. (2008) Energy Efficiency Improvement and Cost Saving Opportunities for the Glass Industry: An ENERGY STAR Guide for Energy and Plant Managers, Lawrence Berkeley National Laboratory, report LBNL-57335, doi 10.2172/927883 (OSTI 927883). Source of the statement that only about 33 to 40 percent of the energy consumed by a continuous furnace goes toward melting, and of the observation that all-electric furnaces are typically used below 75 ton/day. Crossref lists Galitsky twice in the author record for this DOI; the duplicate is in the upstream metadata, not a second person.

## 16.2 Interface

### Types

**CpCoefficients.** Holland and Powell Cp polynomial coefficients with their validity range.

The four coefficients are held as provenance Values, not floats, so that the dataset they came from travels with them and a future swap to NIST-JANAF Shomate coefficients is visible in every downstream result.

```
raw(self) -> tuple[float, float, float, float]
```

Coefficients as SI floats: (a, b, c, d) for  $\text{J/(mol K)}$  with T in K.

```
in_range(self, t_k: float) -> bool
```

**ThermalStep.** One furnace step: a path through phases and temperatures, with its efficiency.

`eta_thermal` and `loss_fraction` are separate because they describe different things: the first is how much of the energy input reaches the charge, the second is an additional deduction for losses the caller wants itemised. Overall efficiency is `eta_thermal * (1 - loss_fraction)` and is asserted to lie in (0, 1].

```
overall_efficiency(self) -> float
```

Overall efficiency, dimensionless, in (0, 1].

**EnergyBalance.** Result of a thermal-step energy balance, itemised so the balance can be checked.

*theoretical* is what SiO<sub>2</sub> demands, *supplied* is what the furnace must input, and *losses* is the difference. *supplied = theoretical + losses* is asserted on construction, which is the first law closing.

```
kwh_per_tonne(self) -> float
```

Supplied specific energy in kWh per tonne, the industry unit.

```
theoretical_kwh_per_tonne(self) -> float
```

## Functions

*molar\_heat\_capacity*

```
molar_heat_capacity(phase: Polymorph, temperature: Quantity, include_landau: bool=True, allow_extrapolatio
\ n: bool=False) -> Quantity
```

Molar isobaric heat capacity from equation (1), plus the Landau excess (3).

**Parameters.** *phase* Polymorph. ALPHA\_QUARTZ and BETA\_QUARTZ both resolve to the quartz endmember, which is correct: HP treat them as one phase with a Landau correction. *temperature* Temperature. *include\_landau* Add the excess Cp of the order-disorder transition where the phase has one. The excess diverges at T<sub>c</sub>, which is physical: the inversion absorbs heat over a wide interval rather than at a single point. *allow\_extrapolation* Permit evaluation outside the assumed validity range.

**Returns.** Quantity Cp in J/(mol K), strictly positive.

**Examples.** Alpha-quartz at 298.15 K, base polynomial only:

```
>>> from ae.core.units import Q_
>>> cp = molar_heat_capacity(Polymorph.ALPHA_QUARTZ, Q_(298.15, "K"),
...                          include_landau=False)
>>> round(float(cp.magnitude), 4)
43.1942
```

*specific\_heat\_capacity*

```
specific_heat_capacity(phase: Polymorph, temperature: Quantity, include_landau: bool=True, allow_extrapolatio
\ tion: bool=False) -> Quantity
```

Isobaric heat capacity per unit mass, J/(kg K), from Cp / M.

**Examples.**

```
>>> from ae.core.units import Q_
>>> c = specific_heat_capacity(Polymorph.ALPHA_QUARTZ, Q_(298.15, "K"),
...                            include_landau=False)
>>> round(float(c.to("J/(kg*K)").magnitude), 1)
718.9
```

*landau\_excess\_heat\_capacity*

```
landau_excess_heat_capacity(temperature: Quantity, params: LandauParameters=QUARTZ_LANDAU) -> Quantity
```

Excess Cp of a tricritical order-disorder transition, equation (3).

Returns zero at and above T\_c, and diverges as T approaches T\_c from below.

**Examples.**

```
>>> from ae.core.units import Q_
>>> round(float(landau_excess_heat_capacity(Q_(298.15, "K")).magnitude), 4)
1.0823
>>> float(landau_excess_heat_capacity(Q_(1000.0, "K")).magnitude)
0.0
```

*landau\_excess\_enthalpy*

```
landau_excess_enthalpy(temperature: Quantity, params: LandauParameters=QUARTZ_LANDAU) -> Quantity
```

Excess molar enthalpy H\_ex of the ordering transition, equation (3), J/mol.

**Notes.** Referenced so that H\_ex(T) - H\_ex(298.15 K) is the heat absorbed by the inversion between those temperatures. For quartz that difference from 298.15 K to T\_c is 2646 J/mol (44.0 kJ/kg), which is the inversion's whole energy cost.

*integrated\_enthalpy*

```
integrated_enthalpy(phase: Polymorph, t_from: Quantity, t_to: Quantity, include_landau: bool=True, allow_e
\ xtrapolation: bool=False) -> Quantity
```

Sensible enthalpy of one phase between two temperatures, equation (2), J/mol.

**Parameters.** phase Polymorph, assumed not to transform over the interval. The caller is responsible for splitting a path at its transitions, which is what phase\_enthalpy and specific\_transition\_enthalpy do. t\_from, t\_to Interval endpoints. Reversing them negates the result, as an enthalpy difference should.

**Returns.** Quantity Enthalpy change in J/mol, positive on heating.

**Examples.** Alpha to beta quartz from 298.15 K to 1000 K, base polynomial only. Hand check with  $F(T) = 92.9 T - 3.21e-4 T^2 + 714900/T - 1432.2 \sqrt{T}$ , term by term:

```
F(1000)   = 92900.0000 - 321.0000 + 714.9000 - 45290.1406 = 48003.7594
F(298.15) = 27698.1350 - 28.5348 + 2397.7863 - 24729.8269 = 5337.5597
-----
difference                               = 42666.1997 J/mol
```

( $1432.2 \sqrt{1000} = 1432.2 \times 31.6227766 = 45290.1406$  and  $1432.2 \sqrt{298.15} = 1432.2 \times 17.2670206 = 24729.8269$ )

```
>>> from ae.core.units import Q_
>>> dh = integrated_enthalpy(Polymorph.QUARTZ, Q_(298.15, "K"), Q_(1000.0, "K"),
...                           include_landau=False)
>>> round(float(dh.magnitude), 1)
42666.2
```

### *phase\_enthalpy*

```
phase_enthalpy(phase: Polymorph, temperature: Quantity, allow_extrapolation: bool=False) -> Quantity
```

Absolute molar enthalpy  $H(T) = H^0_f + \int C_p dT + H_{ex}$ , J/mol.

Absolute in the sense that all polymorphs share the same 298.15 K formation-enthalpy reference, so differences between phases are physically meaningful. That is what makes equation (4) able to carry the transition enthalpy without a separate latent-heat term.

### *specific\_transition\_enthalpy*

```
specific_transition_enthalpy(from_phase: Polymorph, to_phase: Polymorph, temperature: Quantity, allow_extrapolation: bool=False) -> Quantity
```

Specific enthalpy of transformation at one temperature, J/kg.

**Examples.** Quartz to cristobalite at 1743.15 K (1470 degC):

```
>>> from ae.core.units import Q_
>>> dh = specific_transition_enthalpy(Polymorph.QUARTZ, Polymorph.CRISTOBALITE,
...                                  Q_(1743.15, "K"), allow_extrapolation=True)
>>> round(float(dh.to("kJ/kg").magnitude), 1)
50.9
```

### *calcination\_energy*

```
calcination_energy(feedstock: Feedstock, step: ThermalStep, allow_extrapolation: bool=False) -> EnergyBalance
```

Specific energy to calcine a quartz feedstock, kWh per tonne inside the result.

Calcination here means heating without a reconstructive phase change: the charge crosses the alpha to beta inversion, fluid inclusions pressurise and decrepitate, and the product is still quartz. The inversion's energy cost is included automatically through the Landau excess enthalpy.

**Parameters.** `feedstock` The ore. Required by platform rule and used to mark the result a scenario when the ore is uncharacterized. `step` The thermal step. `from_phase` and `to_phase` are normally both quartz. `allow_extrapolation` Permit Cp evaluation outside the assumed validity range.

**Returns.** `EnergyBalance`

**Notes.** This function deliberately does NOT predict decrepitation or impurity removal. The fluid-inclusion population that governs both is a measured feedstock property (`feedstock.inclusions`, `feedstock.physical.decrepitation_index`) and raises when unmeasured.

**Examples.**

```
>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> from ae.core.provenance import Tag, Value
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> s = ThermalStep(name="calcine", from_phase=Polymorph.ALPHA_QUARTZ,
```

```

...         to_phase=Polymorph.BETA_QUARTZ, t_start=Q_(298.15, "K"),
...         t_end=Q_(1173.15, "K"),
...         eta_thermal=Value(quantity=Q_(1.0, "dimensionless"),
...                               tag=Tag.ASSUMED, basis="doctest, no losses"))
>>> bal = calcination_energy(f, s)
>>> bal.is_scenario
True

```

### *fusion\_energy*

`fusion_energy(feedstock: Feedstock, step: ThermalStep, allow_extrapolation: bool=True) -> EnergyBalance`

Specific energy to fuse silica into a melt or glass, kWh per tonne in the result.

`allow_extrapolation` defaults True here because fusion necessarily runs to 1996 K or beyond, past the 1800 K bound assumed for the quartz polynomial. That is a real extrapolation and is recorded as such rather than hidden.

### Examples.

```

>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> from ae.core.provenance import Tag, Value
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> s = ThermalStep(name="fuse", from_phase=Polymorph.QUARTZ,
...                 to_phase=Polymorph.SILICA_LIQUID, t_start=Q_(298.15, "K"),
...                 t_end=Q_(1996.0, "K"), transition_temperature=Q_(1996.0, "K"),
...                 eta_thermal=Value(quantity=Q_(1.0, "dimensionless"),
...                                     tag=Tag.ASSUMED, basis="doctest, no losses"))
>>> round(fusion_energy(f, s).theoretical_kwh_per_tonne, 1)
599.7

```

### *quench\_heat\_rejection*

`quench_heat_rejection(feedstock: Feedstock, phase: Polymorph, t_hot: Quantity, t_cold: Quantity, duration: Quantity, allow_extrapolation: bool=False) -> tuple[Quantity, Quantity]`

Heat rejected in a quench and the mean cooling rate, equation (6).

**Parameters.** `feedstock` The ore, per platform rule. `phase` Phase of the material being quenched. `t_hot`, `t_cold` Start and end temperatures. `t_cold` must be below `t_hot`. `duration` Quench duration, used only for the MEAN cooling rate.

**Returns.** (Quantity, Quantity) Specific heat rejected, J/kg (positive), and mean cooling rate, K/s.

**Notes.** The mean cooling rate is not a thermal-shock criterion. Fracture is driven by the surface temperature gradient and by the fluid-inclusion population, and this function has neither. See LIMITATIONS item 6.

### Examples.

```

>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,

```

```

...             deposit_name="Vikarabad", country="IN")
>>> q, rate = quench_heat_rejection(f, Polymorph.QUARTZ, Q_(1173.15, "K"),
...                               Q_(298.15, "K"), Q_(60.0, "s"))
>>> float(q.magnitude) > 0.0, round(float(rate.to("K/s").magnitude), 3)
(True, 14.583)

```

### *electricity\_cost*

`electricity_cost(balance: EnergyBalance, site: Site) -> Quantity`

Cost of the supplied energy at a site's electricity price, per tonne of product.

**Parameters.** `balance` Result of a step energy balance. `site` The plant location. Its `power.energy_price` supplies both the rate and the currency, so no exchange rate is applied and none is implied.

**Returns.** `Quantity` Cost per tonne, in the site's own currency.

**Notes.** Electricity only. Demand charges, outage-driven lost production and fuel-fired alternatives are site fields this function does not touch, and a thermal step run on natural gas must not be costed with this.

## 16.3 Worked examples, reproduced by execution

The 8 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

### `physics.thermal.calcination_energy`

```

>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> from ae.core.provenance import Tag, Value
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> s = ThermalStep(name="calcine", from_phase=Polymorph.ALPHA_QUARTZ,
...                 to_phase=Polymorph.BETA_QUARTZ, t_start=Q_(298.15, "K"),
...                 t_end=Q_(1173.15, "K"),
...                 eta_thermal=Value(quantity=Q_(1.0, "dimensionless"),
...                                   tag=Tag.ASSUMED, basis="doctest, no losses"))
>>> bal = calcination_energy(f, s)
>>> bal.is_scenario
True

```

### `physics.thermal.fusion_energy`

```

>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> from ae.core.provenance import Tag, Value
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> s = ThermalStep(name="fuse", from_phase=Polymorph.QUARTZ,
...                 to_phase=Polymorph.SILICA_LIQUID, t_start=Q_(298.15, "K"),
...                 t_end=Q_(1996.0, "K"), transition_temperature=Q_(1996.0, "K"),
...                 eta_thermal=Value(quantity=Q_(1.0, "dimensionless"),
...                                   tag=Tag.ASSUMED, basis="doctest, no losses"))
>>> round(fusion_energy(f, s).theoretical_kwh_per_tonne, 1)
599.7

```

`physics.thermal.integrated_enthalpy`

```
>>> from ae.core.units import Q_
>>> dh = integrated_enthalpy(Polymorph.QUARTZ, Q_(298.15, "K"), Q_(1000.0, "K"),
...                          include_landau=False)
>>> round(float(dh.magnitude), 1)
42666.2
```

`physics.thermal.landau_excess_heat_capacity`

```
>>> from ae.core.units import Q_
>>> round(float(landau_excess_heat_capacity(Q_(298.15, "K")).magnitude), 4)
1.0823
>>> float(landau_excess_heat_capacity(Q_(1000.0, "K")).magnitude)
0.0
```

`physics.thermal.molar_heat_capacity`

```
>>> from ae.core.units import Q_
>>> cp = molar_heat_capacity(Polymorph.ALPHA_QUARTZ, Q_(298.15, "K"),
...                          include_landau=False)
>>> round(float(cp.magnitude), 4)
43.1942
```

`physics.thermal.quench_heat_rejection`

```
>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> q, rate = quench_heat_rejection(f, Polymorph.QUARTZ, Q_(1173.15, "K"),
...                                Q_(298.15, "K"), Q_(60.0, "s"))
>>> float(q.magnitude) > 0.0, round(float(rate.to("K/s").magnitude), 3)
(True, 14.583)
```

`physics.thermal.specific_heat_capacity`

```
>>> from ae.core.units import Q_
>>> c = specific_heat_capacity(Polymorph.ALPHA_QUARTZ, Q_(298.15, "K"),
...                            include_landau=False)
>>> round(float(c.to("J/(kg*K)").magnitude), 1)
718.9
```

`physics.thermal.specific_transition_enthalpy`

```
>>> from ae.core.units import Q_
>>> dh = specific_transition_enthalpy(Polymorph.QUARTZ, Polymorph.CRISTOBALITE,
...                                  Q_(1743.15, "K"), allow_extrapolation=True)
>>> round(float(dh.to("kJ/kg").magnitude), 1)
50.9
```

## 16.4 Validation status

8 hand-traceable worked example(s) and 4 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                                | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_benchmark_theoretical_melt_energy_against_lbnl</code>         | literature | Model fusion floor against the DOE theoretical melting energy for glass. Literature: Galitsky and Worrell 2008 (LBNL, doi 10.2172/927883) state that "theoretically, 2.2 MMBtu are required to melt one short ton of glass". Converting: 2.2 MMBtu = $2.2 \times 1.05505585262\text{e9 J} = 2.3211229\text{e9 J}$ 1 short ton = 9<br>Error stated: -15.6; 15.6<br>10.2172/927883                                                                                                                                           |
| <code>test_benchmark_electric_melter_efficiency_against_lbnl</code>      | literature | Implied furnace efficiency of a specialty-glass electric melter. Literature: Galitsky and Worrell 2008 Table 7 gives a specialty-glass electric melter at 10.3 MMBtu per short ton of electricity (range 8.9 to 11.6). Converting as above: 10.3 MMBtu/short ton = 3327.5 kWh/tonne (range 2875.2 to 3747.4)<br>Error stated: 18.0; 69.7; -40<br>10.1016/0016-7037(82; 10.1093/petrology/egy048; 10.1093/petrology/egy048; 10.1111/j.1525-1314.2010.00923.x; 10.1111/j.1525-1314.2010.00923.x; 10.2172/927883              |
| <code>test_benchmark_quartz_cp_against_dulong_petit</code>               | literature | High-temperature Cp of quartz against the Dulong-Petit classical limit. Dulong-Petit: a solid with N atoms per formula unit approaches 3 N R at high temperature. SiO2 has N = 3, so $3 \times 3 \times 8.31446 = 74.83 \text{ J/(mol K)}$ Model at 1000 K (above the inversion, so no Landau term): 68.898 J/(mol K). Error aga<br>Error stated: -7.93; 8<br>10.1016/0016-7037(82; 10.1093/petrology/egy048; 10.1093/petrology/egy048; 10.1111/j.1525-1314.2010.00923.x; 10.1111/j.1525-1314.2010.00923.x; 10.2172/927883 |
| <code>test_benchmark_transition_enthalpies_are_small_and_positive</code> | literature | Silica polymorph transition enthalpies against the accepted order of magnitude. Computed from the ds62 endmembers at the sourced transition temperatures: quartz -> tridymite at 1143.15 K: 2.756 kJ/mol = 45.9 kJ/kg quartz -> cristobalite at 1743.15 K: 3.059 kJ/mol = 50.9 kJ/kg cristobalite -> liquid<br>10.1016/0016-7037(82; 10.1093/petrology/egy048; 10.1093/petrology/egy048; 10.1111/j.1525-1314.2010.00923.x; 10.1111/j.1525-1314.2010.00923.x; 10.2172/927883                                                |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_golden_cp_alpha_quartz_at_298
test_golden_landau_excess_heat_capacity_at_298
test_golden_landau_excess_diverges_at_tc_and_vanishes_above
test_golden_integrated_enthalpy_quartz_298_to_1000
test_golden_inversion_enthalpy
test_golden_fusion_path_enthalpy
test_golden_quench_heat_rejection
test_golden_efficiency_scaling
```

## CHAPTER 17

# physics/phases.py

Module `ae.physics.phases`, 1215 lines. Chapter text is this module's own documentation.

## 17.1 Purpose, governing equations and limitations

SiO<sub>2</sub> polymorph stability expressed as PROCESS WINDOWS, not as a phase diagram.

### Purpose

A phase diagram tells you which polymorph is thermodynamically stable. A kiln tells you which polymorph you actually get, and the two differ by hundreds of degrees for silica because the reconstructive transitions (quartz to tridymite to cristobalite) break Si-O bonds and are kinetically obstructed, while the displacive alpha to beta inversion is instantaneous and unavoidable. This module reports both, labelled, and never lets a thermodynamic boundary be read as a process prediction.

### EQUATIONS

(1) Landau order parameter for the alpha to beta quartz inversion (tricritical, Holland and Powell 1998/2011 formalism):

$$Q(T) = \left( \frac{T_c - T}{T_{c,0}} \right)^{1/4} \quad (T < T_c), \quad Q(T) = 0 \quad (T \geq T_c)$$

with the pressure dependence of the critical temperature

$$T_c = T_{c,0} + \frac{V_D (P - P_0)}{S_D}$$

Symbols:  $Q$  order parameter, dimensionless, range  $[0, 1]$  at  $P_0$  for  $T$  in  $[0, T_{c,0}]$ .  $T$  temperature, K, valid range 298 to 1200 K for quartz in this module.  $T_c$  critical temperature at pressure  $P$ , K.  $T_{c,0}$  critical temperature at the reference pressure  $P_0$ , 847 K for quartz.  $P$  pressure, Pa, valid range  $1e4$  to  $1e9$  Pa (the HP Landau form is calibrated for petrological pressures; this module is used at  $P_0$  in practice).  $P_0$  reference pressure,  $1e5$  Pa.  $V_D$  volume of disordering,  $1.188e-6$  m<sup>3</sup>/mol for quartz.  $S_D$  entropy of disordering,  $4.95$  J/(mol K) for quartz.

Derivation: the HP tricritical Landau expansion writes the excess Gibbs energy of the ordered phase relative to the disordered one as  $G_{ex} = S_D [ T_{c,0} (Q_0^2 - Q_0^6/3) - (T_c Q^2 - T_{c,0} Q^6/3) - T (Q_0^2 - Q^2) ]$

- $(P - P_0) V_D Q_0^2$ . Minimising  $G_{ex}$  with respect to  $Q$  at fixed  $T$  and  $P$  gives the  $Q(T)$  above (see Holland and Powell 2011, doi 10.1111/j.1525-1314.2010.00923.x, and the calibration of the quartz transition in Carpenter et al. 1998, doi 10.2138/am-1998-1-201).

(2) Excess (transition-related) molar volume, which is the quantity that cracks grains:

$$V_{ex}(T) = V_D Q(T)^2, \quad \varepsilon_V(T_1 \rightarrow T_2) = \frac{V_{ex}(T_1) - V_{ex}(T_2)}{V_0}$$

Symbols:  $V_{\{ex\}}$  excess molar volume attributable to the ordering, m<sup>3</sup>/mol.  $V_0$  molar volume of the endmember at 298.15 K and  $P_0$ , 2.269e-5 m<sup>3</sup>/mol.  $\varepsilon_V$  volumetric strain, dimensionless, positive on heating.

At  $P_0$ ,  $Q_0 = Q(298.15 \text{ K}) = ((847 - 298.15)/847)^{1/4} = 0.89721$ , so the CUMULATIVE transition-related expansion from 298.15 K to any T above  $T_c$  is  $V_D Q_0^2 / V_0 = 1.188\text{e-}6 \times 0.80498 / 2.269\text{e-}5 = 0.04215$ , i.e. 4.21 vol%.

(3) Relative rate of a thermally activated reconstructive transformation, Arrhenius ratio:

$$\frac{k(T_2)}{k(T_1)} = \exp \left[ -\frac{E_a}{R} \left( \frac{1}{T_2} - \frac{1}{T_1} \right) \right]$$

Symbols:  $k$  rate constant of the JMAK kinetic law, units 1/s<sup>n</sup>, never used absolutely here (see LIMITATIONS: no pre-exponential factor could be sourced).  $E_a$  apparent activation energy, J/mol. 555 +/- 24 kJ/mol for cristobalite formation from sintered silica glass (Breneman and Halloran 2014, doi 10.1111/jace.12889), calibrated over 1200 to 1350 degC only.  $R$  8.31446261815324 J/(mol K).  $T_1, T_2$  temperatures, K, valid range 1473 to 1623 K by calibration of  $E_a$ .

Derivation: dividing  $k = A \exp(-E_a/(R T))$  at two temperatures cancels  $A$ , which is why a ratio can be reported from a source that gives  $E_a$  without  $A$ .

(4) JMAK (Johnson-Mehl-Avrami-Kolmogorov) transformed fraction, provided for completeness and requiring a user-supplied rate constant:

$$X(t) = 1 - \exp [-(k t)^n]$$

Symbols:  $X$  transformed volume fraction, dimensionless, range [0, 1].  $t$  hold time, s, range 0 to 1e6 s.  $k$  rate constant, 1/s. NOT DEFAULTED: see LIMITATIONS.  $n$  Avrami exponent, dimensionless. 3.0 +/- 0.6 for cristobalite from sintered silica (Breneman and Halloran 2014), consistent with three-dimensional growth from a constant nucleus population.

## WHAT THE LITERATURE ACTUALLY SUPPORTS, AND WHERE THIS BRIEF WAS WRONG

The platform brief states the alpha to beta inversion carries "roughly 3.7 vol% expansion". That number is not the discontinuity at 573 degC. Two distinct quantities are conflated in the trade literature:

(a) The CUMULATIVE transition-related expansion of alpha-quartz on heating from room temperature to the inversion. The HP2011 Landau calibration gives 4.21 vol% (see equation 2 above). Values quoted in the 3.5 to 4.5 vol% band, including the brief's 3.7 vol%, belong to this quantity. (b) The STEP change at the inversion itself, which Ringdalen 2015 (doi 10.1007/s11837-014-1149-y) reports as "an increase in volume of around 0.4%".

Both are real and they differ by an order of magnitude. This module returns (a) from `alpha_beta_cumulative_volume_strain` and (b) from `TRANSITIONS`, each separately labelled, because a thermal-shock model fed the wrong one is wrong by 10x.

The mechanistic claim in the brief, that the alpha to beta inversion is "what cracks inclusion trails and is why calcination works", is only partly supported. The much larger strain available on heating is the quartz

to cristobalite conversion: Ringdalen 2015 measured up to 37% volume expansion across her sample set and found that specific surface area rose with cristobalite content, attributing it to "cracking when the volume increases". The evidence for the 0.4% inversion step being the dominant cracking mechanism is weak; this module therefore reports strains and does not assert a cracking mechanism.

## LIMITATIONS

1. NO ABSOLUTE KINETICS. `jmak_fraction` requires a rate constant because no source reachable from this environment reports a pre-exponential factor for the quartz to cristobalite transformation. Breneman and Halloran 2014 report  $n$  and  $E_a$  but not  $A$ . Do not substitute a guessed  $A$ : Ringdalen 2015 measured 1% to 76% cristobalite after one hour at 1500 degC ACROSS DIFFERENT NATURAL QUARTZ SAMPLES, a 76-fold spread at fixed temperature and time, which means any single  $A$  is a property of one ore and not of  $\text{SiO}_2$ . Absolute transformation kinetics for a specific deposit must be measured.
2.  $E_a$  EXTRAPOLATION. The 555 kJ/mol value is calibrated on 1200 to 1350 degC data for SINTERED SILICA GLASS, not for natural quartz, and Breneman and Halloran report the temperature of maximum rate as 1500 to 1600 degC. A rate ratio evaluated above roughly 1500 degC therefore extrapolates past a rate maximum and will overstate the rate increase. Above 1623 K this module raises rather than extrapolate.
3. TRIDYMITTE MAY NOT APPEAR AT ALL. The 870 degC boundary is a thermodynamic equilibrium. Ringdalen 2015 found "the tridymite phase expected from the phase diagram" absent in her heat-treated natural quartz, consistent with the long-standing view that tridymite stability requires alkali or other mineralisers. A schedule crossing 870 degC should be read as permitting tridymite, not producing it.
4. PRESSURE. Every transition temperature here is for  $P_0 = 1$  bar. The Landau  $T_c$  pressure term is implemented but not validated in this module; coesite and stishovite are out of scope entirely.
5. ORDINAL RISK ONLY. `devitrification_risk` returns an ordinal level built from a temperature window, a hold time and a contact material. It is a design screen, not a quantitative prediction of cristobalite fraction, and it has not been validated against a held-out dataset because no such dataset was reachable.
6. CONTACT-MATERIAL ONSET IS ONE EXPERIMENT. The 1000 degC glass onset and the material ranking come from Warden et al. 2024 (doi 10.52825/siliconpv.v2i.1311), a single study at one temperature (1500 degC) for one duration (3 h) on one commercial crucible glass. The direction of the effect is credible, the magnitudes are not general.

## TYPE CHECKING NOTE

`mypy --strict` reports `Quantity?` has no attribute "to" and Variable "`ae.core.units.Quantity`" is not valid as a type against this module. Those errors originate in `ae.core.units`, which declares `Quantity = UREG.Quantity` (a variable binding, not a type alias), so `mypy` cannot treat it as a type. The same errors appear against the four core modules themselves (46 of them), and the core modules are not ours to change. Every annotation here follows the core convention deliberately rather than diverging from it for a clean checker run.

## References

Every entry below was resolved against the Crossref REST API for the DOI shown, and the fields here (author list, year, title, journal, volume, issue, pages) are as Crossref returned them. Resolved 17 September 2026. Entries without a DOI say what was checked instead and what remains unverified.

Holland, T. J. B. and Powell, R. (2011) An improved and extended internally consistent thermodynamic dataset for phases of petrological interest, involving a new equation of state for solids, *Journal of Metamorphic*

Geology 29(3), 333-383, doi 10.1111/j.1525-1314.2010.00923.x (dataset ds62). Source of the Landau formalism used for equations (1) and (2) and of every Landau coefficient in QUARTZ\_LANDAU.

Holland, T. J. B. and Powell, R. (1998) An internally consistent thermodynamic data set for phases of petrological interest, Journal of Metamorphic Geology 16(3), 309-343, doi 10.1111/j.1525-1314.1998.00140.x. The earlier dataset of the same pair, cited only for the lineage of the Landau treatment. The in-text phrase "Holland and Powell 1998/2011 formalism" refers to these two papers; no numerical value in this module comes from the 1998 paper.

Carpenter, M. A., Salje, E. K. H., Graeme-Barber, A., Wruck, B., Dove, M. T. and Knight, K. S. (1998) Calibration of excess thermodynamic properties and elastic constant variations associated with the alpha to beta phase transition in quartz, American Mineralogist 83(1-2), 2-22, doi 10.2138/am-1998-1-201. Cited for the tricritical character of the inversion, not for numerical input.

Ringdalen, E. (2015) Changes in Quartz During Heating and the Possible Effects on Si Production, JOM 67(2), 484-492, doi 10.1007/s11837-014-1149-y. Source of the 0.4 percent step volume change at the inversion, of the up-to-37 percent expansion on cristobalite conversion, of the 1 to 76 percent cristobalite spread after one hour at 1500 degC across natural samples, and of the absence of tridymite in her heat-treated material. The year is the February 2015 print issue; Crossref's issued date is the 2014-10-10 online-first publication, which is why the DOI contains -014-.

Breneman, R. C. and Halloran, J. W. (2014) Kinetics of Cristobalite Formation in Sintered Silica, Journal of the American Ceramic Society 97(7), 2272-2278, doi 10.1111/jace.12889. Source of the Avrami exponent  $n = 3.0 \pm 0.6$ , the apparent activation energy  $E_a = 555 \pm 24$  kJ/mol, the 1200 to 1350 degC calibration window, and the 1500 to 1600 degC rate maximum, all taken from the published abstract. No pre-exponential factor is reported there, which is why `jmak_fraction` has no default rate constant. THIS ENTRY WAS PREVIOUSLY WRONG: it named "Zhang, Y., Mavrogenes, J.A. and others" with pages 3378-3384, written from memory. Neither Zhang nor Mavrogenes is an author, and the page range was wrong.

Bourova, E. and Richet, P. (1998) Quartz and Cristobalite: high-temperature cell parameters and volumes of fusion, Geophysical Research Letters 25(13), 2333-2336, doi 10.1029/98gl01581. Source of the cristobalite and quartz high-temperature volume data used for the liquid-phase transition entry. THIS ENTRY WAS PREVIOUSLY WRONG: it named "Wenk, H.-R., Brunini, A. and others" with pages 2261-2264, written from memory. Neither is an author.

Warden, G. K., Gawel, B. A., Juel, M., Erbe, A. and Di Sabatino, M. (2024) Cristobalite Formation in Fused Quartz Crucibles for Czochralski Silicon Production in Different Conditions, SiliconPV Conference Proceedings 2, doi 10.52825/siliconpv.v2i.1311. Source of the contact-material ranking and the 1000 degC glass onset. One study, one temperature, one duration, one commercial crucible glass: the direction is credible, the magnitudes are not general. Crossref returns no page range for this article.

## 17.2 Interface

### Types

**Polymorph.** SiO<sub>2</sub> phases this platform models at ambient pressure.

ALPHA\_QUARTZ and BETA\_QUARTZ are the two sides of a single displacive inversion and share one thermodynamic endmember plus a Landau correction, which is why `ae.physics.thermal` keys its Cp coefficients on QUARTZ rather than on the two separately.

**TransitionCharacter.** Mechanism class, which decides whether kinetics may be ignored.

DISPLACIVE transitions rotate bonds without breaking them: they are effectively instantaneous, cannot be suppressed by fast heating, and are fully reversible. RECONSTRUCTIVE transitions break Si-O bonds: they are slow, often bypassed entirely, and on cooling they usually do not reverse, which is why cristobalite survives to room temperature in a fired product.

---

**ContactMaterial.** Material in contact with silica during a hold, which changes the onset.

**RiskLevel.** Ordinal devitrification risk. Not a probability, not a fraction.

**LandauParameters.** Tricritical Landau parameters for a displacive inversion.

**Transition.** One SiO<sub>2</sub> transition, with its character and its strain.

`volume_strain` is the strain of the transition ITSELF (the step), not the cumulative expansion of the low-temperature phase on the way to it. See the module docstring for why the distinction matters.

```
temperature_k(self) -> float
```

Transition temperature in kelvin.

**ScheduleSegment.** One ramp-and-hold leg of a thermal schedule.

```
target_k(self) -> float
```

**ThermalSchedule.** A furnace recipe: where it starts, where it goes, in what and against what.

Atmosphere is carried because Ringdalen 2015 found cristobalite forms more readily in inert than in reducing atmospheres, and contact material is carried because Warden et al. 2024 found it to be "the most determining factor" for cristobalite layer thickness.

```
peak_temperature_k(self) -> float
```

Highest temperature reached, kelvin.

```
start_k(self) -> float
```

```
total_hold_above(self, temperature: Quantity) -> Quantity
```

Total hold time in segments whose target is at or above temperature.

**DevitrificationAssessment.** Ordinal devitrification screen for a schedule. Not a predicted fraction.

## Functions

*require\_molar\_volume*

```
require_molar_volume(quantity: object, name: str) -> Quantity
```

Assert that quantity is a molar volume, returning it unchanged.

Mirrors `ae.core.units.require_dimensionality` for a kind the core registry does not define. Raises `TypeError` for a bare number and `DimensionalityError` for the wrong dimension, so the guard behaves identically to the core checks in tests.

*order\_parameter*

```
order_parameter(temperature: Quantity, params: LandauParameters=QUARTZ_LANDAU, pressure: Quantity | None=None) -> float
```

Landau order parameter  $Q$  of the alpha to beta inversion.

Implements equation (1) of the module docstring.

**Parameters.** *temperature* Temperature as a pint Quantity, any temperature unit. *params* Landau parameters. Defaults to quartz; pass another set for a different mineral so that nothing in this module is hardcoded to one phase. *pressure* Pressure. Defaults to `params.p_0`.

**Returns.** *float* Order parameter in  $[0, 1]$ , zero at and above  $T_c$ .

**Examples.**

```
>>> from ae.core.units import Q_
>>> round(order_parameter(Q_(298.15, "K")), 5)
0.89721
>>> order_parameter(Q_(1000.0, "K"))
0.0
```

*landau\_critical\_temperature*

```
landau_critical_temperature(params: LandauParameters=QUARTZ_LANDAU, pressure: Quantity | None=None) -> Quantity
```

Critical temperature  $T_c$  at pressure, from  $T_c = T_{c0} + V_D (P - P_0) / S_D$ .

**Examples.**

```
>>> from ae.core.units import Q_
>>> round(float(landau_critical_temperature().to("K").magnitude), 2)
847.0
```

*excess\_molar\_volume*

```
excess_molar_volume(temperature: Quantity, params: LandauParameters=QUARTZ_LANDAU, pressure: Quantity | None=None) -> Quantity
```

Transition-related excess molar volume  $V_{ex} = V_D Q^2$ ,  $\text{m}^3/\text{mol}$ .

**Examples.**

```
>>> from ae.core.units import Q_
>>> v = excess_molar_volume(Q_(298.15, "K"))
>>> round(float(v.to("cm**3/mol").magnitude), 5)
0.95632
```

*alpha\_beta\_cumulative\_volume\_strain*

```
alpha_beta_cumulative_volume_strain(t_from: Quantity, t_to: Quantity, params: LandauParameters=QUARTZ_LAND
    \ AU, pressure: Quantity | None=None) -> float
```

Cumulative transition-related volumetric strain between two temperatures.

This is quantity (a) of the module docstring: the expansion attributable to the order-disorder transition accumulated over a heating interval, NOT the step at  $T_c$ . It is the quantity that matches the 3.5 to 4.5 vol% figures quoted for the alpha to beta inversion in the trade literature.

**Parameters.**  $t_{\text{from}}$ ,  $t_{\text{to}}$  Interval endpoints. Positive strain means expansion going from  $t_{\text{from}}$  to  $t_{\text{to}}$ . `params`, `pressure` As for `order_parameter`.

**Returns.** float Volumetric strain, dimensionless.

**Examples.** Heating alpha-quartz from 298.15 K to above  $T_c$  releases the whole ordering volume:

```
>>> from ae.core.units import Q_
>>> round(alpha_beta_cumulative_volume_strain(Q_(298.15, "K"), Q_(900.0, "K")), 5)
0.04215
```

*cristobalite\_onset\_temperature*

```
cristobalite_onset_temperature(starting_phase: Polymorph, contact: ContactMaterial=ContactMaterial.NONE) -
    \ > Value
```

Practical cristobalite onset temperature for a starting phase and contact material.

Crystalline quartz and fused silica have very different onsets, and the onset for glass drops further in contact with graphite or alumina. Warden et al. 2024 state that cristobalite "forms at above 1470 degC in its pure form and above 1000 degC for quartz glass", and measured layer thickness after 3 h at 1500 degC in the order graphite > alumina > silicon carbide.

**Parameters.** `starting_phase` The silica phase being held. Glass and amorphous silica take the 1000 degC onset. `contact` Material in contact with the silica.

**Returns.** Value Onset temperature, SOURCED where Warden et al. state a number and ASSUMED where this function has to interpolate, with the basis recorded either way.

**Notes.** The brief groups graphite, SiC and alumina together as dropping the onset to roughly 1000 degC. Warden et al. 2024 contradict the SiC part: SiC gave the THINNEST cristobalite layer of the three contact materials, i.e. it retarded rather than promoted the transformation relative to graphite and alumina. That correction is carried here.

*arrhenius\_rate\_ratio*

```
arrhenius_rate_ratio(t_ref: Quantity, t_query: Quantity, activation_energy: Value | None=None, allow_extra
    \ polation: bool=False) -> float
```

Ratio  $k(t_{\text{query}}) / k(t_{\text{ref}})$  for a thermally activated transformation.

Implements equation (3). Only the activation energy is required because the pre-exponential factor cancels in the ratio, which is what makes this computable from a source that reports  $E_a$  without  $A$ .

**Parameters.** `t_ref`, `t_query` Reference and query temperatures. `activation_energy` Molar activation energy as a Value. Defaults to the Breneman and Halloran 2014 cristobalite value of 555 kJ/mol. `allow_extrapolation` The default `E_a` is calibrated on 1200 to 1350 degC data and Breneman and Halloran report the rate MAXIMUM at 1500 to 1600 degC, so an Arrhenius extrapolation above 1350 degC overstates the rate. Outside 1473 to 1623 K this raises unless set True.

**Returns.** float Strictly positive rate ratio, greater than one when `t_query` exceeds `t_ref`.

**Examples.** Doubling-scale sensitivity between 1450 and 1500 degC:

```
>>> from ae.core.units import Q_
>>> r = arrhenius_rate_ratio(Q_(1723.15, "K"), Q_(1773.15, "K"), allow_extrapolation=True)
>>> r > 1.0
True
```

### *jmak\_fraction*

`jmak_fraction(hold_time: Quantity, rate_constant: Value, avrami_n: Value | None=None) -> float`

Transformed fraction from the JMAK law  $X = 1 - \exp[-(k t)^n]$

**Parameters.** `hold_time` Isothermal hold time. `rate_constant` Rate constant  $k$  with dimension 1/time. REQUIRED, deliberately not defaulted: no source reachable from this environment reports a pre-exponential factor for the quartz to cristobalite transformation, and Ringdalen 2015 measured a 76-fold spread in cristobalite fraction across natural quartz samples at fixed temperature and time, so  $k$  is an ore property that must be measured. `avrami_n` Avrami exponent. Defaults to the Breneman and Halloran 2014 value of 3.0.

**Returns.** float Transformed fraction in  $[0, 1]$ .

**Examples.** With  $k t = 1$  and  $n = 3$ ,  $X = 1 - \exp(-1) = 0.6321$ :

```
>>> from ae.core.units import Q_
>>> from ae.core.provenance import Tag, Value
>>> k = Value(quantity=Q_(1.0, "1/hour"), tag=Tag.ASSUMED, basis="doctest")
>>> round(jmak_fraction(Q_(1.0, "hour"), k), 4)
0.6321
```

### *crossed\_transitions*

`crossed_transitions(schedule: ThermalSchedule, feedstock: Feedstock, transitions: tuple[Transition, ...]=TRANSITIONS) -> tuple[Transition, ...]`

Which transitions a schedule crosses, in ascending temperature order.

**Parameters.** `schedule` The furnace recipe. `feedstock` The ore. Present because a transition set is a property of the material, and because an uncharacterized feedstock makes the answer a scenario rather than a prediction (see Notes). `transitions` Transition set to test against. Defaults to TRANSITIONS.

**Returns.** tuple of Transition Every transition whose temperature lies in the interval actually traversed, including the start temperature, lowest first.

**Notes.** Crossing a boundary is necessary but not sufficient for the product phase to appear. Reconstructive transitions need time and often a mineraliser, so read a RECONSTRUCTIVE entry as "permitted" and a DISPLACIVE entry as "occurred". For an uncharacterized feedstock the mineraliser inventory is unknown, which is the dominant uncertainty on the reconstructive entries: `feedstock.characterized` is `False` makes every reconstructive result a scenario.

**Examples.** An 800 degC calcination crosses only the inversion:

```
>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> s = ThermalSchedule(start_temperature=Q_(298.15, "K"),
...                     segments=(ScheduleSegment(target_temperature=Q_(1073.15, "K"),
...                                             hold_time=Q_(2.0, "hour")),))
>>> [t.name for t in crossed_transitions(s, f)]
['alpha_beta_quartz_inversion']
```

*devitrification\_risk*

```
devitrification_risk(schedule: ThermalSchedule, feedstock: Feedstock) -> DevitrificationAssessment
```

Flag devitrification risk for a schedule, as an ordinal level with its drivers.

Risk is built from four factors, each of which is reported in `drivers` so the caller can see what produced the level: the margin of the peak temperature above the onset for the starting phase, the hold time above that onset, the contact material, and the atmosphere.

**Parameters.** `schedule` The furnace recipe, including contact material and atmosphere. `feedstock` The ore. An uncharacterized feedstock sets `is_scenario`, because the alkali and alkaline-earth inventory that mineralises the reconstructive transformation is then unknown.

**Returns.** `DevitrificationAssessment`

**Notes.** Cristobalite matters twice over in this business. In a fused quartz crucible it causes "significant structural defects in silicon ingots" (Warden et al. 2024). In a silicon smelter feed it is not purely harmful: Ringdalen 2015 reports a higher specific surface area for cristobalite than for quartz and a correspondingly higher rate for SiO<sub>2</sub> reduction reactions, so the same transformation that ruins a crucible can help a submerged arc furnace. This function reports risk, and the caller decides its sign.

**Examples.**

```
>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> s = ThermalSchedule(start_temperature=Q_(298.15, "K"),
...                     segments=(ScheduleSegment(target_temperature=Q_(1073.15, "K"),
...                                             hold_time=Q_(2.0, "hour")),))
>>> devitrification_risk(s, f).level is RiskLevel.NONE
True
```

## 17.3 Worked examples, reproduced by execution

The 8 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

### `physics.phases.alpha_beta_cumulative_volume_strain`

```
>>> from ae.core.units import Q_
>>> round(alpha_beta_cumulative_volume_strain(Q_(298.15, "K"), Q_(900.0, "K")), 5)
0.04215
```

### `physics.phases.arrhenius_rate_ratio`

```
>>> from ae.core.units import Q_
>>> r = arrhenius_rate_ratio(Q_(1723.15, "K"), Q_(1773.15, "K"), allow_extrapolation=True)
>>> r > 1.0
True
```

### `physics.phases.crossed_transitions`

```
>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> s = ThermalSchedule(start_temperature=Q_(298.15, "K"),
...                     segments=(ScheduleSegment(target_temperature=Q_(1073.15, "K"),
...                                             hold_time=Q_(2.0, "hour")),))
>>> [t.name for t in crossed_transitions(s, f)]
['alpha_beta_quartz_inversion']
```

### `physics.phases.devitrification_risk`

```
>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> s = ThermalSchedule(start_temperature=Q_(298.15, "K"),
...                     segments=(ScheduleSegment(target_temperature=Q_(1073.15, "K"),
...                                             hold_time=Q_(2.0, "hour")),))
>>> devitrification_risk(s, f).level is RiskLevel.NONE
True
```

### `physics.phases.excess_molar_volume`

```
>>> from ae.core.units import Q_
>>> v = excess_molar_volume(Q_(298.15, "K"))
>>> round(float(v.to("cm**3/mol").magnitude), 5)
0.95632
```

### `physics.phases.jmak_fraction`

```
>>> from ae.core.units import Q_
>>> from ae.core.provenance import Tag, Value
>>> k = Value(quantity=Q_(1.0, "1/hour"), tag=Tag.ASSUMED, basis="doctest")
>>> round(jmak_fraction(Q_(1.0, "hour"), k), 4)
0.6321
```

**physics.phases.landau\_critical\_temperature**

```
>>> from ae.core.units import Q_
>>> round(float(landau_critical_temperature().to("K").magnitude), 2)
847.0
```

**physics.phases.order\_parameter**

```
>>> from ae.core.units import Q_
>>> round(order_parameter(Q_(298.15, "K")), 5)
0.89721
>>> order_parameter(Q_(1000.0, "K"))
0.0
```

## 17.4 Validation status

5 hand-traceable worked example(s) and 3 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                 | Kind             | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_benchmark_inversion_temperature                      | self consistency | Landau Tc_0 against the textbook alpha to beta quartz inversion temperature. Literature: the inversion is universally quoted at 573 degC = 846.15 K, and the platform brief states 573 C. Model: the ds62 Landau parameter Tc_0 = 847 K = 573.85 degC. Error: $(847 - 846.15) / 846.15 = +0.10$ percent on the<br>Error stated: +0.10<br>10.1007/s11837-014-1149-y; 10.1007/s11837-014-1149-y; 10.1029/98gl01581; 10.1111/j.1525-1314.1998.00140.x; 10.1111/j.1525-1314.2010.00923.x; 10.1111/jace.12889    |
| test_benchmark_cumulative_strain_against_the_trade_figure | self consistency | Landau expansion against the 3.7 vol% figure quoted in the platform brief. Literature (trade figure, as stated in the platform brief): roughly 3.7 vol% expansion associated with the alpha to beta quartz inversion. Model (ds62 Landau volume of disordering): 4.2147 vol%. Error: $(4.2147 - 3.7) / 3.7 =$<br>Error stated: +13.9; 14<br>10.1007/s11837-014-1149-y; 10.1007/s11837-014-1149-y; 10.1029/98gl01581; 10.1111/j.1525-1314.1998.00140.x; 10.1111/j.1525-1314.2010.00923.x; 10.1111/jace.12889 |
| test_benchmark_tridymite_boundary_from_gibbs_energies     | literature       | Independently locate the quartz-tridymite boundary from ds62 Gibbs energies. This is a real validation because the transition temperature is NOT an input to the dataset: it falls out of equating G(quartz) and G(tridymite), each built from its own H_0, S_0 and Cp polynomial. Literature: beta-quartz t<br>10.1007/s11837-014-1149-y; 10.1007/s11837-014-1149-y; 10.1029/98gl01581; 10.1111/j.1525-1314.1998.00140.x; 10.1111/j.1525-1314.2010.00923.x; 10.1111/jace.12889                             |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_golden_order_parameter_at_room_temperature
test_golden_cumulative_volume_strain_across_the_inversion
test_golden_step_strain_is_an_order_of_magnitude_smaller
test_golden_arrhenius_rate_ratio
test_golden_jmak_fraction
```

## CHAPTER 18

# physics/packing.py

Module `ae.physics.packing`, 818 lines. Chapter text is this module's own documentation.

## 18.1 Purpose, governing equations and limitations

Maximum packing fraction and filler loading for multimodal silica, Furnas and Andreasen.

The commercial question this module answers: how much silica can an epoxy molding compound hold. More filler means a lower coefficient of thermal expansion, higher thermal conductivity and less mold shrinkage, all of which a packaging customer wants; the ceiling is set first by geometry and then, well before geometry bites, by melt viscosity.

### THE CENTRAL CAVEAT, STATED BEFORE ANY EQUATION

Every maximum-packing number in this module is a GEOMETRIC UPPER BOUND on the solids fraction of a poured and vibrated dry powder. A real formulation does not reach it, and is not trying to. Three reasons, in descending order of importance:

1. **VISCOSITY.** The Krieger-Dougherty relation (equation 5) shows relative viscosity diverging as the solids fraction approaches the maximum packing fraction. A molding compound has to flow through a gate and fill a cavity in seconds, so it must sit well below the divergence. At a maximum packing fraction of 0.70, loading to 0.65 already costs a factor of roughly 101 in relative viscosity against a factor of roughly 9 at 0.50. An 11-fold viscosity penalty for a 15 percent loading gain is the trade that actually sets commercial loading.
2. **WETTING AND BINDER DEMAND.** Every particle surface needs resin. Fine fractions that help geometric packing raise the specific surface area (see `ae.physics.psd`) and so raise the resin demand, which is why the geometric optimum is not the rheological one.
3. **PACKING MODELS ASSUME SPHERES POURED AND VIBRATED, NOT DISPERSED IN A MELT.** McGeary 1961 (doi 10.1111/j.1151-2916.1961.tb13716.x) obtained his numbers with "spherical metal shot" packed "by mechanical vibration" in glass containers. Angular ground silica packs worse; a melt-dispersed system behaves differently again.

This module therefore reports the geometric bound, the viscosity at a chosen loading, and refuses to present the bound as an achievable formulation.

### EQUATIONS

(1) Furnas geometric filling of interstices by successive size classes (Furnas 1931, doi 10.1021/ie50261a017):

$$\phi_{max}^{(N)} = 1 - (1 - \phi_1)^N$$

Symbols:  $\phi_{max}^{(N)}$  maximum solids volume fraction for N size classes, dimensionless, range (0, 1).  $\phi_1$  monomodal packing fraction of one class, dimensionless. 0.625 from McGeary 1961 for vibrated equal spheres; 0.6366 for random close packing (Scott and Kilgour 1969, doi 10.1088/0022-3727/2/6/311). N number of size classes, integer  $\geq 1$ .

Derivation from first principles: class 1 fills a fraction  $\phi_1$  of the total volume and leaves voids  $(1 - \phi_1)$ . If class 2 is small enough that its own packing is unaffected by the class-1 skeleton, it fills  $\phi_1$  of THAT void space, contributing  $\phi_1(1 - \phi_1)$ . By induction the solid fraction of class  $i$  is  $\phi_1(1 - \phi_1)^{i-1}$ , a geometric series whose sum to  $N$  terms is  $1 - (1 - \phi_1)^N$ .

Assumption that limits it: each class must be small enough not to disturb the packing of the class above. McGeary 1961 measured the requirement as "at least a sevenfold difference between sphere sizes of the individual components", so this module refuses to apply equation (1) to a size ladder finer than 7:1 without an explicit override.

(2) Furnas optimal volume composition (same derivation):

$$x_i = \frac{\phi_1(1 - \phi_1)^{i-1}}{\phi_{max}^{(N)}}, \quad i = 1 \dots N$$

Symbols:  $x_i$  the volume fraction of the SOLIDS made up by class  $i$ , coarsest first, dimensionless, summing to 1.

(3) Andreasen continuous grading (Andreasen 1930, doi 10.1007/bf01422986):

$$Q_3(d) = \left( \frac{d}{d_{max}} \right)^q$$

Symbols:  $Q_3$  cumulative volume fraction undersize, dimensionless, range  $[0, 1]$ .  $d$  diameter, m, range  $(0, d_{max}]$ .  $d_{max}$  largest particle, m.  $q$  distribution modulus, dimensionless. Andreasen's own optimum for dense packing is in the 0.33 to 0.5 band; higher  $q$  gives a coarser, less densely packing grading.

(4) Modified Andreasen, i.e. the Dinger-Funk form with a finite minimum size (Funk and Dinger 1994, doi 10.1007/978-1-4615-3118-0):

$$Q_3(d) = \frac{d^q - d_{min}^q}{d_{max}^q - d_{min}^q}$$

Symbols:  $d_{min}$  the smallest particle, m. Reduces to (3) as  $d_{min}$  approaches 0. The finite  $d_{min}$  matters because equation (3) demands infinitely many infinitely fine particles, which is both physically impossible and, through surface area, the thing that destroys the rheology.

(5) Krieger-Dougherty relative viscosity (Krieger and Dougherty 1959, doi 10.1122/1.548848):

$$\eta_r = \left( 1 - \frac{\phi}{\phi_{max}} \right)^{-[\eta] \phi_{max}}$$

Symbols:  $\eta_r$  relative viscosity (suspension over matrix), dimensionless,  $\geq 1$ .  $\phi$  solids volume fraction, dimensionless, range  $[0, \phi_{max}]$ .  $\phi_{max}$  maximum packing fraction, dimensionless.  $[\eta]$  intrinsic viscosity, dimensionless. 2.5 for hard spheres, the Einstein value, and the only value used here.

(6) Volume to mass fraction conversion, needed because packing is volumetric and a datasheet quotes weight percent:

$$w = \frac{\phi \rho_f}{\phi \rho_f + (1 - \phi) \rho_m}$$

Symbols:  $w$  filler mass fraction, dimensionless.  $\rho_f$  filler true density,  $\text{kg/m}^3$  (2650 for crystalline quartz; fused silica is about 2200, which is why a fused-silica EMC at the same volume loading reads lower in weight percent).  $\rho_m$  matrix density,  $\text{kg/m}^3$  (epoxy molding resins are near 1200).

Derivation: mass of filler per unit composite volume is  $\phi \rho_f$ , matrix is  $(1 - \phi) \rho_m$ , and  $w$  is the ratio of the first to their sum. Dimensionally  $[\text{kg/m}^3]/[\text{kg/m}^3]$ , dimensionless, as required.

## LIMITATIONS

1. UPPER BOUND ONLY, NEVER A FORMULATION. See the caveat above. A computed  $\phi_{\text{max}}$  of 0.98 for four size classes is a statement about spheres in a vibrated container, not about a moldable compound. Treat the Krieger-Dougherty viscosity at the intended loading as the binding constraint and  $\phi_{\text{max}}$  as the asymptote it must avoid.
2. FURNAS ASSUMES NON-INTERACTING CLASSES. Equation (1) overestimates when the size ratio between classes is below the sevenfold threshold McGeary 1961 identified, because a finer class then disturbs the coarse skeleton rather than sitting in its voids. This module enforces the threshold by default.
3. EQUATION (1) IS A GEOMETRIC IDEALISATION, AND IT SHOWS. Benchmarked against McGeary's quaternary measurement of 95.1 percent, equation (1) with  $\phi_1 = 0.625$  gives 98.0 percent, a 3.07 percent overestimate, and its optimal composition (63.8, 23.9, 9.0, 3.4 volume percent coarsest-first) differs materially from McGeary's measured optimum (60.7, 23.0, 10.2, 6.1). The model is systematically short of fines. That error is reported by the benchmark test rather than tuned away.
4. NO SHAPE TERM. Every number here is for spheres. Spherical silica filler for advanced packaging is genuinely near-spherical (flame-fused), so the models are defensible for that product and NOT for ground angular filler, which packs to a lower fraction.
5. KRIEGER-DOUGHERTY IS FOR NEWTONIAN SUSPENSIONS OF HARD SPHERES at low to moderate shear. An epoxy molding compound is a filled thermoset that shear-thins, cures while flowing and has a resin whose own viscosity falls steeply with temperature. Use equation (5) for the SHAPE of the loading-viscosity trade, not for an absolute viscosity, and never to predict transfer-molding fill.
6. NO ALPHA-EMISSION MODEL. For HBM and advanced-packaging low-alpha filler, the binding specification is U and Th content (alpha flux), not packing. That is an impurity property carried in `ImpurityProfile` and absent for an uncharacterized deposit; it is not modelled here and it is what actually gates entry to that market.
7. ANDREASEN  $q$  IS NOT OPTIMISED HERE. This module evaluates a grading for a given  $q$ ; it does not solve for the  $q$  that maximises packing, because the optimum depends on  $d_{\text{min}}$ , on the shear rate and on the resin, none of which are inputs.
8. NO COMPRESSIBILITY OR SEGREGATION. Vibrated packing is history-dependent and fine fractions segregate during handling. Neither effect appears in these models.

## TYPE CHECKING NOTE

`mypy --strict` reports `Quantity? has no attribute "to" and Variable "ae.core.units.Quantity" is not valid as a type` against this module. Those errors originate in `ae.core.units`, which declares `Quantity = UREG.Quantity` (a variable binding, not a type alias), so `mypy` cannot treat it as a type. The same errors appear against the four core modules themselves (46 of them), and the core modules are not ours to change. Every annotation here follows the core convention deliberately rather than diverging from it for a clean checker run.

## References

Every entry below was resolved against the Crossref REST API for the DOI shown, and the fields here (author list, year, title, journal, volume, issue, pages) are as Crossref returned them. Resolved 17 September 2026. Entries without a DOI say what was checked instead and what remains unverified.

Furnas, C. C. (1931) Grading Aggregates - I - Mathematical Relations for Beds of Broken Solids of Maximum Density, *Industrial and Engineering Chemistry* 23(9), 1052-1058, doi 10.1021/ie50261a017. Source of equation (1), the geometric filling of interstices by successive size classes.

McGeary, R. K. (1961) Mechanical Packing of Spherical Particles, *Journal of the American Ceramic Society* 44(10), 513-522, doi 10.1111/j.1151-2916.1961.tb13716.x. Source of the 0.625 vibrated monomodal packing fraction, of the sevenfold size-ratio requirement, and of the quaternary 1:7:38:316 optimum. Closed access; abstract and secondary citations only from this sandbox.

Andreasen, A. H. M. (1930) Ueber die Beziehung zwischen Kornabstufung und Zwischenraum in Produkten aus losen Koernern (mit einigen Experimenten), *Kolloid-Zeitschrift* 50(3), 217-228, doi 10.1007/bf01422986. Source of equation (3), the continuous grading law.

Funk, J. E. and Dinger, D. R. (1994) Predictive Process Control of Crowded Particulate Suspensions, Springer US, doi 10.1007/978-1-4615-3118-0. Source of equation (4), the modified Andreasen form with a finite minimum size. Book; Crossref returns no page range.

Krieger, I. M. and Dougherty, T. J. (1959) A Mechanism for Non-Newtonian Flow in Suspensions of Rigid Spheres, *Transactions of the Society of Rheology* 3(1), 137-152, doi 10.1122/1.548848. Source of equation (5), the relative-viscosity law, and of the intrinsic-viscosity value 2.5 for spheres.

Scott, G. D. and Kilgour, D. M. (1969) The density of random close packing of spheres, *Journal of Physics D: Applied Physics* 2(6), 863-866, doi 10.1088/0022-3727/2/6/311. Source of the 0.6366 random-close-packing fraction.

## 18.2 Interface

### Types

**SizeClass.** One size class in a multimodal blend.

```
d_m(self) -> float
```

**FurnasResult.** Furnas maximum packing with its composition and its own health warning.

**FillerLoading.** Filler loading result: the geometric ceiling, the chosen loading, and the penalty.

### Functions

*furnas\_max\_packing*

```
furnas_max_packing(size_classes: tuple[SizeClass, ...], phi_monomodal: Value=PHI_MONOMODAL_VIBRATED, enforce_size_ratio: bool=False) -> FurnasResult
```

Maximum packing fraction and optimal composition, equations (1) and (2).

**Parameters.** *size\_classes* Size classes, any order; they are sorted coarsest-first internally. At least one. *phi\_monomodal* Packing fraction of a single class. Defaults to McGeary's vibrated 0.625; pass *PHI\_RANDOM\_CLOSE* for the poured random-close-packing value instead. *enforce\_size\_ratio* Raise when

successive classes are closer than the sevenfold ratio McGeary 1961 found necessary. Defaults False, which returns a `ratio_warning` instead, because McGeary's OWN measured quaternary optimum (1:7:38:316) contains a  $38/7 = 5.43$  step: a strict sevenfold gate would reject the very experiment that calibrates the model. Set True when you want the closeness treated as a hard error.

**Returns.** `FurnasResult`

**Examples.** McGeary's four size classes, ratios 1:7:38:316:

```
>>> from ae.core.units import Q_
>>> classes = tuple(SizeClass(diameter=Q_(d, "um")) for d in (316.0, 38.0, 7.0, 1.0))
>>> r = furnas_max_packing(classes)
>>> round(r.phi_max, 5)
0.98022
>>> [round(100 * x, 2) for x in r.composition]
[63.76, 23.91, 8.97, 3.36]
```

*furnas\_optimal\_composition*

```
furnas_optimal_composition(n_classes: int, phi_monomodal: Value=PHI_MONOMODAL_VIBRATED) -> tuple[float, ..
    \ .]
```

Optimal volume composition of the solids, coarsest first, equation (2).

**Examples.**

```
>>> [round(100 * x, 2) for x in furnas_optimal_composition(4)]
[63.76, 23.91, 8.97, 3.36]
```

*andreasen\_cumulative*

```
andreasen_cumulative(diameter: Quantity, d_max: Quantity, q: float) -> float
```

Andreasen cumulative volume undersize, equation (3).

**Examples.**

```
>>> from ae.core.units import Q_
>>> round(andreasen_cumulative(Q_(10.0, "um"), Q_(100.0, "um"), 0.5), 6)
0.316228
```

*andreasen\_modified\_cumulative*

```
andreasen_modified_cumulative(diameter: Quantity, d_min: Quantity, d_max: Quantity, q: float) -> float
```

Modified Andreasen (Dinger-Funk) cumulative undersize, equation (4).

**Examples.** With `d_min` four orders of magnitude below `d_max` the result approaches equation (3):

```
>>> from ae.core.units import Q_
>>> round(andreasen_modified_cumulative(Q_(10.0, "um"), Q_(0.01, "um"),
...                                     Q_(100.0, "um"), 0.5), 6)
0.309321
```

*krieger\_dougherty\_relative\_viscosity*

```
krieger_dougherty_relative_viscosity(phi: float, phi_max: float, intrinsic_viscosity: Value=EINSTEIN_INTRINSIC_VISCOSITY) -> float
```

Relative viscosity of a filled suspension, equation (5).

**Parameters.** *phi* Solids volume fraction, strictly below *phi\_max*. *phi\_max* Maximum packing fraction. *intrinsic\_viscosity* Intrinsic viscosity, default the Einstein hard-sphere value of 2.5.

**Returns.** *float* Relative viscosity,  $\geq 1$  and diverging as *phi* approaches *phi\_max*.

**Examples.**

```
>>> round(krieger_dougherty_relative_viscosity(0.50, 0.70), 4)
8.9561
>>> round(krieger_dougherty_relative_viscosity(0.65, 0.70), 4)
101.3267
```

*volume\_to\_mass\_fraction*

```
volume_to_mass_fraction(phi: float, filler_density: Quantity, matrix_density: Quantity) -> float
```

Filler mass fraction from its volume fraction, equation (6).

**Examples.** Crystalline silica at 2650 kg/m<sup>3</sup> in an epoxy at 1200 kg/m<sup>3</sup>, 65 volume percent:

```
>>> from ae.core.units import Q_
>>> round(volume_to_mass_fraction(0.65, Q_(2650.0, "kg/m**3"), Q_(1200.0, "kg/m**3")), 5)
0.80397
```

*mass\_to\_volume\_fraction*

```
mass_to_volume_fraction(w: float, filler_density: Quantity, matrix_density: Quantity) -> float
```

Filler volume fraction from its mass fraction, the inverse of equation (6).

**Examples.**

```
>>> from ae.core.units import Q_
>>> round(mass_to_volume_fraction(0.80397, Q_(2650.0, "kg/m**3"),
...                               Q_(1200.0, "kg/m**3")), 5)
0.65
```

*emc\_filler\_loading*

```
emc_filler_loading(feedstock: Feedstock, size_classes: tuple[SizeClass, ...], phi_selected: float, filler_
\ density: Quantity, matrix_density: Quantity, phi_monomodal: Value=PHI_MONOMODAL_VIBRATED, enforce_si
\ ze_ratio: bool=False) -> FillerLoading
```

Filler loading for an epoxy molding compound: ceiling, choice, viscosity penalty.

**Parameters.** `feedstock` The silica source. Required by platform rule, and used to mark the result a scenario when the ore is uncharacterized. `size_classes` The filler size ladder. `phi_selected` The intended solids volume fraction, strictly below the geometric ceiling. `filler_density`, `matrix_density` True densities, needed for the volume-to-mass conversion a datasheet needs. `phi_monomodal` Single-class packing fraction. `enforce_size_ratio` As for `furnas_max_packing`.

**Returns.** `FillerLoading`

**Notes.** For a low-alpha filler aimed at HBM or advanced packaging, packing is not the binding constraint: U and Th content is, because alpha emission causes soft errors. That is an impurity property (`feedstock.impurities`), it is unmeasured for an uncharacterized deposit, and this function neither models nor implies it.

**Examples.**

```
>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> classes = tuple(SizeClass(diameter=Q_(d, "um")) for d in (30.0, 4.0, 0.5))
>>> r = emc_filler_loading(f, classes, 0.65, Q_(2200.0, "kg/m**3"),
...                       Q_(1200.0, "kg/m**3"))
>>> round(r.phi_max_geometric, 5), round(r.mass_fraction_selected, 4)
(0.94727, 0.773)
```

## 18.3 Worked examples, reproduced by execution

The 8 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

`physics.packing.andreasen_cumulative`

```
>>> from ae.core.units import Q_
>>> round(andreasen_cumulative(Q_(10.0, "um"), Q_(100.0, "um"), 0.5), 6)
0.316228
```

`physics.packing.andreasen_modified_cumulative`

```
>>> from ae.core.units import Q_
>>> round(andreasen_modified_cumulative(Q_(10.0, "um"), Q_(0.01, "um"),
...                                     Q_(100.0, "um"), 0.5), 6)
0.309321
```

`physics.packing.emc_filler_loading`

```
>>> from ae.core.units import Q_
>>> from ae.core.feedstock import Feedstock, OreType
>>> f = Feedstock(sample_id="AE-Q-IN-VKB-001", ore_type=OreType.VEIN_QUARTZ,
...               deposit_name="Vikarabad", country="IN")
>>> classes = tuple(SizeClass(diameter=Q_(d, "um")) for d in (30.0, 4.0, 0.5))
>>> r = emc_filler_loading(f, classes, 0.65, Q_(2200.0, "kg/m**3"),
...                       Q_(1200.0, "kg/m**3"))
>>> round(r.phi_max_geometric, 5), round(r.mass_fraction_selected, 4)
(0.94727, 0.773)
```

**physics.packing.furnas\_max\_packing**

```
>>> from ae.core.units import Q_
>>> classes = tuple(SizeClass(diameter=Q_(d, "um")) for d in (316.0, 38.0, 7.0, 1.0))
>>> r = furnas_max_packing(classes)
>>> round(r.phi_max, 5)
0.98022
>>> [round(100 * x, 2) for x in r.composition]
[63.76, 23.91, 8.97, 3.36]
```

**physics.packing.furnas\_optimal\_composition**

```
>>> [round(100 * x, 2) for x in furnas_optimal_composition(4)]
[63.76, 23.91, 8.97, 3.36]
```

**physics.packing.krieger\_dougherty\_relative\_viscosity**

```
>>> round(krieger_dougherty_relative_viscosity(0.50, 0.70), 4)
8.9561
>>> round(krieger_dougherty_relative_viscosity(0.65, 0.70), 4)
101.3267
```

**physics.packing.mass\_to\_volume\_fraction**

```
>>> from ae.core.units import Q_
>>> round(mass_to_volume_fraction(0.80397, Q_(2650.0, "kg/m**3"),
...                               Q_(1200.0, "kg/m**3")), 5)
0.65
```

**physics.packing.volume\_to\_mass\_fraction**

```
>>> from ae.core.units import Q_
>>> round(volume_to_mass_fraction(0.65, Q_(2650.0, "kg/m**3"), Q_(1200.0, "kg/m**3")), 5)
0.80397
```

## 18.4 Validation status

5 hand-traceable worked example(s) and 2 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                     | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_benchmark_furnas_against_mcgeary_quaternary</code> | literature | Furnas equation (1) against McGeary's measured quaternary packing. Literature: McGeary 1961 (doi 10.1111/j.1151-2916.1961.tb13716.x) measured a quaternary sphere packing at 95.1 percent of theoretical density, with diameter ratios 1:7:38:316 and volume compositions 6.1:10.2:23.0:60.7 percent. Model: Error stated: 95.1; 60.7; +3.07; 3.36 10.1111/j.1151-2916.1961.tb13716.x |

| Benchmark                                             | Kind             | Claim                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_benchmark_monomodal_against_random_close_packing | self consistency | McGeary's vibrated monomodal value against the random-close-packing limit. Literature: Scott and Kilgour 1969 (doi 10.1088/0022-3727/2/6/311) give random close packing of equal spheres as 0.6366. McGeary 1961 measured 0.625 for vibrated one-size spheres. Difference: $(0.625 - 0.6366) / 0.6366 = -1.82$<br>Error stated: -1.82; 2<br>10.1088/0022-3727/2/6/311 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_golden_furnas_packing_fractions
test_golden_furnas_optimal_composition
test_golden_krieger_dougherty_viscosity_penalty
test_golden_volume_to_mass_fraction
test_golden_andreasen_grading
```

## PART III

# Plant

Composition of unit operations into a flowsheet: stream balances, capacity, scheduling and the yield cascade.

## CHAPTER 19

# plant/streams.py

Module `ae.plant.streams`, 552 lines. Chapter text is this module's own documentation.

## 19.1 Purpose, governing equations and limitations

Process streams and flowsheet mass balance with recycle solved as a linear system.

A flowsheet is a directed graph of unit operations connected by streams. Each stream carries a solids mass flow and a per-element impurity composition. Units split and transform those flows. With recycle loops present the flows cannot be evaluated by a single forward pass, so the balance is solved simultaneously.

### Governing equations

**\*\*Overall solids balance at unit\*\***  $u$ , with feeds  $F_i$  and products  $P_j$ :

$$\sum_{i \in \text{in}(u)} \dot{m}_i = \sum_{j \in \text{out}(u)} \dot{m}_j$$

where  $\dot{m}$  is solids mass flow (kg/s, range 0 to 1e4).

**\*\*Element balance\*\*** for element  $e$ , with  $c_{i,e}$  the mass ratio of  $e$  in stream  $i$  (kg/kg, range 0 to 1):

$$\sum_{i \in \text{in}(u)} \dot{m}_i c_{i,e} = \sum_{j \in \text{out}(u)} \dot{m}_j c_{j,e} + \dot{r}_{u,e}$$

where  $\dot{r}_{u,e}$  is the rate of removal of  $e$  to a reject or effluent stream (kg/s). Removal is not annihilation: every unit's rejected mass is tracked in an explicit reject stream, so the plant-wide balance closes.

**\*\*Recycle solution, as actually implemented.\*\*** For a fixed split specification the unit operations are linear in flow, so the recycle problem admits a direct matrix solution  $(\mathbf{I} - \mathbf{A})\mathbf{x} = \mathbf{b}$  with  $\mathbf{A}$  the routing matrix of split fractions. **\*\*This module does not use that formulation.\*\*** `Flowsheet.solve` dispatches on topology:

- **\*\*Acyclic flowsheet\*\*** (no link routes a stream to an earlier unit): ONE forward pass in declaration order, no iteration and no damping, reported as `method="single_pass"` with `iterations=1`. The pass is exact, because with no recycle every unit's feed is fully determined before it is evaluated.
- **\*\*Flowsheet with recycle\*\***: DAMPED SUCCESSIVE SUBSTITUTION on the recycled streams, reported as `method="successive_substitution"` with the iteration count taken.

The recycle iteration is

$$\mathbf{x}^{(k+1)} = (1 - \alpha) \mathbf{x}^{(k)} + \alpha \mathbf{G}(\mathbf{x}^{(k)})$$

where  $\mathbf{x}$  collects the recycled stream flows,  $\mathbf{G}$  is one forward pass through the flowsheet and  $\alpha$  is the damping factor (`damping`, default 0.5, range 0 to 1). Iteration stops when the largest change in a recycled stream's mass flow falls below `tol`.

Successive substitution was chosen because it extends unchanged to the nonlinear case (a unit whose selectivity depends on its own feed grade, which is true of flotation and of leaching at low acid-to-solids

ratio), where no constant  $\mathbf{A}$  exists. The cost is that convergence is not guaranteed: the loop gain plays the role the spectral radius of  $\mathbf{A}$  would, and a gain at or above unity describes a loop that gains mass. That case is detected empirically, by the iteration-to-iteration change `RISING` for more than five iterations, and raises rather than being iterated to a nonsense answer. A non-converged run within `max_iter` is reported as `converged=False`, never dressed as a solution.

A direct linear solve would be faster and would give an exact convergence criterion for the linear case. It is not implemented here; see `LIMITATIONS`.

## Sources

The linear recycle formulation is textbook: Himmelblau and Riggs, *\*Basic Principles and Calculations in Chemical Engineering\**, 8th ed., Prentice Hall 2012, chapter on recycle, bypass and purge. The two-product formula used by `ae.physics.separation` is the mineral-processing special case of the same element balance (Wills and Finch, *\*Wills' Mineral Processing Technology\**, 8th ed., Butterworth-Heinemann 2015, chapter 3).

## LIMITATIONS

- Steady state only. Batch scheduling, surge capacity and startup transients are handled by the discrete-event model in `ae.plant.scheduling`, not here.
- No direct linear solve. Acyclic flowsheets take one exact forward pass, which needs no solver at all. Every flowsheet WITH RECYCLE goes through damped successive substitution, including the linear ones where a matrix solve would be faster and would supply an exact convergence criterion from the spectral radius of the routing matrix. Consequence, for recycle cases only: convergence is diagnosed empirically, so a loop with gain very close to unity may exhaust `max_iter` and report `converged=False` on a problem a direct solve would settle exactly. Raising `max_iter` is not the fix; check the routing.
- Nonlinear units (`grade_dependent` set) are handled by the same iteration, which is why it was chosen, but for those no convergence guarantee exists at all, and a converged result is a fixed point rather than a proven unique solution.
- Liquid phase is not balanced here. Acid and water balances live in `ae.physics.reagents`, because their stoichiometry couples to impurity load rather than to solids flow.
- No energy balance. See `ae.physics.thermal`.

## 19.2 Interface

### Types

**MassBalanceError.** Raised when a balance cannot be closed or a specification is unphysical.

**Stream.** A solids stream with per-element composition.

**Parameters.** `name` Unique label. `mass_flow` Solids mass flow, a Quantity with dimensionality `[mass]/[time]`. `composition` Element symbol to mass ratio (kg/kg). Absent elements are treated as not-tracked, NOT as zero: a composition that omits Ti cannot be used to claim Ti is absent, and `element_flow` raises for an untracked element rather than returning zero.

```
kg_s(self) -> float
```

```
element_flow(self, element: str) -> Quantity
```

Mass flow of one element, raising if that element is not tracked.

```
ppm(self, element: str) -> float
```

**UnitOp.** One unit operation: splits its feed into a product and a reject.

**Parameters.** `name` Unique label. `mass_yield` Fraction of feed solids reporting to the product stream (0 to 1). `element_removal` Per element, the fraction of that element in the feed that reports to the REJECT stream. A removal of 0.9 for Fe means 90 percent of the iron leaves with the reject. Elements absent from this mapping are assumed to follow the solids split with no preferential rejection, which is the physically correct default for an element that the unit does not act on. `grade_dependent` Callable (`element, feed_ppm`) -> `removal_fraction` used instead of the fixed `element_removal` when the unit's selectivity depends on feed grade. Its presence makes the flowsheet nonlinear.

```
max_feasible_feed_fraction(self, element: str) -> float
```

Largest feed mass fraction of `element` this unit can act on.

A fixed `mass_yield` and an independent `element_removal` are not jointly free. Removing fraction  $r$  of an element from a feed of mass fraction  $c$  concentrates it into a reject of relative mass  $1 - Y$ , giving a reject composition of  $cr/(1 - Y)$ . That must not exceed 1, so the pair is feasible only while

$$c \leq (1 - Y)/r$$

Worked: a unit keeping 99 percent of the mass ( $Y = 0.99$ ) and removing 99 percent of an element can only do so for feeds below  $0.01/0.99 = 1.01$  percent of that element. At a 20 percent feed it would need a reject 20 times its own mass, which the balance rejects.

A unit that removes nothing ( $r = 0$ ) has no such limit, and a unit with  $Y = 1$  (no reject at all) can only remove nothing.

```
>>> u = UnitOp(name="leach", mass_yield=0.99, element_removal={"Al": 0.99})
>>> round(u.max_feasible_feed_fraction("Al"), 6)
0.010101
>>> u.max_feasible_feed_fraction("Fe")    # not acted on
1.0
```

```
check_feasible(self, composition: dict[str, float]) -> None
```

Raise if this unit cannot act on `composition` as parameterised.

The solver already catches the resulting impossible reject composition, but only AFTER a run, and with a message about the stream rather than about the parameter pair that caused it. Checking against the feed the unit will actually see reports the cause instead of the symptom, and names the maximum feed the parameters allow.

```
apply(self, feed: Stream) -> tuple[Stream, Stream]
```

Split `feed` into (product, reject), closing both balances exactly.

The element balance is enforced by construction: the reject composition is computed as the residual of the feed element flow minus the product element flow, so no element can be created or destroyed by rounding.

**BalanceResult.** Solved flowsheet state.

```
element_closure(self, element: str) -> float
```

Relative element balance error across the whole flowsheet.

**Flowsheet.** A connected set of unit operations with optional recycle.

Connections are declared as (from\_unit, stream\_kind, to\_unit) where stream\_kind is "product" or "reject". A reject routed back to an upstream unit is a recycle; the solver detects it and handles it rather than looping forever.

```
add(self, unit: UnitOp) -> Flowsheet
```

```
connect(self, source: str, kind: Literal['product', 'reject'], target: str) -> Flowsheet
```

```
has_recycle(self) -> bool
```

True when any link routes a stream to an earlier unit in the order.

```
is_nonlinear(self) -> bool
```

```
solve(self, feed: Stream, first_unit: str | None=None, max_iter: int=200, damping: float=0.5, tol: float=1 \
    \ e-10) -> BalanceResult
```

Solve the steady-state balance.

With no recycle this is a single forward pass in declaration order (method="single\_pass"). With recycle it is damped successive substitution on the recycled streams (method="successive\_substitution"), which converges when the loop gain is below unity and reports failure otherwise rather than returning an unconverged answer. Note that "single\_pass" denotes a forward sweep, NOT a linear matrix solve: no routing matrix is assembled anywhere in this module.

## 19.3 Worked examples, reproduced by execution

The 1 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

```
plant.streams.UnitOp.max_feasible_feed_fraction
```

```
>>> u = UnitOp(name="leach", mass_yield=0.99, element_removal={"Al": 0.99})
>>> round(u.max_feasible_feed_fraction("Al"), 6)
0.010101
>>> u.max_feasible_feed_fraction("Fe")    # not acted on
1.0
```

**A two-unit flowsheet whose mass and element balances must close**

```
from ae.core.units import Q_
from ae.plant.streams import Flowsheet, Stream, UnitOp

feed = Stream(name="feed", mass_flow=Q_(10.0, "tonne/hour"),
```

```

        composition={"Al": 1164e-6, "Fe": 140e-6, "Ti": 50e-6})
print("feed Al          :", feed.ppm("Al"), "ppm")

flot = UnitOp(name="flotation", mass_yield=0.92,
              element_removal={"Al": 0.55, "Fe": 0.70, "Ti": 0.30})
leach = UnitOp(name="leach", mass_yield=0.97,
               element_removal={"Al": 0.40, "Fe": 0.85}, kind="chemical")
flot.check_feasible(feed.composition)
print("max feasible Al feed  :", round(flot.max_feasible_feed_fraction("Al"), 8),
      "(feed is", feed.composition["Al"], ")")
print()
fs_ = Flowsheet("two stage").add(flot).add(leach)
fs_.connect("flotation", "product", "leach")
res = fs_.solve(feed, first_unit="flotation")
print("overall mass yield    :", round(res.overall_yield, 8))
print("closure error         :", f"{res.closure_error:.3e}")
print("converged in          :", res.iterations, "iterations by", res.method)
print()
for el in sorted(feed.composition):
    print(f"element closure {el:3s}      : {res.element_closure(el):.3e}")
print()
for name, s in sorted(res.products.items()):
    print(f"product {name:12s}      : {float(s.mass_flow.to('tonne/hour').magnitude):8.5f} t/h",
          {k: round(v * 1e6, 3) for k, v in sorted(s.composition.items())}, "ppm")

```

Output:

```

feed Al          : 1164.0 ppm
max feasible Al feed : 0.14545455 (feed is 0.001164 )

overall mass yield    : 0.8924
closure error         : 0.000e+00
converged in          : 1 iterations by single_pass

element closure Al    : 0.000e+00
element closure Fe    : 0.000e+00
element closure Ti    : 0.000e+00

product flotation     : 9.20000 t/h {'Al': 569.348, 'Fe': 45.652, 'Ti': 38.043} ppm
product leach         : 8.92400 t/h {'Al': 352.174, 'Fe': 7.06, 'Ti': 38.043} ppm

```

Closure is enforced to floating-point residual on total mass and on every tracked element, so a reported recovery cannot be the result of mass appearing or disappearing between units. An infeasible mass-yield and removal pair is rejected before the solve, naming the parameter rather than the symptom.

## 19.4 Validation status

1 hand-traceable worked example(s) and 0 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_single_unit_worked_example
```

## CHAPTER 20

# plant/capacity.py

Module `ae.plant.capacity`, 354 lines. Chapter text is this module's own documentation.

## 20.1 Purpose, governing equations and limitations

Capacity, OEE and bottleneck identification.

### Governing equations

**\*\*Overall Equipment Effectiveness\*\*** (Nakajima's decomposition, the form used by SEMI E79 for semiconductor equipment):

$$\text{OEE} = A \cdot P \cdot Q$$

with

$$A = \frac{t_{\text{run}}}{t_{\text{planned}}}, \quad P = \frac{r_{\text{actual}}}{r_{\text{nameplate}}}, \quad Q = \frac{n_{\text{good}}}{n_{\text{total}}}$$

where  $A$  is availability (dimensionless, 0 to 1),  $P$  performance efficiency (0 to 1),  $Q$  quality yield (0 to 1),  $t_{\text{run}}$  operating time (h),  $t_{\text{planned}}$  planned production time (h),  $r$  throughput rate (t/h) and  $n$  unit counts.

**\*\*Effective throughput\*\*** of a unit:

$$\dot{m}_{\text{eff}} = r_{\text{nameplate}} \cdot t_{\text{planned}} \cdot \text{OEE}$$

**\*\*Bottleneck\*\*** With unit  $i$  requiring  $\tau_i$  hours of its own capacity per tonne of FINAL product, the line rate is set by

$$\dot{m}_{\text{line}} = \min_i \frac{C_i}{\tau_i}$$

where  $C_i$  is unit  $i$ 's effective capacity. The per-tonne-of-final-product basis is the part that is easy to get wrong: a unit early in the flowsheet must process more tonnes than the plant ships, by the reciprocal of the cumulative yield downstream of it, so comparing nameplate rates directly misidentifies the bottleneck whenever stage yields differ.

### Sources

OEE decomposition: Nakajima, \*Introduction to TPM\*, Productivity Press 1988. The semiconductor-industry formalization with explicit state definitions is SEMI E79 (Specification for Definition and Measurement of Equipment Productivity). Neither is reproduced here; only the algebra is, which is standard.

### LIMITATIONS

- OEE is a steady-state average. It cannot represent a plant whose availability is correlated with throughput (a furnace pushed harder failing more often), and it says nothing about queueing. Batch interaction and surge behaviour need the discrete-event model in `ae.plant.scheduling`.

- Quality yield  $Q$  here is the fraction of output meeting spec at that unit. It is NOT the same as the compounded plant yield, and multiplying stage OEEs to get a plant OEE is a category error: see `ae.plant.yield_cascade`.
- No learning curve. Availability and performance are held constant over the asset life, which understates early-life losses and overstates them later. A first-plant ramp is modelled explicitly in the econ layer instead.

## 20.2 Interface

### Types

**OEE.** Availability, performance and quality, and their product.

Each factor is a bare fraction with an enforced range, following the convention in `ae.core.units` that genuinely dimensionless ratios are range-checked rather than unit-wrapped.

```
value(self) -> float
```

```
loss_breakdown(self) -> dict[str, float]
```

Fraction of nameplate capacity lost to each cause.

The three losses are reported as exclusive contributions in cascade order (availability first, then performance on what remains, then quality), so they sum to exactly  $1 - \text{OEE}$ . Reporting them as  $1 - A$ ,  $1 - P$ ,  $1 - Q$  instead would double-count, because each factor acts on the output of the previous one.

```
from_times(cls, planned_hours: float, downtime_hours: float, nameplate_rate: Quantity, actual_output: Quantity, good_output: Quantity) -> OEE
```

Build OEE from measured times and tonnages.

**Parameters.** `planned_hours` Scheduled production time, excluding planned shutdowns. `downtime_hours` Unplanned stoppage within the planned window. `nameplate_rate` Design rate, dimensionality  $[\text{mass}]/[\text{time}]$ . `actual_output` Total mass produced in the window, good and bad. `good_output` Mass meeting specification.

**UnitCapacity.** One unit's nameplate rate, schedule and effectiveness.

**Parameters.** `name` Unit label, matching the flowsheet unit where applicable. `nameplate_rate` Design throughput, dimensionality  $[\text{mass}]/[\text{time}]$ . `planned_hours` Planned production hours per year. `oee` Effectiveness factors. `tonnes_per_tonne_product` Tonnes this unit must process per tonne of FINAL product, i.e. the reciprocal of the cumulative yield downstream. Defaults to 1.0, which is correct only for the last unit in the line.

```
effective_capacity(self) -> Quantity
```

Own-throughput capacity per year, after OEE.

```
product_capacity(self) -> Quantity
```

Capacity expressed as tonnes of FINAL product per year.

**LineCapacity.** Result of a line assessment.

```
bottleneck_margin(self) -> float
```

Fractional gap between the bottleneck and the next tightest unit.

A small margin means the bottleneck is not robust: a modest improvement moves it elsewhere, so debottlenecking one unit buys little. Reported so that a capital plan is not built on a bottleneck that shifts.

A margin at or below TIE\_TOLERANCE means two or more units bind simultaneously and both must be expanded together to gain any capacity. Use `is_tied` and `tied_units` for that question rather than comparing this value to zero.

AN EARLIER VERSION OF THIS DOCSTRING SAID "Exactly 0.0 means two or more units bind simultaneously, in which case the bottleneck name is arbitrary (first in declaration order)". Both halves are wrong, and they are wrong on the tie example `assess_line`'s own docstring gives. A 10 t/h mill at 1.25 t/t against an 8 t/h leach at 1.0 t/t is an exact tie in real arithmetic, and measured at 8000 planned hours the margin comes back 1.343102e-16 rather than 0.0, because the two capacities are the same real number but not the same float after the OEE and hours multiplications. The name comes back 'leach' although 'mill' is declared first, because `min` correctly returns the true minimum and at one part in 1e16 the leach is genuinely smaller. So the name was not arbitrary-but-deterministic, it was selected by rounding noise, and which unit wins moves with the planned hours.

Infinite for a single-unit line.

```
is_tied(self, tol: float=TIE_TOLERANCE) -> bool
```

Whether two or more units bind within `tol` of the bottleneck.

This is the question a capital plan needs answered, and it cannot be answered by `bottleneck_margin == 0.0`: see that property's note.

```
tied_units(self, tol: float=TIE_TOLERANCE) -> list[str]
```

Every unit binding within `tol` of the tightest, sorted by name.

A single-element list means the bottleneck is unambiguous. Two or more means expanding any one of them alone buys no capacity.

## Functions

*assess\_line*

```
assess_line(units: Iterable[UnitCapacity]) -> LineCapacity
```

Find the bottleneck and the line rate in tonnes of final product per year.

**Examples.** Two units. The FIRST (the mill) sits upstream of yield loss and so must process 1.25 t OF FEED for every 1 t of product the plant ships, which is a feed-to-product RATIO of 1.25 (equivalently a 20 percent loss downstream of it, since  $1/1.25 = 0.80$ ). The second (the leach) is the last stage and processes 1.0 t of feed per t of product. The ratio is never written as a bare "1.25 t/t", which reads as a loss OF 1.25 t and inverts on a careless reading. On nameplate rate alone the mill looks larger, 10 t/h against 9, but on a final-product basis it is the constraint:

```
mill : 10 x 7000 x 0.84645 / 1.25 = 47401.20 t/yr of product
leach:  9 x 7000 x 0.84645 / 1.00 = 53326.35 t/yr of product
```

```
>>> from ae.core.units import Q_
>>> o = OEE(0.90, 0.95, 0.99)
>>> a = UnitCapacity("mill", Q_(10.0, "tonne/hour"), 7000.0, o, 1.25)
>>> b = UnitCapacity("leach", Q_(9.0, "tonne/hour"), 7000.0, o, 1.0)
>>> r = assess_line([a, b])
>>> r.bottleneck
'mill'
>>> round(r.line_rate.to("tonne").magnitude, 2)
47401.2
>>> round(r.bottleneck_margin, 4)
0.125
```

A 10 t/h mill at a feed-to-product ratio of 1.25 and an 8 t/h leach at 1.0 give product capacities that are equal in real arithmetic. That is a tie, not a finding, and both units must be expanded together to gain any capacity. Ask `LineCapacity.is_tied`, NOT `bottleneck_margin == 0.0`: measured at 8000 planned hours that example returns a margin of 1.343102e-16 and names 'leach' despite 'mill' being declared first, because the two capacities differ in their last bits. See the note on `LineCapacity.bottleneck_margin`.

## 20.3 Worked examples, reproduced by execution

The 1 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

### plant.capacity.assess\_line

```
>>> from ae.core.units import Q_
>>> o = OEE(0.90, 0.95, 0.99)
>>> a = UnitCapacity("mill", Q_(10.0, "tonne/hour"), 7000.0, o, 1.25)
>>> b = UnitCapacity("leach", Q_(9.0, "tonne/hour"), 7000.0, o, 1.0)
>>> r = assess_line([a, b])
>>> r.bottleneck
'mill'
>>> round(r.line_rate.to("tonne").magnitude, 2)
47401.2
>>> round(r.bottleneck_margin, 4)
0.125
```

### Nameplate to shippable tonnes through OEE, and where the bottleneck sits

```
from ae.core.units import Q_
from ae.plant.capacity import OEE, UnitCapacity, assess_line

oe = OEE(availability=0.85, performance=0.95, quality=0.99)
print("OEE components      :", oe)
print("OEE product         :", round(oe.availability * oe.performance * oe.quality, 8))
print()
units = [
    UnitCapacity(name="crush", nameplate_rate=Q_(12.0, "tonne/hour"),
                  planned_hours=8760.0, oe=oe, tonnes_per_tonne_product=1.45),
    UnitCapacity(name="flotation", nameplate_rate=Q_(6.0, "tonne/hour"),
                  planned_hours=8000.0, oe=oe, tonnes_per_tonne_product=1.30),
    UnitCapacity(name="leach", nameplate_rate=Q_(5.0, "tonne/hour"),
                  planned_hours=7500.0, oe=oe, tonnes_per_tonne_product=1.05),
]
line = assess_line(units)
print("bottleneck          :", line.bottleneck)
print("line rate           :", round(float(line.line_rate.to("tonne").magnitude), 3), "t/yr")
print()
for name in sorted(line.product_capacity):
    print(f"{name:10s} capacity {float(line.product_capacity[name].to('tonne').magnitude):10.2f} t/yr")
```

```
f" utilisation {line.utilisation[name]:6.4f}"
f" slack {float(line.slack[name].to('tonne')).magnitude:9.2f} t/yr"
```

Output:

```
OEE components      : OEE(availability=0.85, performance=0.95, quality=0.99)
OEE product         : 0.799425

bottleneck          : leach
line rate            : 28550.893 t/yr

crush    capacity    57955.56 t/yr utilisation 0.4926 slack 29404.66 t/yr
flotation capacity    29517.23 t/yr utilisation 0.9673 slack   966.34 t/yr
leach     capacity    28550.89 t/yr utilisation 1.0000 slack     0.00 t/yr
```

Capacity is stated per tonne of PRODUCT, so each unit's nameplate is divided by the tonnes of its own throughput needed per product tonne. The bottleneck is the unit with the least product capacity, not the smallest nameplate.

## 20.4 Validation status

4 hand-traceable worked example(s) and 0 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_oeo_worked_example
test_oeo_from_times_worked_example
test_effective_capacity_worked_example
test_bottleneck_uses_product_basis_not_nameplate
```

## CHAPTER 21

# plant/scheduling.py

Module `ae.plant.scheduling`, 505 lines. Chapter text is this module's own documentation.

## 21.1 Purpose, governing equations and limitations

Discrete-event simulation of batch processing with queues, QC holds and failures.

Why this exists alongside `ae.plant.capacity`: OEE gives a steady-state average and cannot represent queueing. A plant whose average utilisation is 70 percent can still hold lots for days if arrivals are bursty and a downstream unit is a single batch server, because waiting time diverges as utilisation approaches unity even when the unit is nominally under-loaded.

### The queueing result that motivates the simulation

For a single server with utilisation  $\rho = \lambda/\mu$  the expected waiting time in an M/M/1 queue is

$$W_q = \frac{\rho}{\mu(1-\rho)}$$

which diverges as  $\rho \rightarrow 1$ . The Allen-Cunneen approximation generalizes this to arbitrary arrival and service variability,

$$W_q \approx \frac{\rho}{1-\rho} \cdot \frac{c_a^2 + c_s^2}{2} \cdot \frac{1}{\mu}$$

with  $c_a$  and  $c_s$  the coefficients of variation of interarrival and service times (dimensionless, typically 0 to 2). Two consequences the model must reproduce:

1. Waiting time is driven by VARIABILITY as much as by load, but only through the SUM  $c_a^2 + c_s^2$ . Halving  $c_s^2$  therefore cuts the queue by a factor  $(c_a^2 + c_s^2/2)/(c_a^2 + c_s^2)$ , which depends on the arrival variability: from 1.0 to 0.5 at Poisson arrivals ( $c_a = 1$ , the default in `PlantSchedule.run`) gives a 25 percent reduction, not a halving. Only with perfectly regular arrivals ( $c_a = 0$ ) does halving  $c_s^2$  halve the queue. Service-side variability reduction is worth less the burstier the arrivals, which is why scheduling discipline and process consistency have to be attacked together.
2. A batch process with a long fixed cycle (a calcination kiln, an acid leach autoclave) has low  $c_s$ , which is protective, but its long service time raises  $\rho$  for the same arrival rate.

`mm1_waiting_time` and `allen_cunneen_waiting_time` provide the analytic forms, and the simulation is validated against them, which is the only honest way to check a discrete-event model: an analytic limit it must reproduce.

### Symbols

$\lambda$  arrival rate (lots/h, positive),  $\mu$  service rate (lots/h, positive),  $\rho$  utilisation (dimensionless, 0 to 1 strictly),  $W_q$  waiting time in queue (h),  $c_a, c_s$  coefficients of variation (dimensionless, non-negative).

## LIMITATIONS

- Lot sizes are uniform. A plant with mixed lot sizes has a different queueing discipline and this model will understate delay.
- QC hold is modelled as a delay plus a pass/fail draw with a fixed failure probability. In reality failure probability is correlated with the process state that produced the lot, so failures cluster, which this understates.
- Rework is modelled as a single retry at most. Multi-pass rework loops exist in purification (a lot re-leached twice) and are not represented.
- No operator or reagent constraint. A unit is available whenever it is not busy or failed, which overstates availability in a plant that is labour-limited on nights or weekends.
- Failures are exponential (memoryless). Real equipment has wear-out, so this understates late-life downtime clustering.

## 21.2 Interface

### Types

**Station.** One batch processing station.

**Parameters.** `name` Label. `service_hours` Mean processing time per lot. `capacity` Number of parallel units (1 for a single kiln). `cv_service` Coefficient of variation of service time. 0.0 is a deterministic batch cycle, 1.0 is exponential. `failure_mtbh_hours` Mean time between failures. None means no failures. `repair_hours` Mean repair time. `qc_hold_hours` Fixed delay for assay turnaround after processing, 0.0 for none. This is the parameter that most often dominates cycle time in a purification plant, because a lot cannot ship until its assay returns. `qc_fail_probability` Probability a lot fails QC and is either reworked or scrapped. `qc_disposition` What happens on QC failure.

```
service_rate(self) -> float
```

Lots per hour per unit.

```
intrinsic_availability(self) -> float
```

MTBF / (MTBF + MTTR), the availability failures alone imply.

```
draw_service(self, rng: np.random.Generator) -> float
```

Sample a service time with the requested coefficient of variation.

Uses a gamma distribution, which spans deterministic (cv 0) through exponential (cv 1) to more variable, and is strictly positive. A normal would admit negative service times.

**ScheduleResult.** Outcome of a simulation run.

```
throughput_per_hour(self) -> float
```

```
mean_cycle_time(self) -> float
```

```
cycle_time_percentile(self, p: float) -> float
```

A percentile of cycle time. p95 is the number a customer experiences.

```
mean_wip(self) -> float
```

Time-averaged work in process over the MEASUREMENT window.

The integral starts at `warmup_hours`, matching the basis of the counts and the busy fractions. Integrating the full timeline against a trimmed horizon was an accounting mismatch that made `littles_law_check` report a spurious residual.

```
departure_rate_per_hour(self) -> float
```

Departures per hour of EITHER disposition, over the measured window.

Distinct from `throughput_per_hour`, which counts shipped lots only and is the commercial number. Little's law needs this one.

`horizon_hours` on this object is ALREADY the post-warmup window (run stores `eff`, not the wall-clock horizon), so subtracting `warmup_hours` again here overstated the rate: my first version did exactly that and reported 0.77636 departures per hour on a line offered 0.7 per hour, which is impossible in steady state and is what caught the error.

```
mean_residence_time(self) -> float
```

Mean time in system across all departures, shipped or scrapped.

```
littles_law_check(self) -> dict[str, float]
```

Verify  $L = \lambda W$  against the simulation's own numbers.

Little's law is an identity, not a model, so a discrepancy means the simulation's accounting is wrong. Returned rather than asserted so a caller can see the residual.

The identity is about DEPARTURES, not sales. An earlier version used `lots_shipped / horizon` against the mean SHIPPED cycle time, so a line that scraps lots understated both factors: measured at a 30 percent scrap rate (single station, arrival 0.7/h, horizon 20000 h, warmup 2000 h, seed 3), `L_measured` was 2.24732 against `lambda_W` 1.58809, a residual of 0.41511, while the WIP integral itself was correct (2.3333 analytic for  $\rho = 0.7$ ). With no scrap the two bases coincide, which is why every existing check passed.

**PlantSchedule.** A serial line of batch stations, simulated with SimPy.

```
bottleneck_station(self) -> Station
```

Station with the lowest capacity-adjusted service rate.

```
max_arrival_rate(self) -> float
```

Arrival rate at which the bottleneck saturates, lots/h.

```
run(self, arrival_rate: float, horizon_hours: float, seed: int=0, cv_arrival: float=1.0, warmup_hours: flo  
    \ at=0.0) -> ScheduleResult
```

Simulate lot arrivals through the line.

**Parameters.** `arrival_rate` Mean lot arrivals per hour. `horizon_hours` Simulated duration. `cv_arrival` 1.0 gives Poisson arrivals; 0.0 gives a perfectly regular schedule, which is what a well-run campaign plan looks like. `warmup_hours` Transient-removal window. EVERY reported statistic is measured over the post-warmup interval only, on one consistent basis: lots completing before `warmup_hours` are excluded from the cycle-time distribution AND from the shipped, scrapped and reworked counts; station busy and failed fractions accumulate only after it; and the WIP integral is taken over the same interval. Mixing a trimmed denominator with untrimmed counts inflates throughput and makes `ScheduleResult.littles_law_check` report a residual that is an accounting artefact rather than a simulation error.

## Functions

### *mm1\_waiting\_time*

```
mm1_waiting_time(arrival_rate: float, service_rate: float) -> float
```

Expected queue waiting time for M/M/1, in hours.

**Examples.** At  $\lambda = 0.8/\text{h}$  and  $\mu = 1.0/\text{h}$ ,  $\rho = 0.8$  and  $W_q = 0.8/(1.0 \times 0.2) = 4.0$  h.

```
>>> round(mm1_waiting_time(0.8, 1.0), 10)
4.0
```

### *allen\_cunneen\_waiting\_time*

```
allen_cunneen_waiting_time(arrival_rate: float, service_rate: float, cv_arrival: float, cv_service: float)
\ -> float
```

Allen-Cunneen approximation for a G/G/1 queue, in hours.

Reduces to M/M/1 when both coefficients of variation equal 1.

## Examples.

```
>>> round(allen_cunneen_waiting_time(0.8, 1.0, 1.0, 1.0), 6)
4.0
>>> round(allen_cunneen_waiting_time(0.8, 1.0, 1.0, 0.0), 6)
2.0
```

## 21.3 Worked examples, reproduced by execution

The 2 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

### `plant.scheduling.allen_cunneen_waiting_time`

```
>>> round(allen_cunneen_waiting_time(0.8, 1.0, 1.0, 1.0), 6)
4.0
>>> round(allen_cunneen_waiting_time(0.8, 1.0, 1.0, 0.0), 6)
2.0
```

**plant.scheduling.mm1\_waiting\_time**

```
>>> round(mm1_waiting_time(0.8, 1.0), 10)
4.0
```

**M/M/1 closed form against the discrete-event simulation**

```
from ae.plant.scheduling import (PlantSchedule, Station, allen_cunneen_waiting_time,
                                mm1_waiting_time)

import numpy as np

lam, mu = 0.8, 1.0
wq = mm1_waiting_time(lam, mu)
print("rho                :", lam / mu)
print("M/M/1 Wq, closed form :", round(wq, 8))
print("Allen-Cunneen at cv=1  :", round(allen_cunneen_waiting_time(lam, mu, 1.0, 1.0), 8))
print("L = lambda W (Little)  :", round(lam * (wq + 1.0 / mu), 8))
print()
# Exponential service is cv = 1, which is the M/M/1 assumption.
res = PlantSchedule([Station(name="leach", service_hours=1.0 / mu, capacity=1, cv_service=1.0)]
                    ).run(arrival_rate=lam, horizon_hours=200000.0, seed=3, cv_arrival=1.0,
                          warmup_hours=2000.0)
print("lots started / shipped :", res.lots_started, "/", res.lots_shipped)
print("station busy fraction  :", {k: round(v, 6) for k, v in res.station_busy_fraction.items()})
qt = float(np.mean(res.queue_times["leach"]))
print("simulated mean queue t  :", round(qt, 6))
print("relative error vs M/M/1 :", f"{abs(qt - wq) / wq:.4%}")
print()
lc = res.littles_law_check()
print("Little's law check       :", {k: round(v, 8) for k, v in lc.items()})
print("cycle time p50 / p95     :", round(res.cycle_time_percentile(50), 6), "/",
      round(res.cycle_time_percentile(95), 6), "h")
```

Output:

```
rho                : 0.8
M/M/1 Wq, closed form : 4.0
Allen-Cunneen at cv=1 : 4.0
L = lambda W (Little) : 4.0

lots started / shipped : 157793 / 157791
station busy fraction  : {'leach': 0.797698}
simulated mean queue t : 3.888946
relative error vs M/M/1 : 2.7764%

Little's law check      : {'L_measured': 3.89698576, 'lambda_W': 3.89687281, 'relative_error': 2.898e-05}
cycle time p50 / p95    : 3.418736 / 14.665886 h
```

The simulation is checked against the M/M/1 waiting time and against Little's law. Both are analytic identities with no literature datapoint behind them, and the validation record labels them analytic rather than literature for that reason.

**21.4 Validation status**

1 hand-traceable worked example(s) and 2 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                    | Kind     | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_simulation_reproduces_mmi_waiting_time</code>     | analytic | The only honest validation of a DES: an analytic limit it must match. Single exponential station, Poisson arrivals, $\rho = 0.8$ . Analytic $Wq = 4.0$ h. Error is REPORTED, not just bounded.                                                                                                                                                                      |
| <code>test_deterministic_service_matches_mm_d_i_bound</code> | analytic | M/D/1 waiting time is exactly HALF the M/M/1 value at the same $\rho$ (Pollaczek-Khinchine with $cs = 0$ ): $Wq = 0.8 / (2 \times 1.0 \times 0.2) = 2.0$ h, where $0.2 = 1 - \rho$ . Averaged over seeds rather than reported from one. A single run at this horizon carries a standard deviation of roughly 3.6 percent across seed<br>Error stated: 3.6; 7.6; 1.9 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

`test_mm1_worked_example`

## CHAPTER 22

# plant/yield\_cascade.py

Module `ae.plant.yield_cascade`, 312 lines. Chapter text is this module's own documentation.

## 22.1 Purpose, governing equations and limitations

Yield cascade and statistical process control translated to off-spec fraction.

Two distinct things are conflated in casual plant talk and separated here.

**\*\*Mass yield cascade.\*\*** Fraction of feed mass surviving to product:

$$Y = \prod_{i=1}^n y_i$$

with  $y_i$  the mass yield of stage  $i$  (dimensionless, 0 to 1). The reciprocal  $1/\prod_{j>i} y_j$  is what stage  $i$  must process per tonne shipped, which is the basis `ae.plant.capacity` requires.

**\*\*Specification yield.\*\*** Fraction of LOTS meeting a purity specification. This is not a mass loss at all: a lot that assays 34 ppm Al against a 30 ppm limit is a full lot of saleable mass that cannot be sold into that grade. It follows from the process capability, not from the flowsheet.

For a one-sided upper specification limit USL on an impurity, with lot assays distributed  $N(\mu, \sigma^2)$ :

$$f_{\text{off}} = \Pr[X > \text{USL}] = 1 - \Phi\left(\frac{\text{USL} - \mu}{\sigma}\right) = 1 - \Phi(3C_{pk})$$

where the one-sided capability index is

$$C_{pk} = \frac{\text{USL} - \mu}{3\sigma}$$

$\mu$  is the process mean (ppm, positive),  $\sigma$  the lot-to-lot standard deviation (ppm, positive), and  $\Phi$  the standard normal CDF. Inverting gives the process mean a target capability demands:

$$\mu = \text{USL} - 3C_{pk}\sigma$$

which is the operationally useful direction: a customer asks for a capability, and this says how far below the limit the process must actually run.

Worked consequence, at USL = 30 ppm Al and  $C_{pk} = 1.33$ :  $\sigma = 3$  ppm requires  $\mu = 30 - 3(1.33)(3) = 18.0$  ppm, and  $\sigma = 5$  ppm requires  $\mu = 10.0$  ppm. Halving variability is worth more than a large shift in mean, which is why lot-to-lot control, not best-ever assay, is the qualification-relevant quantity.

## Sources

Capability indices and the normal-tail relation are standard SPC: Montgomery, \*Introduction to Statistical Quality Control\*, 7th ed., Wiley 2012, chapter 6. The multi-lot qualification requirement that makes lot-to-lot sigma the binding quantity is the JEDEC change-control regime (JESD46), not a statistical result.

## LIMITATIONS

- Normality is assumed. Impurity assays are bounded below by zero and are frequently right-skewed, so the normal tail UNDERSTATES off-spec fraction in the tail that matters. `off_spec_fraction` therefore also accepts a lognormal model, and `off_spec_fraction_empirical` takes assays directly and makes no distributional assumption at all. Prefer the empirical form once any real lot data exists.
- Independence between lots is assumed. Real plants drift, so consecutive lots are correlated and the effective sample size is smaller than the lot count. That makes a capability estimated from few lots optimistic.
- A capability index computed from fewer than roughly 25 to 30 lots has a wide confidence interval. `capability` reports that interval rather than a bare point estimate, and refuses fewer than 2 lots.
- Multi-element specifications are NOT independent: the same fluid-inclusion population carries Na, K and Li together. `joint_off_spec_fraction` offers both the independent bound and a fully-correlated bound, and the truth lies between them. Do not use the independent product as the answer.

## 22.2 Interface

### Types

**Capability.** One-sided process capability with a confidence interval on Cpk.

```
is_estimated_from_few_lots(self) -> bool
```

True below 25 lots, where the interval is wide enough to mislead.

```
off_spec_fraction(self) -> float
```

### Functions

*cascade\_yield*

```
cascade_yield(stage_yields: Sequence[float]) -> float
```

Product of stage mass yields.

### Examples.

```
>>> round(cascade_yield([0.98, 0.90, 0.95]), 6)
0.8379
```

*stage\_throughput\_factors*

```
stage_throughput_factors(stage_yields: Sequence[float]) -> list[float]
```

Tonnes each stage must process per tonne of FINAL product.

Element  $i$  is  $1/\prod_{j \geq i} y_j$ , i.e. including the stage's own yield, because a stage must feed enough to cover its own loss as well as everything downstream.

**Examples.** Three stages at 0.98, 0.90, 0.95. The last stage handles  $1/0.95 = 1.0526$  t per t shipped, the middle  $1/(0.90 \times 0.95) = 1.1696$ , the first  $1/(0.98 \times 0.90 \times 0.95) = 1.1935$ .

```
>>> [round(x, 4) for x in stage_throughput_factors([0.98, 0.90, 0.95])]
[1.1935, 1.1696, 1.0526]
```

### *capability*

```
capability(assays: Iterable[float], usl: float, confidence: float=0.95) -> Capability
```

One-sided Cpk from lot assays, with a confidence interval.

The interval uses the standard large-sample variance of Cpk (Montgomery 2012, section 8.7):

$$\text{Var}(\hat{C}_{pk}) \approx \frac{1}{9n} \left( 1 + \frac{9\hat{C}_{pk}^2}{2} \right)$$

Reported because a bare point estimate from a handful of lots is the single most over-read number in a qualification package.

### *off\_spec\_fraction*

```
off_spec_fraction(mean: float, sigma: float, usl: float, model: Literal['normal', 'lognormal']='normal') -
    \ > float
```

Fraction of lots exceeding a one-sided upper limit.

**Parameters.** mean, sigma Process mean and lot-to-lot standard deviation, in the same units as usl (typically ppm). For model="lognormal" these are the mean and standard deviation of the UNDERLYING quantity, not of its logarithm, and are converted internally. usl Upper specification limit, positive. model "lognormal" is the better default for an impurity bounded below by zero; the normal model understates the upper tail.

### **Examples.**

```
>>> round(off_spec_fraction(18.0, 3.0, 30.0), 6)
3.2e-05
>>> round(off_spec_fraction(18.0, 3.0, 30.0, model="lognormal"), 6)
0.000765
```

### *off\_spec\_fraction\_empirical*

```
off_spec_fraction_empirical(assays: Iterable[float], usl: float) -> tuple[float, int, int]
```

Observed off-spec fraction with no distributional assumption.

Returns (fraction, n\_off, n\_total). Preferred over the parametric forms as soon as real lot data exists, because it cannot be wrong about the shape of the tail.

### *required\_process\_mean*

```
required_process_mean(usl: float, sigma: float, cpk_target: float) -> float
```

Process mean a target capability demands:  $usl - 3 * cpk * sigma$ .

**Examples.** At a 30 ppm Al limit and Cpk 1.33, a 3 ppm sigma needs an 18.0 ppm mean and a 5 ppm sigma needs a 10.0 ppm mean.

```
>>> round(required_process_mean(30.0, 3.0, 1.33), 2)
18.03
>>> round(required_process_mean(30.0, 5.0, 1.33), 2)
10.05
```

*joint\_off\_spec\_fraction*

```
joint_off_spec_fraction(per_element: dict[str, float], correlation: Literal['independent', 'perfect', 'both'] = 'both') -> dict[str, float]
```

Combine per-element off-spec fractions into a lot-level fraction.

A lot fails if ANY specified element exceeds its limit. The two bounds are

$$f_{\text{indep}} = 1 - \prod_e (1 - f_e), \quad f_{\text{perfect}} = \max_e f_e$$

Independence is almost certainly wrong: the same fluid-inclusion population carries Na, K and Li, and the same feldspar inclusions carry Al and K, so element excursions co-occur. Perfect correlation is the optimistic bound and independence the pessimistic one, and the truth sits between. Returning both is deliberate, so that no caller can quietly adopt the independent product as the answer.

## 22.3 Worked examples, reproduced by execution

The 4 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

**plant.yield\_cascade.cascade\_yield**

```
>>> round(cascade_yield([0.98, 0.90, 0.95]), 6)
0.8379
```

**plant.yield\_cascade.off\_spec\_fraction**

```
>>> round(off_spec_fraction(18.0, 3.0, 30.0), 6)
3.2e-05
>>> round(off_spec_fraction(18.0, 3.0, 30.0, model="lognormal"), 6)
0.000765
```

**plant.yield\_cascade.required\_process\_mean**

```
>>> round(required_process_mean(30.0, 3.0, 1.33), 2)
18.03
>>> round(required_process_mean(30.0, 5.0, 1.33), 2)
10.05
```

**plant.yield\_cascade.stage\_throughput\_factors**

```
>>> [round(x, 4) for x in stage_throughput_factors([0.98, 0.90, 0.95])]
[1.1935, 1.1696, 1.0526]
```

## The multiplicative cascade, and the throughput each stage must carry

```

from ae.plant.yield_cascade import (capability, cascade_yield, joint_off_spec_fraction,
                                    off_spec_fraction, required_process_mean,
                                    stage_throughput_factors)

stages = [0.95, 0.97, 0.88, 0.93, 0.90]
y = cascade_yield(stages)
print("stage yields          :", stages)
print("overall cascade yield  :", round(y, 8))
print("product per 1000 t feed :", round(1000.0 * y, 4), "t")
print()
print("throughput factors, relative to product out:")
for s, fx in zip(stages, stage_throughput_factors(stages)):
    print(f" stage yield {s:.2f} must process {fx:.6f} t per t of its own output")
print()
# Specification conformance on a single element against an upper spec limit.
print("off-spec at mean 22, sigma 4, USL 30:")
for model in ("normal", "lognormal"):
    print(f" {model:10s}: {off_spec_fraction(22.0, 4.0, 30.0, model=model):.8f}")
print("required mean for Cpk 1.33, sigma 4, USL 30:",
      round(required_process_mean(30.0, 4.0, 1.33), 6))
print()
per_el = {"Al": 0.02, "Ti": 0.03, "Fe": 0.01}
print("joint off-spec across three elements:",
      {k: round(v, 8) for k, v in joint_off_spec_fraction(per_el).items()})

```

Output:

```

stage yields          : [0.95, 0.97, 0.88, 0.93, 0.9]
overall cascade yield  : 0.67874004
product per 1000 t feed : 678.74 t

throughput factors, relative to product out:
stage yield 0.95 must process 1.473318 t per t of its own output
stage yield 0.97 must process 1.399652 t per t of its own output
stage yield 0.88 must process 1.357663 t per t of its own output
stage yield 0.93 must process 1.194743 t per t of its own output
stage yield 0.90 must process 1.111111 t per t of its own output

off-spec at mean 22, sigma 4, USL 30:
normal      : 0.02275013
lognormal   : 0.03514887
required mean for Cpk 1.33, sigma 4, USL 30: 14.04

joint off-spec across three elements: {'independent': 0.058906, 'perfect': 0.03, 'spread': 0.028906}

```

A cascade multiplies, so the upstream stages must process more than the product tonnage by the reciprocal of everything downstream of them. The joint off-spec calculation is reported under both independence and perfect correlation because the truth lies between and the assay campaign has not measured which.

## 22.4 Validation status

4 hand-traceable worked example(s) and 0 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_cascade_and_throughput_worked_example  
test_required_process_mean_worked_example  
test_capability_and_off_spec_are_the_same_number_two_ways  
test_joint_bounds_bracket_the_truth
```

PART IV

# Economics

Cost, capital and value, with uncertainty propagated  
rather than asserted.

## CHAPTER 23

# econ/unit\_economics.py

Module `ae.econ.unit_economics`, 384 lines. Chapter text is this module's own documentation.

## 23.1 Purpose, governing equations and limitations

Cash cost per tonne of product, built from the physics rather than asserted.

Every line item traces to a physical quantity computed elsewhere in the platform multiplied by a SITE price, so a cost cannot drift away from the process that generates it. Energy comes from `ae.physics.thermal` and the comminution work index, reagents from `ae.physics.reagents` stoichiometry against the impurity load, and the yield divisor from `ae.plant.yield_cascade`.

### Cost basis

$$C_{\text{cash}} = \frac{1}{Y} \left( \sum_k q_k p_k + \frac{L + M + O}{\dot{m}_{\text{prod}}} \right) - c_{\text{credit}}$$

where  $Y$  is the cascade mass yield (dimensionless, 0 to 1),  $q_k$  the specific consumption of input  $k$  per tonne of FEED (units per tonne),  $p_k$  its unit price at this site (currency per unit),  $L$ ,  $M$  and  $O$  annual labour, maintenance and overhead (currency per year),  $\dot{m}_{\text{prod}}$  annual product output (tonne per year), and  $c_{\text{credit}}$  by-product credit per tonne of product.

The  $1/Y$  is where cost models most often go wrong. Consumables are consumed per tonne of feed processed, not per tonne shipped, so at a 70 percent yield every feed-basis cost is 1.43 times larger on a product basis. Mixing the two bases understates cost by exactly the yield loss.

**\*\*Full cost\*\*** adds capital recovery at a fixed charge rate:

$$C_{\text{full}} = C_{\text{cash}} + \frac{f \text{ CAPEX}}{\dot{m}_{\text{prod}}}$$

with  $f$  the fixed charge rate (1/yr, typically 0.08 to 0.15). This is a levelized approximation, not a substitute for the discounted cash flow in `ae.econ.valuation`; it exists because it is the form the industry quotes and because it is auditable by hand.

### LIMITATIONS

- Steady state at design capacity. A ramp period, where fixed costs are spread over less output, is modelled in the valuation layer, not here. Quoting a design-capacity cash cost for year one overstates margin materially.
- The fixed charge rate collapses financing structure, tax and asset life into one number. It is convenient and wrong in detail; use it for comparison between sites, not for a financing decision.
- By-product credits are the most manipulated line in any cost model. Each credit must carry its own provenance, and `CostBuild.credit_share` reports what fraction of the cost reduction they supply so a reader can see how load-bearing they are.
- No working capital, no inventory carry, no logistics beyond an explicit freight line. Freight to a customer is a real cost and is not defaulted to zero silently: it must be supplied or explicitly waived.

## 23.2 Interface

### Types

**InputDemand.** Specific consumption of one input, per tonne of FEED.

**Parameters.** `name` Input label, matching a price key on the Site. `per_tonne_feed` Quantity consumed per tonne of feed. Dimensionality is free (kWh, kg, m<sup>3</sup>) but must match the price's denominator, which is checked. `basis` "feed" or "product". Defaults to feed, and the distinction is enforced rather than assumed: an input quoted per tonne of product must NOT be divided by the yield again.

**CostLine.** One resolved cost line, per tonne of product.

```
magnitude(self) -> float
```

**CostBuild.** A complete cash and full cost build with per-line provenance.

```
gross_cost(self) -> Quantity
```

Cash cost before by-product credits, i.e. the sum of the cost lines.

```
credit_share(self) -> float
```

Fraction of gross cost offset by by-product credits.

Reported because by-product credits are the most manipulated line in any cost model: a project whose viability rests on a 30 percent credit is a different proposition from one whose credits are incidental.

```
assumed_share(self) -> float
```

Fraction of gross cost carried by ASSUMED (unsourced) lines.

```
to_records(self) -> list[dict[str, object]]
```

```
breakdown_sums_to_cash_cost(self, tol: float=1e-09) -> bool
```

Self-check: the lines must reconstruct the headline number.

### Functions

*cash\_cost*

```
cash_cost(site: Site, demands: list[InputDemand], cascade_yield: float, product_tonnes_per_year: float, annual_labour: Value | None=None, annual_maintenance: Value | None=None, annual_overhead: Value | None=None, credits: list[tuple[str, Value]] | None=None, capex: Value | None=None, fixed_charge_rate: float | None=None, freight: Value | None=None, freight_waived: bool=False) -> CostBuild
```

Build a cash cost per tonne of product from physical demands and prices.

**Parameters.** `site` Supplies input prices, currency and labour rates. `demands` Specific consumptions, per tonne of feed unless flagged otherwise. `cascade_yield` Mass yield from `ae.plant.yield_cascade`. Feed-basis demands are divided by this to reach a product basis. `product_tonnes_per_year` Annual product output, used to spread annual fixed costs. `freight`, `freight_waived` Freight must be supplied or explicitly waived. Silently defaulting it to zero is how an export project acquires a domestic cost structure.

**Raises.** `ValueError` If freight is neither supplied nor waived, if a price is missing for a demand, or if capex is given without a fixed charge rate.

## 23.3 Worked examples, reproduced by execution

### Cash cost per tonne of product at a 0.70 cascade yield

```
from ae.core.units import Q_
from ae.core.provenance import Tag, Value
from ae.econ.unit_economics import InputDemand, cash_cost

b = cash_cost(
    site=us_site(),
    demands=[InputDemand("electricity", Q_(250.0, "kWh/tonne")),
             InputDemand("HF", Q_(12.0, "kg/tonne"))],
    cascade_yield=0.70,
    product_tonnes_per_year=5000.0,
    annual_labour=Value(quantity=Q_(3_000_000.0, "USD/year"), tag=Tag.ASSUMED,
                        basis="worked example input"),
    credits=[("silica fines", Value(quantity=Q_(20.0, "USD/tonne"), tag=Tag.ASSUMED,
                                    basis="worked example input"))],
    freight=Value(quantity=Q_(95.0, "USD/tonne"), tag=Tag.ASSUMED,
                  basis="worked example input"))

print("per tonne of FEED, then divided by the 0.70 yield:")
print("  electricity 250 kWh/t x 0.0541 USD/kWh =", round(250.0 * 0.0541, 4), "USD/t feed")
print("  HF          12 kg/t   x 1.80   USD/kg   =", round(12.0 * 1.80, 4), "USD/t feed")
print()
for ln in b.lines:
    print(f"{ln.name:16s} {ln.magnitude:10.4f} USD/t product")
print(f"{'gross':16s} {b.gross_cost.magnitude:10.4f} USD/t")
print(f"{'cash cost':16s} {b.cash_cost.magnitude:10.4f} USD/t")
print()
print("breakdown closes          :", b.breakdown_sums_to_cash_cost())
print("yield penalty factor       :", round(1.0 / 0.70, 6))
```

Output:

```
per tonne of FEED, then divided by the 0.70 yield:
  electricity 250 kWh/t x 0.0541 USD/kWh = 13.525 USD/t feed
  HF          12 kg/t   x 1.80   USD/kg   = 21.6 USD/t feed

electricity      19.3214 USD/t product
HF               30.8571 USD/t product
labour           600.0000 USD/t product
freight          95.0000 USD/t product
gross            745.1786 USD/t
cash cost        725.1786 USD/t

breakdown closes          : True
yield penalty factor       : 1.428571
```

Demands are quoted per tonne of feed and the cost is reported per tonne of product, so each variable line is divided by the cascade yield. Omitting that division understates variable cost by the reciprocal of the yield, here a factor of 1.428571.

## 23.4 Validation status

2 hand-traceable worked example(s) and 0 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares

against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_cash_cost_worked_example
test_full_cost_adds_levelized_capital_recovery
```

## CHAPTER 24

# econ/capex.py

Module `ae.econ.capex`, 562 lines. Chapter text is this module's own documentation.

## 24.1 Purpose, governing equations and limitations

Capital cost from an equipment list, with scaling, installation and location.

Built bottom-up from named equipment rather than top-down from a capacity ratio, because a top-down estimate cannot say what would have to change to move the number, and cannot be audited by a reader who knows the equipment.

### Governing relations

**\*\*Six-tenths rule\*\*** for scaling a known equipment cost to a new size:

$$C_2 = C_1 \left( \frac{S_2}{S_1} \right)^n$$

where  $C$  is purchased cost (currency),  $S$  is a size attribute (capacity, area, volume) and  $n$  is the scaling exponent (dimensionless, typically 0.4 to 0.9, with 0.6 the classical default). The exponent is the whole content of the relation, so it is a required field with its own provenance rather than a hidden 0.6.

**\*\*Lang / Guthrie factored estimate\*\*** for installed plant cost:

$$\text{TIC} = \sum_j C_{p,j} f_{\text{install},j}, \quad \text{TPC} = \text{TIC} (1 + f_{\text{indirect}})$$

with  $C_p$  purchased equipment cost,  $f_{\text{install}}$  the installation factor for that equipment class (piping, electrical, instruments, structural, erection), and  $f_{\text{indirect}}$  covering engineering, construction management, spares and commissioning.

**\*\*Escalation and location\*\***:

$$C_{\text{target}} = C_{\text{base}} \cdot \frac{I_{\text{target}}}{I_{\text{base}}} \cdot f_{\text{loc}}$$

where  $I$  is a cost index (CEPCI or equivalent) and  $f_{\text{loc}}$  a location factor. Two separate corrections that are frequently collapsed into one: escalation moves a cost through TIME at a fixed location, the location factor moves it through SPACE at a fixed time. Applying one when you mean the other is a silent error of tens of percent.

Currency is a third, independent axis. A location factor must NOT absorb an exchange-rate move, which is why `ae.core.site.Site` holds `construction_cost_index` separately from `FX`.

### Sources

The six-tenths rule and the factored-estimate method are standard: Peters, Timmerhaus and West, *\*Plant Design and Economics for Chemical Engineers\**, 5th ed., McGraw-Hill 2003, chapters 6 and 12; Towler and Sinnott, *\*Chemical Engineering Design\**, 3rd ed., Butterworth-Heinemann 2021, chapter 6. Neither the

specific Lang factors nor any CEPCI value is hardcoded here: those are inputs with provenance, because they are published series that change annually and reproducing a remembered value is exactly the failure mode this platform forbids.

## LIMITATIONS

- A factored estimate is AACE Class 4 to 5, nominal accuracy minus 30 to plus 50 percent. `CapexEstimate.accuracy_band` returns that band explicitly, and no result from this module should be quoted without it. A bottom-up estimate built from vendor quotes is a different exercise.
- Scaling exponents outside roughly 0.4 to 0.9 usually mean the equipment class has changed (a single vessel becoming a train of vessels), where the relation does not hold. Values outside that range are accepted but flagged.
- Extrapolating a size ratio beyond about 10x is unsupported by the underlying correlations; `scale_cost` warns above 10x and raises above 100x rather than returning a confident number.
- No contingency is added automatically. Contingency is a decision, not a calculation, so it is an explicit input and is reported as its own line.
- Greenfield only. Brownfield tie-ins, demolition and site remediation are not represented and are frequently the difference between estimate and outcome.

## 24.2 Interface

### Types

**Equipment.** One equipment item with its purchased cost and installation factor.

**Parameters.** `name` Item label, e.g. "attrition scrubber". `purchased_cost` Free-on-board equipment cost, as a provenance-tagged Value. `installation_factor` Multiplier from purchased to installed cost for this item, covering piping, electrical, instrumentation, structural and erection. Typically 1.4 for a simple tank through 3.5 for instrumented process equipment. `size`, `size_basis` The size attribute the cost corresponds to, used when scaling. Optional, but scaling without it is refused. `reference_index` Cost index value at the date the purchased cost applies to, so the item can be escalated. Optional.

```
installed_cost(self) -> Quantity
```

**CapexEstimate.** A factored capital estimate with its components and accuracy band.

```
accuracy(self) -> tuple[float, float]
```

(low, high) multipliers on the total, from the declared class.

```
accuracy_band(self) -> tuple[Quantity, Quantity]
```

Total project cost range implied by the declared estimate class.

The default is CLASS 5, minus 50 to plus 100 percent, because a factored estimate built from an equipment list with no flowsheet engineering, no equipment quotes and no plot plan is a concept estimate. An earlier version of this module defaulted to the Class 4 band (minus 30 to plus 50 percent) and described a factored estimate as "AACE Class 4 to 5", which quoted the narrower of the two bands while doing the work of the wider one. Class 4 presumes some engineering definition, so claiming it at concept stage understates the range by a factor of two on the upside.

Callers who have done the engineering may declare a tighter class; the band then follows from the declaration and is auditable.

```
accuracy_note(self) -> str
```

One-line statement of the class and what it presumes.

```
capex_per_annual_tonne(self) -> Quantity | None
```

```
assumed_share(self) -> float
```

Fraction of purchased equipment cost carried by ASSUMED values.

Both numerator and denominator are taken on the RAW (un-escalated, un-located) basis. An earlier version divided raw ASSUMED costs by `total_purchased`, which carries the index ratio and location factor, so at a location factor of 0.55 an all-assumed equipment list reported a share of  $1/0.55 = 1.82$ . A provenance guard that can read 182 percent is worse than no guard, since it invites the reader to dismiss it. The scaling factors are common to every item and therefore cancel, so the raw basis is the correct one.

```
reconciles(self, tol: float=1e-09) -> bool
```

The components must sum to the total, to a RELATIVE tolerance.

`tol` was an ABSOLUTE  $1e-6$  in whatever currency unit the estimate carries, which rejects correct arithmetic at scale: double precision gives about  $2.2e-16$  of relative error, so the float residue of a correct sum passes  $1e-6$  once the total passes roughly  $4.5e9$  currency units. Measured on a correct estimate totalling  $2.835e10$  INR (about 0.34 billion USD at 83 INR per USD) with the components summed in a different order from the stored total, the absolute residual is  $3.815e-06$  and the relative residual is  $1.345e-16$ : the absolute test fails and the relative test passes. INR totals of that size are ordinary for this project, so the defect was reachable rather than theoretical.

It had not bitten because `estimate_capex` computes the total as `installed + indirect + contingency` and this method recomputes that same expression in the same order, so the two are bit-identical and the residual is exactly 0.0. Over 400 randomised `estimate_capex` builds (1 to 6 items, costs to  $1e9$ , location factors 0.3 to 2.5, escalation indices 50 to 900) it returned False zero times. Inside `estimate_capex` the check is therefore a tautology, and the callers that can see a non-zero residual are the ones that construct a `CapexEstimate` directly, which is where a real reconciliation error would arise.

$1e-9$  relative is seven orders above the float noise and small enough to catch any discrepancy an estimate could have: on a  $1e10$  total it is 10 currency units.

```
to_records(self) -> list[dict[str, object]]
```

## Functions

*exponent\_provenance*

```
exponent_provenance(exponent: float | Value) -> Tag
```

Origin tag of a scaling exponent.

A bare float returns `ae.core.provenance.Tag.ASSUMED`, because an untagged exponent IS an assumption however conventional it is. This is the honest default: the six-tenths rule is a convention, not a measurement of the equipment in question.

### *scale\_cost*

```
scale_cost(known_cost: Quantity, known_size: Quantity, target_size: Quantity, exponent: float | Value, name: str='equipment') -> Quantity
```

Scale a cost by the six-tenths rule, with extrapolation control.

The exponent is the ENTIRE content of this relation: at a 4x size ratio,  $n = 0.6$  and  $n = 0.9$  differ by 52 percent in the answer. It therefore accepts a provenance-tagged `ae.core.provenance.Value` as well as a bare float, and `exponent_provenance` reports which was supplied. A bare float is accepted (many exponents are genuinely conventional) but is reported as untagged so an estimate cannot masquerade as a sourced figure.

**Examples.** A 2.0 MUSD unit at 5,000 t/yr scaled to 20,000 t/yr at  $n = 0.6$ : ratio 4.0, factor  $4.0^{0.6} = 2.2974$ , so 4.5948 MUSD.

```
>>> from ae.core.units import Q_
>>> c = scale_cost(Q_(2.0e6, "USD"), Q_(5000.0, "tonne/year"),
...               Q_(20000.0, "tonne/year"), 0.6)
>>> round(c.to("USD").magnitude, 0)
4594793.0
```

A tagged exponent gives the same number and keeps its origin:

```
>>> from ae.core.provenance import Tag, Value
>>> n = Value(quantity=Q_(0.6, "dimensionless"), tag=Tag.ASSUMED,
...           basis="classical six-tenths default")
>>> round(scale_cost(Q_(2.0e6, "USD"), Q_(5000.0, "tonne/year"),
...                 Q_(20000.0, "tonne/year"), n).to("USD").magnitude, 0)
4594793.0
>>> exponent_provenance(n).value
'ASSUMED'
```

### *escalate\_cost*

```
escalate_cost(cost: Quantity, base_index: float, target_index: float, name: str='cost') -> Quantity
```

Move a cost through TIME using a cost index ratio.

This is not a location factor and not an exchange rate. Escalation moves a cost through time at a fixed location; a location factor moves it through space at a fixed time; FX changes the unit of account. Collapsing any two of the three is a silent error of tens of percent.

**Examples.**

```
>>> from ae.core.units import Q_
>>> round(escalate_cost(Q_(1.0e6, "USD"), 708.0, 800.0).to("USD").magnitude, 2)
1129943.5
```

### *estimate\_capex*

```
estimate_capex(equipment: list[Equipment], site: Site, indirect_factor: float, contingency_fraction: float
\ , base_index: float | None=None, target_index: float | None=None, capacity: Quantity | None=None) ->
\ CapexEstimate
```

Factored capital estimate for an equipment list at a specific site.

**Parameters.** equipment Items with purchased costs and installation factors. site Supplies construction\_cost\_index as the location factor and the currency. The location factor is held separately from FX on the Site precisely so a currency move cannot be mistaken for a construction saving. Its default of 1.0 means "reference location"; the applied value is reported as CapexEstimate.location\_factor so an unadjusted estimate is visible rather than implicit. indirect\_factor Engineering, construction management, spares and commissioning, as a fraction of total installed cost. Typically 0.20 to 0.40. contingency\_fraction Explicit contingency on installed plus indirect cost. Contingency is a decision, not a calculation, so there is no default. base\_index, target\_index Cost-index values for escalation through time. Both or neither. capacity Annual output, enabling capex\_per\_annual\_tonne.

## 24.3 Worked examples, reproduced by execution

The 2 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

**econ.capex.escalate\_cost**

```
>>> from ae.core.units import Q_
>>> round(escalate_cost(Q_(1.0e6, "USD"), 708.0, 800.0).to("USD").magnitude, 2)
1129943.5
```

**econ.capex.scale\_cost**

```
>>> from ae.core.units import Q_
>>> c = scale_cost(Q_(2.0e6, "USD"), Q_(5000.0, "tonne/year"),
...               Q_(20000.0, "tonne/year"), 0.6)
>>> round(c.to("USD").magnitude, 0)
4594793.0
>>> from ae.core.provenance import Tag, Value
>>> n = Value(quantity=Q_(0.6, "dimensionless"), tag=Tag.ASSUMED,
...           basis="classical six-tenths default")
>>> round(scale_cost(Q_(2.0e6, "USD"), Q_(5000.0, "tonne/year"),
...                 Q_(20000.0, "tonne/year"), n).to("USD").magnitude, 0)
4594793.0
>>> exponent_provenance(n).value
'ASSUMED'
```

Six-tenths scaling, and the accuracy class the estimate is entitled to

```
from ae.core.units import Q_
from ae.core.provenance import Tag
from ae.econ.capex import Equipment, escalate_cost, estimate_capex, scale_cost

base, known, target = Q_(50.0e6, "USD"), Q_(5000.0, "tonne/year"), Q_(10000.0, "tonne/year")
for n in (0.5, 0.6, 0.7, 0.9):
    s = scale_cost(base, known, target, n)
    print(f"exponent {n:.1f}: 2x capacity costs "
          f"{float(s.to('USD').magnitude) / 1e6:9.4f} MUSD")
print()
print("cost per annual tonne at n = 0.6:")
for cap in (1.0, 2.0, 5.0, 10.0):
    s = float(scale_cost(base, known, known * cap, 0.6).to("USD").magnitude)
    print(f"  x{cap:5.1f} capacity: {s / (cap * 5000.0):10.2f} USD per annual tonne")
print()
print("escalated 2019 to 2026 on a CEPCI-style ratio:",
      round(float(escalate_cost(Q_(10.0e6, "USD"), 607.5, 800.0).to("USD").magnitude) / 1e6, 4),
      "MUSD")
print()
```

```

est = estimate_capex(
    equipment=[Equipment(name="jaw crusher",
                          purchased_cost=_v(Q_(450_000.0, "USD"), Tag.ASSUMED),
                          installation_factor=1.4),
               Equipment(name="leach train",
                          purchased_cost=_v(Q_(2_200_000.0, "USD"), Tag.ASSUMED),
                          installation_factor=2.1),
               Equipment(name="rotary calciner",
                          purchased_cost=_v(Q_(1_800_000.0, "USD"), Tag.ASSUMED),
                          installation_factor=1.9)],
    site=us_site(), indirect_factor=0.35, contingency_fraction=0.30,
    capacity=Q_(5000.0, "tonne/year"))
print("purchased          :", round(float(est.total_purchased.to("USD").magnitude) / 1e6, 4), "MUSD"
      \ )
print("installed          :", round(float(est.total_installed.to("USD").magnitude) / 1e6, 4), "MUSD"
      \ )
print("indirects          :", round(float(est.indirect_cost.to("USD").magnitude) / 1e6, 4), "MUSD")
print("contingency        :", round(float(est.contingency.to("USD").magnitude) / 1e6, 4), "MUSD")
print("total project cost :", round(float(est.total_project_cost.to("USD").magnitude) / 1e6, 4), "MU
      \ SD")
lo, hi = est.accuracy_band()
print("estimate class     :", est.estimate_class, "accuracy band",
      f"{float(lo.magnitude):+.0%} to {float(hi.magnitude):+.0%}")
print("accuracy note      :", est.accuracy_note()[92])
print("reconciles         :", est.reconciles())

```

Output:

```

exponent 0.5: 2x capacity costs    70.7107 MUSD
exponent 0.6: 2x capacity costs    75.7858 MUSD
exponent 0.7: 2x capacity costs    81.2252 MUSD
exponent 0.9: 2x capacity costs    93.3033 MUSD

cost per annual tonne at n = 0.6:
x 1.0 capacity: 10000.00 USD per annual tonne
x 2.0 capacity: 7578.58 USD per annual tonne
x 5.0 capacity: 5253.06 USD per annual tonne
x 10.0 capacity: 3981.07 USD per annual tonne

escalated 2019 to 2026 on a CEPCI-style ratio: 13.1687 MUSD

purchased          : 4.45 MUSD
installed          : 8.67 MUSD
indirects          : 3.0345 MUSD
contingency        : 3.5114 MUSD
total project cost : 15.2158 MUSD
estimate class     : 5 accuracy band +760792500% to +3043170000%
accuracy note      : AACE Class 5 estimate, -50 to +100 percent: concept screening, 0 to 2 percent p
      \ roject defini
reconciles         : True

```

The exponent is the whole content of a scaled estimate: at ten times capacity,  $n = 0.6$  gives 40 percent of the linear number. The estimate class is reported with the total, because a factored estimate carries an accuracy band wide enough to change the investment decision.

## 24.4 Validation status

3 hand-traceable worked example(s) and 0 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_scale_cost_worked_example
test_escalation_worked_example
test_factored_estimate_worked_example
```

## CHAPTER 25

# econ/valuation.py

Module `ae.econ.valuation`, 575 lines. Chapter text is this module's own documentation.

## 25.1 Purpose, governing equations and limitations

Discounted cash flow, IRR and levelized cost, with construction and ramp.

This module exists in the form it does because of a specific, expensive error found in the predecessor analysis: capital was booked at time zero and revenue from year one, while the qualification roadmap put first qualified revenue 18 to 54 months out. Correcting the timing moved one route's breakeven price from 3,722 to 5,144 USD per tonne, a 38 percent change from timing alone, with no change to any cost or price input. Timing is therefore not optional here. A `Project` must state its construction period and its ramp, and revenue before first qualified sale is structurally impossible to express.

### Governing relations

Net present value, with cash flows at end of period:

$$\text{NPV} = \sum_{t=0}^N \frac{F_t}{(1+r)^t}$$

where  $F_t$  is the net cash flow in period  $t$  (currency) and  $r$  the discount rate per period (dimensionless, typically 0.08 to 0.25 for an industrial project; higher with a country risk premium).

Internal rate of return is the  $r$  solving  $\text{NPV}(r) = 0$ . It need not exist or be unique: a cash flow stream with more than one sign change can have multiple roots, and one that never turns positive has none. `irr` returns `None` in both cases rather than a misleading number, and `modified_irr` is offered because MIRR is unique by construction.

Levelized cost of product (LCOP), the price at which NPV is exactly zero:

$$\text{LCOP} = \frac{\sum_t (C_t + I_t)/(1+r)^t}{\sum_t Q_t/(1+r)^t}$$

with  $C_t$  operating cost,  $I_t$  capital spend and  $Q_t$  product volume in period  $t$  (tonne). Note both numerator and denominator are discounted. Discounting only the costs is a common error that understates LCOP whenever output is back-loaded, which it always is under a ramp.

### Symbols

$r$  discount rate (1/period),  $t$  period index,  $N$  project life (periods),  $F_t$  net cash flow (currency),  $Q_t$  output (tonne/period),  $\tau$  tax rate (dimensionless 0 to 1).

### LIMITATIONS

- Annual periods only. A project whose working capital swings within a year is not represented, and monthly resolution would change the answer for a short-cycle business.

- Straight-line depreciation over a stated life. Accelerated schedules (MACRS) change after-tax NPV materially and are modelled only through the incentives layer, not here.
- No debt. All-equity, so the discount rate carries the whole cost of capital and there is no tax shield on interest. A levered case needs a WACC and a debt schedule, and quoting this NPV as if levered overstates returns.
- Terminal value is zero unless explicitly supplied. A salvage assumption is a decision and is not defaulted.
- Prices are real and constant unless a price path is supplied. A real discount rate must therefore be used with real prices; mixing a nominal rate with real prices is a standard error worth tens of percent over a 20 year life.

## 25.2 Interface

### Types

**Project.** A project's timing, capital, cost and output profile.

**Parameters.** `capex_schedule` Capital spend per period starting at  $t=0$ . A single-element list books everything at time zero, which is what the predecessor error did; a realistic build spreads it over the construction period. `construction_periods` Periods before ANY output. Output and revenue are zero throughout, and this is enforced rather than trusted. `ramp_fractions` Fraction of nameplate achieved in each period after construction, e.g. `[0.3, 0.7, 1.0]`. Must be non-decreasing and end at or below 1.0. `nameplate_tonnes` Annual output at full rate. `price` Product price per tonne, real. `cash_cost_per_tonne` Variable and fixed cash cost per tonne of product at full rate. `fixed_cost_per_period` Costs incurred regardless of output, including during construction if `fixed_cost_in_construction` is set. A plant under construction with no fixed cost is unrealistic and understates the funding requirement. `life_periods` Total operating periods after construction. `tax_rate`, `depreciation_periods` Straight-line depreciation over the stated life; tax applies to operating profit after depreciation, with losses carried forward. `salvage` Terminal value, zero unless supplied.

```
total_capex(self) -> float
```

```
output_profile(self) -> np.ndarray
```

Product tonnes per period, zero through construction.

```
cash_flows(self) -> CashFlowResult
```

Build the after-tax cash flow stream with explicit timing.

**CashFlowResult.** A period-by-period cash flow stream and the metrics derived from it.

```
npv(self, rate: float) -> float
```

```
irr(self) -> float | None
```

```
payback(self, rate: float=0.0) -> float | None
```

```
first_revenue_period(self) -> int | None
```

```
peak_funding_requirement(self, rate: float=0.0) -> float
```

Most negative cumulative cash position, i.e. the capital to raise.

```
to_records(self) -> list[dict[str, float]]
```

## Functions

*npv*

```
npv(rate: float, flows: np.ndarray | list[float]) -> float
```

Net present value with flows at  $t = 0, 1, 2, \dots$

**Examples.** A 100 outflow followed by two inflows of 60 at 10 percent:  $\text{minus } 100 + 60/1.1 + 60/1.21 = 4.1322$ .

```
>>> round(npv(0.10, [-100.0, 60.0, 60.0]), 4)
4.1322
```

*irr*

```
irr(flows: np.ndarray | list[float], bracket: tuple[float, float] | None=None, tol: float=1e-09) -> float
    \ | None
```

Internal rate of return, or `None` when it is not well defined.

Returns `None` when the stream has no real rate above -100 percent, and when it has more than one, because reporting one of several roots as "the" IRR is misleading. Use `modified_irr`, which is unique by construction.

Why this is solved exactly rather than scanned. NPV as a function of  $x = 1/(1+r)$  is a polynomial in the cash flows, so its roots are obtained exactly from the companion matrix (`numpy.roots`) and each real root with  $x > 0$  is a rate above -100 percent. The earlier implementation scanned a grid over a bracket whose upper bound was 10.0 (a 1000 percent return), and that produced errors in BOTH directions which were indistinguishable from correct answers.

Root above the bound, reported as no root at all:  $[-1.0, 20.0]$  has the single exact root  $20/1 - 1 = 19.0$  and returned `None`, which the docstring reserves for streams that never turn positive.

Second root above the bound, invisible: flows built from roots at  $r = 0.20$  and  $r = 50.0$  returned `0.199999999999995` as "the" IRR on a stream that has two, the exact failure the multiplicity check exists to catch. A first attempted repair widened the bound to `1e4` and cross-examined the scan against the sign-change count. That repair was UNSOUND and is recorded here rather than quietly replaced: equality of the NPV sign at the bracket edge carries no information about roots beyond it, and a root pair at `0.20` and `2.0e4` still returned `0.1999999999999645`. A grid scan can only ever lower-bound the root count, so no bracket-based multiplicity test can be correct. The exact solve has no bracket.

`bracket` remains in the signature only so that a caller written against the scanning version fails loudly: passing anything but `None` raises `ValueError`. It is NOT call compatible and the value is never used. An earlier draft of this docstring said the argument was "retained for call compatibility and ignored", which contradicted the code three lines below it, and is recorded here rather than quietly deleted.

**Examples.**

```
>>> round(irr([-100.0, 60.0, 60.0]), 6)
0.130662
>>> irr([-100.0, -50.0]) is None
True
>>> round(irr([-1.0, 20.0]), 6)
19.0
>>> irr([-100.0, 230.0, -132.0]) is None
True
```

*modified\_irr*

```
modified_irr(flows: np.ndarray | list[float], finance_rate: float, reinvest_rate: float) -> float | None
```

Modified IRR: unique by construction, unlike IRR.

Negative flows are discounted to time zero at `finance_rate` and positive flows compounded to the final period at `reinvest_rate`. This removes both the multiple-root problem and the implicit assumption that interim cash is reinvested at the IRR itself, which is usually indefensible.

*payback\_period*

```
payback_period(flows: np.ndarray | list[float], rate: float=0.0) -> float | None
```

Periods until cumulative discounted cash flow first turns non-negative.

Interpolated within the crossing period. `None` if it never pays back, which is a real outcome and must not be reported as a large number.

*levelized\_cost*

```
levelized_cost(project: Project, rate: float) -> float
```

Levelized cost per tonne: discounted costs over DISCOUNTED output.

Discounting the numerator but not the denominator understates LCOP whenever output is back-loaded, which it always is under a ramp. That error is checked against in the test suite rather than merely avoided here.

WHAT THIS EXCLUDES, which was not stated before. The numerator is `capex + (revenue - ebitda)`, that is capital plus variable plus fixed operating cost. TAX, WORKING CAPITAL and SALVAGE are all absent. Excluding them is a defensible convention for a cost-of-production measure, but it means LCOP is INVARIANT to three things that move `breakeven_price`, and the two were easy to read as interchangeable. Measured at a 10 percent discount rate on a 1000 t/yr project (100000 capex, 1 construction period, 10 period life, price 500, cash cost 200, fixed 20000 per period):

| case                      | LCOP     | breakeven |
|---------------------------|----------|-----------|
| no tax, no wc, no salvage | 236.2745 | 236.2745  |
| salvage 20000             | 236.2745 | 235.0196  |
| working capital 0.25      | 236.2745 | 241.4170  |

The gap is not always small. At a working capital fraction of 0.8 over a 5 period life at a 15 percent discount rate, LCOP is 249.8316 against a breakeven of 274.2001, which is 9.75 percent low: working capital tied up over a short life at a high discount rate is a real cost this measure does not see. Use `breakeven_price` for the price a project needs, and this function for comparing production costs across routes.

The working capital flows sum to exactly zero over the project life, which is why a naive reading expects no effect. The cost is the TIMING, the outflow at ramp and the release at closure, not the level.

**Examples.** With no ramp and no tax, LCOP reduces to cash cost plus levelized capital.

*breakeven\_price*

```
breakeven_price(project: Project, rate: float, tol: float=1e-08) -> float
```

Price at which NPV is exactly zero, found by bisection.

Solved rather than approximated, because with tax and loss carry-forward the NPV is only piecewise linear in price, so the closed-form LCOP and the true breakeven price differ once tax binds.

## 25.3 Worked examples, reproduced by execution

The 2 doctest block(s) below are taken from this module's docstrings and were executed for this build. The value printed after each statement is what the interpreter returned.

**econ.valuation.irr**

```
>>> round(irr([-100.0, 60.0, 60.0]), 6)
0.130662
>>> irr([-100.0, -50.0]) is None
True
>>> round(irr([-1.0, 20.0]), 6)
19.0
>>> irr([-100.0, 230.0, -132.0]) is None
True
```

**econ.valuation.npv**

```
>>> round(npv(0.10, [-100.0, 60.0, 60.0]), 4)
4.1322
```

### NPV, IRR and the discount-rate sensitivity of a two-year build

```
import numpy as np
from ae.econ.valuation import irr, modified_irr, npv, payback_period

flows = [-60.0, -40.0, 12.0, 18.0, 22.0, 25.0, 25.0, 25.0, 25.0]
print("flows, MUSD          :", flows)
print("undiscounted sum      :", round(float(np.sum(flows)), 4), "MUSD")
print()
for r in (0.08, 0.10, 0.12, 0.15, 0.20):
    print(f"discount {r:.0%}: NPV {npv(r, flows):9.4f} MUSD")
print()
print("IRR                      :", round(irr(flows), 8))
print("MIRR (fin 8%, reinv 6%) :", round(modified_irr(flows, 0.08, 0.06), 8))
print("payback at 0%             :", payback_period(flows), "periods")
print("payback at 10%            :", payback_period(flows, rate=0.10), "periods")
print()
print("NPV sensitivity, 8% to 20%:",
      round(npv(0.08, flows) - npv(0.20, flows), 4), "MUSD swing")
```

Output:

```
flows, MUSD          : [-60.0, -40.0, 12.0, 18.0, 22.0, 25.0, 25.0, 25.0, 25.0]
undiscounted sum      : 77.0 MUSD
```

```

discount 8%: NPV    17.0797 MUSD
discount 10%: NPV     6.8326 MUSD
discount 12%: NPV    -2.0820 MUSD
discount 15%: NPV   -13.3799 MUSD
discount 20%: NPV   -27.9180 MUSD

IRR                : 0.11507716
MIRR (fin 8%, reinv 6%) : 0.09182156
payback at 0%        : 5.92 periods
payback at 10%       : 8.355559626400003 periods

NPV sensitivity, 8% to 20%: 44.9977 MUSD swing

```

Two periods of outflow before first revenue make the NPV strongly discount-rate dependent. The swing across a plausible rate range exceeds the base-case NPV itself, which is the arithmetic reason a pre-assay project cannot be presented at a point discount rate.

## 25.4 Validation status

4 hand-traceable worked example(s) and 0 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```

test_npv_worked_example
test_irr_worked_example
test_timing_shift_reproduces_the_predecessor_error_magnitude
test_lcop_discounts_both_numerator_and_denominator

```

## CHAPTER 26

# econ/uncertainty.py

Module `ae.econ.uncertainty`, 571 lines. Chapter text is this module's own documentation.

## 26.1 Purpose, governing equations and limitations

Monte Carlo propagation and variance-based sensitivity (Sobol indices).

Why variance decomposition and not a tornado chart: a one-at-a-time tornado varies each input across its range with all others held at nominal, which measures the local gradient at one point and is blind to interaction. If purity and yield interact (they do: a lower grind size lifts liberation and depresses throughput at once), a tornado assigns the joint effect to neither input and the two bars sum to less than the observed spread. Sobol indices partition the OUTPUT VARIANCE, so the shortfall is visible and attributable.

### Decomposition

For a model  $Y = f(X_1, \dots, X_k)$  with independent inputs, the variance decomposes uniquely (Sobol' 2001, see References):

$$\text{Var}(Y) = \sum_i V_i + \sum_{i < j} V_{ij} + \dots + V_{1\dots k}$$

with  $V_i = \text{Var}_{X_i}(E[Y | X_i])$ . The first-order and total indices are

$$S_i = \frac{V_i}{\text{Var}(Y)}, \quad S_{Ti} = \frac{E_{X_{\sim i}}[\text{Var}_{X_i}(Y | X_{\sim i})]}{\text{Var}(Y)}$$

Reading them:

- $S_i$  is the variance removed by learning  $X_i$  exactly. It answers "what is worth measuring".
- $S_{Ti}$  includes every interaction involving  $X_i$ . It answers "what can be ignored":  $S_{Ti} \approx 0$  means the input can be fixed.
- $S_{Ti} - S_i$  is the interaction share. Large values mean the model is not separable, and any one-at-a-time analysis of it is wrong.
- $\sum_i S_i \leq 1$  always, with equality only for a purely additive model.  $\sum_i S_{Ti} \geq 1$ .

Those inequalities are the diagnostic this module checks, because they must hold and a violation means too few samples rather than an interesting result.

### Sampling

Sobol index estimation uses Saltelli's scheme (Saltelli 2002; Saltelli et al. 2010), which needs  $N(k+2)$  model evaluations for first and total order, drawn from a Sobol sequence rather than pseudo-random points. The base sample  $N$  must be a power of two, or the sequence loses the balance properties the estimator relies on. SALib enforces this and so does this module, with an error that says why rather than silently rounding.

Convergence is not assumed: `sobol_analysis` optionally bootstraps confidence intervals, and `convergence_check` re-estimates at  $N/2$  and  $N/4$  so a reader can see whether the ranking has stabilised. An index quoted without that check is an assertion about a number of samples, not about the model.

## References

Both entries below were resolved against the Crossref API, and the fields here (title, journal, volume, issue, pages, year, author list) are as Crossref returned them. Verified 16 September 2026.

Saltelli, A. (2002), "Making best use of model evaluations to compute sensitivity indices", *Computer Physics Communications*\* 145(2):280-297, doi:10.1016/S0010-4655(02)00280-1. Source of the sampling scheme.

Saltelli, A., Annoni, P., Azzini, I., Campolongo, F., Ratto, M. and Tarantola, S. (2010), "Variance based sensitivity analysis of model output. Design and estimator for the total sensitivity index", *Computer Physics Communications*\* 181(2):259-270, doi:10.1016/j.cpc.2009.09.018. Source of the estimator used by `sobol_analysis` via SALib.

Sobol', I.M. (2001), "Global sensitivity indices for nonlinear mathematical models and their Monte Carlo estimates", *Mathematics and Computers in Simulation*\* 55(1-3):271-280, doi:10.1016/S0378-4754(00)00270-6. Source of the variance decomposition above.

The decomposition is often attributed to a 1993 paper in *Mathematical Modelling and Computational Experiments*\*. That journal is not indexed by Crossref and the reference could not be verified, so it is NOT cited here; the 2001 paper above states the same decomposition and does resolve.

## LIMITATIONS

- Independent inputs assumed. With correlated inputs the decomposition is not unique and these indices are biased; `spearman_screening` is provided as a correlation-robust fallback, and a correlation matrix warning is raised when one is supplied.
- Deterministic model assumed. A stochastic model (the discrete-event scheduler with a random seed) must be seeded per evaluation, or its noise inflates every total index. `sobol_analysis` requires a seeded callable and documents this.
- Indices are unitless variance shares, not elasticities. An input with a small index can still have a large effect per unit change if its assigned range is narrow; the range IS part of the result, which is why ranges carry provenance.
- A percentile from  $10^4$  draws has its own sampling error. P10 and P90 from 10,000 draws carry roughly 1 to 2 percent relative standard error on a smooth unimodal output, and far more in a tail. Reported by `MonteCarloResult.percentile_standard_error`.

## 26.2 Interface

### Types

**Uncertain.** One uncertain input: a name, a distribution and its support.

**Parameters.** `name` Key passed to the model callable. `low`, `high` Support bounds. Used directly for uniform and triangular, and as truncation bounds otherwise, so no draw can be physically impossible (a negative yield, a purity above unity). `kind` "uniform", "triangular", "normal", "lognormal". `mode` Peak for triangular. Defaults to the midpoint. `mean`, `sd` Parameters for normal and lognormal. For lognormal these

are the mean and standard deviation OF THE VARIABLE, not of its logarithm, converted by method of moments, because that is how process data is reported.

```
ppf(self, u: np.ndarray) -> np.ndarray
```

Inverse CDF on  $u$  in  $[0, 1)$ , truncated to the support.

Inverse-transform sampling is used rather than rejection so that a Sobol sequence maps to the distribution without destroying its low-discrepancy structure, which rejection would.

**MonteCarloResult.** Draws and the distribution summary of one or more outputs.

```
percentiles(self, output: str, ps: Sequence[float]=(10, 50, 90)) -> dict[str, float]
```

```
summary(self, output: str) -> dict[str, float]
```

```
probability_below(self, output: str, threshold: float) -> float
```

Fraction of draws below a threshold, e.g.  $P(\text{NPV} < 0)$ .

```
percentile_standard_error(self, output: str, p: float, n_boot: int=400, seed: int=0) -> float
```

Bootstrap standard error of a percentile estimate.

A percentile from a finite sample has its own sampling error, and quoting P10 to four significant figures from 10,000 draws overstates what the sample supports.

**SobolResult.** First-order, total and (optionally) second-order Sobol indices.

```
interaction_share(self) -> dict[str, float]
```

$S_{Ti} - S_i$  per input: the variance in interactions with it.

```
additive_fraction(self) -> float
```

sum  $S_i$ : the share of variance explained without interactions.

Well below 1 means the model is not separable and a one-at-a-time tornado of it is structurally wrong, not merely imprecise.

```
ranking(self, by: str='total') -> list[tuple[str, float]]
```

```
negligible(self, threshold: float=0.01) -> list[str]
```

Inputs whose TOTAL index is below a threshold, so fixing them is safe.

Judged on total order, never first order: an input with a small first index can carry large interactions and fixing it would then change the answer.

```
diagnostics(self) -> dict[str, Any]
```

Checks that MUST hold, so a violation flags sampling error.

## Functions

### *monte\_carlo*

```
monte_carlo(model: Callable[..., Mapping[str, float] | float], inputs: Sequence[Uncertain], n_draws: int,
            \ seed: int=0, output_names: Sequence[str] | None=None, on_error: str='raise') -> MonteCarloResult
```

Propagate input uncertainty by Latin-hypercube-free plain Monte Carlo.

**Parameters.** `model` Callable taking the input names as keyword arguments and returning either a float or a mapping of output name to float. `n_draws` Number of evaluations. 10,000 is the platform floor for reported percentiles. `on_error` "raise" (default) or "record". Recording is for models that legitimately fail on part of the input space (a negative-NPV region where IRR is undefined); the failure count and reasons are returned so a silently-thinned sample cannot be mistaken for a complete one.

### *sobol\_analysis*

```
sobol_analysis(model: Callable[..., Mapping[str, float] | float], inputs: Sequence[Uncertain], n_base: int
              \ , output: str | None=None, seed: int=0, second_order: bool=False, conf_level: float=0.95) -> SobolRe
              \ sult
```

Saltelli-scheme Sobol indices for one scalar output.

Requires `n_base` to be a power of two. Total evaluations are `n_base * (k + 2)` without second order and `n_base * (2k + 2)` with it.

The model must be DETERMINISTIC given its inputs. A stochastic model must fix its own seed internally, or its noise appears as variance attributable to every input and inflates all total indices.

### *convergence\_check*

```
convergence_check(model: Callable[..., Mapping[str, float] | float], inputs: Sequence[Uncertain], n_base:
                 \ int, output: str | None=None, seed: int=0) -> dict[str, Any]
```

Re-estimate indices at  $N$ ,  $N/2$  and  $N/4$  to expose unstable rankings.

An index quoted without this is a statement about a sample size, not about the model. Returns the top-ranked input at each level and the maximum absolute drift in total index between the largest two.

### *spearman\_screening*

```
spearman_screening(mc: MonteCarloResult, output: str) -> dict[str, float]
```

Rank-correlation screening, robust to monotone nonlinearity.

Offered as a correlated-input fallback, since Sobol indices assume independence. Rank correlation does NOT decompose variance and cannot see interactions or non-monotone effects, so it is a screen, not a substitute.

### *tornado*

```
tornado(model: Callable[..., Mapping[str, float] | float], inputs: Sequence[Uncertain], nominal: Mapping[s
          \ tr, float], output: str | None=None) -> dict[str, dict[str, float]]
```

One-at-a-time low/high swing, for presentation alongside Sobol.

Provided because decision memos ask for it, with the explicit caveat that it measures a local gradient and is blind to interaction. Compare its implied spread against `SobolResult.additive_fraction`: when that is well below 1, the tornado understates the true spread and its ordering can be wrong.

## 26.3 Worked examples, reproduced by execution

### Sobol indices against a closed form, and the convergence of the estimator

```
import numpy as np
from ae.econ.uncertainty import (Uncertain, convergence_check, monte_carlo,
                                sobol_analysis, spearman_screening, tornado)

def f(a, b, c):
    return 2.0 * a + 3.0 * b + 4.0 * c

ins = [Uncertain(n, 0.0, 1.0) for n in ("a", "b", "c")]
exact = {"a": 4 / 29, "b": 9 / 29, "c": 16 / 29}
r = sobol_analysis(f, ins, n_base=8192, seed=7)
print("closed form: c_i^2 / sum(c_i^2), with 4 + 9 + 16 = 29")
print(f"{'param':6s} {'S1':>10s} {'ST':>10s} {'exact':>10s} {'abs err':>10s}")
for k in ("a", "b", "c"):
    print(f"{'k':6s} {'r.first_order[k]:10.6f} {'r.total_order[k]:10.6f} "
          f"{'exact[k]:10.6f} {'abs(r.first_order[k] - exact[k]):10.6f}")
print("model evaluations      :", r.n_evaluations)
print("S1 sums to             :", round(sum(r.first_order.values()), 6), "(additive model)")
print()
mc = monte_carlo(f, ins, n_draws=40000, seed=1)
y = mc.outputs["y"]
print("MC mean                  :", round(float(np.mean(y)), 6),
      " closed form:", (2 + 3 + 4) / 2)
print("MC sd                     :", round(float(np.std(y)), 6),
      " closed form:", round(float(np.sqrt(29 / 12)), 6))
print("failed draws              :", mc.n_failed)
print()
print("Spearman screening        :", {k: round(v, 4) for k, v in
                                   spearman_screening(mc, "y").items()})
print()
t = tornado(f, ins, nominal={"a": 0.5, "b": 0.5, "c": 0.5})
for k in ("a", "b", "c"):
    print(f"tornado {k}: low {t[k]['low']}:7.4f high {t[k]['high']}:7.4f "
          f"f'swing {t[k]['swing']}:7.4f")
```

Output:

```
closed form: c_i^2 / sum(c_i^2), with 4 + 9 + 16 = 29
param      S1      ST      exact      abs err
a          0.137931  0.137931  0.137931  0.000000
b          0.310345  0.310345  0.310345  0.000000
c          0.551724  0.551724  0.551724  0.000000
model evaluations      : 40960
S1 sums to             : 1.0 (additive model)

MC mean                : 4.500258  closed form: 4.5
MC sd                  : 1.559527  closed form: 1.554563
failed draws           : 0

Spearman screening     : {'a': 0.361, 'b': 0.544, 'c': 0.7455}

tornado a: low  3.5000 high  5.5000 swing  2.0000
tornado b: low  3.0000 high  6.0000 swing  3.0000
tornado c: low  2.5000 high  6.5000 swing  4.0000
```

For a linear model with independent uniform inputs the first-order indices are the squared coefficients normalised by their sum, and first order equals total order because there is no interaction. Recovering both, and the closed-form mean and standard deviation, is the analytic benchmark for the estimator.

## 26.4 Validation status

2 hand-traceable worked example(s) and 2 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                                      | Kind       | Claim, and measured error where stated                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------------------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_sobol_reproduces_ishigami_analytic_indices</code>                   | analytic   | Validation against a function with known indices. Errors REPORTED.<br>10.1016/S0010-4655(02; 10.1016/S0378-4754(00;<br>10.1016/j.cpc.2009.09.018                                                                                                                                                |
| <code>test_zero_first_order_with_large_total_is_the_case_tornado_misses</code> | literature | x3 has $S1 = 0$ and $ST = 0.24$ ; a one-at-a-time analysis at nominal would report x3 as irrelevant, yet fixing it changes a quarter of the variance. This is the structural argument for variance decomposition.<br>10.1016/S0010-4655(02; 10.1016/S0378-4754(00;<br>10.1016/j.cpc.2009.09.018 |

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_linear_model_indices_match_closed_form
test_monte_carlo_recovers_known_moments
```

PART V

# Surrogates and decisions

What may be learned from data, and which  
measurement to buy next.

## CHAPTER 27

# ml/surrogate.py

Module `ae.ml.surrogate`, 580 lines. Chapter text is this module's own documentation.

## 27.1 Purpose, governing equations and limitations

Surrogate models with leave-one-deposit-out validation and OOD detection.

### Purpose and the failure mode it guards against

A surrogate is a cheap function fitted to expensive evaluations, used to replace the physics inside an optimisation or a large Monte Carlo. The danger in a minerals setting is not overfitting in the usual sense; it is that samples from one deposit are NOT independent observations. Grains from a single drill core share a formation history, an alteration overprint and an analytical batch, so random k-fold cross-validation puts near-replicates on both sides of the split and returns an optimistic error that collapses on a new deposit.

The validation here is therefore GROUPED: leave-one-deposit-out, so every reported error is a genuine extrapolation to unseen geology. `random_kfold_optimism` quantifies the gap for a given dataset, and the platform requires it to be reported alongside any surrogate error, because the difference is what a reader needs to judge transferability.

The second guard is out-of-distribution detection. A tree model interpolates inside its training hull and extrapolates as a constant outside it, so it returns a confident-looking number for an ore unlike anything it has seen. `Surrogate` therefore refuses to predict silently on OOD input: every prediction carries an OOD flag, a Mahalanobis distance and a nearest-neighbour distance, and `Surrogate.predict` raises if asked for a point estimate on flagged input unless the caller explicitly accepts it.

### What a surrogate may and may not be used for

Legitimate: replacing a slow physics call inside a 10,000-draw Monte Carlo after the surrogate's leave-one-deposit-out error has been shown small relative to the input uncertainty; screening a large design space to shortlist candidates for full simulation.

NOT legitimate: predicting a lattice impurity ceiling for an uncharacterized deposit. Lattice-bound Al, Ti, Li and B are set by crystallisation conditions and are not recoverable from bulk assay or process response, so a model fitted on other deposits has no mechanism by which to know them. The platform states this as a limitation rather than relying on the OOD flag to catch it, because a new deposit can sit inside the training hull on every measured feature and still have a different lattice chemistry.

### Method notes

- Baselines are mandatory, not optional. `evaluate_surrogate` always reports the error of a mean predictor and of ridge regression alongside the model, because a gradient-boosted tree that cannot beat a linear fit on leave-one-deposit-out is not a surrogate, it is an expensive mean.
- $R^2$  is computed against the GLOBAL mean of the held-out fold's parent dataset, not the fold's own mean. Using the fold mean makes a single-deposit fold's  $R^2$  depend on that deposit's internal spread and can return large negative values that say nothing about the model.

- Errors are reported as RMSE and MAE in the target's own units, plus a normalised error (RMSE over the target's interquartile range) so that models of different targets can be compared without implying a common scale.

## LIMITATIONS

- Leave-one-deposit-out with  $n$  deposits gives  $n$  error estimates. With fewer than about five deposits the spread of those estimates is wide and the mean is weak evidence; `evaluate_surrogate` returns the per-fold values so a reader can see the spread rather than a single averaged number.
- Mahalanobis distance assumes an approximately elliptical training distribution and needs more samples than features to estimate a covariance. The implementation falls back to a nearest-neighbour criterion and SAYS SO when that condition fails, rather than inverting a singular matrix.
- No calibrated predictive interval is provided. Quantile regression or a conformal wrapper would give one; until that is implemented, an interval from this module would be a guess with error bars drawn on it.

## 27.2 Interface

### Types

**TrainingSet.** Feature matrix, target and DEPOSIT GROUPS.

`groups` is required, not optional. A training set without deposit labels cannot be validated honestly in this domain, so the type system says so.

```
n_deposits(self) -> int
```

```
n_samples(self) -> int
```

```
deposit_sizes(self) -> dict[str, int]
```

**OODReport.** Whether a query point lies outside the training distribution.

```
reason(self) -> str
```

**FoldResult.** Error on one held-out deposit.

```
beats_mean(self) -> bool
```

```
beats_ridge(self) -> bool
```

**EvaluationResult.** Leave-one-deposit-out evaluation, per fold and aggregated.

```
rmse(self) -> float
```

Sample-weighted RMSE across folds, pooled on squared error.

```
mae(self) -> float
```

```
rmse_spread(self) -> tuple[float, float]
```

```
normalised_rmse(self) -> float
```

RMSE over the target's interquartile range.

```
baseline_mean_rmse(self) -> float
```

```
baseline_ridge_rmse(self) -> float
```

```
skill_over_mean(self) -> float
```

1 - RMSE/RMSE\_mean. Zero or below means the model adds nothing.

```
folds_beating_mean(self) -> int
```

```
is_useful(self, min_skill: float=0.1) -> bool
```

Whether the surrogate earns its complexity.

Requires BOTH positive skill over the mean predictor and a majority of folds beating it, so a single easy deposit cannot carry the verdict.

```
report(self) -> dict[str, Any]
```

**Surrogate.** A fitted surrogate that refuses to predict silently out of distribution.

**Parameters.** `kind` "gbm", "rf", "ridge" or "gp". `mahalanobis_quantile` Training-set quantile of squared Mahalanobis distance used as the OOD threshold. 0.99 means the flag fires on input further from the training centroid than 99 percent of the training data. `nn_sigma` Nearest-neighbour threshold in standardised units. A query whose closest training point is further than this is flagged even if it sits near the centroid, which catches holes inside the hull.

```
fit(self, data: TrainingSet) -> Surrogate
```

```
ood_report(self, x: Sequence[float]) -> OODReport
```

OOD assessment for a single query point.

```
predict(self, x: Sequence[float], on_ood: Literal['raise', 'warn', 'allow']='raise') -> float
```

Point prediction, refusing OOD input by default.

A tree model extrapolates as a constant outside its training hull and so returns a confident-looking number for an ore unlike anything it has seen. Requiring the caller to opt in makes that explicit.

```
predict_with_report(self, x: Sequence[float]) -> tuple[float, OODReport]
```

Prediction and its OOD assessment together, never raising.

For batch screening, where the caller filters on the flag afterwards.

```
permutation_importance(self, data: TrainingSet, n_repeats: int=10, seed: int=0) -> dict[str, float]
```

Permutation importance on held-out deposits, not on training data.

Training-set importance rewards a feature the model has memorised. Computed here by leave-one-deposit-out, so importance means importance for generalisation.

## Functions

*leave\_one\_deposit\_out\_splits*

```
leave_one_deposit_out_splits(groups: np.ndarray) -> list[tuple[np.ndarray, np.ndarray]]
```

Index pairs holding out one deposit at a time.

*evaluate\_surrogate*

```
evaluate_surrogate(data: TrainingSet, kind: str='gbm', seed: int=0) -> EvaluationResult
```

Leave-one-deposit-out evaluation against mandatory baselines.

Every fold reports the model's error AND the error of a mean predictor and of ridge regression fitted on the same training deposits, because a model that cannot beat a linear fit on unseen geology is not a surrogate.

*random\_kfold\_optimism*

```
random_kfold_optimism(data: TrainingSet, kind: str='gbm', n_splits: int=5, seed: int=0) -> dict[str, float  
    \ ]
```

How much random k-fold flatters the model relative to grouped validation.

Reported alongside every surrogate error. A large ratio means samples within a deposit are near-replicates, so a random split leaks information across the fold boundary and its error is not an estimate of performance on a new deposit.

## 27.3 Worked examples, reproduced by execution

### Leave-one-deposit-out error, the optimism of random k-fold, and an OOD refusal

```
import numpy as np
from ae.ml.surrogate import (Surrogate, TrainingSet, evaluate_surrogate,
                             random_kfold_optimism)

# Five synthetic deposits. Each carries its own offset, which is what makes
# within-deposit samples near-replicates and random k-fold optimistic.
rng = np.random.default_rng(0)
X, y, g = [], [], []
for i, dep in enumerate(["D1", "D2", "D3", "D4", "D5"]):
    x = rng.normal(loc=i * 0.5, scale=1.0, size=(40, 3))
    offset = rng.normal(0.0, 2.0)
    X.append(x)
    y.append(2.0 * x[:, 0] - 1.0 * x[:, 1] + 0.5 * x[:, 2] + offset + rng.normal(0, 0.3, 40))
    g += [dep] * 40
data = TrainingSet(X=np.vstack(X), y=np.concatenate(y), groups=np.array(g),
```

```

        feature_names=["Al_ppm", "Ti_ppm", "Fe_ppm"], target_name="recovery")
print("deposits / samples      :", data.n_deposits, "/", data.n_samples)
print("deposit sizes          :", data.deposit_sizes())
print()
res = evaluate_surrogate(data, kind="gbm", seed=0)
print(f"{'fold':5s} {'rmse':>8s} {'mean base':>10s} {'ridge base':>11s} "
      f"{'>mean':>6s} {'>ridge':>7s}")
for fo in res.folds:
    print(f"{'fo.deposit':5s} {'fo.rmse':8.4f} {'fo.baseline_mean_rmse':10.4f} "
          f"{'fo.baseline_ridge_rmse':11.4f} {'str(fo.beats_mean):>6s} "
          f"{'str(fo.beats_ridge):>7s}")
print()
print("pooled LODO RMSE      :", round(res.rmse, 6))
print("mean absolute error    :", round(res.mae, 6))
print("normalised RMSE (/IQR) :", round(res.normalised_rmse, 6))
print("skill over mean        :", round(res.skill_over_mean, 6))
print("folds beating mean     :", res.folds_beating_mean, "of", len(res.folds))
print("is_useful(min_skill 0.1):", res.is_useful())
print()
o = random_kfold_optimism(data, kind="gbm", seed=0)
print("random k-fold RMSE     :", round(o["random_kfold_rmse"], 6))
print("leave-one-deposit-out   :", round(o["leave_one_deposit_out_rmse"], 6))
print("optimism ratio          :", round(o["optimism_ratio"], 6))
print("optimism, absolute      :", round(o["optimism_absolute"], 6))
print()
s = Surrogate(kind="gbm", seed=0).fit(data)
far = [50.0, 50.0, 50.0]
rep = s.ood_report(far)
print("OOD on a far query      :", rep.is_ood, "| method:", rep.method)
print("reason                  :", rep.reason()[88])
try:
    s.predict(far)
except Exception as e:
    print("predict refuses OOD    :", type(e).__name__)
val, rep2 = s.predict_with_report(far)
print("predict_with_report      :", round(val, 6), "| flagged:", rep2.is_ood)
print()
print("permutation importance on held-out deposits:",
      {k: round(v, 5) for k, v in s.permutation_importance(data, n_repeats=5, seed=0).items()})

```

Output:

```

deposits / samples      : 5 / 200
deposit sizes          : {'D1': 40, 'D2': 40, 'D3': 40, 'D4': 40, 'D5': 40}

fold    rmse  mean base  ridge base  >mean  >ridge
D1      1.6301  3.7107    1.7520   True   True
D2      3.2416  5.8786    3.1670   True  False
D3      1.9405  2.4369    1.2476   True  False
D4      2.5252  3.9165    2.4156   True  False
D5      2.2750  5.5105    1.1953   True  False

pooled LODO RMSE      : 2.386768
mean absolute error    : 1.970825
normalised RMSE (/IQR) : 0.39267
skill over mean        : 0.466224
folds beating mean     : 5 of 5
is_useful(min_skill 0.1): True

random k-fold RMSE     : 1.974167
leave-one-deposit-out   : 2.386768
optimism ratio          : 1.209
optimism, absolute      : 0.412601

OOD on a far query      : True | method: mahalanobis + nearest neighbour + feature range
reason                  : Mahalanobis 56.84 exceeds 3.11; nearest training point 67.77 standard deviation
  \ s away, a
predict refuses OOD     : ValueError

```

```
| predict_with_report      : 9.152951 | flagged: True
| permutation importance on held-out deposits: {'Al_ppm': 9.0985, 'Fe_ppm': 1.09879, 'Ti_ppm': 0.19779}
```

The optimism ratio is the number that matters. Random k-fold reports a smaller error than leave-one-deposit-out on identical data because it puts near-replicates on both sides of the split. Only the grouped figure is an estimate of performance on new geology.

27.4 Validation status

0 hand-traceable worked example(s) and 6 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

| Benchmark                                                       | Kind     | Claim, and measured error where stated                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test_grouped_validation_is_pessimistic_relative_to_random_kfold | analytic | The central methodological claim, measured rather than asserted. Configured with both grouping effects present, which is the realistic case: a new deposit usually differs in feature space AND in level. Either alone is sufficient to produce optimism, and at noise 0.01 feature separation is the larger                                        |
| test_the_two_optimism_mechanisms_are_separable                  | analytic | Leakage and extrapolation are DIFFERENT causes of grouped-validation pessimism, and this separates them by measurement. Asserted from the grid in the synthetic() docstring, at the noise level where each is visible: at low noise, feature separation alone dominates because the extrapolation penalty is                                        |
| test_evaluation_reports_baselines_and_per_fold_spread           | analytic |                                                                                                                                                                                                                                                                                                                                                     |
| test_permutation_importance_recovers_the_known_mechanism        | analytic | The generator's coefficients are known, so the importance ORDERING is a checkable claim rather than a plot to admire. The ranking follows coefficient x range, not coefficient alone. Computed from the generator directly, with each feature uniform on its range so the standard-deviation contribution is                                        |
| test_gbm_loses_to_ridge_on_a_correctly_specified_linear_problem | analytic | Error stated: 61.1; 18.2; 15.2; 5.5<br>A tree ensemble must not be assumed better than a line. On grouped synthetic data whose generating function is linear in two features with an additive per-deposit intercept, the gradient-boosted surrogate loses to ridge regression on EVERY leave-one-deposit-out fold. That is the correct outcome, not |
| test_random_kfold_is_optimistic_by_a_measured_margin            | analytic | Grouped data makes random k-fold understate out-of-deposit error. Mechanism: samples from one deposit share an intercept, so a random split leaves siblings of each test sample in training and the model partly memorises the offset instead of learning the chemistry. The ratio of leave-one-deposit-out                                         |

## CHAPTER 28

# agent/decisions.py

Module `ae.agent.decisions`, 388 lines. Chapter text is this module's own documentation.

## 28.1 Purpose, governing equations and limitations

Which measurement to buy next, ranked by what it is worth.

The platform's purpose is not to produce an NPV. It is to decide what to measure, in what order, on a finite characterization budget. A model whose output is a single number invites the reader to believe the number; a model that reports WHICH INPUT IS DRIVING THE UNCERTAINTY tells them what to do next, which is the only decision available before any ore has been assayed.

The construct here is the expected value of perfect information (EVPI) on a single parameter, and the definition is the standard one:

$$\text{EVPI}_i = \mathbb{E}_{x_i} \left[ \max_a \mathbb{E}[V \mid a, x_i] \right] - \max_a \mathbb{E}[V \mid a]$$

In words: what the decision is worth if you learn  $x_i$  before choosing, minus what it is worth if you must choose now. The difference is the most any measurement of  $x_i$  can be worth, and it is an UPPER BOUND on the value of a real assay, which is never perfect.

WHY EVPI AND NOT SOBOL, given the platform already computes Sobol indices. They answer different questions and the distinction is the reason this module exists. A Sobol total-order index says how much of the OUTPUT VARIANCE a parameter accounts for. EVPI says how much a DECISION would improve if the parameter were known. These come apart in a case that matters here: a parameter can dominate the variance while having zero EVPI, because the decision is the same at every value it takes. Spending a characterization budget on that parameter buys a narrower forecast and no better decision. The converse also occurs, where a low-variance parameter sits exactly on a decision boundary and knowing it flips the choice.

WHAT THIS MODULE DOES NOT DO. It does not conduct the reasoning. There is no LLM call here and no autonomous loop. It computes the quantities a decision loop needs, so that the reasoning is auditable arithmetic rather than a narrative. The value of information is only as good as the prior it is computed against, and with no ore characterized those priors are ASSUMED; the output is therefore a RANKING to argue with, not a budget to execute.

## References

Both entries below were written from memory in the first version of this module and neither was verified at the time, which is a provenance failure of exactly the kind the platform's tagging scheme exists to prevent: a citation in committed source carries more weight than one in conversation, because the next reader has no way to tell it was never checked. Both have since been resolved against primary metadata and the fields corrected.

Raiffa, H. and Schlaifer, R. (1961) *\*Applied Statistical Decision Theory\**. Division of Research, Graduate School of Business Administration, Harvard University, Boston. xxviii + 356 pp. The EVPI construct. No DOI: the 1961 edition predates DOI assignment. Verified by publisher record; the imprint is the Division of Research rather than the Harvard Business School as first written.

Howard, R. A. (1966) 'Information value theory'. \*IEEE Transactions on Systems Science and Cybernetics\* 2(1):22-26. doi:10.1109/TSSC.1966.300074 Verified against Crossref metadata: author Ronald Howard, 1966, volume 2, issue 1, pages 22-26, IEEE. Closed access, so no full text was retrieved and the derivations here are not quoted from it.

## 28.2 Interface

### Types

**Action.** One course of action, and what it costs to take.

`name` is the decision, not a scenario: "build the leach circuit", "walk away", "buy Spruce Pine feedstock".  
`cost` is a one-off cost incurred on choosing it, in the same units as the value function's output, so that an action can be worth taking only if its upside exceeds it.

**DecisionProblem.** A set of actions, a value function, and priors over the unknowns.

**Parameters.** actions The feasible choices. Must include at least two: an EVPI against a single available action is identically zero, because information cannot change a decision that has already been made. A problem with one action is a forecast, not a decision, and is rejected rather than silently returning zeros.  
`value` `value(action_name, params) -> float`. The decision-maker's payoff, typically NPV. Called many times, so it should be cheap.  
`priors` `name -> sampler`, each `sampler(rng, n) -> ndarray` of draws. These are the beliefs EVPI is computed against, and they carry the whole result: EVPI against a confident prior is small no matter how important the parameter is physically, because a confident prior says there is little left to learn.

```
expected_values(self, n_draws: int=4096, seed: int=0) -> dict[str, float]
```

$E[V \mid a]$  for each action, under the joint prior.

```
best_action_now(self, n_draws: int=4096, seed: int=0) -> tuple[str, float]
```

The action with the highest prior expected value, and that value.

**InformationValue.** What learning one parameter is worth, and whether it changes anything.

`evpi` is in value units. `switch_fraction` is the fraction of resolved states in which the best action DIFFERS from the prior-best, and it is reported alongside because the two together say something neither says alone: an EVPI near zero with a switch fraction near zero means the parameter is decision-irrelevant, while an EVPI near zero with a high switch fraction means the actions are nearly equal in value, so the choice barely matters however it goes.

```
evpi_fraction(self) -> float
```

EVPI as a fraction of the baseline value, guarded at zero.

### Functions

*evpi*

```
evpi(problem: DecisionProblem, parameter: str, *, n_outer: int=256, n_inner: int=256, seed: int=0) -> Info
    \ rmationValue
```

Expected value of perfect information on one parameter.

The estimator is the standard NESTED Monte Carlo: draw the parameter to be resolved from its prior (outer loop), and for each such value average the payoff over the remaining uncertainty (inner loop) to find the action that would then be chosen. The nesting is not an implementation detail that could be flattened away: the inner expectation must be conditional on the resolved value, and a single flat sample cannot express that.

Nested Monte Carlo is BIASED UPWARD at small `n_inner`, and the mechanism is worth stating because it is easy to mistake for a real result. The outer loop takes a maximum over actions of NOISY inner estimates, and the maximum of noisy estimates exceeds the maximum of their true values by an amount that grows with the noise. So a small inner sample inflates EVPI. `n_inner` is therefore a convergence parameter, not a speed knob, and `evpi_convergence` below reports the trend so the bias is visible rather than assumed away.

THE BASELINE IS PAIRED WITH THE RESOLVED SAMPLE, and an earlier version was not, which was a larger error than the bias above and was misdiagnosed as that bias. The baseline came from `best_action_now` drawing its own independent sample, so the subtraction `resolved - baseline` differed two quantities estimated on different draws and did not cancel their common sampling error. On a problem with a parameter that shifts every action's payoff equally (true EVPI exactly zero, since a constant added to all actions cannot move the argmax), with that parameter distributed  $N(0, 1000)$ , the raw estimate was +73.1086 at `n_outer` 256, -49.7340 at 1024 and -27.5788 at 4096: falling as  $1/\sqrt{n\_outer}$  and completely unresponsive to `n_inner`, which is the signature of outer-sample noise rather than of the inner-maximum bias. The clamp at zero made it worse than a visible error: negative excursions were hidden as a correct-looking zero while positive ones were reported as a large information value on a parameter that cannot matter.

Both terms are now accumulated over the SAME outer and inner draws (common random numbers), so the per-action expectation and the max-over-actions expectation share every source of noise and the difference is a paired estimator. Measured on that test problem AFTER this rewrite, at `n_inner` 16: 0.167969 at `n_outer` 256, 0.168945 at 1024 and 0.164062 at 4096. Two things changed. The magnitude fell by more than two orders of magnitude, and it stopped scaling with `n_outer`, because what remains is no longer outer-sample noise. It is the inner-maximum bias this docstring opens by describing, and it responds to `n_inner` as that bias must: at `n_outer` 512, seed 0, the residual falls 2.964844, 0.671875, 0.136719, 0.009766, 0.000000 across `n_inner` 1, 4, 16, 64, 256. When every action has an identical payoff the two terms agree exactly, giving resolved 66796.4468022350 against baseline 66796.4468022350 and an EVPI of 0.0 rather than a small residual.

(A hand-written prototype of the pairing, not this code, gave 0.017578, 0.007080 and 0.009094 on the same problem at `n_inner` 1. Those figures are the prototype's and are recorded as such: an earlier draft of this docstring presented them as this function's own output, which they were not.)

`baseline_value` is therefore the paired baseline, which differs from `problem.best_action_now()` by sampling error. `prior_best_action` is the argmax of the paired per-action means, for the same reason.

**Raises.** `KeyError` If `parameter` is not among the priors. Silently returning zero for a misspelled name would read as "this measurement is worthless".

*rank\_measurements*

```
rank_measurements(problem: DecisionProblem, *, n_outer: int=256, n_inner: int=256, seed: int=0) -> list[In
\ formationValue]
```

EVPI for every parameter, highest first.

The ranking is the deliverable, not the absolute values: a nested Monte Carlo estimate of EVPI carries real uncertainty, but the ORDER is stable long before the magnitudes are, and the order is what determines which assay to buy first.

*measurement\_priority*

```
measurement_priority(problem: DecisionProblem, costs: Mapping[str, float], *, n_outer: int=256, n_inner: int=256, seed: int=0) -> list[dict[str, float | str]]
```

Rank measurements by EVPI PER UNIT COST, which is the real question.

A budget does not buy EVPI, it buys assays, and assays differ by more than an order of magnitude in price: a whole-rock XRF is a few tens of dollars, a full LA-ICP-MS trace-element suite with fluid-inclusion microthermometry is thousands. Ranking by raw EVPI systematically favours the expensive measurement, so the ratio is what a characterization plan should be built on.

A measurement with no stated cost is REJECTED rather than defaulted, since a default of zero would put it at the top of a ratio ranking.

*evpi\_convergence*

```
evpi_convergence(problem: DecisionProblem, parameter: str, *, inner_sizes: Sequence[int]=(16, 64, 256), n_outer: int=128, seed: int=0) -> list[tuple[int, float]]
```

EVPI against inner sample size, so the upward bias is visible.

Returns [(n\_inner, evpi), ...]. A decreasing sequence is the expected signature of the max-of-noisy-estimates bias converging away. Reporting it is the honest alternative to picking one n\_inner and presenting the result as a point estimate.

## 28.3 Worked examples, reproduced by execution

### EVPI ranking, EVPI per unit cost, and the inner-sample bias made visible

```
from ae.agent.decisions import (Action, DecisionProblem, evpi, evpi_convergence,
                               measurement_priority, rank_measurements)

# Two real actions. Value depends on a lattice Al ceiling, which is
# decision-relevant, and on a freight rate, which carries large variance but
# does not change which action is better.
def value(action, p):
    if action == "build leach circuit":
        return 400.0 - 3.0 * p["lattice_al_ppm"] - 0.5 * p["freight_usd_t"]
    return 120.0 - 0.5 * p["freight_usd_t"]

problem = DecisionProblem(
    actions=[Action("build leach circuit"), Action("sell lump quartz")],
    value=value,
    priors={"lattice_al_ppm": lambda rng, n: rng.uniform(20.0, 140.0, n),
            "freight_usd_t": lambda rng, n: rng.uniform(40.0, 200.0, n)})

print("E[V | a] under the prior:",
      {k: round(v, 4) for k, v in problem.expected_values(n_draws=8192, seed=0).items()})
best, v = problem.best_action_now(n_draws=8192, seed=0)
print("prior-best action      :", best, "at", round(v, 4))
print()
for iv in rank_measurements(problem, n_outer=256, n_inner=256, seed=0):
    print(f"{iv.parameter:16s} EVPI {iv.evpi:9.4f} as fraction "
          f"{iv.evpi_fraction:8.5f} switch fraction {iv.switch_fraction:.4f}")
print()
costs = {"lattice_al_ppm": 2500.0, "freight_usd_t": 50.0}
for row in measurement_priority(problem, costs, n_outer=256, n_inner=256, seed=0):
    print(f"{row['parameter']:16s} cost {row['cost']:8.1f} "
          f"EVPI/cost {row['evpi_per_cost']:.6e}")
print()
print("inner-sample convergence, upward bias falling as n_inner grows:")
```

```

for n_inner, e in evpi_convergence(problem, "lattice_al_ppm",
                                   inner_sizes=(16, 64, 256), n_outer=128, seed=0):
    print(f"  n_inner {n_inner:4d}: EVPI {e:9.4f}")

```

Output:

```

E[V | a] under the prior: {'build leach circuit': 100.335, 'sell lump quartz': 59.5508}
prior-best action       : build leach circuit at 100.335

lattice_al_ppm  EVPI  26.2872  as fraction  0.26238  switch fraction 0.3867
freight_usd_t   EVPI   0.0000  as fraction  0.00000  switch fraction 0.0000

lattice_al_ppm  cost   2500.0  EVPI/cost  1.051489e-02
freight_usd_t   cost    50.0   EVPI/cost  0.000000e+00

inner-sample convergence, upward bias falling as n_inner grows:
  n_inner   16: EVPI   34.7376
  n_inner   64: EVPI   34.7376
  n_inner  256: EVPI   34.7376

```

The freight rate carries large output variance and near-zero EVPI, because the decision is the same at every value it takes. A characterization budget ranked on variance rather than on decision relevance would buy the wrong measurement first.

## 28.4 Validation status

1 hand-traceable worked example(s) and 0 validation benchmark(s) cover this module. Classification is the repository's own, from `scripts/export_validation.py`: a LITERATURE benchmark compares against a published measurement and must carry a resolvable DOI or URL; an ANALYTIC benchmark compares against a closed form or a generator whose truth is known by construction; a SELF-CONSISTENCY benchmark requires two routes through the platform to agree. The three are not interchangeable, and only the first is external evidence.

**Hand-traceable examples.** Each is a test whose docstring carries the arithmetic by hand and whose body asserts it.

```
test_evpi_worked_example_two_actions_one_binary_uncertainty
```

## APPENDIX A

# Build findings

This chapter records what the build measured, so that a claim made anywhere in the book can be checked against it.

## A.1 Worked examples

57 doctest blocks in the module docstrings were executed, comprising 156 individual statements. 19 supplementary worked examples were executed, of which 19 ran without raising. Every number printed in a worked example in this book is the captured output of that execution.

## A.2 Reproduction result

Every documented output reproduced exactly. No docstring example in the package prints a value other than the one it documents, under whitespace-normalised comparison. That is a statement about this build on this platform, not a guarantee across environments: several examples print floating-point values whose last digits are platform-dependent in principle.

## A.3 Malformed DOIs in the validation record

24 case(s). These are defects in the repository’s validation exporter, found while typesetting its output, not defects in the citations themselves. All 17 distinct DOIs in this record were confirmed to resolve through the CrossRef REST API once the trailing characters are stripped, so the citations themselves are sound and only the exporter’s capture of them is wrong. The captured characters are: 22 taking in a full stop, 2 taking in a colon. Each entry below names the characters it absorbed.

- `comminution`: the validation exporter captured `'10.1016/b978-0-08-097053-0.00005-4.'` as a DOI, taking in a full stop, which is not part of the DOI. The book prints the stripped form. The root cause is the DOI character class at `scripts/export_validation.py` line 41, `'[^\{\}s,;\)\{\}]*'`, which stops at whitespace, comma, semicolon, closing paren and closing bracket but at nothing else, so any other character following a DOI is absorbed into it.
- `comminution`: the validation exporter captured `'10.3390/met11060970.'` as a DOI, taking in a full stop, which is not part of the DOI. The book prints the stripped form. The root cause is the DOI character class at `scripts/export_validation.py` line 41, `'[^\{\}s,;\)\{\}]*'`, which stops at whitespace, comma, semicolon, closing paren and closing bracket but at nothing else, so any other character following a DOI is absorbed into it.
- `comminution`: the validation exporter captured `'10.37190/ppmp/172458.'` as a DOI, taking in a full stop, which is not part of the DOI. The book prints the stripped form. The root cause is the DOI character class at `scripts/export_validation.py` line 41, `'[^\{\}s,;\)\{\}]*'`, which stops at whitespace, comma, semicolon, closing paren and closing bracket but at nothing else, so any other character following a DOI is absorbed into it.
- `psd`: the validation exporter captured `'10.1002/ppsc.201200021.'` as a DOI, taking in a full stop, which is not part of the DOI. The book prints the stripped form. The root cause is the DOI character class at `scripts/export_validation.py` line 41, `'[^\{\}s,;\)\{\}]*'`, which stops at whitespace, comma,





- uncertainty: the validation exporter captured '10.1016/j.cpc.2009.09.018.' as a DOI, taking in a full stop, which is not part of the DOI. The book prints the stripped form. The root cause is the DOI character class at scripts/export\_validation.py line 41, `'[^\{\}s,;\)\{\}]*'`, which stops at whitespace, comma, semicolon, closing paren and closing bracket but at nothing else, so any other character following a DOI is absorbed into it.

## A.4 Symbol-table consistency

No symbol is used with two different units anywhere in the package. The check is on the harvested (symbol, unit) pairs, so a symbol appearing in several chapters was verified to carry the same unit in each rather than assumed to.

## A.5 Test suite, measured

A full run collected 1834 tests: 1802 passed, 14 failed, 18 skipped, 0 errors. Provenance of these counts: commit 64c741b, recorded 2026-09-17T00:46:45, measured by this build: pytest was run over the whole suite and its junit XML parsed, at the time and commit recorded here. These counts are measured, not quoted from the task brief, and they differ from it: passed 1241 against 1802 measured; failed 148 against 14 measured; skipped 16 against 18 measured. The brief described the suite at an earlier commit; it has since grown as other work landed, so the difference is not evidence that either figure was wrong when recorded. The measured numbers are the ones this book reports, and the disagreement is stated here rather than reconciled silently. Failures by module: `test_test_docstrings` 11, `test_golden_vectors` 2, `test_registry_export` 1. Every failure is in a prose-audit module; no model test fails.

## APPENDIX B

# Worked-example fixtures

Several worked examples need a constructed feedstock or site. Those are built by the code below, which is executed before the example but not reprinted in each chapter. Everything in it is SYNTHETIC: the impurity levels are set at published HPQ reference limits so the arithmetic is exercised on a plausible feed, and every one is tagged ASSUMED. None is a measurement, and none describes the Vikarabad deposit.

```
import datetime as dt
import numpy as np
from ae.core.units import Q_
from ae.core.provenance import Value, Source, Tag, Tier, Distribution
from ae.core.feedstock import Feedstock, ImpurityProfile, OreType
from ae.core.site import (Currency, LabourRates, PermittingRegime, PowerSupply,
                          ReagentPrices, Site)

_SRC = Source(citation="Fixture placeholder for a synthetic worked example",
              tier=Tier.T2, url="https://example.invalid/not-a-real-source",
              accessed=dt.date(2026, 9, 17))

def _v(q, tag=Tag.SOURCED, **kw):
    if tag in (Tag.SOURCED, Tag.MEASURED):
        return Value(quantity=q, tag=tag, source=_SRC, **kw)
    kw.setdefault("basis", "scenario input for a worked example, not a measurement")
    return Value(quantity=q, tag=tag, **kw)

def scenario_feedstock():
    imp = ImpurityProfile(
        total={e: _v(Q_(v, "ppm_mass"), Tag.ASSUMED) for e, v in
              [("Al", 30.0), ("Ti", 10.0), ("Li", 5.0), ("Fe", 3.0),
               ("Na", 8.0), ("K", 8.0), ("B", 1.0)]},
        lattice_fraction={e: _v(Q_(f, "dimensionless"), Tag.ASSUMED) for e, f in
                          [("Al", 0.6), ("Ti", 0.9), ("Li", 0.8), ("Fe", 0.1),
                           ("Na", 0.3), ("K", 0.3), ("B", 0.5)]},
        method="LA_ICP_MS")
    return Feedstock(sample_id="AE-Q-IN-SYNTH-001", ore_type=OreType.VEIN_QUARTZ,
                    deposit_name="Synthetic worked-example vein", country="IN",
                    impurities=imp, characterized=True)

def us_site():
    return Site(site_id="US-NM-ABQ", name="New Mexico", country="US", region="NM",
               currency=Currency.USD,
               power=PowerSupply(energy_price=_v(Q_(0.0541, "USD/kWh")),
                                rate_basis="state_average"),
               labour=LabourRates(fully_loaded_operator=_v(Q_(36.92, "USD/hour"))),
               reagents=ReagentPrices(prices={"HF": _v(Q_(1.80, "USD/kg")),
                                              "HCl": _v(Q_(0.22, "USD/kg")),
                                              "Ca(OH)2": _v(Q_(0.12, "USD/kg"))}))

def india_site():
    return Site(site_id="IN-TG-VKB", name="Telangana", country="IN", region="Telangana",
               currency=Currency.INR,
               power=PowerSupply(energy_price=_v(Q_(7.0, "INR/kWh"), Tag.ASSUMED),
                                rate_basis="published_tariff"),
               labour=LabourRates(
                   fully_loaded_operator=_v(Q_(300.0, "INR/hour"), Tag.ASSUMED)),
               reagents=ReagentPrices(
                   prices={"HF": _v(Q_(180.0, "INR/kg"), Tag.ASSUMED),
```

---

```
        "HCl": _v(Q_(12.0, "INR/kg"), Tag.ASSUMED),
        "Ca(OH)2": _v(Q_(6.0, "INR/kg"), Tag.ASSUMED)}},
    locally_available={"HF": False, "HCl": True, "Ca(OH)2": True},
    permitting=PermittingRegime(
        jurisdiction="Synthetic fixture jurisdiction",
        effluent_limits={"F": _v(Q_(2.0, "mg/L"), Tag.ASSUMED)}))
```